Large Language Models: Foundations, Architecture, Training, and Evaluation

Sepehr Kerachi

Copyright © 2026 Sepehr Kerachi. All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without prior written permission of the copyright holder, except as permitted by law.

Edition: First Edition

Contents

I Foundations and Architecture **1**

1 Introduction **2**

2 Architecture **3**

2.1 Tokenization . . . 3
2.1.1 From Written Text to Token Sequences . . . 3
2.1.2 From Word-Level to Subword-Level Representations . . . 5
2.1.3 Balancing Tokens and Types: The Principle of Least Effort . . . 6
2.1.4 A Stochastic Framework for Tokenization . . . 9
2.1.5 Statistical and Computational Properties . . . 12
2.1.6 Limitations and Connection to Positional Encoding . . . 14
2.2 Positional Encoding . . . 15
2.2.1 Why Self-Attention Requires Positional Information . . . 15
2.2.2 Absolute Positional Encoding . . . 17
2.2.3 Combining Content and Position: Addition and Concatenation . . . 19
2.2.4 Relative Positional Encoding and Rotary Position Embedding . . . 20
2.2.5 Long-Context Position Rescaling and Multi-Scale Positional Encoding 21
2.2.6 Limitations and Connection to the Next Architectural Components . 25
2.3 Feed-Forward Networks . . . 25
2.3.1 Position-Wise Feed-Forward Networks . . . 27
2.3.2 Parameterization and Computational Complexity . . . 28
2.3.3 Gated Feed-Forward Networks and Their Internal Units . . . 30
2.3.4 Feed-Forward Networks Within Transformer Blocks . . . 31
2.3.5 From Dense FFNs to Mixture-of-Experts Layers . . . 32
2.3.6 Switch Routing, Expert Capacity, and Load Balancing . . . 33
2.3.7 Fine-Grained Segmentation and Shared Experts . . . 35
2.3.8 Expert Structure, Specialization, and Practical Constraints . . . 38
2.3.9 Limitations and Connection to Activation Functions . . . 39
2.4 Activation Functions . . . 39
2.4.1 Why nonlinear transformations are indispensable . . . 40
2.4.2 Differentiability and gradient propagation . . . 41
2.4.3 Classical and smooth activation functions . . . 42
2.4.4 Gated activation functions . . . 44
2.4.5 Learnable rational activation functions . . . 45
2.4.6 Polynomial composition activation functions . . . 48
2.4.7 Approximation theory and effective dimensionality . . . 50

2.4.8 Empirical performance and computational cost . . . 52
2.4.9 Limitations and connection to normalization . . . 53
2.5 Normalization . . . 54
2.5.1 Architectural motivation and statistical foundations . . . 54
2.5.2 Layer Normalization: formulation and geometry . . . 56
2.5.3 RMSNorm and normalization within residual Transformer blocks . . . 58
2.5.4 Observed input-output behavior of trained normalization layers . . . 60
2.5.5 Dynamic Tanh: formulation, derivatives, and implementation . . . 62
2.5.6 Computational analysis and a worked numerical example . . . 64
2.5.7 Language-model evaluation and initialization . . . 65
2.5.8 Normalization choices in Small Language Models . . . 67
2.5.9 Limitations and connection to the attention mechanism . . . 68
2.6 Attention Mechanisms . . . 69
2.6.1 The role of attention in sequence representation . . . 69
2.6.2 Query, key, and value representations . . . 70
2.6.3 Scaled dot-product attention . . . 71
2.6.4 Self-attention, cross-attention, and causal masking . . . 73
2.6.5 Multi-head attention . . . 74
2.6.6 Backpropagation through attention . . . 76
2.6.7 Attention-head interpretation and functional specialization . . . 78
2.6.8 Computational and memory complexity of dense attention . . . 81
2.6.9 Multi-query and grouped-query attention . . . 82
2.6.10 FlashAttention and FlashAttention-2 . . . 84
2.6.11 Linear attention . . . 87
2.6.12 Sparse attention . . . 90
2.6.13 Hybrid attention architectures . . . 92
2.6.14 Limitations and connection to the next topic . . . 93

II Optimization and Scaling **95**

3 Optimization **96**
3.1 Optimization Algorithms . . . 96
3.1.1 Optimization as a Statistical Learning Problem . . . 96
3.1.2 Stochastic and Batch Gradient Methods . . . 98
3.1.3 First-Order and Adaptive Optimization Algorithms . . . 101
3.1.4 Memory-Efficient and Alternative Update Rules . . . 103
3.1.5 Curvature-Aware and Second-Order Optimization . . . 105
3.1.6 Structured Matrix and Tensor Preconditioning: Shampoo . . . 106
3.1.7 SOAP: Adam in the Shampoo Eigenbasis . . . 109
3.1.8 Low-Rank and Matrix-Orthogonalized Optimizers . . . 110
3.1.9 Systems Constraints and Numerical Implementation . . . 111
3.1.10 Empirical Evaluation and Optimizer Selection . . . 112
3.1.11 Limitations and Connection to the Next Topic . . . 115
3.2 Loss Functions . . . 115
3.3 Quantization . . . 115
3.3.1 Numerical representation and the need for quantization . . . 116
3.3.2 Uniform affine quantization . . . 117

3.3.3 Quantization error and its propagation through neural networks . . 120
3.3.4 Quantization as part of the optimization process 121
3.3.5 Quantization targets and the lifecycle of a language model 124
3.3.6 Numerical formats, mixed precision, and hardware efficiency 127
3.3.7 Evaluation, limitations, and connection to the next topic 128
3.4 Knowledge Distillation 130
3.4.1 Predictive distributions and the mathematical objective of distillation 131
3.4.2 Knowledge interfaces: white-box and black-box distillation 134
3.4.3 Post-training and pre-training distillation 136
3.4.4 On-policy distillation and the student-state distribution 139
3.4.5 Distillation scaling laws and compute allocation 141
3.4.6 Dataset distillation and its integration with model distillation 143
3.4.7 Evaluation of distilled language models 144
3.4.8 Limitations and connection to parameter-efficient fine-tuning 145
3.5 Parameter-Efficient Fine-Tuning 146
3.5.1 From Full Fine-Tuning to Parameter-Efficient Adaptation 146
3.5.2 Additive and Prompt-Based Adaptation 150
3.5.3 Selective and Reparameterized Adaptation 152
3.5.4 Sparse Matrix Tuning and Comparative Analysis 155
3.5.5 Limitations, Research Directions, and Connection to Scaling 160

4 Scaling Laws **163**
4.0.1 Predictive loss and the fundamental scaling variables 164
4.0.2 Power-law formulations and compute-optimal allocation 167
4.0.3 Empirical fitting and validation 170
4.0.4 Inference-efficient and architecture-aware scaling 173
4.0.5 Joint scaling for mixture-of-experts models 177
4.0.6 Limitations, synthesis, and transition 178

III Reinforcement Learning and Neuroscience **180**

5 Reinforcement Learning **181**
5.1 Theoretical Foundations 181
5.1.1 Markov decision processes, policies, and value functions 181
5.1.2 Bellman operators and dynamic programming 184
5.1.3 Policy improvement, performance difference, and approximation . . 185
5.1.4 Information geometry and covariant policy search 188
5.1.5 Trust-region policy optimization 191
5.1.6 Regularized Markov decision processes 194
5.1.7 Mirror descent and modified policy iteration 196
5.1.8 Worked example and connection to autoregressive language models 198
5.1.9 Limitations and connection to the next topic 199
5.2 Actor–Critic Methods 200
5.2.1 Sequential decision-making and the policy represented by the actor 200
5.2.2 Value functions, the critic, and the advantage 202
5.2.3 Temporal-difference estimation and generalized advantage estimation 203
5.2.4 The actor–critic objective in Proximal Policy Optimization 205

5.2.5 Mapping the actor–critic architecture onto language models 207
5.2.6 Computational limitations and the transition beyond a learned critic 208
5.3 Reinforcement Learning with Verifiable Rewards 210
5.3.1 From autoregressive language modeling to outcome-based learning 211
5.3.2 Verifiable rewards and the KL-regularized objective 212
5.3.3 PPO optimization, value estimation, and implementation 214
5.3.4 Verifiable data, training recipe, and empirical behavior 217
5.3.5 Limitations, interpretation, and connection to GRPO 222
5.4 Group Relative Policy Optimization for Verifiable Rewards 223

6 Neuroscience and Cognitive Foundations 224
6.1 Introduction 224
6.2 From System 1 to System 2 224

IV Data and Training 225

7 Training Data 226
7.1 Pre-Training Data 226
7.2 Post-Training Data 226

8 Training Methods 227
8.1 Pre-Training 227
8.2 Post-Training 227
8.3 Machine Unlearning 227

V Evaluation, Impact, Philosophy, and Challenges 228

9 Evaluation 229

10 Environmental Impacts 230

11 Philosophy 231

12 Challenges 232

Part I

Foundations and Architecture

1. Introduction

2. Architecture

2.1 Tokenization

The architecture of a language model begins with a representational question: what are the discrete units over which the model will operate? A neural network does not receive written language as an unstructured sequence of words in their ordinary typographical form. It receives a numerical representation of a sequence of discrete units. Tokenization constructs this representation by mapping character strings into sequences drawn from a vocabulary. The choice of these units affects the length of the sequences processed by the model, the size of its vocabulary, the linguistic regularities available to it, and the probability distribution that it learns to estimate. Tokenization is therefore not merely an incidental preprocessing operation; it is the first representational decision in the architecture considered in this chapter.

Two complementary perspectives are needed to understand this decision. The cognitive perspective examines how the number and nature of linguistic units affect the effort required to process language. In particular, it considers the trade-off between the number of token occurrences and the number of distinct vocabulary types. The formal perspective treats a tokenizer as a pair of mappings between character strings and token sequences and asks whether these mappings preserve the statistical and computational properties required by language modeling. The former perspective motivates flexible vocabularies containing units of different sizes, while the latter establishes conditions under which a model trained over token sequences remains a sound estimator of a distribution over original texts (**yang2024rethinking**; **gastaldi2025foundations**).

2.1.1 From Written Text to Token Sequences

A formal description of tokenization begins by distinguishing the elementary symbols of a text from the units selected for a model's vocabulary. Let Σ be a finite, nonempty alphabet of elementary symbols. Depending on the textual representation, these symbols may be characters or other elementary textual units. For each nonnegative integer n, let Σ^n denote the set of strings containing exactly n symbols from Σ. The set of all finite strings over the alphabet is the Kleene closure

$$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n. \tag{2.1}$$

It includes the empty string, denoted by ε_Σ. The length of a string $\sigma \in \Sigma^*$ is written as $|\sigma|$. If $\sigma_1, \sigma_2 \in \Sigma^*$, their concatenation is written as $\sigma_1 \cdot \sigma_2$. Concatenation is associative and has the empty string as its identity:

$$(\sigma_1 \cdot \sigma_2) \cdot \sigma_3 = \sigma_1 \cdot (\sigma_2 \cdot \sigma_3), \qquad \varepsilon_\Sigma \cdot \sigma = \sigma \cdot \varepsilon_\Sigma = \sigma. \tag{2.2}$$

Thus, Σ^*, equipped with concatenation and the empty string, is a free monoid. This structure is useful because it describes strings without imposing a particular linguistic segmentation on them (**gastaldi2025foundations**).

A tokenizer introduces a second set, denoted by Δ, whose elements are the units in the tokenizer's vocabulary. In contrast to Σ, whose elements are elementary symbols, Δ may contain units of different lengths. A vocabulary may include individual characters, subword units, complete words, and, in some designs, multiword expressions. The set of all finite sequences over this vocabulary is

$$\Delta^* = \bigcup_{n=0}^{\infty} \Delta^n. \tag{2.3}$$

An element of Σ^* is a character string or text, whereas an element of Δ^* is a token sequence. The distinction is mathematical rather than hierarchical: the formal framework does not assume that characters are intrinsically more fundamental than tokens. It distinguishes the two sets because tokenization studies the mappings between them (**gastaldi2025foundations**).

It is also necessary to distinguish a token from a token type. A token is an occurrence of a unit in a particular tokenized text. A token type is a distinct unit that can occur in the vocabulary. If a corpus contains the word *language* several times, each occurrence contributes a token, while the word itself contributes only one type. Consequently, the number of tokens measures the length of the corpus after segmentation, whereas the number of types measures the size of the set of distinct units used to represent it.

Let a corpus be

$$\mathcal{C} = \{\sigma_1, \sigma_2, \ldots, \sigma_M\},$$

where each $\sigma_m \in \Sigma^*$, and let T be a deterministic tokenizer. The total number of token occurrences produced by T is

$$N_T(\mathcal{C}) = \sum_{m=1}^{M} |T(\sigma_m)|. \tag{2.4}$$

If $\mathrm{Types}(T(\sigma_m))$ denotes the set of distinct token types appearing in the tokenization of σ_m, the number of types observed in the corpus is

$$V_T(\mathcal{C}) = \left| \bigcup_{m=1}^{M} \mathrm{Types}(T(\sigma_m)) \right|. \tag{2.5}$$

These quantities describe different properties of a representation. Reducing $N_T(\mathcal{C})$ does not necessarily reduce $V_T(\mathcal{C})$, and reducing the vocabulary size does not necessarily shorten the tokenized corpus. The tension between these objectives is central to tokenizer design (**yang2024rethinking**).

2.1.2 From Word-Level to Subword-Level Representations

Word-level tokenization treats complete words as the primary units. In writing systems that use spaces to separate words, a tokenizer can use spaces and punctuation as practical segmentation cues. This representation is intuitive because its units often correspond to familiar lexical items. However, the vocabulary required to cover a language is not fixed. Morphological inflections, newly coined words, spelling variation, and domain-specific terminology continually introduce forms that may not have appeared in the training corpus. A word-level tokenizer must either add these forms to its vocabulary or handle them as out-of-vocabulary (OOV) items. The problem is particularly visible in languages whose writing systems do not mark word boundaries with spaces, because segmentation cannot be obtained from whitespace alone (**yang2024rethinking**).

Character-level tokenization takes the opposite approach. It represents a text as a sequence of elementary characters. This substantially limits the number of distinct vocabulary units and allows an unfamiliar word to be represented whenever its constituent characters are available. Its cost is sequence length: a word that would be one token under a word-level representation may require several character tokens. Longer sequences require the language model to process and predict more units. Character-level tokenization therefore reduces vocabulary complexity while increasing the number of token occurrences.

Subword tokenization occupies an intermediate position. It constructs units that may be larger than individual characters but smaller than complete words. Frequently occurring character sequences can be represented as single vocabulary entries, while rare or unseen words can be decomposed into smaller units. The result is an open-vocabulary strategy: the tokenizer does not need a separate entry for every possible complete word, provided that the relevant elementary symbols and subword components can be represented.

Byte Pair Encoding (BPE) is a prominent example. Its underlying pair-merging procedure originated in data compression and was adapted to subword representation. In its basic form, the corpus is initialized as sequences of small units. The algorithm repeatedly identifies frequent adjacent pairs and merges selected pairs into larger units. Repeating the operation builds a vocabulary containing units of different lengths. Frequent sequences can then be represented compactly, while less frequent forms remain expressible through smaller components. WordPiece is another subword approach. Its maximal-munch encoding procedure repeatedly selects the longest vocabulary prefix matching the remaining input. Unigram tokenization can represent a text through alternative subword segmentations; when regularization is used, different segmentations can be sampled during training. Yang also discusses SentencePiece as a language-independent subword framework, including its relevance to languages that do not use spaces as word boundaries (**yang2024rethinking**; **gastaldi2025foundations**).

The principal distinction among these approaches is not simply the length of the resulting units. Their segmentation procedures determine which sequences are valid representations, whether the encoder is deterministic or stochastic, and how many alternative token sequences may correspond to the same text. These differences become important when the tokenizer is treated as part of a probabilistic language model rather than as a stand-alone text-processing utility.

Subword segmentation also does not resolve every linguistic representation problem. Multiword expressions (MWEs) are expressions composed of multiple words whose con-

ventional meaning may not be reducible to the literal meanings of their components. The expression *kick the bucket*, for example, has an idiomatic meaning that differs from its literal composition. If such an expression is represented only through its constituent words or subwords, the model must learn its conventional meaning from contextual examples. Adding an MWE as a single vocabulary unit may make its holistic meaning more directly available, but it can also increase the number of types. Rare or domain-specific MWEs may not occur often enough in training data to support a reliable representation. Yang therefore treats direct MWE representation as a vocabulary-design trade-off rather than as a rule that every multiword expression should become a token (**yang2024rethinking**).

2.1.3 Balancing Tokens and Types: The Principle of Least Effort

The difference between token count and vocabulary size can be quantified. Yang reports a comparison on a German corpus in which word-level segmentation produces 100 million tokens and $1{,}750{,}000$ types, BPE produces 111 million tokens and $82{,}000$ types, and character-level segmentation produces 550 million tokens and $3{,}000$ types. These values are reproduced in table 2.1.

Table 2.1: Token and type counts for three segmentation methods on the German corpus reported by Yang.

Segmentation method	Number of tokens	Number of types
Words	100 million	$1{,}750{,}000$
BPE	111 million	$82{,}000$
Characters	550 million	$3{,}000$

Source: **yang2024rethinking**, reproducing the comparison attributed there to the BPE literature.

The BPE vocabulary contains

$$\frac{82{,}000}{1{,}750{,}000} \approx 0.0469 \tag{2.6}$$

of the word-level types, or approximately 4.7%. This corresponds to a reduction of approximately 95.3% in the number of types. Its token count, however, is

$$\frac{111}{100} = 1.11 \tag{2.7}$$

times the word-level count, an increase of approximately 11%. Character-level segmentation reduces the vocabulary to approximately 0.17% of the word-level type count, but increases the number of tokens to 5.5 times the word-level count. The comparison illustrates the central compromise: word-level tokenization uses relatively few tokens but requires a large vocabulary; character-level tokenization uses a small vocabulary but creates long sequences; subword tokenization can sharply reduce the vocabulary while increasing the token count much less severely (**yang2024rethinking**).

The trade-off is affected by the language being represented. In morphologically rich languages, subword units can capture recurring components across inflected forms, reducing the need to store every form independently. In Chinese, the absence of whitespace

word boundaries and the prevalence of compounds create different segmentation challenges. Yang notes that Chinese BPE units are often individual characters and reports that a Chinese sentence may require approximately 1.7 times as many tokens as a comparable English sentence, while Burmese or Amharic may require as many as ten times the number. These observations indicate that a tokenizer's efficiency depends not only on its algorithm but also on the writing system and statistical structure of the corpus (**yang2024rethinking**).

Yang interprets the token–type trade-off through the Principle of Least Effort (PLE). In this account, language processing involves both immediate and future effort. Fewer token occurrences reduce the burden associated with maintaining and processing units during immediate comprehension; Yang relates this burden to working memory. Fewer distinct types reduce the burden of storing and retrieving different units; this is associated with long-term memory. The two objectives can conflict: merging units into longer tokens may shorten a sequence but introduce additional vocabulary types, while removing types may require a text to be represented through more tokens.

The relationship also has a qualitative connection to Minimum Description Length (MDL). In an MDL interpretation, fewer types correspond to a more compact description of the lexicon or encoder model, whereas fewer tokens correspond to a more compact description of the encoded corpus. Yang emphasizes an important distinction: the cognitive account is not simply a single global minimization of the sum of token count and type count. It treats the minimization of the two burdens as distinct processes whose interaction produces a balance. The corresponding design objectives can be written schematically as

$$\min_T N_T(\mathcal{C}) \qquad \text{and} \qquad \min_T V_T(\mathcal{C}), \tag{2.8}$$

with the understanding that these objectives are generally competing rather than independently attainable (**yang2024rethinking**).

The Less-is-Better model. The Less-is-Better (LiB) model is Yang's proposed implementation of PLE. Its vocabulary is not restricted to a single linguistic level: it can contain subwords, words, and MWEs. The model has two interacting mechanisms, called the Memorizer and the Forgetter. It initially divides the input corpus into small units. The Memorizer repeatedly merges adjacent units into longer units and stores the resulting units in the vocabulary. Longer units can reduce the number of tokens required to represent a text, but every newly stored unit can increase the number of types. The Forgetter removes less useful vocabulary entries, including infrequent types and types whose use increases the number of tokens needed for sentences. Removing types reduces vocabulary size but can require the corresponding text to be represented by smaller units. The two mechanisms therefore act in opposite directions and are intended to approach a relatively stable vocabulary (**yang2024rethinking**).

The LiB paper identifies larger-first selection and counterfactual evaluation as segmentation strategies, and memorizing, re-ranking, and forgetting as vocabulary-update strategies. It does not provide a fully parameterized optimization objective, a universal update schedule, or a numerical convergence threshold in the paper considered here. The mechanism should therefore be understood as the proposed learning-and-forgetting design, not as a completely specified algorithm with a uniquely determined implementa-

tion. The interaction between corpus segmentation and vocabulary updating is summarized in figure 2.1.

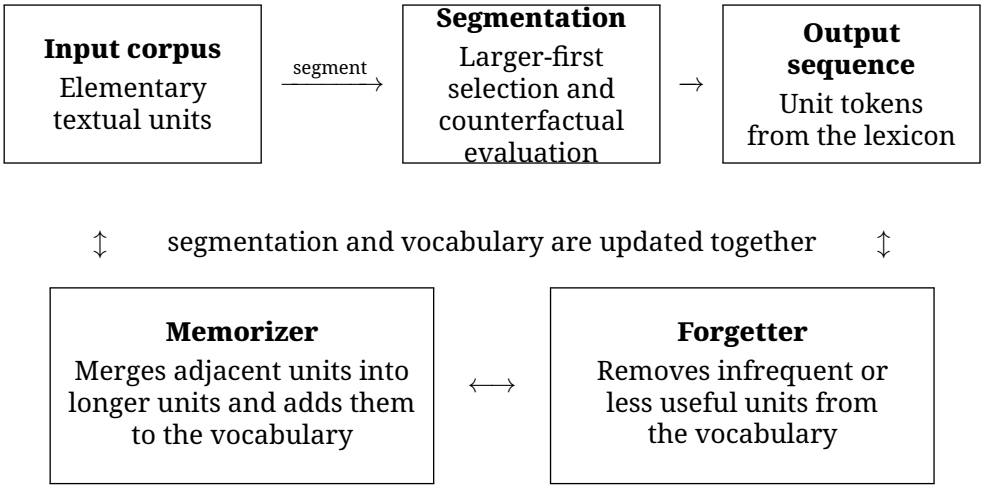


Figure 2.1: Conceptual flow of the Less-is-Better tokenizer. The Memorizer introduces longer units, which can reduce token count while increasing vocabulary size. The Forgetter removes less useful types, reducing vocabulary size while potentially increasing the number of tokens needed for a text. Their interaction is intended to approach a relatively stable vocabulary. Original schematic based on the mechanism described by **yang2024rethinking**.

Yang reports LiB segmentations that combine units of different sizes. For example, the English input ‘allrightwhydon’tweputhimawaynow‘ is represented as ‘allright | whydon’t | we | puthimaway | now‘, rather than as the nine word-level units ‘all | right | why | don’t | we | put | him | away | now‘. The example illustrates that LiB does not impose a strict boundary between subwords, words, and multiword expressions. It searches for units that function usefully within the corpus under its learning-and-forgetting procedure (**yang2024rethinking**).

The reported vocabulary and average-token-length measurements are shown in table 2.2. The BRphono average length is measured in English phonemes, while the CTB8 average length is measured in Chinese characters. The results show that LiB learns longer average units than the other listed methods in both corpora. Its vocabulary size is smaller than BPE’s in BRphono, but larger than the word vocabulary; in CTB8, its vocabulary is smaller than the word vocabulary but larger than the BPE vocabulary. Thus, the measurements support a different balance between unit length and vocabulary size, rather than a claim that LiB has the smallest vocabulary under every comparison.

The paper also reports bits-per-character (BPC) scores from simple 2-gram and 3-gram language-model evaluations. If x is an evaluated character sequence containing N_c characters and $p_\theta(x)$ is the probability assigned to it by a model, its description length per character in bits is

$$\mathrm{BPC}(x) = -\frac{1}{N_c} \log_2 p_\theta(x). \tag{2.9}$$

A lower score indicates a shorter model-based description of the evaluated sequence. The reported results are reproduced in table 2.3.

Table 2.2: Average token length and vocabulary size reported for BRphono and CTB8.

Corpus	Measure	Characters	BPE	Words	LiB
BRphono (English)	Average token length	1.0	2.8	2.9	3.6
BRphono (English)	Vocabulary size	50	5,574	1,321	1,869
CTB8 (Chinese)	Average token length	1.0	1.4	1.7	1.9
CTB8 (Chinese)	Vocabulary size	4,697	7,980	65,410	39,320

Source: **yang2024rethinking**. Average length is measured in English phonemes for BRphono and Chinese characters for CTB8.

Table 2.3: Bits-per-character scores reported for 2-gram and 3-gram models under different tokenizations.

Corpus	Model	Characters	BPE	Words	LiB
CTB8	2-gram	3.558	2.788	2.333	**2.095**
CTB8	3-gram	2.025	1.193	1.163	**0.903**
BRphono	2-gram	2.221	1.003	0.977	**0.791**
BRphono	3-gram	1.371	0.563	0.584	**0.484**

Source: **yang2024rethinking**.

LiB obtains the lowest reported BPC in each of these four comparisons. The scope of the result must nevertheless be kept precise. Yang describes LiB as a cognitive model that was not optimized specifically for large language models; the reported language-model evaluations use simple 2-gram and 3-gram models. The results therefore motivate further investigation of PLE-based tokenizers, but they do not establish that LiB outperforms established tokenizers in large-scale transformer language models (**yang2024rethinking**).

2.1.4 A Stochastic Framework for Tokenization

The token–type trade-off concerns the efficiency and linguistic suitability of a representation. A separate question is whether the representation preserves the probability distribution of the original text. A language model is intended to estimate probabilities of linguistic sequences, but after tokenization it estimates probabilities over token sequences. To understand the relationship between these two distributions, Gastaldi et al. formulate tokenization through stochastic maps (**gastaldi2025foundations**).

A stochastic map from a countable set X to a countable set Y, written $f : X \rightsquigarrow Y$, assigns a probability distribution over Y to every input $x \in X$. Its transition probabilities satisfy

$$f(y \mid x) \geq 0, \qquad \sum_{y \in Y} f(y \mid x) = 1 \quad \text{for every } x \in X. \tag{2.10}$$

A deterministic function is a special case in which all probability mass for an input is concentrated on one output. The stochastic-map formulation therefore includes deterministic tokenizers as well as tokenizers that permit alternative segmentations.

Definition 2.1 **Tokenizer model.** A tokenizer model from Σ^* to Δ^* is a pair

$$T = (\tau, \kappa),$$

where

$$\tau : \Sigma^* \rightsquigarrow \Delta^* \qquad \text{and} \qquad \kappa : \Delta^* \rightsquigarrow \Sigma^*$$

are stochastic maps. The map τ is the encoder and the map κ is the decoder.

The encoder and decoder are defined as composable mappings between the two string spaces. No additional relationship between the alphabet Σ and the vocabulary Δ is assumed by the general definition. This breadth is deliberate: it allows the statistical consequences of the complete encoding–decoding process to be studied independently of the particular segmentation algorithm.

Let p^* be a reference probability distribution over Σ^*. Applying the encoder produces a probability distribution over token sequences, called the image or pushforward distribution:

$$q^* = \tau p^*. \tag{2.11}$$

For an individual token sequence $\delta \in \Delta^*$, this means

$$q^*(\delta) = \sum_{\sigma \in \Sigma^*} \tau(\delta \mid \sigma) p^*(\sigma). \tag{2.12}$$

The language model estimates q^* from tokenized data. Let $\{q_n\}$ be a sequence of estimators, where n indexes the amount of information used for estimation. If the estimators are consistent, then q_n converges to q^*. Decoding the estimated token distribution gives

$$p_n = \kappa q_n, \qquad p_n(\sigma) = \sum_{\delta \in \Delta^*} \kappa(\sigma \mid \delta) q_n(\delta). \tag{2.13}$$

The target of interest is whether p_n converges to the original distribution p^*, not merely whether q_n converges to the token-level distribution q^*.

The key mathematical property is that stochastic maps preserve consistency of estimators on the countable spaces considered by Gastaldi et al. In particular, if probability distributions p_n converge pointwise to a probability distribution p, then their L^1 distance converges to zero:

$$\lim_{n \to \infty} \sum_{x \in X} |p_n(x) - p(x)| = 0. \tag{2.14}$$

For a stochastic map $f : X \rightsquigarrow Y$, the triangle inequality and the normalization in equation (2.10) give

$$\begin{aligned} \sum_{y \in Y} |(fp_n)(y) - (fp)(y)| &= \sum_{y \in Y} \left| \sum_{x \in X} f(y \mid x)(p_n(x) - p(x)) \right| \\ &\leq \sum_{y \in Y} \sum_{x \in X} f(y \mid x) |p_n(x) - p(x)| \\ &= \sum_{x \in X} |p_n(x) - p(x)| \sum_{y \in Y} f(y \mid x) \\ &= \sum_{x \in X} |p_n(x) - p(x)|. \end{aligned} \tag{2.15}$$

Thus, applying a stochastic map cannot increase the L^1 distance between the distributions. This contraction property explains why consistency is preserved when an estimator is transformed through a stochastic map (**gastaldi2025foundations**).

Theorem 2.1 **Fundamental Principle of Tokenization.** Let p^* be a reference probability distribution over Σ^*, let $T = (\tau, \kappa)$ be a tokenizer, and let $\{q_n\}$ be a consistent sequence of estimators of $q^* = \tau p^*$. Then the decoded estimators $\{\kappa q_n\}$ are consistent estimators of p^* if and only if

$$\boxed{\kappa\tau p^* = p^*.} \tag{2.16}$$

Proof. Because $\{q_n\}$ is consistent for q^*, preservation of consistency under stochastic maps gives

$$\kappa q_n \longrightarrow \kappa q^*.$$

By the definition $q^* = \tau p^*$, the limiting distribution is

$$\kappa q^* = \kappa\tau p^*.$$

The decoded estimators therefore converge to p^* exactly when their limiting distribution equals p^*, which is the condition $\kappa\tau p^* = p^*$. This proves both directions. □

The theorem separates two kinds of correctness. A language model may estimate the token-level distribution consistently, yet the resulting character-level estimator may still be inconsistent if the tokenizer changes the reference distribution. The condition in equation (2.16) is therefore a property of the tokenizer together with the target distribution; it is not guaranteed solely by successful optimization of the language model.

Definition 2.2 **Consistency and exactness.** A tokenizer $T = (\tau, \kappa)$ is consistent with respect to a probability distribution p over Σ^* if

$$\kappa\tau p = p.$$

It is exact if

$$\kappa\tau = \mathrm{id}_{\Sigma^*}, \tag{2.17}$$

where id_{Σ^*} is the identity map on character strings.

Exactness implies consistency for every probability distribution over Σ^*. The converse does not hold if consistency is established only for one particular distribution: two strings with equal probability may have their probability assignments interchanged without changing that distribution. Gastaldi et al. establish that exactness is equivalent to consistency with respect to every probability distribution over Σ^*. They also show that, for an exact tokenizer, the decoder is deterministic on the support of the encoder. In the corresponding sense, the encoder is injective: distinct original texts cannot assign positive encoding probability to a common token sequence. This is the formal reason that lossless recovery and statistical consistency are connected (**gastaldi2025foundations**).

The two spaces and the statistical requirement are summarized in figure 2.2. The diagram makes explicit that the model estimates a distribution in token space and that the decoder must return this estimate to the original string space without changing the target distribution.

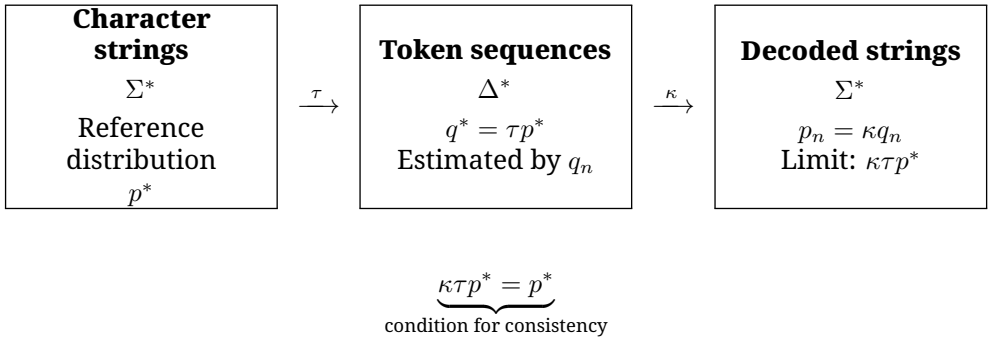


Figure 2.2: The statistical path induced by a tokenizer. The encoder maps the reference distribution over character strings to a distribution over token sequences; the language model estimates the token-level distribution; and the decoder maps that estimate back to character space. Consistency with the original distribution requires the encoder–decoder composition to preserve p^*, as established by **gastaldi2025foundations**.

2.1.5 Statistical and Computational Properties

The formal conditions above identify failure modes that can arise in actual tokenizers. Gastaldi et al. organize these concerns into statistical problems, especially inconsistency and ambiguity, and computational problems, especially finiteness and sequentiality (**gastaldi2025foundations**).

Noninjective encoding and inconsistency. An encoder is noninjective when distinct character strings can be mapped to the same token sequence. For example, preprocessing operations such as lowercasing, stripping accents, removing punctuation, or standardizing whitespace can cause distinct original strings to receive identical representations. If the decoder does not retain the information removed by preprocessing, the original string cannot in general be recovered. The composition $\kappa\tau$ is then not the identity, and the consistency condition may fail.

The use of an unknown symbol can create a related problem. If multiple out-of-vocabulary symbols or strings are mapped to one distinguished unknown representation, their probability mass is combined. A decoder that has no information distinguishing the original inputs cannot generally reconstruct their separate probabilities. Including the elementary alphabet in the vocabulary can support open-vocabulary encoding, but out-of-alphabet symbols at training or inference time still require careful treatment. The relevant requirement is not merely that an input can be assigned a token; it is that the complete encoding–decoding process preserves the distinctions needed for the target distribution (**gastaldi2025foundations**).

Noninjective decoding and ambiguity. A decoder is noninjective when more than one token sequence maps to the same character string. For example, if the vocabulary contains the units `t`, `h`, `e`, and `he`, the text `the` can be represented as `t | h | e` or as `t | he`. A deterministic encoder may select only one representation, while the decoder accepts both. If a language model assigns positive probability to token sequences outside the image of the encoder, the decoded text distribution can receive probability from al-

ternative sequences. Gastaldi et al. call this effect spurious ambiguity when it arises from estimation rather than from the reference tokenization process.

Softmax-based neural models are relevant here. For finite logits $z_1, \ldots, z_K$, the softmax probability of token i is

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}. \tag{2.18}$$

Every such probability is strictly positive. Consequently, a neural model may assign positive probability to token sequences that a deterministic encoder would never produce. This can reintroduce ambiguity even when the tokenizer itself is bijective over the encoder's image.

Ambiguity can also be deliberate. Stochastic tokenization and subword regularization expose a model to multiple segmentations of the same text during training. The intended benefit is to improve the handling of compositional structure, noise, and tokenization variation. Spurious ambiguity and intentional stochastic ambiguity have different origins, but both require the probability of a character string to account for all token sequences that decode to it. For a deterministic decoder, this marginal probability is

$$p_n(\sigma) = \sum_{\delta:\, \kappa(\delta)=\sigma} q_n(\delta). \tag{2.19}$$

For a stochastic decoder, the corresponding expression is the sum in equation (2.13). The sum can be computationally demanding when the set of preimages is large or infinite (**gastaldi2025foundations**).

Multiplicativity and finite preimages. A decoder is multiplicative if it respects concatenation. Writing token-sequence concatenation as $\odot$, the condition is

$$\kappa(\delta' \odot \delta'') = \kappa(\delta') \cdot \kappa(\delta''). \tag{2.20}$$

The commonly used BPE, WordPiece, and Unigram tokenizers discussed by Gastaldi et al. have multiplicative decoders. A multiplicative decoder has a trivial kernel if it maps every nonempty token sequence to a nonempty character string:

$$\delta \neq \varepsilon_\Delta \implies \kappa(\delta) \neq \varepsilon_\Sigma. \tag{2.21}$$

A sufficient condition is that no individual vocabulary token is erased during decoding.

Under multiplicativity and a trivial kernel, each token contributes at least one character to the decoded text. Therefore,

$$\kappa(\delta) = \sigma \implies |\delta| \leq |\sigma|. \tag{2.22}$$

If $|\sigma| = n$, every token sequence that decodes to σ has length at most n. For a finite vocabulary, the number of possible sequences of these lengths is bounded by

$$|\kappa^{-1}(\sigma)| \leq \sum_{i=1}^{n} |\Delta|^i, \tag{2.23}$$

for nonempty σ. Thus, the set of preimages is finite. The bound is exponential in the length of the text, however, so finiteness alone does not guarantee an efficient marginalization algorithm. Gastaldi et al. note that the desired computational complexity of the

complete composition $\kappa\tau$ should ideally be at most linear in the input length. For exact deterministic tokenizers such as BPE and WordPiece, the composition can be computed in linear time when the encoder and decoder themselves admit linear-time implementations, although the rigorous treatment of spurious ambiguity remains a separate challenge (**gastaldi2025foundations**).

Prefix monotonicity and sequential generation. A decoder has prefix monotonicity when decoding any token prefix yields a prefix of the complete decoded text:

$$\delta' \preceq \delta \implies \kappa(\delta') \preceq \kappa(\delta). \tag{2.24}$$

Here, $\preceq$ denotes the prefix relation. Multiplicativity implies prefix monotonicity under the framework considered by Gastaldi et al. This property matters for autoregressive language models because they generate token sequences incrementally. A prefix-monotone decoder preserves the relationship between a generated token prefix and the corresponding character prefix.

Bounded variation and sequentiality. For strings $\gamma, \gamma' \in \Gamma^*$, let $\gamma \wedge \gamma'$ be their longest common prefix. The left distance between the strings is

$$d_{\mathrm{L}}(\gamma, \gamma') = |\gamma| + |\gamma'| - 2|\gamma \wedge \gamma'|. \tag{2.25}$$

A function $f : \Gamma^* \to B^*$ has bounded variation if, for every $k \geq 0$, there is a finite constant $C_k \geq 0$ such that

$$d_{\mathrm{L}}(\gamma, \gamma') \leq k \implies d_{\mathrm{L}}(f(\gamma), f(\gamma')) \leq C_k. \tag{2.26}$$

In other words, inputs that differ by a bounded amount outside their longest common prefix cannot produce outputs whose corresponding difference is arbitrarily large.

Gastaldi et al. connect bounded variation with subsequential functions from formal language theory. Under the additional condition that the function preserves rational sets, bounded variation implies that the function is subsequential. A subsequential function can be implemented by a deterministic finite-state transducer with an additional output function for terminal states. Since subsequential functions are closed under composition, the encoder and decoder can be studied through the finite-state realizability of their individual mappings.

WordPiece's maximal-munch procedure repeatedly selects the longest vocabulary prefix that matches the remaining input. The maximum length of vocabulary units bounds this prefix search, supporting a subsequential realization. Gastaldi et al. also discuss results that construct deterministic finite automata for BPE encoders under specific conditions on the merge rules. These connections show that a tokenizer's practical suitability depends not only on the number of tokens it produces but also on whether its mappings can be implemented through tractable sequential computation (**gastaldi2025foundations**).

2.1.6 Limitations and Connection to Positional Encoding

The two perspectives developed in this section address different but complementary requirements. The Principle of Least Effort motivates a vocabulary that balances the number of token occurrences against the number of distinct types. The LiB model illustrates

how memorization and forgetting can construct units of different sizes, including subwords, words, and multiword expressions. Its reported 2-gram and 3-gram results are encouraging within the scope of the experiments, but they do not constitute a large-scale transformer evaluation. The numerical evidence should therefore be treated as motivation for further research rather than as a universal ranking of tokenizer algorithms (**yang2024rethinking**).

The formal framework establishes a separate condition: a consistent estimator over token sequences yields a consistent estimator over original strings only when the tokenizer's encoder–decoder composition preserves the target distribution. Noninjective encoding can destroy distinctions between texts; noninjective decoding can create ambiguity; and marginalizing over alternative token sequences can be computationally expensive. Multiplicativity, a trivial decoder kernel, prefix monotonicity, and bounded variation provide formal tools for analyzing finiteness and sequential implementation (**gastaldi2025foundations**).

Tokenization consequently determines the discrete units and sequence length on which the remaining architecture operates, but it does not itself specify how the model represents the positions or order of those units. Once a text has been converted into a token sequence, the architecture must provide a representation of where each token occurs in that sequence. This is the next representational problem: positional encoding supplies information about token positions and their relationships, allowing subsequent computational layers to process not only token identities but also their order.

2.2 Positional Encoding

Once tokenization has converted an input text into a sequence of discrete tokens, the Transformer must construct numerical representations that retain the information needed for subsequent computation. Token embeddings represent the identities and learned content of the tokens, but a sequence is not defined by token identity alone. The order in which tokens occur, and the distances separating them, are also essential to linguistic structure. The next architectural question is therefore how a Transformer can represent sequence position and make that information available to its attention mechanism.

This requirement follows from the computational organization of the Transformer. Unlike recurrent architectures, which propagate information through an explicitly ordered sequence of state updates, self-attention computes interactions among token representations through query, key, and value projections. This design enables parallel processing and direct interactions between distant tokens, but the attention operation does not, by itself, provide a complete representation of the positions occupied by those tokens. Positional encoding supplies this missing structural information. The distinction between token content and token position, and the way in which they are combined, consequently form part of the mathematical definition of a Transformer layer.

2.2.1 Why Self-Attention Requires Positional Information

Let an input sequence contain L tokens, and let the embedding of the token at position i be $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$. Collecting the embeddings as rows gives

$$X = \begin{bmatrix} \mathbf{x}_1^\mathsf{T} \\ \mathbf{x}_2^\mathsf{T} \\ \vdots \\ \mathbf{x}_L^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}. \tag{2.27}$$

For one attention head, the query, key, and value matrices are obtained through learned linear projections,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{2.28}$$

where $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. The query and key dimension is d_k, and the value dimension is d_v. The scaled dot-product attention scores and attention weights are

$$S = \frac{QK^\mathsf{T}}{\sqrt{d_k}}, \qquad A = \text{softmax}_{\text{row}}(S), \qquad Z = AV. \tag{2.29}$$

The row-wise softmax normalizes each query's scores over the keys that it is permitted to attend to. In unmasked self-attention, each row of A sums to one. The factor $\sqrt{d_k}$ controls the scale of the dot products as the query and key dimension changes. Without this scaling, the variance of the dot products can grow with d_k, pushing softmax toward saturated distributions and making optimization less well-conditioned. The formulation in equation (2.29) is the self-attention basis from which the positional mechanisms considered in this section are developed (**zhou2024rethinking**).

The limitation of content-only self-attention can be made precise through permutation equivariance. Let $P \in \mathbb{R}^{L \times L}$ be a permutation matrix. Left multiplication by P rearranges the rows of a sequence matrix. If the input is permuted, then

$$X' = PX, \qquad Q' = PQ, \qquad K' = PK, \qquad V' = PV. \tag{2.30}$$

For unmasked self-attention, the score matrix becomes

$$S' = \frac{Q'K'^\mathsf{T}}{\sqrt{d_k}} \tag{2.31}$$

$$= \frac{(PQ)(PK)^\mathsf{T}}{\sqrt{d_k}} \tag{2.32}$$

$$= P\frac{QK^\mathsf{T}}{\sqrt{d_k}}P^\mathsf{T} \tag{2.33}$$

$$= PSP^\mathsf{T}. \tag{2.34}$$

Because softmax is applied independently to each row, permuting both the query rows and key columns permutes the attention matrix in the same way: $A' = PAP^\mathsf{T}$. The output is therefore

$$Z' = A'V' \tag{2.35}$$
$$= PAP^{\mathsf{T}}PV \tag{2.36}$$
$$= PAV \tag{2.37}$$
$$= PZ. \tag{2.38}$$

Thus, content-only, unmasked self-attention is permutation-equivariant: permuting the input representations permutes the outputs, but the operation does not independently encode the original sequential positions. In an autoregressive language model, a causal mask additionally restricts each query to preceding positions. The mask establishes a direction of information flow, but positional representations are still needed to provide the attention computation with a more expressive account of absolute positions and relative displacements. Positional encoding addresses this requirement by making position-dependent information available to the network.

2.2.2 Absolute Positional Encoding

An absolute positional encoding assigns a representation to each position independently of the positions occupied by other tokens. If the token at position p has content embedding $\mathbf{x}_p$, its positional representation is denoted by $\mathbf{e}_p$. In the conventional additive formulation, the input to the subsequent Transformer computation is

$$\mathbf{h}_p = \mathbf{x}_p + \mathbf{e}_p, \qquad \mathbf{x}_p, \mathbf{e}_p \in \mathbb{R}^{d_{\text{model}}}. \tag{2.39}$$

The positional vector must have the same dimensionality as the token embedding for this addition to be defined. Absolute positional representations can be generated by predetermined functions or learned as model parameters. The two alternatives considered by **zhou2024rethinking** are sinusoidal positional encoding and learnable absolute positional encoding.

Sinusoidal positional encoding. Sinusoidal positional encoding constructs each positional vector from sine and cosine functions with different frequencies. For an embedding dimension d_{model}, the components are

$$\text{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2.40}$$
$$\text{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2.41}$$

where p is the position index and i indexes a pair of embedding dimensions. Defining the angular frequency

$$\omega_i = 10000^{-2i/d_{\text{model}}}, \tag{2.42}$$

the two components associated with frequency ω_i can be written as

$$\mathbf{p}_i(p) = \begin{bmatrix} \sin(p\omega_i) \\ \cos(p\omega_i) \end{bmatrix}. \tag{2.43}$$

The frequencies decrease geometrically as the pair index increases. The lower-index pairs therefore change more rapidly with position, while the higher-index pairs change more slowly. The complete positional vector combines these frequency scales, giving the model a deterministic multiscale representation of position.

For example, when $d_{\text{model}} = 4$, there are two frequency pairs. Their frequencies are

$$\omega_0 = 10000^0 = 1, \qquad \omega_1 = 10000^{-1/2} = 0.01. \tag{2.44}$$

At position $p = 2$, the positional vector is

$$\text{PE}(2) = \begin{bmatrix} \sin(2) \\ \cos(2) \\ \sin(0.02) \\ \cos(0.02) \end{bmatrix} \tag{2.45}$$

$$\approx \begin{bmatrix} 0.9093 \\ -0.4161 \\ 0.0200 \\ 0.9998 \end{bmatrix}. \tag{2.46}$$

The first pair varies substantially over a small positional change, whereas the second pair varies slowly. Their combination allows positions to be represented through multiple periodic scales rather than through a single monotonically increasing coordinate.

A useful property of this construction is that a fixed positional displacement can be represented by a linear transformation. For a fixed offset k, the trigonometric addition identities give

$$\mathbf{p}_i(p+k) = \begin{bmatrix} \cos(k\omega_i) & \sin(k\omega_i) \\ -\sin(k\omega_i) & \cos(k\omega_i) \end{bmatrix} \mathbf{p}_i(p). \tag{2.47}$$

The transformation in equation (2.47) depends on the offset k, not on the absolute position p. Although the encoding is defined using absolute positions, this property gives a model a mathematical route to learning relationships associated with relative displacements. The sinusoidal construction is deterministic and does not require a separately learned positional parameter vector for every position (**zhou2024rethinking**).

Learnable absolute positional encoding. An alternative is to learn a positional vector for each position. For a maximum represented sequence length L_{max}, the positional parameters can be arranged as

$$E_{\text{pos}} \in \mathbb{R}^{L_{\text{max}} \times d_{\text{model}}}, \qquad \mathbf{e}_p = E_{\text{pos}}[p, :]. \tag{2.48}$$

The token and positional vectors are then combined using equation (2.39). The number of positional parameters is

$$N_{\text{pos}} = L_{\text{max}} d_{\text{model}}. \tag{2.49}$$

Unlike the sinusoidal method, the learned table does not impose a predetermined frequency structure across positions. Its positional vectors are optimized with the other model parameters. This flexibility makes the representation data-dependent, while the

explicit table also ties the learned parameters to the positions represented during training. The two methods therefore differ in how positional structure is supplied: sinusoidal encoding imposes a fixed multiscale construction, whereas learnable encoding estimates the positional vectors through optimization.

2.2.3 Combining Content and Position: Addition and Concatenation

The construction of a positional vector is only one part of the design. The model must also determine how positional information is combined with token content before the attention projections are applied. Addition is the conventional method, but concatenation provides an alternative that keeps the two components in distinct parts of the representation.

Under addition, the query and key vectors at positions i and j are

$$\mathbf{q}_i = (\mathbf{x}_i + \mathbf{p}_i)W_Q, \qquad \mathbf{k}_j = (\mathbf{x}_j + \mathbf{p}_j)W_K. \tag{2.50}$$

Let $W_{QK} = W_Q W_K^\mathsf{T}$. The unnormalized dot product can be expanded into four interactions:

$$\begin{aligned}\mathbf{q}_i\mathbf{k}_j^\mathsf{T} &= (\mathbf{x}_i + \mathbf{p}_i)W_{QK}(\mathbf{x}_j + \mathbf{p}_j)^\mathsf{T} \\ &= \mathbf{x}_i W_{QK}\mathbf{x}_j^\mathsf{T} + \mathbf{p}_i W_{QK}\mathbf{x}_j^\mathsf{T} \\ &\quad + \mathbf{x}_i W_{QK}\mathbf{p}_j^\mathsf{T} + \mathbf{p}_i W_{QK}\mathbf{p}_j^\mathsf{T}.\end{aligned} \tag{2.51}$$

The first term is a content-to-content interaction. The second is a position-to-content interaction, the third is a content-to-position interaction, and the fourth is a position-to-position interaction. In the additive formulation, these interactions share the same learned query-key interaction matrix W_{QK}. They are consequently not parameterized as four independent transformations.

zhou2024rethinking use this decomposition to motivate a concatenation-based alternative. To maintain the same total input dimension, the content and positional components are each assigned half of the representation dimension:

$$\mathbf{x}'_i, \mathbf{p}'_i \in \mathbb{R}^{d_{\text{model}}/2}, \qquad \mathbf{h}'_i = \mathbf{x}'_i \oplus \mathbf{p}'_i \in \mathbb{R}^{d_{\text{model}}}, \tag{2.52}$$

where $\oplus$ denotes vector concatenation. The concatenated representation is projected into queries and keys in the usual way. If the resulting query-key interaction matrix is partitioned into four blocks,

$$W_{QK} = \begin{bmatrix} W_{cc} & W_{cp} \\ W_{pc} & W_{pp} \end{bmatrix}, \tag{2.53}$$

then the score contains separately parameterized content-to-content, content-to-position, position-to-content, and position-to-position interactions:

$$\begin{aligned}\mathbf{q}'_i\mathbf{k}'^\mathsf{T}_j &= \mathbf{x}'_i W_{cc}\mathbf{x}'^\mathsf{T}_j + \mathbf{x}'_i W_{cp}\mathbf{p}'^\mathsf{T}_j \\ &\quad + \mathbf{p}'_i W_{pc}\mathbf{x}'^\mathsf{T}_j + \mathbf{p}'_i W_{pp}\mathbf{p}'^\mathsf{T}_j.\end{aligned} \tag{2.54}$$

The block decomposition makes the distinction between the two methods explicit. Addition uses a shared interaction matrix for the four terms, whereas concatenation allows the corresponding blocks to be learned independently. The dimension reduction in equation (2.52) keeps the concatenated input dimension equal to that of the additive representation. The conceptual difference is illustrated in figure 2.3.

Zhou et al. argue that content and positional vectors represent distinct types of information and that independently parameterized interaction spaces can therefore be beneficial. Their experiments compare addition and concatenation in several single-scale and multiscale Vision Transformer models. They report performance differences ranging from approximately 0.1 to 0.5 percentage points in top-1 ImageNet-1K accuracy in favor of concatenation for several evaluated models, without a systematic increase in computation overhead. These are results from vision architectures and image-classification experiments; they should not be interpreted as direct evidence that concatenation is superior for language models. The contribution relevant to language-model architecture is the analysis of how the choice of combination operation changes the parameterization of content-position interactions (**zhou2024rethinking**).

2.2.4 Relative Positional Encoding and Rotary Position Embedding

Absolute positional encoding assigns a representation to each individual position. Many relationships between tokens, however, are naturally described by the displacement between their positions. If two tokens occur at positions m and n, their signed relative displacement is

$$\Delta = m - n. \tag{2.55}$$

For example, the pairs $(m, n) = (10, 7)$ and $(m, n) = (100, 97)$ have different absolute positions but the same displacement, $\Delta = 3$. A relative positional method aims to make the interaction between the tokens depend on this relationship. One family of relative methods uses learnable representations indexed by relative displacement. Rotary Position Embedding (RoPE) achieves a related objective by rotating query and key representations according to their absolute positions so that their inner product depends on the relative displacement (**zhou2024rethinking**; **zhang2024foundmiddle**).

Two-dimensional rotations. The mathematical basis of RoPE is a rotation in a two-dimensional real vector space. For an angle ϕ, define

$$R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}. \tag{2.56}$$

For a vector $\mathbf{x} \in \mathbb{R}^2$, the rotated vector is $R(\phi)\mathbf{x}$. The matrix is orthogonal:

$$R(\phi)^\mathsf{T} R(\phi) = I, \qquad R(\phi)^\mathsf{T} = R(-\phi). \tag{2.57}$$

It follows that rotations preserve the Euclidean norm,

$$\|R(\phi)\mathbf{x}\|_2^2 = \mathbf{x}^\mathsf{T} R(\phi)^\mathsf{T} R(\phi)\mathbf{x} = \|\mathbf{x}\|_2^2. \tag{2.58}$$

The composition of two rotations adds their angles:

$$R(\alpha)R(\beta) = R(\alpha + \beta). \tag{2.59}$$

These identities are the essential algebraic properties used by RoPE. Equivalently, a two-dimensional vector can be represented as a complex number $z = x_1 + \mathrm{i}x_2$, and multiplication by $e^{\mathrm{i}\phi}$ performs the same rotation through Euler's identity.

Frequency-dependent rotations. Let the query and key dimension of one attention head be d_h, assumed even. RoPE groups the vector coordinates into $d_h/2$ pairs and assigns a different angular frequency to each pair. With the frequency convention used in the papers considered here,

$$\theta_j = 10000^{-2j/d_h}, \qquad j = 0, 1, \ldots, \frac{d_h}{2} - 1. \tag{2.60}$$

For position m, the complete rotation is a block-diagonal matrix,

$$R_m = \mathrm{diag}\left(R(m\theta_0), R(m\theta_1), \ldots, R(m\theta_{d_h/2-1})\right). \tag{2.61}$$

The transformed query and key vectors are

$$\widetilde{\mathbf{q}}_m = R_m \mathbf{q}_m, \qquad \widetilde{\mathbf{k}}_n = R_n \mathbf{k}_n. \tag{2.62}$$

The query-key inner product is then

$$\begin{aligned} \widetilde{\mathbf{q}}_m^{\mathsf{T}} \widetilde{\mathbf{k}}_n &= (R_m \mathbf{q}_m)^{\mathsf{T}} (R_n \mathbf{k}_n) && (2.63)\\ &= \mathbf{q}_m^{\mathsf{T}} R_m^{\mathsf{T}} R_n \mathbf{k}_n && (2.64)\\ &= \mathbf{q}_m^{\mathsf{T}} R_{n-m} \mathbf{k}_n. && (2.65) \end{aligned}$$

The last equality follows from the orthogonality and composition properties in equations (2.57) and (2.59). It establishes the central property of RoPE: although each query and key is transformed using its own absolute position, their attention interaction contains a rotation determined by the relative displacement $n - m$. The sign of the displacement depends on the convention used for the rotation matrix; the essential property is dependence on the difference between positions. The structure is summarized in figure 2.4.

The multiple frequencies in equation (2.60) cause different coordinate pairs to rotate at different rates. Higher-frequency components change phase more rapidly as position changes, while lower-frequency components change more gradually. RoPE therefore introduces positional information at multiple frequency scales directly into query-key interactions. It does not require a separate learned positional embedding table, because the transformations are determined by the position indices and the frequency schedule (**zhang2024foundmiddle**).

2.2.5 Long-Context Position Rescaling and Multi-Scale Positional Encoding

The ability to encode relative displacement does not guarantee that a language model will use all positions in a long input equally well. **zhang2024foundmiddle** study the "lost-in-the-middle" phenomenon: relevant information can be overlooked when it occurs in the

middle of a long context, even when the model can process the complete sequence. They identify two contributing factors. First, causal attention can favor initial tokens because these tokens are available to a larger number of subsequent queries. Second, the long-distance behavior associated with RoPE can reduce attention to distant tokens, including tokens that contain relevant information.

The latter observation should be interpreted as an empirical property of the resulting attention behavior, not as a theorem that every individual RoPE query-key score decreases monotonically with distance. The rotation matrices themselves preserve vector norms, and their inner-product contributions vary with relative phase. The paper's motivation is that the combined behavior of RoPE-based attention in the evaluated models can contribute to reduced use of distant context.

Uniform positional rescaling. Since RoPE uses position indices to determine rotation angles, the effective positional distances can be changed by rescaling the indices. For a positive scaling factor r, replace position m by

$$m' = \frac{m}{r}. \tag{2.66}$$

For two positions m and n, the rescaled displacement becomes

$$\Delta' = \frac{m}{r} - \frac{n}{r} = \frac{m-n}{r} = \frac{\Delta}{r}. \tag{2.67}$$

When $r > 1$, the effective positional distance is compressed. When $0 < r < 1$, it is expanded, and $r = 1$ recovers the original RoPE positions. For example, two tokens separated by $3{,}000$ positions have an effective separation of

$$\Delta' = \frac{3000}{1.5} = 2000 \tag{2.68}$$

when $r = 1.5$. The sequence itself is not shortened and no tokens are removed; only the positional indices used by the rotary transformation are remapped. This can reduce the phase difference between distant positions and potentially alleviate the long-distance decay observed by the authors.

Uniform rescaling has a trade-off. A factor that is too large can alter the positional patterns learned during pre-training and can change which regions of the context the model favors. Fractional rescaled positions may also differ from the integer position indices encountered during pre-training, creating a positional out-of-distribution concern. In the experiments reported by Zhang et al., small scaling factors such as 0.5 favor more recent context, whereas larger factors such as 2.5 can favor the beginning of the context. A factor near 1.5 improves average performance in their initial rescaling experiments, but a single factor does not account for the different positional behavior of individual attention heads.

Position-awareness across attention heads. The motivation for Multi-Scale Positional Encoding (Ms-PoE) is the observation that attention heads differ in their ability to follow relevant information when its position changes. Some heads shift their attention toward the relevant content as that content moves through the input. Other heads remain more strongly concentrated on the beginning and end regions. Zhang et al. use

the term *position-aware* for heads that can capture relevant information across these positional changes.

For a layer k, let the attention distribution of head h for the most recent query be

$$\mathbf{A}_{k,h} = [A_{k,h,1}, A_{k,h,2}, \ldots, A_{k,h,L}], \qquad \sum_{t=1}^{L} A_{k,h,t} = 1. \tag{2.69}$$

Its mean component is $1/L$. The Position-Awareness Score is defined as

$$S^{P}_{k,h} = \frac{1}{L} \sum_{t=1}^{L} \mathbb{I}\left(A_{k,h,t} \geq \alpha \frac{1}{L} \sum_{j=1}^{L} A_{k,h,j} \right), \tag{2.70}$$

where $\mathbb{I}(\cdot)$ is the indicator function and α is a threshold hyperparameter. The experiments use $\alpha = 3$. Because the attention weights sum to one, the threshold in this case is $3/L$. The score is the fraction of positions whose attention weights are at least three times the mean attention weight. A larger value indicates that a larger fraction of positions receive attention above this threshold. This score is an operational measure introduced by the authors; it is not itself a direct measure of semantic understanding or retrieval accuracy.

For each layer, heads are sorted in descending order of their Position-Awareness Scores. The most position-aware head receives a rescaling factor near one, while heads with lower scores receive larger factors. The authors use a linearly spaced set of factors from 1.2 to 1.8. For the head at sorted rank i, the assigned factor is

$$r_i = 1.2 + \frac{(i-1)(1.8-1.2)}{n_h - 1}, \qquad i = 1, \ldots, n_h, \tag{2.71}$$

where n_h is the number of attention heads in the layer. Thus, the first head in the descending positional-awareness ordering receives $r_1 = 1.2$, and the last receives $r_{n_h} = 1.8$. The factors are assigned separately within each layer. The resulting head-specific positional transformation is

$$\widetilde{\mathbf{q}}_{k,h,m} = R\left(\frac{m}{r_{k,h}}\right) \mathbf{q}_{k,h,m}, \qquad \widetilde{\mathbf{k}}_{k,h,n} = R\left(\frac{n}{r_{k,h}}\right) \mathbf{k}_{k,h,n}, \tag{2.72}$$

where $R(\cdot)$ denotes the frequency-dependent block rotation defined earlier. The relative displacement used by head h in layer k is consequently $(m-n)/r_{k,h}$. Different heads therefore operate with different effective positional scales. The arrangement is illustrated in figure 2.5.

Inference procedure and computational considerations. Ms-PoE is applied to a pretrained model that already uses RoPE. During prefilling, the model obtains the attention distribution of the most recent query for every head in every layer. It computes the Position-Awareness Scores, sorts the heads within each layer, and assigns the linearly spaced rescaling factors. These factors remain fixed during the subsequent decoding stage. The procedure does not require additional fine-tuning or additional trainable positional parameters (**zhang2024foundmiddle**).

Algorithm 1: Head-wise positional rescaling in Ms-PoE

Input: Pre-trained RoPE-based language model; input sequence; number of layers n_l; number of heads n_h; threshold α; scaling interval $[1.2, 1.8]$.

Output: Generated sequence using head-specific positional rescaling.

for $k = 1$ **to** n_l **do**
  Obtain the most recent query and the keys for layer k;
  for $h = 1$ **to** n_h **do**
    Obtain the attention distribution $\mathbf{A}_{k,h}$;
    Compute $S^P_{k,h}$ using equation (2.70);
  Sort the heads by $S^P_{k,h}$ in descending order;
  Assign the factors r_i using equation (2.71);
  Associate each assigned factor with its sorted head;
Replace the original RoPE position indices by the corresponding head-specific rescaled indices during decoding;
Generate tokens autoregressively using the fixed factors;

The attention vectors used for the score are already part of the prefilling attention computation. Given these vectors, evaluating the threshold indicator for all heads and positions requires $O(n_l n_h L)$ comparisons. Sorting the n_h heads in each layer requires $O(n_l n_h \log n_h)$ comparisons. Storing the scores and assigned factors requires $O(n_l n_h)$ scalar values. These costs describe the additional score-reduction and ordering operations implied by the procedure; the method does not introduce a new trainable parameter matrix or a new attention layer. Once the factors are fixed, the decoding stage uses the modified RoPE transformations for the existing heads.

Experimental evaluation. Zhang et al. evaluate Ms-PoE on long-context generation and retrieval tasks using Llama-2, StableBeluga, and Vicuna models. On the ZeroSCROLLS benchmark, they report average improvements over the corresponding baselines for the evaluated models. The reported average gain is 1.6 points for Llama-2-7B-Chat, 0.9 for Llama-2-13B-Chat, 2.6 for StableBeluga-7B, 0.6 for StableBeluga-13B, 3.8 for Vicuna-7B, and 3.3 for Vicuna-7B-16K. The results indicate improvements across the reported model sizes and context configurations, although the magnitude varies by model and task (**zhang2024foundmiddle**).

The multi-document question-answering and key-value retrieval experiments vary the location of the relevant document or key-value pair. For Vicuna-7B, the average multi-document question-answering accuracy increases from 61.12% for the baseline to 65.04% with Ms-PoE. For key-value retrieval, the corresponding average increases from 36.24% to 79.96%. The authors average results over 500 samples and use the difference between the best and worst accuracy across relevant-information positions as a measure of positional sensitivity:

$$\text{GapAccuracy} = \max_p \text{Acc}(p) - \min_p \text{Acc}(p), \tag{2.73}$$

where $\text{Acc}(p)$ is the accuracy when the relevant information is placed at position p. A smaller gap indicates less variation in accuracy as the location of relevant information changes. The results show that Ms-PoE improves average retrieval performance and reduces the positional gap in the evaluated settings (**zhang2024foundmiddle**).

The ablation experiments also examine the way scaling factors are assigned. In the reported multi-document question-answering experiment, the position-awareness ordering achieves an average accuracy of 65.3%, compared with 61.7% for random assignment, 57.8% for sequential assignment, and 62.3% for attention-entropy-based assignment. The baseline average is 62.1%. These results support the authors' claim that head-wise positional-awareness information provides a more effective assignment criterion than the tested alternatives. They do not establish that the same ordering criterion is optimal for every architecture or task.

2.2.6 Limitations and Connection to the Next Architectural Components

Positional encoding determines how a Transformer represents token locations and how positional information enters attention interactions. The methods considered here make different design choices. Sinusoidal encoding imposes a fixed multiscale trigonometric structure; learnable absolute encoding estimates a positional table during training; addition and concatenation determine how content and position are combined; RoPE uses position-dependent rotations whose query-key interaction depends on relative displacement; and Ms-PoE modifies RoPE at inference by assigning different rescaling factors to different attention heads.

Each construction has limitations. A learned absolute positional table is tied to the positions represented by its parameters. Sinusoidal encoding supplies a fixed frequency structure, but the model must still learn to use that structure through its projections. Concatenation changes the parameterization of content-position interactions, but the experimental comparison reported by Zhou et al. concerns vision architectures and cannot by itself establish an advantage for language models. RoPE provides a relative-position interaction through rotations, but its behavior over long distances can still be associated with reduced utilization of distant context. Ms-PoE addresses this issue in the evaluated models, but its head-wise score is a threshold-based proxy, its factors are selected from a specified interval, and rescaling can create positional patterns that differ from those seen during pre-training.

The central architectural conclusion is that positional encoding is not merely an auxiliary vector added to token embeddings. It determines how sequence order and relative displacement influence the attention computation, and therefore affects how a language model can represent and use information distributed across a sequence. With token content and positional relationships now specified as inputs to the attention mechanism, the next architectural question is how each token representation is further transformed after attention. This leads to the feed-forward component of the Transformer, which applies position-wise nonlinear transformations to the representations constructed by attention.

2.3 Feed-Forward Networks

Positional encoding supplies token representations with information about their positions in a sequence. Position information, however, does not by itself specify how the features represented by each token are transformed into increasingly useful internal representations. A Transformer therefore requires computational components that operate on the feature dimensions of token representations in addition to components that es-

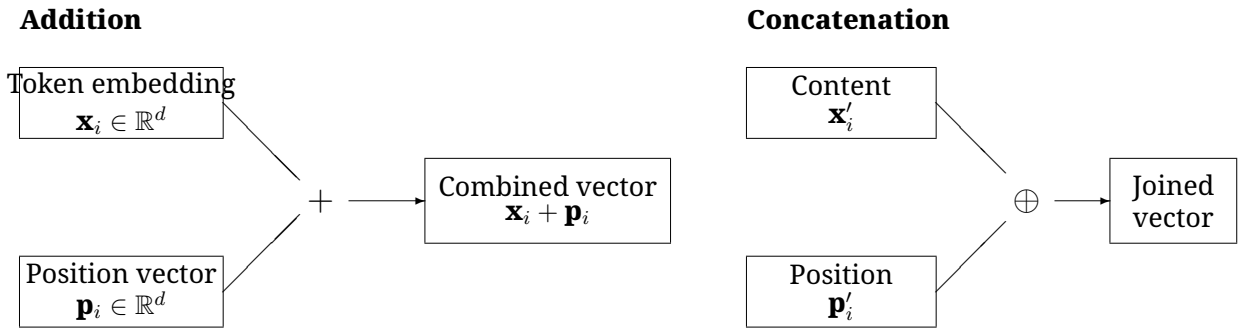
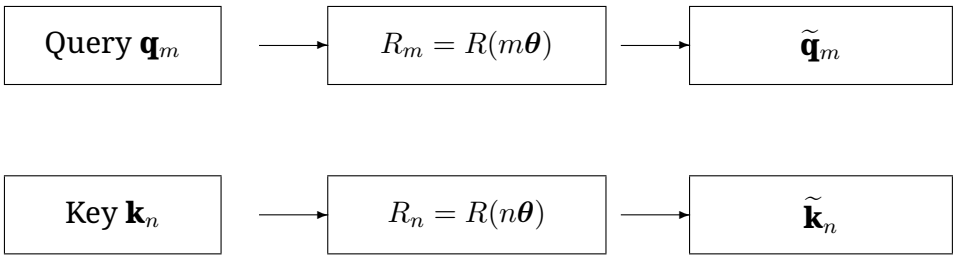


Figure 2.3: Two ways of incorporating positional information into token representations. Addition superimposes full-dimensional vectors, whereas concatenation joins half-dimensional content and positional components while retaining the same total dimension. Adapted conceptually from the comparison in **zhou2024rethinking**.



$$\widetilde{\mathbf{q}}_m^{\mathsf{T}} \widetilde{\mathbf{k}}_n = \mathbf{q}_m^{\mathsf{T}} R_{n-m} \mathbf{k}_n$$

Figure 2.4: Rotary Position Embedding applies position-dependent rotations to queries and keys. Their inner product contains a rotation determined by the relative displacement between positions. Schematic based on **zhang2024foundmiddle**.

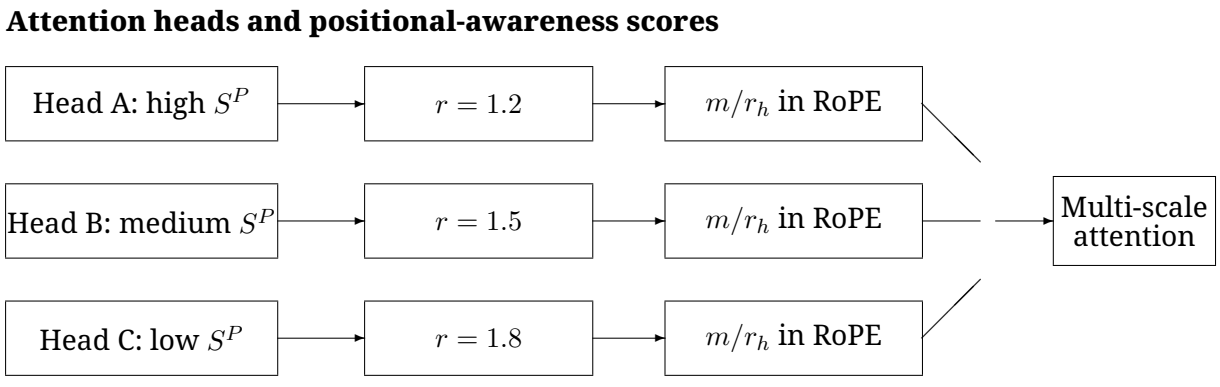


Figure 2.5: Conceptual flow of Ms-PoE. Heads are ranked by their position-awareness scores and receive different rescaling factors; higher-scoring heads are changed less, while lower-scoring heads receive stronger positional compression. The values shown are illustrative points in the interval used by **zhang2024foundmiddle**.

tablish relationships between different token positions. The feed-forward network (FFN) provides this feature-transformation mechanism. In a conventional Transformer block, the FFN is applied independently to each token representation, using the same learned parameters at every sequence position (**dai2024deepseekmoe**; **cai2025survey**).

The distinction between position-wise processing and sequence-wise interaction is fundamental. An FFN transforms the feature vector associated with one token without directly combining it with the feature vectors of other tokens. The parameters are shared across positions, so the same nonlinear mapping is applied throughout the sequence. This arrangement complements the self-attention sublayer, which supplies interactions between token positions. The present section develops the mathematical structure of the FFN, examines its parameter and computational requirements, and then follows the architectural transition from dense FFNs to sparsely activated mixtures of FFN experts.

2.3.1 Position-Wise Feed-Forward Networks

Let a sequence contain T tokens, and let d_{model} denote the dimensionality of each token representation. The representation of token t is a vector $\mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}$. A sequence can be represented by a matrix

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_T \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times T},$$

where each column corresponds to one token. This column-wise convention will be used throughout the mathematical development.

A standard two-projection FFN first maps the input representation into an intermediate feature space, applies an element-wise nonlinear activation, and then maps the result back into the model hidden space. For one token, the computation is

$$\mathbf{a}_t = \mathbf{W}_1 \mathbf{x}_t + \mathbf{b}_1, \tag{2.74}$$

$$\widetilde{\mathbf{a}}_t = \phi(\mathbf{a}_t), \tag{2.75}$$

$$\mathbf{y}_t = \mathbf{W}_2 \widetilde{\mathbf{a}}_t + \mathbf{b}_2. \tag{2.76}$$

Here, d_{ff} is the intermediate hidden dimension, $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the input-projection matrix, $\mathbf{b}_1 \in \mathbb{R}^{d_{\text{ff}}}$ is its bias vector, $\mathbf{W}_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ is the output-projection matrix, and $\mathbf{b}_2 \in \mathbb{R}^{d_{\text{model}}}$ is the output bias. The activation function ϕ acts element-wise on its vector argument. The dimensions therefore satisfy

$$\mathbf{x}_t \in \mathbb{R}^{d_{\text{model}}}, \quad \mathbf{a}_t, \widetilde{\mathbf{a}}_t \in \mathbb{R}^{d_{\text{ff}}}, \quad \mathbf{y}_t \in \mathbb{R}^{d_{\text{model}}}.$$

For a bias-free formulation, the bias vectors are omitted. The essential structure is the same: an input projection, an element-wise nonlinearity, and an output projection. The conventional FFN is consequently a two-layer multilayer perceptron whose input and output dimensions agree, while its intermediate dimension may be wider. The description of standard FFN experts as linear–activation–linear networks is also used in the MoE literature (**dai2024deepseekmoe**; **cai2025survey**).

The complete sequence is processed by applying the same matrices and bias vectors to every column:

$$\mathbf{A} = \mathbf{W}_1 \mathbf{X} + \mathbf{b}_1 \mathbf{1}_T^\top, \tag{2.77}$$

$$\widetilde{\mathbf{A}} = \phi(\mathbf{A}), \tag{2.78}$$

$$\mathbf{Y} = \mathbf{W}_2 \widetilde{\mathbf{A}} + \mathbf{b}_2 \mathbf{1}_T^\top, \tag{2.79}$$

where $\mathbf{1}_T \in \mathbb{R}^T$ is the all-ones vector. Thus, $\mathbf{A}, \widetilde{\mathbf{A}} \in \mathbb{R}^{d_{\text{ff}} \times T}$ and $\mathbf{Y} \in \mathbb{R}^{d_{\text{model}} \times T}$. The matrix multiplication acts on the feature dimension of each column; there is no multiplication between different token columns. Therefore, for every t, the output $\mathbf{y}_t$ depends on $\mathbf{x}_t$ and the shared FFN parameters, but not directly on $\mathbf{x}_s$ for $s \neq t$.

The nonlinearity is necessary for the composition to represent a nonlinear mapping. If ϕ were the identity function and the biases were zero, then

$$\mathbf{y}_t = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x}_t.$$

The product $\mathbf{W}_2 \mathbf{W}_1$ is itself a matrix, so the two projections would collapse to one linear transformation. An intervening nonlinear activation prevents this reduction in general and allows the FFN to implement a nonlinear feature transformation. The specific mathematical properties of the activation function are treated in the subsequent section on activation functions.

2.3.2 Parameterization and Computational Complexity

The dimensional structure of an FFN determines its parameter count. Ignoring bias parameters, the input-projection matrix contains $d_{\text{ff}} d_{\text{model}}$ entries, and the output-

projection matrix contains $d_{\text{model}}d_{\text{ff}}$ entries. The total number of weight parameters is therefore

$$P_{\text{FFN}} = d_{\text{ff}}d_{\text{model}} + d_{\text{model}}d_{\text{ff}} \tag{2.80}$$
$$= 2d_{\text{model}}d_{\text{ff}}. \tag{2.81}$$

When both bias vectors are included, the total becomes

$$P_{\text{FFN,bias}} = 2d_{\text{model}}d_{\text{ff}} + d_{\text{ff}} + d_{\text{model}}. \tag{2.82}$$

It is useful to express the intermediate dimension relative to the model dimension. Define the expansion ratio

$$r = \frac{d_{\text{ff}}}{d_{\text{model}}}, \qquad d_{\text{ff}} = r\, d_{\text{model}}. \tag{2.83}$$

Substitution into equation (2.81) gives

$$P_{\text{FFN}} = 2r\, d_{\text{model}}^2. \tag{2.84}$$

For a fixed expansion ratio, the number of FFN weight parameters therefore grows quadratically with the model hidden dimension. The intermediate dimension is an architectural choice rather than a universal constant; its value depends on the model configuration.

The dominant arithmetic cost is incurred by the two matrix multiplications. Let M denote the total number of token vectors processed by the FFN in a batch. The first projection requires $Md_{\text{model}}d_{\text{ff}}$ multiply–accumulate operations (MACs), and the second projection requires the same number. Thus,

$$C_{\text{FFN}} = 2Md_{\text{model}}d_{\text{ff}} \quad \text{MACs}. \tag{2.85}$$

When a multiplication and an addition are counted as separate floating-point operations, the corresponding leading-order arithmetic count is

$$F_{\text{FFN}} \approx 4Md_{\text{model}}d_{\text{ff}} \quad \text{FLOPs}. \tag{2.86}$$

This estimate counts the two dominant projections. Bias additions and element-wise activation operations add work of order $O(Md_{\text{ff}})$, which is lower order than the matrix multiplications when the model dimensions are large. The estimate also uses the conventional dense-matrix arithmetic count; actual latency depends on hardware, precision, memory movement, and kernel efficiency.

The distinction between parameter count and arithmetic cost is important. The matrices are shared across all token positions, so changing the sequence length does not change the number of FFN parameters. It does, however, change the number of times those parameters are used and therefore changes the computation approximately linearly in M. Increasing d_{model} or d_{ff}, in contrast, changes both parameter count and per-token computation.

2.3.3 Gated Feed-Forward Networks and Their Internal Units

The two-projection FFN is a useful basic model, but modern language-model FFNs can introduce an additional projection that controls the intermediate features. Lo et al. examine the three-projection expert structure used in LLaMA-style MoE models and write its computation as (**lo2025closer**):

$$E(\mathbf{x}) = \mathbf{W}_{\text{down}} \left[\left(\mathbf{W}_{\text{up}} \mathbf{x} \right) \odot \sigma \left(\mathbf{W}_{\text{act}} \mathbf{x} \right) \right]. \tag{2.87}$$

Here, $\mathbf{x} \in \mathbb{R}^{d_{\text{hid}}}$, $\mathbf{W}_{\text{up}}, \mathbf{W}_{\text{act}} \in \mathbb{R}^{d_{\text{mid}} \times d_{\text{hid}}}$, and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{mid}}}$. The symbol $\odot$ denotes element-wise multiplication, and σ is an element-wise activation function. Both the input and output of the expert belong to $\mathbb{R}^{d_{\text{hid}}}$, while the two intermediate projections produce vectors in $\mathbb{R}^{d_{\text{mid}}}$.

The up projection produces

$$\mathbf{a} = \mathbf{W}_{\text{up}} \mathbf{x}, \tag{2.88}$$

whereas the activation projection produces

$$\mathbf{g} = \sigma \left(\mathbf{W}_{\text{act}} \mathbf{x} \right). \tag{2.89}$$

Their element-wise product is

$$\mathbf{z} = \mathbf{a} \odot \mathbf{g}, \qquad z_i = a_i g_i, \quad i = 1, \ldots, d_{\text{mid}}. \tag{2.90}$$

The down projection then maps this modulated intermediate representation back to the hidden dimension:

$$E(\mathbf{x}) = \mathbf{W}_{\text{down}} \mathbf{z}. \tag{2.91}$$

The additional projection is not merely a different choice of activation function. It supplies a second learned transformation of the input, and its output controls the intermediate features through element-wise multiplication.

Ignoring biases, the three matrices contain

$$P_{\text{gated}} = d_{\text{mid}} d_{\text{hid}} + d_{\text{mid}} d_{\text{hid}} + d_{\text{hid}} d_{\text{mid}} \tag{2.92}$$

$$= 3 d_{\text{hid}} d_{\text{mid}} \tag{2.93}$$

parameters. This is 50% more projection parameters than a two-projection FFN with the same input, output, and intermediate dimensions. For M tokens, the three projections require approximately $3 M d_{\text{hid}} d_{\text{mid}}$ MACs, or $6 M d_{\text{hid}} d_{\text{mid}}$ FLOPs when each multiply–accumulate is counted as two FLOPs, in addition to element-wise operations.

The dimensions reported by Lo et al. for the Mistral 7B configuration include $d_{\text{hid}} = 4096$ and $d_{\text{mid}} = 14336$. Using these dimensions in the three-projection expression gives

$$P_{\text{gated}} = 3(4096)(14336) \tag{2.94}$$

$$= 176\,160\,768. \tag{2.95}$$

This is the weight count for the three projection matrices under the bias-free formulation. It is not a count of every parameter in the Transformer block or the complete language model (**lo2025closer**).

The three-projection formulation also permits the contribution of each intermediate unit to be examined explicitly. Let $\mathbf{W}_{\text{up}}[i, :]$ and $\mathbf{W}_{\text{act}}[i, :]$ denote the i-th rows of the up

and activation matrices, and let $\mathbf{W}_{\text{down}}[:, i]$ denote the i-th column of the down matrix. The contribution of unit i is determined by

$$z_i = \left(\mathbf{W}_{\text{up}}[i, :]\mathbf{x}\right) \sigma\left(\mathbf{W}_{\text{act}}[i, :]\mathbf{x}\right). \tag{2.96}$$

The complete expert output can consequently be decomposed as

$$E(\mathbf{x}) = \sum_{i=1}^{d_{\text{mid}}} \mathbf{W}_{\text{down}}[:, i] \left[\left(\mathbf{W}_{\text{up}}[i, :]\mathbf{x}\right) \sigma\left(\mathbf{W}_{\text{act}}[i, :]\mathbf{x}\right)\right]. \tag{2.97}$$

Each intermediate unit therefore combines two input-side projections with an output-side vector. The input projections determine the input-dependent scalar activation of that unit, and the corresponding column of the down matrix determines the direction of its contribution in the output space.

Lo et al. use this decomposition to motivate the interpretation of FFN neurons as fine-grained computational experts. This is a functional interpretation of the internal units, not a claim that each neuron is an independent expert network with its own router. The activation projection controls intermediate neurons within one FFN, whereas an MoE router selects among complete expert modules. These are distinct levels of computation (**lo2025closer**).

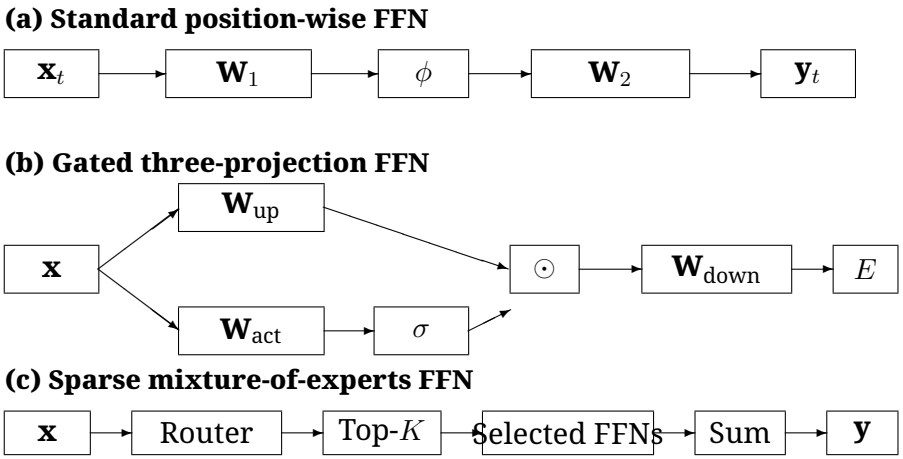


Figure 2.6: Schematic comparison of a standard position-wise FFN, a three-projection gated FFN, and a sparsely routed MoE layer. In the standard FFN, the input is projected, transformed element-wise, and projected back to the model dimension. In the gated FFN, parallel up and activation projections are combined element-wise before the down projection. In the sparse MoE layer, a router selects a subset of FFN experts whose weighted outputs are aggregated.

2.3.4 Feed-Forward Networks Within Transformer Blocks

An FFN is a component of a larger Transformer block rather than an independent sequence-processing stage. In the formulation used by Dai et al., the self-attention sublayer first produces a representation with a residual connection, after which the

FFN transforms each token representation and adds another residual connection (**dai2024deepseekmoe**). Let $\mathbf{h}_{1:T}^{l-1}$ denote the token representations entering block l, and let $\mathbf{u}_{1:T}^{l}$ denote the representations after its self-attention sublayer. The simplified computation is

$$\mathbf{u}_{1:T}^{l} = \text{SelfAtt}\left(\mathbf{h}_{1:T}^{l-1}\right) + \mathbf{h}_{1:T}^{l-1}. \tag{2.98}$$

The FFN then operates on each token separately:

$$\mathbf{h}_{t}^{l} = \text{FFN}\left(\mathbf{u}_{t}^{l}\right) + \mathbf{u}_{t}^{l}. \tag{2.99}$$

The residual addition requires the FFN output to have the same dimension as its input. This is why the second FFN projection returns the intermediate representation to d_{model} dimensions.

A residual connection adds the input of a sublayer to the transformation computed by that sublayer. It therefore preserves a direct path for the incoming representation while allowing the learned transformation to contribute an update. The equations above follow the simplified formulation in Dai et al.; that paper omits layer-normalization operations from these equations for brevity. The equations should therefore be read as a description of the FFN and residual structure, not as a complete specification of every normalization convention used by all Transformer implementations (**dai2024deepseekmoe**).

The two sublayers have complementary roles. Self-attention explicitly combines information across token positions, whereas the position-wise FFN applies the same learned nonlinear feature transformation to each token. The FFN does not itself create cross-token interactions; it transforms the representation that is available at its input.

2.3.5 From Dense FFNs to Mixture-of-Experts Layers

The parameter count of an FFN can be substantial because both projection matrices scale with the model and intermediate dimensions. Cai et al. report that, in the 540-billion-parameter PaLM configuration discussed in their survey, approximately 90% of the parameters are located in FFN layers. They also summarize an empirical analysis reporting lower active expert engagement in FFN layers than in self-attention layers for the particular model and task examined. These observations motivate the investigation of conditional computation in FFN modules, but the reported proportions are specific to the cited experiments and should not be treated as universal constants for Transformer models (**cai2025survey**).

A mixture-of-experts (MoE) layer replaces one FFN with a collection of N expert networks,

$$\mathcal{E} = \{E_1, E_2, \ldots, E_N\},$$

where each expert is itself an FFN. A gating network, also called a router, assigns scores to the experts for each input token. In a linear-router formulation, the router logits for input $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ are

$$\mathbf{q}(\mathbf{x}) = \mathbf{W}_r \mathbf{x}, \qquad \mathbf{W}_r \in \mathbb{R}^{N \times d_{\text{model}}}. \tag{2.100}$$

The logits are transformed into routing probabilities by the softmax:

$$p_i(\mathbf{x}) = \frac{\exp(q_i(\mathbf{x}))}{\sum_{j=1}^{N} \exp(q_j(\mathbf{x}))}, \qquad \sum_{i=1}^{N} p_i(\mathbf{x}) = 1. \tag{2.101}$$

The probabilities are nonnegative and sum to one, so they define a distribution over the available experts.

In a dense MoE layer, all experts are evaluated and their outputs are combined using their routing probabilities:

$$\mathrm{MoE}_{\mathrm{dense}}(\mathbf{x}) = \sum_{i=1}^{N} p_i(\mathbf{x}) E_i(\mathbf{x}). \tag{2.102}$$

In a sparse MoE layer, only the K experts with the largest routing scores are evaluated. Let $\mathcal{T}_K(\mathbf{x})$ denote the set of their indices. The sparse gate is

$$g_i(\mathbf{x}) = \begin{cases} p_i(\mathbf{x}), & i \in \mathcal{T}_K(\mathbf{x}), \\ 0, & i \notin \mathcal{T}_K(\mathbf{x}). \end{cases} \tag{2.103}$$

The sparse MoE output is

$$\mathrm{MoE}_{\mathrm{sparse}}(\mathbf{x}) = \sum_{i=1}^{N} g_i(\mathbf{x}) E_i(\mathbf{x}) = \sum_{i \in \mathcal{T}_K(\mathbf{x})} p_i(\mathbf{x}) E_i(\mathbf{x}). \tag{2.104}$$

The distinction is computational: in the sparse formulation, experts whose gate values are zero need not perform their FFN computation for that token. The linear-router formulation above follows the routing description in Fedus et al.; Dai et al. also describe token-to-expert affinity scores based on the similarity between a token representation and an expert centroid (**fedus2022switch**; **dai2024deepseekmoe**).

If each expert contains P_E parameters and all experts have the same size, the total expert parameter count is

$$P_{\mathrm{experts,total}} = N P_E. \tag{2.105}$$

When K experts are active for a token, the number of active expert parameters is

$$P_{\mathrm{experts,active}} = K P_E. \tag{2.106}$$

The ratio is therefore

$$\frac{P_{\mathrm{experts,active}}}{P_{\mathrm{experts,total}}} = \frac{K}{N}. \tag{2.107}$$

These expressions concern expert parameters and exclude router parameters and any other shared components. They show how a sparse MoE layer can expose a large total pool of FFN parameters while activating only a subset for an individual token. The total model capacity and the expert computation per token are consequently different quantities.

2.3.6 Switch Routing, Expert Capacity, and Load Balancing

Fedus et al. introduce the Switch Transformer as a simplified sparse MoE design in which each token is routed to one expert rather than to multiple experts. The selected expert is

$$i^* = \arg\max_{1 \le i \le N} p_i(\mathbf{x}), \tag{2.108}$$

and the output of the Switch FFN is

$$\text{SwitchFFN}(\mathbf{x}) = p_{i^*}(\mathbf{x}) E_{i^*}(\mathbf{x}). \tag{2.109}$$

The expert output is scaled by the selected router probability. Top-1 routing reduces the number of expert computations per token and simplifies token dispatch relative to a routing rule that activates multiple experts. Fedus et al. report that this simplification can preserve model quality in their experiments while reducing routing and communication requirements (**fedus2022switch**).

Sparse routing creates a finite-capacity allocation problem. Let M be the number of tokens presented to one MoE layer in a batch. Under uniform allocation across N experts, the expected number of tokens per expert is M/N. Switch Transformer defines expert capacity as

$$C = \left(\frac{M}{N} \right) c_{\text{factor}}, \tag{2.110}$$

where c_{factor} is the capacity factor. A factor greater than one provides buffer for nonuniform routing. In an implementation, the resulting capacity must be represented by an admissible integer tensor dimension.

For example, consider $M = 1024$ tokens and $N = 16$ experts. The uniform allocation is $1024/16 = 64$ tokens per expert. With $c_{\text{factor}} = 1.5$, the capacity is

$$C = 64 \times 1.5 = 96.$$

An expert can therefore accept up to 96 tokens in this example. If more tokens are routed to it, the excess tokens are dropped from that expert computation in the Switch implementation and their representations are passed to the next layer through the residual connection. A larger capacity factor reduces the risk of overflow, but it also allocates more buffer and can increase memory and computation spent on unused capacity. Capacity is therefore a trade-off between token overflow and resource utilization (**fedus2022switch**).

The router may not distribute tokens uniformly. To encourage a more balanced allocation, Switch Transformer adds an auxiliary loss to the main training objective. For a batch $\mathcal{B}$ containing M tokens, define the fraction of tokens dispatched to expert i as

$$f_i = \frac{1}{M} \sum_{\mathbf{x} \in \mathcal{B}} \mathbb{1} \left[\arg\max_j p_j(\mathbf{x}) = i \right], \tag{2.111}$$

where $\mathbb{1}[\cdot]$ is the indicator function. Define the average router probability assigned to expert i as

$$P_i = \frac{1}{M} \sum_{\mathbf{x} \in \mathcal{B}} p_i(\mathbf{x}). \tag{2.112}$$

The auxiliary load-balancing loss is

$$\mathcal{L}_{\text{aux}} = \alpha N \sum_{i=1}^{N} f_i P_i, \tag{2.113}$$

where α controls the contribution of this term to the training objective. The vector f records the realized dispatch frequencies, whereas P records the average probability

mass assigned by the router. These quantities are not interchangeable: one describes discrete assignments and the other describes the router's probability outputs.

Uniform allocation corresponds to $f_i = P_i = 1/N$ for every expert. Under this condition,

$$\mathcal{L}_{\text{aux}} = \alpha N \sum_{i=1}^{N} \frac{1}{N^2} = \alpha.$$

The objective is minimized at the uniform allocation. Although the dispatch indicator in f_i is discrete, the probability term P_i depends on the softmax router outputs and supplies a differentiable training signal to the router. The auxiliary loss is added to the model's main loss; it does not replace the language-modeling objective (**fedus2022switch**).

2.3.7 Fine-Grained Segmentation and Shared Experts

A conventional sparse MoE layer may contain a limited number of relatively large FFN experts. Dai et al. identify two potential limitations of this organization. Knowledge hybridity occurs when one expert must process tokens associated with diverse types of knowledge. Knowledge redundancy occurs when different experts independently acquire overlapping information that is useful across multiple contexts. DeepSeekMoE addresses these problems through fine-grained expert segmentation and shared expert isolation (**dai2024deepseekmoe**).

In fine-grained expert segmentation, each original expert is divided into m smaller experts by reducing its intermediate dimension by a factor of m. If the original architecture has N experts, each with intermediate dimension d_{ff}, the fine-grained architecture has

$$N_{\text{fine}} = mN, \qquad d_{\text{ff,fine}} = \frac{d_{\text{ff}}}{m}. \tag{2.114}$$

To preserve the dominant expert computation per token, the number of activated experts is increased from K to mK:

$$K_{\text{fine}} = mK. \tag{2.115}$$

These relations assume that the intermediate dimension can be divided as specified and that the expert projection costs scale linearly with that dimension.

If an original expert contains P_E projection parameters, each fine-grained expert contains approximately P_E/m such parameters. The total expert parameter count is therefore preserved:

$$P_{\text{fine,total}} = mN \left(\frac{P_E}{m} \right) \tag{2.116}$$

$$= NP_E. \tag{2.117}$$

Likewise, if the dominant computation of one original expert is C_E, then the active expert computation is

$$C_{\text{fine,active}} = mK \left(\frac{C_E}{m} \right) \tag{2.118}$$

$$= KC_E. \tag{2.119}$$

Thus, the segmentation increases the number of available experts and the number selected for each token while preserving the leading-order expert parameter count and active expert computation.

The principal change is the number of possible expert combinations. If K experts are selected from N experts, the number of unordered combinations is

$$\binom{N}{K} = \frac{N!}{K!(N-K)!}. \tag{2.120}$$

After segmentation, the corresponding count is

$$\binom{mN}{mK}. \tag{2.121}$$

Dai et al. give the example $N = 16$ and $K = 2$, for which the conventional architecture permits

$$\binom{16}{2} = 120$$

possible combinations. With $m = 4$, the fine-grained architecture contains 64 experts and activates eight, giving

$$\binom{64}{8} = 4\,426\,165\,368$$

possible combinations. This is a count of possible subsets, not a claim that all combinations are equally likely or are observed during training. The larger combination space gives the router more ways to compose specialized expert subsets (**dai2024deepseekmoe**).

Fine-grained segmentation does not, by itself, eliminate redundancy between experts. DeepSeekMoE therefore introduces shared expert isolation. A designated set of K_s shared experts is activated for every token, while the remaining experts are routed conditionally. The number of routed experts is $mN - K_s$, and the number of routed experts selected for each token is $mK - K_s$. A simplified representation of the layer output is

$$\begin{aligned}\mathbf{h}_t^l = &\sum_{j=1}^{K_s} E_j^{\text{shared}}\left(\mathbf{u}_t^l\right) \\ &+ \sum_{i=1}^{mN-K_s} g_{i,t} E_i^{\text{routed}}\left(\mathbf{u}_t^l\right) + \mathbf{u}_t^l.\end{aligned} \tag{2.122}$$

The shared experts provide a common computational pathway for information that is useful across contexts. The routed experts can then specialize more strongly in input-dependent information without each having to reproduce all common knowledge.

For the routing formulation described by Dai et al., the affinity between token t and expert i in layer l is computed from the token representation and the expert centroid $\mathbf{e}_i^l$:

$$s_{i,t} = \text{Softmax}_i\left((\mathbf{u}_t^l)^\top \mathbf{e}_i^l\right). \tag{2.123}$$

The sparse gate retains the mK largest affinities in the fine-grained formulation, with shared experts handled separately. The centroid-affinity expression is a particular routing parameterization and should not be conflated with the linear router used in the generic Switch formulation.

The DeepSeekMoE paper distinguishes balancing at the expert level from balancing at the device level. Let

$$N' = mN - K_s, \qquad K' = mK - K_s$$

denote the number of routed experts and the number of routed experts selected for each token, respectively. For a batch of T tokens, the normalized dispatch fraction for routed expert i is

$$f_i = \frac{N'}{K'T} \sum_{t=1}^{T} \mathbb{1}\left(\text{token } t \text{ selects routed expert } i\right), \tag{2.124}$$

and the average affinity assigned to that expert is

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t}. \tag{2.125}$$

The expert-level balancing objective is

$$\mathcal{L}_{\text{ExpBal}} = \frac{\alpha_1}{N'} \sum_{i=1}^{N'} f_i P_i, \tag{2.126}$$

where α_1 controls the strength of expert-level balancing. The normalization in f_i makes its value equal to one under uniform routed-expert allocation.

Balancing individual experts does not guarantee balanced accelerator workloads. Suppose the routed experts are partitioned into D groups, with group $\mathcal{G}_d$ deployed on device d. The normalized dispatch fraction and aggregate affinity for device group d are

$$f'_d = \frac{1}{|\mathcal{G}_d|} \sum_{j \in \mathcal{G}_d} f_j, \tag{2.127}$$

$$P'_d = \sum_{j \in \mathcal{G}_d} P_j. \tag{2.128}$$

The corresponding device-level balancing objective is

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{d=1}^{D} f'_d P'_d, \tag{2.129}$$

where α_2 is the device-level balance factor. Dai et al. describe using a relatively small expert-level balance factor to reduce routing collapse while assigning a larger weight to device-level balancing, because overly strict expert-level balancing can compromise model performance. These two objectives address different bottlenecks: one concerns token allocation among experts, and the other concerns the aggregate computational load assigned to devices (**dai2024deepseekmoe**).

Dai et al. report that DeepSeekMoE 2B achieves performance comparable to GShard 2.9B despite the latter having approximately 1.5 times the expert parameters and computation. They also report that DeepSeekMoE 16B achieves performance comparable to LLaMA2 7B using about 40% of the computation in their reported comparison. These are empirical results for the configurations and evaluations in that paper, rather than general guarantees for every fine-grained MoE design (**dai2024deepseekmoe**).

2.3.8 Expert Structure, Specialization, and Practical Constraints

The presence of multiple experts does not by itself establish that they have learned distinct functions. Experts may acquire overlapping parameters or exhibit similar output behavior. Lo et al. investigate this issue by analyzing the projection matrices $\mathbf{W}_{\text{up}}$, $\mathbf{W}_{\text{act}}$, and $\mathbf{W}_{\text{down}}$ in several MoE models (**lo2025closer**).

One way to compare two parameter matrices is to flatten them into vectors and calculate cosine similarity. For nonzero vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, cosine similarity is

$$\text{CosSim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}, \tag{2.130}$$

where

$$\|\mathbf{a}\|_2 = \sqrt{\sum_{j=1}^{n} a_j^2}$$

is the Euclidean norm. Cosine similarity measures angular alignment and is invariant to positive rescaling of either vector. It does not, however, measure equality of magnitudes or guarantee equality of the functions implemented by the corresponding networks.

Flattened-matrix comparison is also sensitive to neuron ordering. Two experts may contain similar units in different orders, while a direct element-by-element comparison pairs units that do not correspond. Lo et al. therefore investigate neuron-level comparisons, including a reordering procedure based on the Jonker–Volgenant algorithm for a linear assignment problem. Their analysis treats a gated FFN neuron as the combination of the corresponding rows of $\mathbf{W}_{\text{up}}$ and $\mathbf{W}_{\text{act}}$, together with the corresponding column of $\mathbf{W}_{\text{down}}$. This connects the neuron decomposition in equation (2.97) to a more detailed analysis of expert similarity (**lo2025closer**).

In their investigated models, Lo et al. report that matrix-level similarities differ across architectures and layers. For Mixtral, they report cosine similarities primarily in the range 0.2–0.4 between experts, while similarities between Mixtral experts and the dense Mistral FFN are approximately 0.6 in the examined comparisons. They also report that similarities between the routed experts of DeepSeekMoE and the experts of Grok-1 are close to zero in their analysis. These observations are specific to the models, layers, matrices, and comparison procedures studied. They should not be interpreted as a universal numerical measure of expert specialization.

The choice of expert count, expert dimension, number of active experts, and frequency of MoE layers also affects the architecture. Cai et al. identify these as important design variables and describe configurations in which MoE layers replace every second FFN layer, every fourth FFN layer, or the FFN in every layer. A larger number of experts can increase total parameter capacity, while a larger expert dimension increases the capacity and cost of each expert. Increasing the number of active experts changes the computation performed per token. These choices must therefore be considered together rather than optimized independently (**cai2025survey**).

The theoretical arithmetic savings of sparse FFNs do not automatically translate into proportional reductions in wall-clock latency. Routing requires score calculation and expert selection. The implementation must also dispatch tokens to the selected experts and combine their outputs. When experts are distributed across accelerators, this can introduce communication and synchronization costs. Zhang et al. review practical issues

including irregular memory access, cross-device communication, fragmented workloads, and hardware underutilization. These system effects can reduce the practical benefit of sparse computation, particularly under latency or memory constraints (**zhang2025moe**).

The architecture must consequently balance representational capacity, expert specialization, routing quality, load distribution, memory requirements, and the communication pattern imposed by the hardware. An MoE layer is not efficient merely because it activates a small fraction of its parameters; its routing and execution mechanisms must also use the available computational resources effectively.

2.3.9 Limitations and Connection to Activation Functions

The development of FFN architectures reveals a progression from a shared nonlinear transformation applied to every token, through gated intermediate representations, to conditional computation over multiple FFN experts. The conventional FFN provides a position-wise mapping whose parameter count and arithmetic cost are determined by its input, intermediate, and output dimensions. Gated FFNs introduce an additional learned projection that modulates intermediate features. MoE architectures extend the same basic feed-forward computation by selecting a subset of expert networks for each token.

These architectures introduce different trade-offs. Increasing the intermediate dimension increases the capacity and cost of a dense FFN. Adding a gate projection increases the parameterization and computation of a gated FFN. Increasing the number of MoE experts increases total expert capacity, but sparse execution introduces routing, capacity, load-balancing, and communication constraints. Fine-grained segmentation and shared expert isolation provide strategies for organizing expert parameters, but they do not remove the need to evaluate the quality of specialization or the practical cost of routing.

The mathematical role of the activation function remains central throughout this progression. In a conventional FFN, it supplies the nonlinearity between the two projections. In a gated FFN, an activation function transforms the control projection before it is combined with the up-projection. In an MoE architecture, the router also uses a probability transformation to produce expert-selection weights. These operations may all involve nonlinear functions, but they act on different representations and serve different architectural purposes. The next section examines activation functions as mathematical operators, establishing the properties needed to understand their effects on feature transformations, gated computation, and gradient-based learning.

2.4 Activation Functions

The preceding section established the feed-forward network as the component that transforms each contextual token representation through learned projections. Its central operation is a composition of affine mappings with a nonlinear transformation between them. The mathematical role of that intermediate transformation is the subject of the present section. An activation function is not merely a convenient operation inserted between matrix multiplications: it determines which nonlinear mappings the network can represent, how derivatives propagate during optimization, and what numerical and computational costs are incurred. These properties become particularly consequential

in a Transformer, where the same feed-forward architecture is applied repeatedly across a deep stack of layers (**han2024review**; **zhuo2025polycom**).

2.4.1 Why nonlinear transformations are indispensable

An affine transformation maps an input vector to an output vector by multiplying it by a matrix and adding a bias. Under the row-vector convention used in the preceding feed-forward discussion, let $x \in \mathbb{R}^{1\times H}$, let $W \in \mathbb{R}^{H\times D}$, and let $b \in \mathbb{R}^{1\times D}$. The affine mapping is

$$z = xW + b, \qquad z \in \mathbb{R}^{1\times D}. \tag{2.131}$$

The word *affine* is important: when $b \neq 0$, the mapping is not linear in the strict algebraic sense, but it is a linear transformation followed by a translation.

Composing affine transformations does not, by itself, create a nonlinear function. To see this, consider two successive mappings,

$$z_1 = xW_1 + b_1, \tag{2.132}$$
$$z_2 = z_1W_2 + b_2. \tag{2.133}$$

Substitution of the first expression into the second gives

$$\begin{aligned} z_2 &= (xW_1 + b_1)W_2 + b_2 \\ &= x(W_1W_2) + (b_1W_2 + b_2). \end{aligned} \tag{2.134}$$

Defining

$$W_{\text{eff}} = W_1W_2, \qquad b_{\text{eff}} = b_1W_2 + b_2, \tag{2.135}$$

reduces the composition to

$$z_2 = xW_{\text{eff}} + b_{\text{eff}}. \tag{2.136}$$

It is therefore still one affine transformation. The same reasoning extends to any finite sequence of affine layers: without an intervening nonlinear operation, depth alone does not enlarge the class of functions beyond affine mappings.

An activation function changes this conclusion. Let $\rho : \mathbb{R} \to \mathbb{R}$ be a scalar nonlinear function. For a vector input, the standard element-wise convention applies ρ to every coordinate independently. A two-projection feed-forward network can then be written as

$$\text{FFN}_\rho(x) = \rho(xW_1 + b_1)W_2 + b_2. \tag{2.137}$$

Here $W_1 \in \mathbb{R}^{H\times D_{\text{ff}}}$, $W_2 \in \mathbb{R}^{D_{\text{ff}}\times H}$, $b_1 \in \mathbb{R}^{1\times D_{\text{ff}}}$, and $b_2 \in \mathbb{R}^{1\times H}$. The intermediate dimension D_{ff} is the number of features to which the activation is applied. The bias-free form

$$\text{FFN}_\rho(x) = \rho(xW_1)W_2 \tag{2.138}$$

is the formulation used by **zhuo2025polycom**; the inclusion of biases does not alter the argument about the need for nonlinearity.

For a complete sequence tensor $X \in \mathbb{R}^{B\times S\times H}$, where B is batch size and S is sequence length, the operation acts on the final feature dimension:

$$Z = XW_1, \qquad Z \in \mathbb{R}^{B\times S\times D_{\text{ff}}}, \tag{2.139}$$
$$H_\rho = \rho(Z), \qquad H_\rho \in \mathbb{R}^{B\times S\times D_{\text{ff}}}, \tag{2.140}$$
$$Y = H_\rho W_2, \qquad Y \in \mathbb{R}^{B\times S\times H}. \tag{2.141}$$

The activation changes the feature coordinates of each token representation. For element-wise functions such as ReLU, GELU, and SiLU, it does not itself mix different sequence positions. The attention sublayer is responsible for cross-position information exchange, whereas the feed-forward activation contributes to the nonlinear transformation of the representation available at each position (**han2024review**; **zhuo2025polycom**).

The contrast between an affine composition and a nonlinear feed-forward mapping is summarized in figure 2.7. The upper path can be collapsed into one affine mapping. The lower path cannot generally be collapsed in this way because the activation intervenes between the two projections.

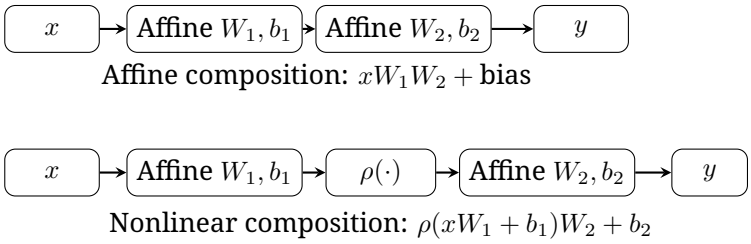


Figure 2.7: Affine and nonlinear compositions in a feed-forward network. Without an activation, two affine projections collapse into one affine mapping. An intervening nonlinear function changes the class of representable mappings. Author-generated schematic based on the feed-forward formulation in **zhuo2025polycom**.

2.4.2 Differentiability and gradient propagation

The representational role of an activation function is only one part of its behavior. A language model is trained by minimizing an objective function with respect to its parameters. In gradient-based optimization, the derivatives of the activation function determine how the training signal passes through the feed-forward computation.

Let the pre-activation vector be

$$z = xW_1 + b_1 \tag{2.142}$$

and the activated representation be

$$h = \rho(z). \tag{2.143}$$

The output is $y = hW_2 + b_2$, and let $\mathcal{L}$ denote a differentiable training loss. The chain rule gives

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial y} W_2^\top, \tag{2.144}$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial h} \odot \rho'(z), \tag{2.145}$$

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial z} W_1^\top. \tag{2.146}$$

The symbol $\odot$ denotes the Hadamard product: for equal-dimensional vectors u and v, $(u \odot v)_i = u_i v_i$. The derivative $\rho'(z)$ is evaluated coordinate by coordinate. Equivalently, the Jacobian of an element-wise activation is a diagonal matrix,

$$D_{\rho'}(z) = \operatorname{diag}\big(\rho'(z_1), \ldots, \rho'(z_{D_{\text{ff}}})\big). \tag{2.147}$$

This diagonal structure follows from the fact that each output coordinate depends only on the corresponding input coordinate.

Across a deep network, the chain rule produces products of weight matrices and activation Jacobians. In operator-norm form, a typical gradient bound contains factors of the form

$$\left\| \frac{\partial \mathcal{L}}{\partial h^{(0)}} \right\|_2 \leq \left\| \frac{\partial \mathcal{L}}{\partial h^{(L)}} \right\|_2 \prod_{\ell=1}^{L} \left\| W^{(\ell)} \right\|_2 \left\| D_{\rho'}(z^{(\ell)}) \right\|_2 . \tag{2.148}$$

Here L is the number of transformations, $W^{(\ell)}$ is the corresponding weight matrix, and $\|\cdot\|_2$ is the spectral norm for matrices. For a diagonal activation Jacobian, $\|D_{\rho'}(z)\|_2 = \max_i |\rho'(z_i)|$. If the factors in the product are repeatedly small, gradients can diminish as they are propagated toward earlier layers; this is the vanishing-gradient phenomenon. If the products are repeatedly large, gradients can grow excessively, contributing to exploding gradients. The activation function is therefore part of the optimization dynamics, not merely a forward-pass nonlinearity (**lukac2026llms**).

2.4.3 Classical and smooth activation functions

The sigmoid function maps the real line to the interval $(0, 1)$:

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}. \tag{2.149}$$

Differentiating gives

$$\text{sigmoid}'(x) = \text{sigmoid}(x)\big(1 - \text{sigmoid}(x)\big). \tag{2.150}$$

The derivative reaches its maximum value $1/4$ at $x = 0$ and approaches zero as x tends to either positive or negative infinity. In these extreme regions the function saturates: large changes in the input produce only small changes in the output. Repeated multiplication by small activation derivatives can attenuate gradients.

The hyperbolic tangent function is

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}, \tag{2.151}$$

with derivative

$$\tanh'(x) = 1 - \tanh^2(x). \tag{2.152}$$

Its output lies in $(-1, 1)$ and is zero-centered, but the derivative again approaches zero for large absolute input values. Sigmoid and hyperbolic tangent are therefore useful bounded nonlinear functions, while their saturation behavior is a relevant consideration when they are used repeatedly in deep networks (**fang2023raft**; **lukac2026llms**).

The Rectified Linear Unit (ReLU) is defined by

$$\text{ReLU}(x) = \max\{0, x\} = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases} \tag{2.153}$$

For $x \neq 0$, its derivative is

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases} \tag{2.154}$$

ReLU is not classically differentiable at zero; implementations conventionally choose a subgradient there. Its positive branch has a constant derivative of one, avoiding saturation on that side. Its negative branch outputs zero and has zero derivative, however, so a unit that remains in the negative region receives no gradient through that activation. This behavior is commonly described as the dying-ReLU problem. ReLU is computationally simple and can produce sparse intermediate vectors because negative coordinates are set exactly to zero (**fang2023raft**; **zhuo2025polycom**).

The Gaussian Error Linear Unit (GELU) uses the cumulative distribution function of a standard normal random variable. Let

$$\phi(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \tag{2.155}$$

be the standard normal density, and let

$$\Phi(x) = \int_{-\infty}^{x} \phi(t)\, dt \tag{2.156}$$

be its cumulative distribution function. GELU is

$$\mathrm{GELU}(x) = x\Phi(x). \tag{2.157}$$

Applying the product rule and using $\Phi'(x) = \phi(x)$ gives

$$\mathrm{GELU}'(x) = \Phi(x) + x\phi(x). \tag{2.158}$$

GELU weights the input by a smooth, input-dependent factor. For large positive x, $\Phi(x)$ approaches one and GELU approaches the identity function. For large negative x, $\Phi(x)$ approaches zero and the output approaches zero, but negative inputs are not removed by a hard threshold as they are in ReLU.

Using the error function,

$$\mathrm{erf}(u) = \frac{2}{\sqrt{\pi}} \int_0^u \exp(-t^2)\, dt, \tag{2.159}$$

the same activation can be written as

$$\mathrm{GELU}(x) = \frac{x}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right]. \tag{2.160}$$

A commonly used approximation replaces the error-function expression with a hyperbolic tangent:

$$\mathrm{GELU}_{\tanh}(x) = \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right]. \tag{2.161}$$

The approximation is useful when a computationally convenient expression is preferred to direct evaluation of the Gaussian cumulative distribution function. **fang2023raft** report that GELU was used by 80% of the 20 Transformer language models they surveyed after BERT, while 45% of those models did not explicitly specify their activation function. These percentages describe the paper's sample, not every Transformer model.

The Sigmoid Linear Unit (SiLU), also called Swish when its scaling parameter is one, is

$$\mathrm{SiLU}(x) = x\,\mathrm{sigmoid}(x) = \frac{x}{1+\exp(-x)}. \tag{2.162}$$

Its derivative follows from the product rule:

$$\mathrm{SiLU}'(x) = \mathrm{sigmoid}(x) + x\,\mathrm{sigmoid}(x)\big(1-\mathrm{sigmoid}(x)\big). \tag{2.163}$$

The generalized Swish function is

$$\mathrm{Swish}_\beta(x) = x\,\mathrm{sigmoid}(\beta x), \tag{2.164}$$

where β controls the transition profile. SiLU is smooth and can produce negative outputs for some negative inputs; it is not simply a smooth version of a hard zero threshold. Its importance in Transformer architectures is also connected to its use as the gating nonlinearity in SwiGLU (**zhuo2025polycom**; **lukac2026llms**).

Table 2.4: Definitions and principal mathematical characteristics of commonly discussed scalar activation functions.

Function	Definition	Mathematical characteristic
Sigmoid	$1/(1+\exp(-x))$	Bounded in $(0,1)$; saturating derivative.
tanh	$(\exp(x)-\exp(-x))/(\exp(x)+\exp(-x))$	Zero-centered and bounded in $(-1,1)$; saturating derivative.
ReLU	$\max\{0,x\}$	Piecewise linear; zero output and zero derivative for negative input.
GELU	$x\Phi(x)$	Smooth Gaussian-CDF weighting of the input.
SiLU	$x\,\mathrm{sigmoid}(x)$	Smooth, non-monotonic input-dependent gating profile.

2.4.4 Gated activation functions

A conventional feed-forward network applies one activation function to a single intermediate representation. A gated linear unit (GLU) changes this structure by constructing two learned representations and combining them multiplicatively. Let

$$v = xW_v + b_v, \tag{2.165}$$

$$g = xW_g + b_g, \tag{2.166}$$

where v is the value branch and g is the gate branch. For an element-wise gating function ρ, the gated transformation is

$$\mathrm{GLU}_\rho(x) = v \odot \rho(g). \tag{2.167}$$

For two vectors of the same dimension, the Hadamard product is defined coordinatewise by

$$(v \odot q)_i = v_i q_i. \tag{2.168}$$

The gate therefore controls the contribution of each value coordinate through a corresponding coordinate of $\rho(g)$.

The original sigmoid-gated form uses

$$\text{GLU}(x) = (xW_v + b_v) \odot \text{sigmoid}(xW_g + b_g). \tag{2.169}$$

Replacing the sigmoid gate with GELU gives GEGLU:

$$\text{GEGLU}(x) = (xW_v + b_v) \odot \text{GELU}(xW_g + b_g). \tag{2.170}$$

Replacing it with SiLU gives SwiGLU:

$$\text{SwiGLU}(x) = (xW_v + b_v) \odot \text{SiLU}(xW_g + b_g). \tag{2.171}$$

The feed-forward block includes an output projection:

$$\text{FFN}_{\text{SwiGLU}}(x) = [(xW_v + b_v) \odot \text{SiLU}(xW_g + b_g)]\, W_o + b_o. \tag{2.172}$$

If the model hidden dimension is H and the gated intermediate dimension is D_g, the matrices have shapes

$$W_v, W_g \in \mathbb{R}^{H \times D_g}, \qquad W_o \in \mathbb{R}^{D_g \times H}. \tag{2.173}$$

The two parallel input projections and the output projection contain approximately $3HD_g$ weights, ignoring biases.

For comparison, a conventional two-projection FFN with intermediate dimension D_{ff} contains approximately $2HD_{\text{ff}}$ weights. Matching the projection-parameter counts requires

$$3HD_g = 2HD_{\text{ff}}, \tag{2.174}$$

$$D_g = \frac{2}{3} D_{\text{ff}}. \tag{2.175}$$

When $D_{\text{ff}} = 4H$, this gives

$$D_g = \frac{8}{3} H. \tag{2.176}$$

This parameter matching matters when activation architectures are compared experimentally. Giving a gated network the same intermediate dimension as a conventional network would generally give it a larger projection parameter budget. **zhuo2025polycom** use an intermediate dimension equal to two-thirds of that used by the other activation configurations in their parameter-matched experiments.

The architecture of SwiGLU is shown in figure 2.8. Its two branches should not be confused with two successive activations: they are parallel learned projections whose outputs are combined by an element-wise product. Gated activations therefore modify both the nonlinear operation and the projection structure of the feed-forward network (**zhuo2025polycom**; **lukac2026llms**).

2.4.5 Learnable rational activation functions

In a conventional Transformer, the activation function is selected before pre-training and remains fixed while the weight matrices are optimized. This separates the choice of activation from the learning process. It also creates a search problem if different functions

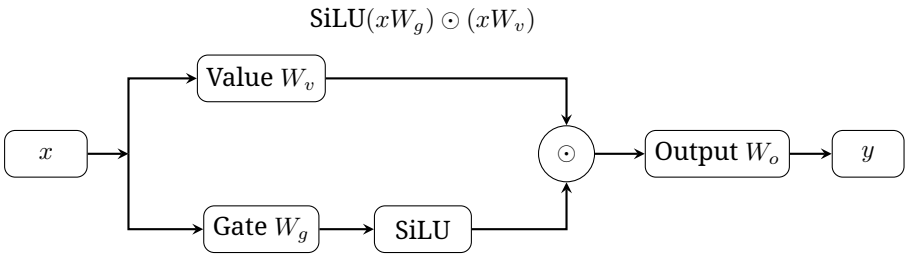


Figure 2.8: SwiGLU feed-forward computation. The input is mapped through parallel value and gate projections. The gate is transformed by SiLU, multiplied element-wise by the value representation, and projected back to the model hidden dimension. Author-generated schematic based on the formulation discussed by **zhuo2025polycom**.

may be selected independently for different layers. If k candidate functions are available for each of L layers, the number of possible layer-wise configurations is

$$N_{\text{config}} = k^L. \tag{2.177}$$

For $k = 3$ choices and $L = 12$ layers,

$$N_{\text{config}} = 3^{12} = 531{,}441. \tag{2.178}$$

fang2023raft use this example to illustrate why exhaustive selection of layer-specific activation functions is impractical for compute-intensive pre-training.

Fang and colleagues investigate rational activation functions (RAFs), which replace a fixed activation with a rational function whose coefficients are learned from data. A rational function is a ratio of polynomials. Their safe parameterization is

$$F(x) = \frac{P(x)}{Q_{\text{safe}}(x)} = \frac{\displaystyle\sum_{j=0}^{m} a_j x^j}{1 + \left| \displaystyle\sum_{k=0}^{n} b_k x^k \right|}, \tag{2.179}$$

where a_j and b_k are trainable coefficients, while m and n are the polynomial degrees. Because

$$1 + \left| \sum_{k=0}^{n} b_k x^k \right| \geq 1, \tag{2.180}$$

the denominator in this parameterization cannot be zero. This is the stability motivation for the safe rational formulation adopted in **fang2023raft**.

The degrees control the flexibility of the rational function. Increasing them can improve its ability to fit a target activation, but it also introduces more coefficients and computational work. Fang and colleagues evaluate degrees in the range $\{4, 5\}$ and select $m = 5$ and $n = 4$ because this configuration closely fits GELU on the interval $[-3, 3]$. The rational function is presented as a universal function approximator capable of fitting existing activation profiles. They report nine additional learnable parameters per implemented rational activation and 108 additional parameters in their 12-layer Transformer,

less than 0.000098% of the original model parameter count. The paper's reported implementation count is retained here rather than inferred by counting every coefficient in the generic displayed expression.

The optimization problem changes when the activation coefficients are trainable. Let θ denote the ordinary Transformer parameters and let α collect the rational activation coefficients. Instead of minimizing only over θ, training jointly optimizes

$$(\theta^*, \alpha^*) = \arg\min_{\theta,\alpha} \mathcal{L}(\theta, \alpha; \mathcal{D}), \tag{2.181}$$

where $\mathcal{D}$ is the training data and $\mathcal{L}$ is the training objective. A gradient-based update can be written as

$$\theta_{t+1} = \theta_t - \eta_\theta \nabla_\theta \mathcal{L}, \tag{2.182}$$

$$\alpha_{t+1} = \alpha_t - \eta_\alpha \nabla_\alpha \mathcal{L}. \tag{2.183}$$

The two learning rates need not be equal because the ordinary network parameters and the activation coefficients can have different optimization dynamics.

fang2023raft instantiate this approach in a BERT-like encoder-only Transformer with 12 layers, hidden size 768, and 12 attention heads. They compare a fixed-GELU Transformer (FAFT) with a rational-activation Transformer (RAFT). Their models are pre-trained from scratch on English Wikipedia using dynamic masked language modeling. The paper reports an AdamW configuration with $\beta_1 = 0.9$, $\beta_2 = 0.98$, and weight decay 0.01. The learning rate for the ordinary model parameters is 7×10^{-4}, whereas the learning rate for RAF coefficients is 5×10^{-3}. Both are warmed up during the first one percent of training steps; the ordinary parameter learning rate then decays linearly, while the RAF learning rate remains constant.

The pre-training validation results are

Model	Validation loss	Perplexity
FAFT	1.645	5.18
RAFT	1.611	5.00

(2.184)

The lower validation loss and perplexity indicate an improvement in the reported masked-language-modeling evaluation. The authors also evaluate downstream fine-tuning. With 100 labeled GLUE examples, RAFT improves the average score over FAFT by 5.31 points when all model parameters, including RAF coefficients, are fine-tuned ($\text{RAFT}_{\text{full}}$), and by 5.71 points when the pre-trained RAFs are held fixed ($\text{RAFT}_{\text{fixed}}$). With 300 examples, the reported average improvements are 0.98 and 1.59 points, respectively. In the full-data setting, the corresponding improvements are 0.70 and 0.58 points. The paper's abstract reports a 2.05-point improvement on SQuAD when all available data are used (**fang2023raft**).

The authors also combine RAFT with BitFit, a parameter-efficient fine-tuning method that updates bias parameters. In the full-data setting, the RAFT-based BitFit variants improve over the FAFT-based BitFit baseline by 2.87 and 3.08 points, depending on whether the pre-trained RAFs are fixed or fine-tuned. In the low-data setting, the reported improvements are 3.95 and 4.15 points. The study further examines fine-tuning only the RAF parameters; it reports 1,178 RAF parameters in that configuration, while the classification head is also updated in the experimental setup. These results suggest that learned

activation functions can be useful both as trainable task-adaptation parameters and as pre-trained transformations that are subsequently kept fixed.

The learned rational functions differ across Transformer layers after pre-training. During downstream fine-tuning, the upper layers change more substantially and develop more task-specific activation profiles than the lower layers. The authors also report cases in which the output of a rational activation is close to zero, suggesting that the corresponding feed-forward branch contributes relatively little to the layer output. They present the layer- and task-dependent shapes as a possible additional way to investigate the internal organization of pre-trained Transformers; similarity between activation profiles, however, is not by itself proof that two layers encode the same linguistic properties.

RAF flexibility has a computational cost. In the reported implementation, training is 36.8% slower during pre-training and 14.8% slower during fine-tuning than the fixed-activation baseline, because the rational coefficients must also be updated. With their CUDA extension, the authors report inference speedups of 13.8% during pre-training measurements and 3.9% during fine-tuning measurements. These values are implementation- and experiment-specific, not hardware-independent properties of rational functions. The authors also identify limited training resources as a limitation: their BERT-like experiment is not a direct comparison with substantially larger publicly available pre-trained models. They note that the effect of the additional flexibility on susceptibility to learned data biases requires further investigation (**fang2023raft**).

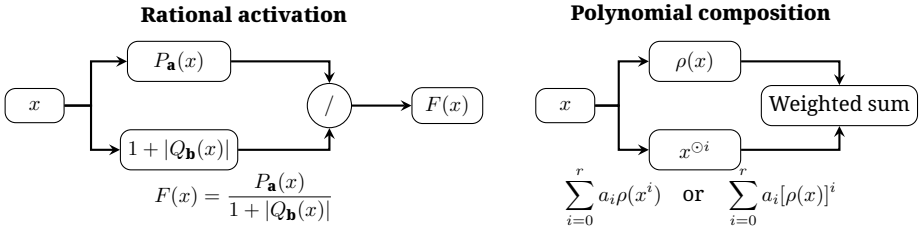


Figure 2.9: Conceptual comparison of rational and polynomial-composition activation constructions. A rational activation learns the coefficients of a numerator and denominator, whereas PolyCom combines powers and another function through learned coefficients. Author-generated schematic based on **fang2023raft** and **zhuo2025polycom**.

2.4.6 Polynomial composition activation functions

Learnable rational functions change the coefficients of a rational approximation during training. A different approach is to construct a more expressive activation from polynomial terms combined with another function. **zhuo2025polycom** introduce Polynomial Composition Activations (PolyCom) for this purpose. Their motivation is that pure polynomial activations have limitations in the approximation setting they discuss and have also shown weak empirical performance in some deep-learning applications. PolyCom therefore combines polynomial operations with another function rather than relying on a polynomial alone.

Let ρ denote a function such as ReLU, PReLU, sigmoid, SiLU, or a normalization operation. Let $r \in \mathbb{N}$ be the polynomial order and let $a_i \in \mathbb{R}$ be trainable coefficients. The authors define two composition types:

$$\text{PolyCom}_{\text{I}}(x) = \sum_{i=0}^{r} a_i [\rho(x)]^i, \tag{2.185}$$

$$\text{PolyCom}_{\text{II}}(x) = \sum_{i=0}^{r} a_i \rho(x^i). \tag{2.186}$$

For vector inputs, powers are applied element-wise. The distinction is the order of operations. Type I first applies ρ and then takes powers of the resulting values. Type II first takes powers of the input and then applies ρ to each powered representation. In the practical experiments, the authors use third-order PolyCom, $r = 3$, with $a_0 = 0$ and $a_i = 1/r$ for $i = 1, \ldots, r$ at initialization; the coefficients are subsequently trainable.

PolyReLU. PolyReLU is a Type I composition in which ρ is ReLU:

$$\text{PolyReLU}(x) = \sum_{i=0}^{r} a_i \big[\text{ReLU}(x)\big]^i, \qquad \text{ReLU}(x) = \max\{x, 0\}. \tag{2.187}$$

For $r = 3$, the expression is

$$\text{PolyReLU}(x) = a_0 + a_1 \,\text{ReLU}(x) + a_2 \,\text{ReLU}(x)^2 + a_3 \,\text{ReLU}(x)^3. \tag{2.188}$$

For negative inputs, all positive powers of ReLU are zero and the output is a_0. For positive inputs, the function is a polynomial in x. The learnable coefficients therefore control the shape of the positive branch. The paper establishes that ReLU, squared ReLU, and polynomial activations can be represented as special cases of PolyReLU (**zhuo2025polycom**).

PolyNorm. PolyNorm is a Type II composition in which the powers are normalized by their Euclidean norms. For a vector $x = (x_1, \ldots, x_d)^\top$, define its element-wise i-th power as

$$x^{\odot i} = (x_1^i, \ldots, x_d^i)^\top. \tag{2.189}$$

The Euclidean norm of a vector $u \in \mathbb{R}^d$ is

$$\|u\|_2 = \sqrt{\sum_{j=1}^{d} u_j^2}. \tag{2.190}$$

PolyNorm is then

$$\text{PolyNorm}(x) = \sum_{i=0}^{r} a_i \frac{x^{\odot i}}{\|x^{\odot i}\|_2}. \tag{2.191}$$

Each powered vector is rescaled before the weighted terms are combined. This is intended to prevent different polynomial orders from having excessively large or small magnitudes and thereby to stabilize training. Unlike scalar element-wise activations such as ReLU and GELU, the normalization denominator depends on the full feature vector.

Thus, each output coordinate of a normalized term depends on the other coordinates through the common norm. The operation remains local to the token's feature vector; it does not itself exchange information between sequence positions. The displayed expression requires a defined convention if a powered vector has zero norm; the paper's formula alone does not specify a particular numerical safeguard for that edge case.

Both PolyReLU and PolyNorm replace the activation in the conventional feed-forward expression

$$\mathrm{FFN}_\rho(x) = \rho(xW_1)W_2. \tag{2.192}$$

The overall two-projection FFN structure is retained, while the nonlinear transformation is changed. This differs from SwiGLU, which adds a parallel projection and a multiplicative gate.

2.4.7 Approximation theory and effective dimensionality

The theoretical analysis in **zhuo2025polycom** examines the expressivity of PolyReLU. A first result is that a ReLU network can be represented exactly by a PolyReLU network without increasing the asymptotic network size. Let $f : [-1,1]^d \to [-1,1]$ be a ReLU network with depth L and width K. The paper establishes the existence of a PolyReLU network g such that

$$f(x) = g(x) \quad \text{for all } x \in [-1,1]^d, \qquad \mathrm{Size}(g) = O(LK). \tag{2.193}$$

The construction follows from the fact that ReLU is a special case of PolyReLU: the corresponding activation in each layer can be replaced without changing the represented function.

The reverse approximation direction is not equivalent in complexity. The paper shows that a ReLU network can approximate a fixed-order PolyReLU activation on a bounded interval with an approximation error below ϵ, using a network whose size is $O(\log^2(1/\epsilon))$ for the scalar activation approximation. At the network level, the authors give an upper bound of the form

$$O\left(LK \log^2\left(\frac{L\alpha}{\epsilon}\right)\right), \tag{2.194}$$

under the theorem's assumptions. Here L is depth, K is width, α is a Lipschitz constant for the PolyReLU activation, and $\epsilon \in (0,1)$ is the target uniform approximation tolerance. A Lipschitz constant bounds how much a function can change when its input changes:

$$|\rho(x) - \rho(y)| \le \alpha|x - y|. \tag{2.195}$$

The authors also establish a lower-bound result of order $\Omega(LK\log(1/\epsilon))$ for the corresponding approximation problem under their stated assumptions. These results concern function representation and approximation complexity; they do not by themselves imply that every PolyReLU language model will outperform every ReLU language model on a particular training distribution.

The paper further studies approximation of smooth functions in Sobolev spaces. Let d be the input dimension and let s denote the smoothness order. A multi-index $\gamma = (\gamma_1, \ldots, \gamma_d)$ has order $|\gamma|_1 = \sum_{j=1}^d \gamma_j$, and $D^\gamma f$ denotes the corresponding weak derivative. The Sobolev space $W^{s,\infty}([-1,1]^d)$ contains functions whose weak derivatives through order s are essentially bounded. A corresponding norm can be written as

$$\|f\|_{W^{s,\infty}} = \max_{|\gamma|_1 \le s} \operatorname{ess\,sup}_{x \in [-1,1]^d} |D^\gamma f(x)|. \tag{2.196}$$

The essential supremum is the smallest bound that holds almost everywhere, allowing exceptions on sets of measure zero. Consider the unit ball

$$\mathcal{F}_{s,d} = \left\{ f \in W^{s,\infty}([-1,1]^d) : \|f\|_{W^{s,\infty}} \leq 1 \right\}. \tag{2.197}$$

The paper establishes that, for every $f \in \mathcal{F}_{s,d}$ and every $\epsilon \in (0,1)$, there exists a PolyReLU network g such that

$$\sup_{x \in [-1,1]^d} |f(x) - g(x)| < \epsilon, \qquad \text{Size}(g) = O(\epsilon^{-d/s}). \tag{2.198}$$

The paper compares this rate with the following reported rates for other activation families:

$$N_{\text{ReLU}}(\epsilon) = O\left(\epsilon^{-d/s} \log \frac{1}{\epsilon}\right), \tag{2.199}$$

$$N_{\text{rational}}(\epsilon) = O\left(\epsilon^{-d/s} \log\log \frac{1}{\epsilon}\right). \tag{2.200}$$

Since a corresponding lower bound of order $\Omega(\epsilon^{-d/s})$ applies to the function class under consideration, the PolyReLU rate is optimal in this Sobolev approximation setting. The conclusion is specific to the function class, domain, and assumptions of the theorem; it should not be interpreted as a general theorem about language-model quality or semantic performance (**zhuo2025polycom**).

The authors also examine the effective rank of feed-forward weight matrices. Ordinary matrix rank counts the number of nonzero singular values, but it does not describe how evenly the matrix magnitude is distributed across singular directions. Let

$$A = U\Sigma V^\top \tag{2.201}$$

be a singular value decomposition, where the diagonal entries of Σ are singular values $\sigma_1 \geq \cdots \geq \sigma_n \geq 0$. Define

$$p_i = \frac{\sigma_i}{\sum_{j=1}^n \sigma_j}, \qquad i = 1, \ldots, n, \tag{2.202}$$

for a nonzero matrix. The values p_i form a probability distribution over singular directions. The effective rank is

$$\text{erank}(A) = \exp\left(-\sum_{i=1}^n p_i \ln p_i\right), \tag{2.203}$$

where the convention $0 \ln 0 = 0$ is used. The exponent is the Shannon entropy of the normalized singular-value distribution. A more uniform distribution produces a larger effective rank, whereas concentration in a few singular directions produces a smaller value. **zhuo2025polycom** report higher effective ranks for PolyReLU and PolyNorm FFN weight matrices than for ReLU, GELU, and SwiGLU in their experiments. This is a structural observation about the learned matrices, not a direct measure of semantic understanding.

2.4.8 Empirical performance and computational cost

zhuo2025polycom evaluate PolyCom in a dense model and a Mixture-of-Experts (MoE) model. Their dense model has approximately 1.3 billion parameters, 24 Transformer layers, hidden size 2,048, and 16 attention heads. It is trained from scratch on RedPajama-1T. The reported comparison is made after processing 250 billion training tokens. The MoE model uses the OLMoE framework and has approximately 6.9 billion total parameters, of which about 1.3 billion are active. It contains 16 layers, hidden size 2,048, 16 attention heads, and 64 experts. The MoE comparison is trained on 200 billion tokens. The dense experiments use 32 NVIDIA A100-80G GPUs, and the MoE experiments use 64 such GPUs.

The dense-model results are shown in table 2.5. PolyReLU and PolyNorm achieve lower training loss and validation perplexity than the compared ReLU, GELU, and SwiGLU configurations. PolyNorm has the highest average downstream accuracy in the table. Its average is 1.21 percentage points above SwiGLU in the six reported downstream tasks. These are results under the paper's parameter-matched configurations and training setup, not a universal ranking independent of model size or data.

Table 2.5: Results for the approximately 1.3-billion-parameter dense model after 250 billion training tokens. PPL denotes validation perplexity; downstream accuracy is averaged across six tasks. Source: **zhuo2025polycom**, Table 1.

Activation	Training loss	Validation PPL	Average accuracy
SwiGLU	2.19	3.22	57.47%
GELU	2.20	3.24	56.98%
ReLU	2.21	3.26	57.18%
PolyReLU	2.17	3.18	58.22%
PolyNorm	2.17	3.17	58.68%

For the MoE comparison, the paper evaluates PolyNorm against SwiGLU. PolyNorm obtains an average validation loss of 2.39, compared with 2.41 for SwiGLU across the 11 reported validation datasets. Its average downstream accuracy is 63.37%, compared with 62.78% for SwiGLU across the reported downstream tasks. PolyNorm outperforms SwiGLU on eight of the ten downstream tasks reported in the table. Because the MoE experiment compares only these two activation choices, it does not establish a ranking of all PolyCom variants against all possible gated activations (**zhuo2025polycom**).

The polynomial order controls the number of terms in PolyCom. The ablation study compares second-, third-, and fourth-order PolyReLU. Increasing the order improves convergence speed in the reported experiment, but the difference between orders three and four is not substantial. Higher orders also increase computation and can create overflow problems, particularly with low-precision arithmetic. The authors therefore use third-order PolyCom as a practical compromise. Their composition ablation finds that PolyNorm gives the lowest training loss among the tested PolyCom variants, while combining ReLU with normalization does not improve on PolyNorm. This indicates that additional compositional complexity is not automatically beneficial.

Activation cost must be compared with that of the feed-forward matrix operations. For an input tensor $X \in \mathbb{R}^{B\times S\times H}$, **zhuo2025polycom** use $24BSH^2$ as the matrix-computation baseline in their activation-cost analysis and define the ratio

$$R_{\text{FLOPs}} = \frac{\text{FLOPs}_{\text{activation}}}{24BSH^2}. \tag{2.204}$$

This denominator is the source-specific baseline used in their appendix. It should not be confused with a separate derivation that counts only two bias-free projection matrix multiplications.

For their illustrative configuration $H = 1024$, $B = 4$, $S = 4096$, BF16 precision, and third-order PolyCom, the paper reports the values in table 2.6. The ratios are small relative to the stated matrix-computation baseline, although the activation-related memory overhead differs across functions. The values depend on the tensor dimensions, input-scale assumption, precision, and implementation.

Table 2.6: Activation FLOPs and memory overhead without gradient checkpointing for the configuration reported by **zhuo2025polycom**, Table 8.

Activation	Activation FLOPs	FLOPs ratio	Memory overhead
ReLU	$4BSH$	0.016%	128 MB
GELU	$72BSH$	0.29%	320 MB
SwiGLU	$\frac{112}{3}BSH$	0.15%	256 MB
ReLU^2	$8BSH$	0.032%	256 MB
PolyNorm	$72BSH$	0.29%	384 MB
PolyReLU	$40BSH$	0.16%	256 MB

The same appendix reports a separate analysis with gradient checkpointing. In that setting, the additional activation-memory overhead is reduced to zero in the table, while activation FLOPs increase because intermediate values are recomputed during the backward pass. Checkpointing therefore exchanges computation for memory. The paper's comparison also adjusts the SwiGLU intermediate dimension to $8H/3$ for a parameter-matched comparison with the $4H$ intermediate dimension used by the other activations (**zhuo2025polycom**).

2.4.9 Limitations and connection to normalization

The activation functions considered in this section differ in the architectural components they modify. ReLU, GELU, and SiLU are scalar nonlinear transformations applied to intermediate features. GEGLU and SwiGLU use a gating branch and therefore change the projection structure as well as the nonlinear operation. Rational activation functions make the activation coefficients trainable, allowing the shape of the transformation to adapt during training. PolyCom combines polynomial powers with another function, while PolyNorm additionally normalizes each powered feature vector.

These approaches involve trade-offs rather than a single universally optimal choice. ReLU is computationally simple but has zero derivative on its negative branch. GELU and SiLU provide smooth alternatives, while gated units add parallel projections and multiplicative operations. RAFs introduce a small number of additional parameters in the reported implementation but increase training computation. PolyCom provides stronger

approximation results under the assumptions studied by its authors and improves the reported dense- and MoE-model results, but polynomial order must be controlled to avoid unnecessary computation and numerical overflow.

The evidence also has scope limitations. The RAF study uses a BERT-like encoder-only model and explicitly notes its limited pre-training resources; its results should not be transferred without qualification to every decoder-only or multimodal architecture. The PolyCom study covers substantial dense and MoE training configurations, but its experiments still represent a finite set of model sizes, datasets, optimization settings, and evaluation tasks. Neither study establishes that one activation function dominates in every architecture and training regime.

The normalization inside PolyNorm should be distinguished from the normalization sublayers used in Transformer blocks. PolyNorm normalizes powered feature vectors as part of the activation construction. Layer normalization and related operations have a broader architectural role in controlling the statistics and scale of hidden representations. The activation function and the normalization sublayer therefore remain conceptually distinct, even though both can influence numerical stability and optimization.

The central result of this section is that the activation function determines a substantial part of the nonlinear computation performed by the feed-forward network. Its definition affects the representable mapping, its derivative affects gradient propagation, and its implementation affects training cost and numerical stability. Gated, rational, and polynomial-composition activations extend the design space beyond a fixed scalar nonlinearity, but their benefits must be evaluated under explicit architectural and computational constraints. Having established how intermediate features are transformed, the next step is to examine normalization mechanisms, which regulate the scale and statistical properties of representations as they pass through the Transformer stack.

2.5 Normalization

The preceding discussion established how feed-forward transformations and activation functions modify the feature representation of each token. These operations are repeatedly composed with attention sublayers throughout a Transformer. Their repeated application raises a further architectural question: how should the numerical scale of the intermediate representations be controlled as information passes through a deep sequence of transformations? Normalization addresses this question by transforming activations according to statistical quantities calculated from selected dimensions of the representation. Its position within Transformer blocks, its mathematical properties, and its alternatives are therefore necessary parts of the architecture before the attention mechanism is considered in full.

2.5.1 Architectural motivation and statistical foundations

A neural network transforms its input through a sequence of parameterized operations. In a Transformer, the representations passed between these operations are vectors associated with individual tokens. Each vector contains features that encode information accumulated by the model at a particular stage of computation. The numerical values of these features are called activations. Their magnitudes and distributions affect the

inputs received by subsequent transformations and the derivatives through which the parameters are optimized.

Consider a batch of token sequences represented by the activation tensor

$$\mathbf{X} \in \mathbb{R}^{B\times T\times C}, \tag{2.205}$$

where B denotes the batch size, T the number of token positions in each sequence, and C the hidden or embedding dimension. The scalar activation x_{ijk} is the value for sample i, token position j, and feature channel k. The representation of one token is therefore

$$\mathbf{x}_{ij} = \begin{bmatrix} x_{ij1} & x_{ij2} & \cdots & x_{ijC} \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^{C}. \tag{2.206}$$

The normalization methods most relevant to Transformers calculate their statistics over the feature dimension of this token vector. The batch and sequence dimensions are retained, but they do not determine the statistics used to normalize a particular token.

The statistical quantities used in normalization can be introduced for a generic vector $\mathbf{x} = [x_1, \ldots, x_C]^{\mathsf{T}} \in \mathbb{R}^C$. Its arithmetic mean is

$$\mu = \frac{1}{C}\sum_{k=1}^{C} x_k. \tag{2.207}$$

The mean measures the central location of the feature values. Subtracting this value from every component gives the centered vector

$$\widetilde{x}_k = x_k - \mu. \tag{2.208}$$

The population variance across the C features is

$$\sigma^2 = \frac{1}{C}\sum_{k=1}^{C}(x_k - \mu)^2, \tag{2.209}$$

and the standard deviation is its nonnegative square root,

$$\sigma = \sqrt{\sigma^2}. \tag{2.210}$$

The factor $1/C$, rather than $1/(C-1)$, is used because the features constitute the complete set over which the normalization statistics are calculated; the expression is not an estimate of a population variance from a separate statistical sample.

Subtracting the mean and dividing by the standard deviation produces the standardized value

$$\widehat{x}_k = \frac{x_k - \mu}{\sigma}. \tag{2.211}$$

When $\sigma > 0$, this transformation has zero mean and unit variance over the feature dimension. In numerical implementations, the denominator is modified by a small positive constant ε:

$$\widehat{x}_k = \frac{x_k - \mu}{\sqrt{\sigma^2 + \varepsilon}}. \tag{2.212}$$

The constant $\varepsilon > 0$ prevents division by zero when all feature values are identical and reduces numerical sensitivity when the variance is very small. With nonzero ε, the resulting variance is not exactly one; it is $\sigma^2/(\sigma^2+\varepsilon)$, which approaches one when ε is negligible relative to σ^2.

Most normalization operations can be expressed using the common affine formulation

$$\mathrm{Norm}(\mathbf{x}) = \boldsymbol{\gamma} \odot \frac{\mathbf{x} - \boldsymbol{\mu}}{\sqrt{\boldsymbol{\sigma}^2 + \varepsilon}} + \boldsymbol{\beta}, \tag{2.213}$$

where the mean and variance are broadcast to the shape of the input, $\boldsymbol{\gamma} \in \mathbb{R}^C$ and $\boldsymbol{\beta} \in \mathbb{R}^C$ are learnable feature-wise scale and shift parameters, and $\odot$ denotes element-wise multiplication. The scale and shift parameters allow the network to learn the output range required by subsequent layers. Thus, normalization does not permanently constrain the final output to have zero mean and unit variance; those properties describe the standardized intermediate values before the affine transformation. This common formulation and the dependence of different normalization methods on the dimensions used to calculate their statistics are described by **zhu2025dyt**.

The choice of aggregation dimensions distinguishes normalization methods. For example, Batch Normalization calculates channel-wise statistics across the batch and token dimensions. For channel k, its mean and variance are

$$\mu_k^{\mathrm{BN}} = \frac{1}{BT} \sum_{i=1}^{B} \sum_{j=1}^{T} x_{ijk}, \tag{2.214}$$

$$(\sigma_k^{\mathrm{BN}})^2 = \frac{1}{BT} \sum_{i=1}^{B} \sum_{j=1}^{T} (x_{ijk} - \mu_k^{\mathrm{BN}})^2. \tag{2.215}$$

Other methods used in convolutional architectures, including Group Normalization and Instance Normalization, use different groupings and aggregation dimensions. They share the broad objective of transforming activations using statistical quantities, but the axes over which those quantities are calculated differ. For Transformer architectures, the two principal statistical methods considered here are Layer Normalization and Root Mean Square Normalization (RMSNorm) (**zhu2025dyt**; **corradini2025slmreview**).

2.5.2 Layer Normalization: formulation and geometry

Layer Normalization (LayerNorm or LN) calculates the mean and variance independently for each token in each sample. For the token vector $\mathbf{x}_{ij}$, its mean is

$$\mu_{ij} = \frac{1}{C} \sum_{k=1}^{C} x_{ijk}, \tag{2.216}$$

and its variance is

$$\sigma_{ij}^2 = \frac{1}{C} \sum_{k=1}^{C} (x_{ijk} - \mu_{ij})^2. \tag{2.217}$$

The normalized output for feature k, including the learnable affine parameters, is

$$\boxed{y_{ijk}^{\mathrm{LN}} = \gamma_k \frac{x_{ijk} - \mu_{ij}}{\sqrt{\sigma_{ij}^2 + \varepsilon}} + \beta_k.} \tag{2.218}$$

Here, γ_k and β_k are shared across samples and token positions, but each feature channel has its own scale and shift parameters. The statistics, in contrast, are calculated separately for every pair (i, j). The normalized representation of one token therefore does not depend on the statistics of another token or another sample in the batch.

The distinction between the statistical transformation and the affine parameters is essential. Before the affine transformation, the feature values have zero mean and approximately unit variance. After multiplication by γ_k and addition of β_k, these properties need not hold. The learnable parameters restore flexibility to the representation and allow subsequent attention or feed-forward operations to receive feature values with scales and offsets selected through training.

Geometric interpretation. The centering operation can be expressed as a projection in $\mathbb{R}^C$. Let $\mathbf{1} \in \mathbb{R}^C$ be the all-ones vector and let $\mathbf{I}$ be the $C \times C$ identity matrix. Define the centering matrix

$$\mathbf{P} = \mathbf{I} - \frac{1}{C}\mathbf{1}\mathbf{1}^\mathsf{T}. \tag{2.219}$$

The mean of $\mathbf{x}$ is $\mu = C^{-1}\mathbf{1}^\mathsf{T}\mathbf{x}$, so the centered vector is

$$\mathbf{x}_c = \mathbf{P}\mathbf{x}. \tag{2.220}$$

The matrix $\mathbf{P}$ is symmetric and idempotent:

$$\mathbf{P}^\mathsf{T} = \mathbf{P}, \qquad \mathbf{P}^2 = \mathbf{P}. \tag{2.221}$$

It projects the vector onto the subspace orthogonal to $\mathbf{1}$, whose dimension is $C - 1$ for $C > 1$. The variance can consequently be written as

$$\sigma^2 = \frac{1}{C}\mathbf{x}_c^\mathsf{T}\mathbf{x}_c = \frac{1}{C}\|\mathbf{P}\mathbf{x}\|_2^2. \tag{2.222}$$

Ignoring ε, the standardized vector is

$$\widehat{\mathbf{x}} = \frac{\sqrt{C}\,\mathbf{P}\mathbf{x}}{\|\mathbf{P}\mathbf{x}\|_2}, \tag{2.223}$$

provided that $\mathbf{P}\mathbf{x} \neq \mathbf{0}$. It therefore lies on a sphere of radius $\sqrt{C}$ inside the zero-mean subspace. LayerNorm first removes the component parallel to the all-ones vector and then fixes the magnitude of the remaining centered vector.

This representation also reveals two invariance properties of the statistical part of LayerNorm. For $a > 0$ and scalar b, and when ε is neglected,

$$\mathrm{LN}_{\mathrm{stat}}(a\mathbf{x} + b\mathbf{1}) = \mathrm{LN}_{\mathrm{stat}}(\mathbf{x}). \tag{2.224}$$

Adding the same scalar to all features is removed by centering. Multiplying all features by a positive scalar changes the centered vector and its standard deviation by the same factor, so their ratio is unchanged. The learnable affine transformation is not included in $\mathrm{LN}_{\mathrm{stat}}$ in equation (2.224).

Differential behavior. The Jacobian of a transformation describes how small changes in its input affect its output. It is therefore central to understanding how gradients are propagated through normalization during backpropagation. Define

$$s = \sqrt{\frac{1}{C}\mathbf{x}_c^\mathsf{T}\mathbf{x}_c + \varepsilon}, \qquad \widehat{\mathbf{x}} = \frac{\mathbf{x}_c}{s}. \tag{2.225}$$

Since $\mathbf{x}_c = \mathbf{P}\mathbf{x}$, differentiating the normalized vector with respect to the input gives

$$\boxed{\frac{\partial \widehat{\mathbf{x}}}{\partial \mathbf{x}} = \frac{\mathbf{P}}{s} - \frac{\mathbf{x}_c \mathbf{x}_c^\mathsf{T}}{C s^3}.} \tag{2.226}$$

The first term represents centering and scaling of input perturbations. The second term accounts for the fact that the denominator also depends on the input vector. A change in one feature can therefore alter the normalized values of the other features through the shared mean and variance.

Let $\mathbf{\Gamma} = \mathrm{diag}(\gamma_1, \ldots, \gamma_C)$. The Jacobian of the complete affine LayerNorm operation is

$$\frac{\partial \mathbf{y}^{\mathrm{LN}}}{\partial \mathbf{x}} = \mathbf{\Gamma}\left(\frac{\mathbf{P}}{s} - \frac{\mathbf{x}_c \mathbf{x}_c^\mathsf{T}}{C s^3}\right). \tag{2.227}$$

The shift vector $\boldsymbol{\beta}$ does not appear because its derivative with respect to the input is zero. The feature-wise scale parameters modify the gradient magnitude of the corresponding output features. These expressions are direct mathematical consequences of the LayerNorm formulation in equation (2.218); they are not additional empirical results reported by the source paper.

2.5.3 RMSNorm and normalization within residual Transformer blocks

Root Mean Square Normalization (RMSNorm) simplifies LayerNorm by removing the mean-centering step. Instead of calculating squared deviations from the mean, it calculates the mean of the squared feature values. For a token representation $\mathbf{x}_{ij}$, the mean square is

$$m_{ij}^{(2)} = \frac{1}{C}\sum_{k=1}^{C} x_{ijk}^2, \tag{2.228}$$

and the root mean square is

$$r_{ij} = \sqrt{\frac{1}{C}\sum_{k=1}^{C} x_{ijk}^2}. \tag{2.229}$$

The method can be expressed within the common affine formulation by setting the mean to zero and using the mean square as the variance-like quantity:

$$\boxed{y_{ijk}^{\mathrm{RMS}} = \gamma_k \frac{x_{ijk}}{\sqrt{\frac{1}{C}\sum_{\ell=1}^{C} x_{ij\ell}^2 + \varepsilon}} + \beta_k.} \tag{2.230}$$

The defining distinction is that RMSNorm does not subtract the token's feature mean. The common affine expression in equation (2.213) is the parameterization used for

the normalization formulation in **zhu2025dyt**; implementations may use a scale-only affine parameterization. The statistical distinction between LayerNorm and RMSNorm is the use of the root mean square without mean-centering, as also emphasized by **corradini2025slmreview**.

The relation between variance and mean square follows by expanding the definition of variance:

$$\begin{aligned}\frac{1}{C}\sum_{k=1}^{C} x_k^2 &= \frac{1}{C}\sum_{k=1}^{C}\big[(x_k-\mu)+\mu\big]^2 \\ &= \frac{1}{C}\sum_{k=1}^{C}(x_k-\mu)^2+\mu^2 \\ &= \sigma^2+\mu^2. \end{aligned} \tag{2.231}$$

Thus, mean square and variance are equal only when the feature mean is zero. RMSNorm controls the root mean square magnitude of the vector, but it does not explicitly force the mean of its features to zero.

Geometric interpretation and Jacobian. Ignoring ε, the root mean square is $r = \|\mathbf{x}\|_2/\sqrt{C}$, and the normalized vector is

$$\widehat{\mathbf{x}}^{\mathrm{RMS}} = \frac{\sqrt{C}}{\|\mathbf{x}\|_2}\mathbf{x}. \tag{2.232}$$

Its Euclidean norm is $\sqrt{C}$, but it is not restricted to the zero-mean subspace. RMSNorm rescales the original vector without first projecting out its component in the direction of $\mathbf{1}$. It is invariant to positive scalar multiplication when ε is neglected, but it is not invariant to adding a common scalar to all features.

For

$$r = \sqrt{\frac{1}{C}\mathbf{x}^{\mathsf{T}}\mathbf{x}+\varepsilon}, \qquad \widehat{\mathbf{x}}^{\mathrm{RMS}} = \frac{\mathbf{x}}{r}, \tag{2.233}$$

the Jacobian is

$$\boxed{\frac{\partial \widehat{\mathbf{x}}^{\mathrm{RMS}}}{\partial \mathbf{x}} = \frac{\mathbf{I}}{r} - \frac{\mathbf{x}\mathbf{x}^{\mathsf{T}}}{Cr^3}.} \tag{2.234}$$

As in LayerNorm, the denominator couples the feature values of a token: changing one feature changes the root mean square used to normalize all features. The difference is that RMSNorm does not contain the centering projection $\mathbf{P}$.

Residual connections. Normalization is used together with residual connections in Transformer blocks. Let $\mathbf{H}$ be the input to a sublayer and let F denote the transformation performed by that sublayer, such as an attention or feed-forward operation. A residual addition forms

$$\mathbf{Z} = \mathbf{H} + F(\mathbf{H}). \tag{2.235}$$

The derivative of this addition with respect to its input is

$$\frac{\partial \mathbf{Z}}{\partial \mathbf{H}} = \mathbf{I} + \mathbf{J}_F, \tag{2.236}$$

where $\mathbf{J}_F$ is the Jacobian of F. The identity term provides a direct contribution to the gradient independently of the derivative of the sublayer transformation.

In the post-residual arrangement described in the systematic review, normalization is applied after the sublayer output has been added to its input:

$$\mathbf{H}_{\text{next}} = \mathrm{LN}\big(\mathbf{H} + F(\mathbf{H})\big). \tag{2.237}$$

An RMSNorm operation can occupy the corresponding normalization position in architectures that use RMSNorm. Residual addition and normalization therefore have complementary roles in this described block arrangement: the residual path provides a direct route for information and gradients, while normalization controls the numerical scale of the representation passed to subsequent computation. The equation specifies the arrangement discussed by the review and should not be read as a claim that every Transformer implementation uses an identical normalization placement.

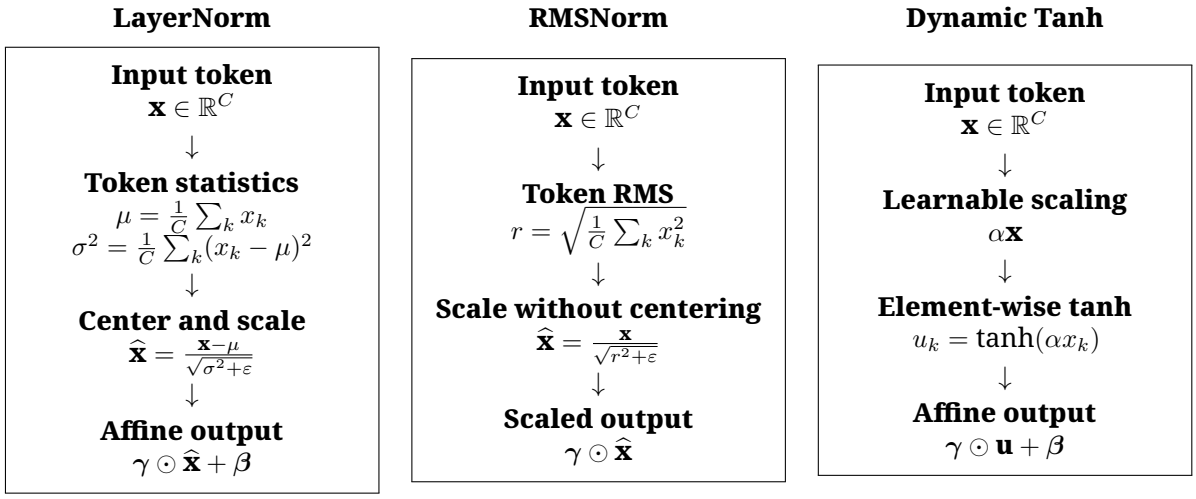


Figure 2.10: Schematic comparison of the operations performed by LayerNorm, RMSNorm, and Dynamic Tanh. LayerNorm calculates token-wise mean and variance and centers the feature vector; RMSNorm calculates the token-wise root mean square without centering; DyT applies a learnable element-wise scaling followed by a hyperbolic tangent. The comparison follows the formulations in **zhu2025dyt** and **corradini2025slmreview**.

2.5.4 Observed input-output behavior of trained normalization layers

The statistical definitions explain how LayerNorm and RMSNorm are computed, but they do not by themselves describe the aggregate input-output behavior observed across a trained Transformer. To investigate this behavior, **zhu2025dyt** examine normalization layers in three trained architectures: a Vision Transformer (ViT-B) trained on ImageNet-1K, a wav2vec 2.0 Large Transformer trained on LibriSpeech, and a Diffusion Transformer (DiT-XL) trained on ImageNet-1K. In the examined models, LayerNorm is applied within the Transformer blocks and before the final linear projection.

The authors perform forward passes on mini-batches and record the tensor elements immediately before and after selected LayerNorm operations, before the learnable affine

transformation. Because LayerNorm preserves the tensor dimensions, each input activation can be paired with its corresponding normalized output. Plotting these pairs reveals that the relationships in earlier normalization layers are predominantly linear, whereas deeper layers often exhibit full or partial S-shaped curves.

An important distinction is required when interpreting these plots. For one token, the mean and standard deviation are fixed across its feature channels during the normalization operation. Its input-output relation is therefore

$$y_k = \frac{x_k - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \tag{2.238}$$

which is affine in x_k for that token. Different tokens, however, have different means and standard deviations. Their token-specific affine relations consequently have different slopes and intercepts. When activations from many tokens are plotted together, the collection of these different lines can form an aggregate nonlinear pattern.

The token-grouped visualization reported by **zhu2025dyt** shows approximately straight relationships within individual tokens, with different slopes associated with different token variances. The channel- grouped visualization shows that channels can span substantially different input ranges. A small number of channels contain extreme values, and these values are strongly reduced by the normalization operation. The aggregate input-output relationship therefore resembles a scaled hyperbolic tangent, even though the operation applied to each individual token is based on centering and division by its standard deviation.

The authors report that approximately 99% of the activation points in their analysis fall within the central, approximately linear part of the observed S-shaped relationship. Nevertheless, some activations are extreme; in the examined ViT, examples include values greater than 50 or smaller than -50. The normalization operation reduces the relative magnitude of these outlying values and brings them closer to the range occupied by the majority of activations. This disproportionate treatment of extreme values is not reproduced by a single affine transformation with a constant slope. The observation motivates the hypothesis that nonlinear suppression of extreme activations is an important functional effect of normalization in the examined trained networks (**zhu2025dyt**).

The hyperbolic tangent function provides a simple mathematical model of the observed S-shaped relation. It is defined by

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \tag{2.239}$$

and maps real inputs into the bounded interval $(-1, 1)$. Its limiting behavior is

$$\lim_{x \to +\infty} \tanh(x) = 1, \qquad \lim_{x \to -\infty} \tanh(x) = -1, \qquad \tanh(0) = 0. \tag{2.240}$$

Near the origin, its Taylor expansion is

$$\tanh(x) = x - \frac{x^3}{3} + \frac{2x^5}{15} - \frac{17x^7}{315} + \cdots, \tag{2.241}$$

so the function is approximately linear when $|x|$ is small. Its derivative is

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) = \operatorname{sech}^2(x). \tag{2.242}$$

The derivative is largest at the origin and approaches zero as the magnitude of the input increases. Thus, the central region transmits small changes approximately linearly, while the saturated regions reduce the sensitivity of the output to further increases in the input magnitude.

A scaled hyperbolic tangent,

$$f_\alpha(x) = \tanh(\alpha x), \tag{2.243}$$

changes the width of the transition region. For $\alpha > 0$, increasing α causes saturation to occur over a narrower input range, whereas decreasing α expands the approximately linear region. This scaling property is the mathematical basis for the learnable parameter introduced by Dynamic Tanh.

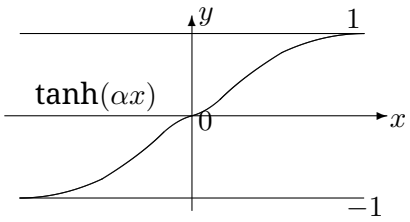


Figure 2.11: Schematic S-shaped input-output relation of the hyperbolic tangent. The central region is approximately linear, while the output approaches bounded positive and negative limits for large input magnitudes. The role of the scale parameter is discussed by **zhu2025dyt**.

2.5.5 Dynamic Tanh: formulation, derivatives, and implementation

The S-shaped aggregate input-output relationships observed in trained normalization layers motivate an alternative to statistical normalization. Rather than calculating the mean and variance of each token representation, Dynamic Tanh (DyT) applies a learnable element-wise nonlinear transformation. For an input tensor $\mathbf{X}$, the operation introduced by **zhu2025dyt** is

$$\boxed{\mathrm{DyT}(\mathbf{X}) = \boldsymbol{\gamma} \odot \tanh(\alpha \mathbf{X}) + \boldsymbol{\beta}.} \tag{2.244}$$

The scalar α is learnable and shared across the elements of a DyT layer. The vectors $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^C$ are learnable channel-wise scale and shift parameters. The hyperbolic tangent is applied element-wise, and the channel-wise parameters are broadcast across the batch and token dimensions. For one activation, the operation is

$$y_{ijk} = \gamma_k \tanh(\alpha x_{ijk}) + \beta_k. \tag{2.245}$$

The scalar α controls the scale of the input before the nonlinear function. The parameter γ_k controls the output scale of channel k, and β_k controls its output displacement. If $\alpha > 0$, the output of channel k approaches $\beta_k + \gamma_k$ for large positive inputs and $\beta_k - \gamma_k$ for large negative inputs. More generally, its bounded range is determined by β_k and $|\gamma_k|$. Unlike LayerNorm and RMSNorm, DyT does not calculate a mean, variance, or root mean square from the current input tensor.

Local behavior and saturation. For one feature, define

$$y_k = \gamma_k \tanh(\alpha x_k) + \beta_k. \tag{2.246}$$

Its derivative with respect to the input is

$$\boxed{\frac{\partial y_k}{\partial x_k} = \gamma_k \alpha \left[1 - \tanh^2(\alpha x_k)\right].} \tag{2.247}$$

At the origin, the derivative is $\gamma_k \alpha$. For small $|\alpha x_k|$, the Taylor expansion gives

$$y_k \approx \gamma_k \alpha x_k + \beta_k. \tag{2.248}$$

For large $|\alpha x_k|$, the derivative approaches zero and the output saturates. This supplies an element-wise mechanism for reducing the sensitivity of the output to extreme input values.

The derivative with respect to the shared scale parameter is

$$\frac{\partial y_k}{\partial \alpha} = \gamma_k x_k \left[1 - \tanh^2(\alpha x_k)\right]. \tag{2.249}$$

For a scalar loss $\mathcal{L}$, the parameter gradients accumulate contributions from the batch and sequence dimensions:

$$\frac{\partial \mathcal{L}}{\partial \beta_k} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial y_{ijk}}, \tag{2.250}$$

$$\frac{\partial \mathcal{L}}{\partial \gamma_k} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial y_{ijk}} \tanh(\alpha x_{ijk}), \tag{2.251}$$

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \sum_{i,j,k} \frac{\partial \mathcal{L}}{\partial y_{ijk}} \gamma_k x_{ijk} \left[1 - \tanh^2(\alpha x_{ijk})\right]. \tag{2.252}$$

These expressions show how the scalar parameter is learned from the activation elements throughout the layer. Contributions from highly saturated activations are reduced by the factor $1 - \tanh^2(\alpha x_{ijk})$.

For a token vector, define $\mathbf{\Gamma} = \mathrm{diag}(\gamma_1, \ldots, \gamma_C)$. The Jacobian of DyT is diagonal:

$$\boxed{\mathbf{J}_{\mathrm{DyT}} = \mathbf{\Gamma}\, \mathrm{diag}\left(\alpha \left[1 - \tanh^2(\alpha \mathbf{x})\right]\right).} \tag{2.253}$$

The expression inside the second diagonal matrix is understood element-wise. The diagonal structure follows from the fact that each output activation depends directly only on its corresponding input activation. This differs from LayerNorm and RMSNorm, whose shared token-level denominators couple the feature values in the Jacobian. The derivative comparison is a mathematical consequence of the respective forward equations.

Implementation and initialization. For an input tensor of shape $[B, T, C]$, the implementation in **zhu2025dyt** initializes the scalar parameter to a chosen value α_0, initializes the channel-wise scale vector to ones, and initializes the channel-wise shift vector to zeros:

$$\alpha = \alpha_0, \qquad \boldsymbol{\gamma} = \mathbf{1}_C, \qquad \boldsymbol{\beta} = \mathbf{0}_C. \tag{2.254}$$

The forward pass first computes

$$\mathbf{U} = \tanh(\alpha \mathbf{X}), \tag{2.255}$$

and then applies

$$\mathbf{Y} = \boldsymbol{\gamma} \odot \mathbf{U} + \boldsymbol{\beta}. \tag{2.256}$$

The operation replaces one normalization layer with one DyT layer. It can be inserted at normalization locations in attention blocks, feed-forward blocks, and before the final linear projection. The original activation functions in the architecture, such as GELU or ReLU, are not replaced by DyT; the method replaces the normalization operation while leaving those activation functions unchanged (**zhu2025dyt**).

The learnable scalar α is not calculated from the current token's statistics. The analysis in **zhu2025dyt** nevertheless finds that its value tracks the inverse standard deviation of the input activations during training and is correlated with that quantity in trained models. This indicates that α can partially learn an activation-scale adjustment. However, it is shared across the input activations of a layer, rather than being calculated separately for every token. The scalar alone therefore cannot reproduce token-specific statistical normalization or the nonlinear suppression of extreme values; the latter is supplied by the hyperbolic tangent function.

2.5.6 Computational analysis and a worked numerical example

The three methods differ in the operations required for each token representation. LayerNorm calculates the feature mean, subtracts it, calculates the mean squared centered deviation, adds the stability constant, takes a square root, and scales the centered features. RMSNorm calculates the mean of squared features, adds the stability constant, takes a square root, and scales the original features. DyT performs a scalar multiplication, an element-wise hyperbolic tangent, and a channel-wise affine transformation.

For a tensor with BT token representations and C features per token, each method requires work proportional to the number of activation elements. Their leading arithmetic complexity is therefore

$$\mathcal{O}(BTC). \tag{2.257}$$

This asymptotic expression does not imply equal execution time. LayerNorm and RMSNorm require reductions over the feature dimension, whereas DyT uses element-wise operations and a nonlinear function. Actual latency depends on hardware, memory traffic, kernel fusion, numerical precision, and compiler optimization. In particular, **zhu2025dyt** report that DyT does not provide a speedup over properly compiled and optimized normalization layers, despite its simpler conceptual formulation.

A numerical example makes the transformations explicit. Consider the four-dimensional token vector

$$\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}^{\mathsf{T}}, \qquad C = 4. \tag{2.258}$$

For this example, set $\varepsilon = 0$ to keep the arithmetic transparent, and set the affine scale to one and the affine shift to zero.

The LayerNorm mean is

$$\mu = \frac{1 + 2 + 3 + 4}{4} = 2.5. \tag{2.259}$$

The variance is

$$\sigma^2 = \frac{(1-2.5)^2 + (2-2.5)^2 + (3-2.5)^2 + (4-2.5)^2}{4} = \frac{2.25 + 0.25 + 0.25 + 2.25}{4} = 1.25, \tag{2.260}$$

and the standard deviation is

$$\sigma = \sqrt{1.25} \approx 1.1180. \tag{2.261}$$

The resulting LayerNorm vector is approximately

$$\widehat{\mathbf{x}}^{\mathrm{LN}} = \begin{bmatrix} -1.3416 & -0.4472 & 0.4472 & 1.3416 \end{bmatrix}^{\mathsf{T}}. \tag{2.262}$$

For RMSNorm, the mean square is

$$m^{(2)} = \frac{1^2 + 2^2 + 3^2 + 4^2}{4} = \frac{30}{4} = 7.5, \tag{2.263}$$

so that

$$r = \sqrt{7.5} \approx 2.7386. \tag{2.264}$$

The RMS-normalized vector is consequently

$$\widehat{\mathbf{x}}^{\mathrm{RMS}} \approx \begin{bmatrix} 0.3651 & 0.7303 & 1.0954 & 1.4606 \end{bmatrix}^{\mathsf{T}}. \tag{2.265}$$

Unlike LayerNorm, RMSNorm does not subtract the mean, so the signs and relative direction of the original vector are retained.

For DyT, choose $\alpha = 0.5$, $\boldsymbol{\gamma} = \mathbf{1}$, and $\boldsymbol{\beta} = \mathbf{0}$. Its output is

$$\mathrm{DyT}(\mathbf{x}) = \tanh(0.5\mathbf{x}) \approx \begin{bmatrix} 0.4621 & 0.7616 & 0.9051 & 0.9640 \end{bmatrix}^{\mathsf{T}}. \tag{2.266}$$

The fourth component is close to the positive saturation limit, whereas the first component remains in the more nearly linear region. This example illustrates the different mathematical operations; it is not an experimental result reported by either source paper.

2.5.7 Language-model evaluation and initialization

The replacement of a normalization layer is useful only if it preserves the behavior of the complete model during training and evaluation. **zhu2025dyt** investigate this question by replacing RMSNorm with DyT in LLaMA models of different sizes. The reported configurations are LLaMA 7B, 13B, 34B, and 70B. The models are pretrained on The Pile using 200 billion tokens and follow the LLaMA training recipe. The DyT experiments also introduce a learnable scalar parameter after the initial embedding layer and tune the initial value of α.

The authors report the final training loss and average performance over 15 zero-shot tasks from the lm-eval framework. The results are summarized in table 2.7. The zero-shot scores are identical at the reported precision for the RMSNorm and DyT versions of all four models. The final losses differ by 0.01 for the 7B and 13B models and are identical at the reported precision for the 34B and 70B models. The training curves are also closely aligned across the four model sizes (**zhu2025dyt**).

Table 2.7: LLaMA pretraining loss and average performance on 15 zero-shot lm-eval tasks when using RMSNorm or DyT. Values are reproduced from Table 4 of **zhu2025dyt**.

Model	RMSNorm score	DyT score	RMSNorm loss	DyT loss
LLaMA 7B	0.513	0.513	1.59	1.60
LLaMA 13B	0.529	0.529	1.53	1.54
LLaMA 34B	0.536	0.536	1.50	1.50
LLaMA 70B	0.549	0.549	1.45	1.45

The role of the initial value of α. The authors find that an initial value of $\alpha_0 = 0.5$ is generally sufficient for the non-language-model tasks examined in their study. The situation differs for LLM training: tuning α_0 can substantially improve the training loss. Their LLaMA experiments indicate that larger models require smaller initial values and that DyT layers in attention blocks benefit from larger initial values than those in feed-forward blocks or before the final output projection.

The model-specific values reported in Table 10 of **zhu2025dyt** are shown in table 2.8. The distinction between the attention and other locations becomes particularly important as model width increases.

Table 2.8: Reported initial values of the DyT scaling parameter for attention blocks and other DyT locations in LLaMA models. Values are reproduced from Table 10 of **zhu2025dyt**.

Model	Width	Depth	α_0 (attention / other)
LLaMA 7B	4096	32	$0.8/0.20$
LLaMA 13B	5120	40	$0.6/0.15$
LLaMA 34B	8196	48	$0.2/0.05$
LLaMA 70B	8196	80	$0.2/0.05$

To separate the influence of model width and depth, the authors also report optimal initializations across combinations of widths and depths. The values are reproduced in table 2.9. The selected initialization remains constant across the tested depths at a given width, while it decreases as width increases. The authors therefore identify width as the more important factor in these experiments and report comparatively little influence from depth.

Table 2.9: Reported optimal α_0 values (attention / other locations) across model widths and depths. Values are reproduced from Table 11 of **zhu2025dyt**.

Width	Depth 8	Depth 16	Depth 32	Depth 64
1024	$1.0/1.0$	$1.0/1.0$	$1.0/1.0$	$1.0/1.0$
2048	$1.0/0.5$	$1.0/0.5$	$1.0/0.5$	$1.0/0.5$
4096	$0.8/0.2$	$0.8/0.2$	$0.8/0.2$	$0.8/0.2$
8192	$0.2/0.05$	$0.2/0.05$	$0.2/0.05$	$0.2/0.05$

The dependence on α_0 is also related to training stability. In the supervised ViT experiments, larger models are more prone to instability, and reducing either the learning rate or the initial α_0 improves stability. The authors report that a default $\alpha_0 = 0.5$ gives stability comparable to LayerNorm for the non-LLM settings they examine. For LLMs, however, their results motivate separate tuning of the initial values at different architectural locations rather than relying on one universal value (**zhu2025dyt**).

Ablation of the squashing function and the learnable scale. The role of the hyperbolic tangent is tested by replacing it with hardtanh, sigmoid, or the identity function. In the ViT-S and ViT-B experiments, using the identity function leads to training divergence, whereas the squashing functions permit stable training. Among the tested squashing functions, tanh achieves the highest reported accuracy. The results are reproduced in table 2.10.

Table 2.10: ImageNet-1K classification accuracy with alternative DyT squashing functions. The identity-function experiments diverged; the preceding accuracy is the highest reached before divergence. Values are reproduced from Table 7 of **zhu2025dyt**.

Model	Identity	Tanh	Hardtanh	Sigmoid
ViT-S	$58.5\% \rightarrow$ failed	80.3%	79.9%	79.6%
ViT-B	$61.0\% \rightarrow$ failed	82.5%	82.2%	81.6%

The learnable scale is examined separately by comparing the same squashing functions with and without α. For ViT-B, removing α reduces the reported accuracy for all three nonlinear functions. With tanh, the accuracy decreases from 82.5% to 81.1%; with hardtanh, it decreases from 82.2% to 80.7%; and with sigmoid, it decreases from 81.6% to 80.7%. These results indicate that the squashing function and its learnable input scale make complementary contributions in the tested setting (**zhu2025dyt**).

2.5.8 Normalization choices in Small Language Models

The choice between LayerNorm and RMSNorm is also visible in the architectural trends of Small Language Models. These models are designed to balance language-model capability against constraints on parameter count, memory, inference time, and deployment hardware. In this setting, the computational structure of repeatedly applied Transformer components becomes an important design consideration.

The systematic review by **corradini2025slmreview** analyzes 70 English-language studies published between January 2023 and January 2025, focusing on recent SLM developments and primarily on models with up to seven billion parameters. Its architectural analysis identifies RMSNorm as a frequently adopted component. The review reports RMSNorm in approximately 66% of the analyzed model designs and describes its prevalence as a preference for a computationally lighter alternative to standard LayerNorm.

This prevalence is consistent with the mathematical distinction already established. RMSNorm omits the mean-centering operation and uses the root mean square of the token features to determine the scale. Repeating this simpler statistical calculation across many tokens and layers can be relevant to the computational design of compact language

models. However, the reported 66% is a frequency within the review's selected research corpus; it is not a universal estimate for every existing SLM or language model architecture. Nor does the frequency alone establish a controlled performance advantage over LayerNorm (**corradini2025slmreview**).

The three methods therefore represent different choices about how activation behavior is controlled. LayerNorm uses token-specific centering and variance scaling. RMSNorm retains token-specific statistical scaling but removes mean-centering. DyT does not calculate token statistics and instead uses a learnable element-wise nonlinear transformation. Their common architectural purpose does not make their mathematical operations interchangeable in every setting.

Table 2.11: Conceptual comparison of the three methods based on the formulations discussed by **zhu2025dyt** and **corradini2025slmreview**.

Property	LayerNorm	RMSNorm	Dynamic Tanh
Mean calculation	Yes	No	No
Variance or RMS calculation	Variance	Root mean square	No input statistic
Mean-centering	Yes	No	No
Input scaling	Token-specific standard deviation	Token-specific root mean square	Learnable layer-wise scalar
Nonlinear saturation	Indirect aggregate behavior	No explicit squashing function	Explicit hyperbolic tangent
Feature coupling in the forward Jacobian	Yes	Yes	No; diagonal Jacobian

2.5.9 Limitations and connection to the attention mechanism

The results of **zhu2025dyt** challenge the assumption that statistical normalization layers are indispensable for training modern Transformer architectures. They do not, however, establish that DyT is a universal replacement for every normalization method or architecture. The experiments primarily investigate networks that use LayerNorm or RMSNorm. The authors report that preliminary experiments do not support a direct replacement of Batch Normalization by DyT in conventional ResNet architectures. The applicability of DyT to other normalization methods therefore remains unresolved.

The computational implications also require qualification. Although DyT removes the statistical reductions used by LayerNorm and RMSNorm, the experiments report no speedup over optimized normalization layers. Actual latency depends on hardware and implementation; a simpler expression does not guarantee lower deployment latency (**zhu2025dyt**).

The evidence from the two provided studies supports a more precise understanding of normalization as an architectural design choice. The systematic review identifies RMSNorm as a prevalent option in recent SLM research, while the DyT study demonstrates that a learned, bounded element-wise transformation can achieve comparable performance to RMSNorm in the evaluated LLaMA configurations. These results do not show that all normalization mechanisms have identical effects. Rather, they show that the

control of activation scale and the suppression of extreme values can be implemented through different mathematical operations.

Normalization is therefore best understood not merely as a procedure for forcing activations to have a prescribed statistical distribution, but as a component that affects the numerical behavior and differential structure of the transformations composing a deep network. Its interaction with residual pathways, attention sublayers, and feed-forward transformations determines how intermediate representations are passed through the architecture.

With the role of normalization established, the next architectural component to examine is attention. Whereas normalization controls aspects of the numerical representation within a token, attention provides the mechanism through which information is exchanged across different token positions. Understanding both operations is necessary for reconstructing how a Transformer block processes a sequence.

2.6 Attention Mechanisms

The preceding discussion examined normalization as a means of controlling the scale and statistical behavior of intermediate representations. A further architectural question remains: how can the representation of one token incorporate information carried by other tokens in the same sequence? Token embeddings and positional information establish the initial numerical representations, while normalization and position-wise transformations operate on those representations. None of these operations, by itself, specifies a content-dependent mechanism through which one position can retrieve information from another. The attention mechanism supplies this missing operation. It computes relationships between token representations and uses those relationships to determine how information is exchanged across sequence positions. Attention is therefore the next step in the architectural account of how a language model converts a sequence of token representations into context-dependent representations (**ansar2024transformers**; **dao2023flashattention2**).

2.6.1 The role of attention in sequence representation

Consider a sequence containing N tokens. After tokenization, embedding, and the addition or incorporation of positional information, each token is represented by a vector in a d_{model}-dimensional real vector space. Using a row-vector convention, the sequence can be written as

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \in \mathbb{R}^{N \times d_{\text{model}}}, \qquad x_i \in \mathbb{R}^{1 \times d_{\text{model}}}. \tag{2.267}$$

Each row is the current representation of one token position. The word “current” is important: in a deep Transformer, a row of X is not necessarily the original embedding of the token. It may already contain information produced by preceding layers.

The representation required at position i generally depends on more than the token at that position. A word may be disambiguated by another word in the sentence; a pronoun may refer to an entity introduced earlier; and a question may require information

distributed across several parts of a document. A model that transforms each token independently cannot directly exchange information between positions through that transformation alone. Attention addresses this requirement by allowing each position to retrieve a content-dependent combination of information from other positions.

The mechanism separates two questions. The first is which positions are relevant to the current position. The second is what information should be transferred from those positions. Queries and keys determine the first relationship, while values carry the information used in the second operation. This separation is the basis of the query–key–value formulation of attention described in the supplied Transformer survey (**ansar2024transformers**).

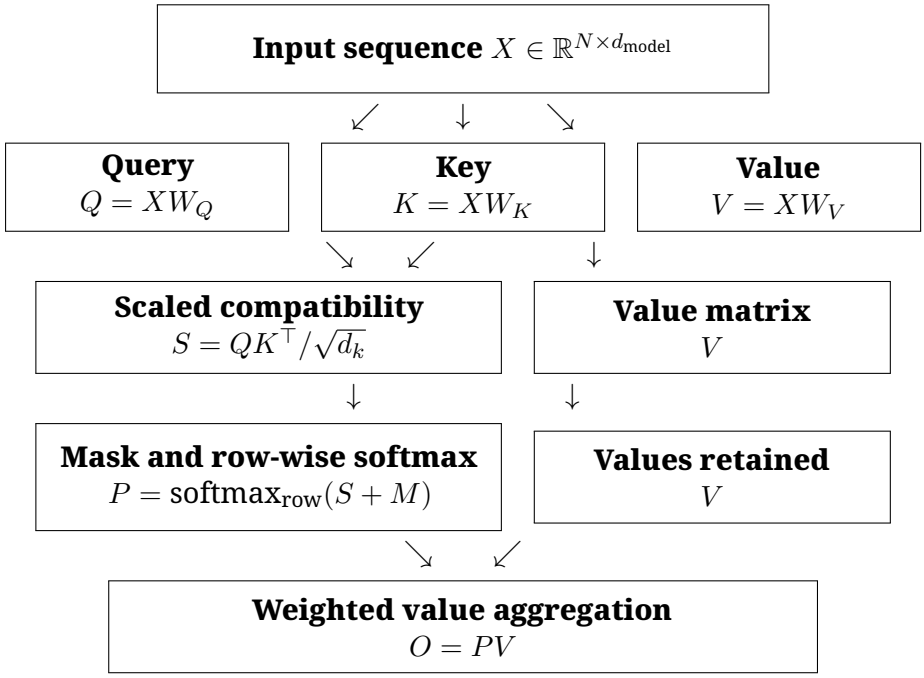


Figure 2.12: Computational flow of scaled dot-product attention. The input representations are projected into queries, keys, and values; query–key compatibility scores are scaled and normalized; and the resulting attention weights determine the weighted aggregation of value vectors. The formulation follows the attention descriptions in **ansar2024transformers** and **dao2023flashattention2**.

2.6.2 Query, key, and value representations

Attention begins by applying three learned linear projections to the input matrix X. For one attention head, define

$$Q = XW_Q, \qquad W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \tag{2.268}$$

$$K = XW_K, \qquad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \tag{2.269}$$

$$V = XW_V, \qquad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \tag{2.270}$$

The resulting matrices have dimensions

$$Q, K \in \mathbb{R}^{N \times d_k}, \qquad V \in \mathbb{R}^{N \times d_v}. \tag{2.271}$$

Here, d_k is the query and key dimension, and d_v is the value dimension. The matrices W_Q, W_K, and W_V are trainable parameters. The i-th rows of Q, K, and V are denoted by q_i, k_i, and v_i, respectively:

$$q_i = x_i W_Q, \qquad k_i = x_i W_K, \qquad v_i = x_i W_V. \tag{2.272}$$

A query is the projected representation used by a position to express what information it seeks. A key is the projected representation against which queries are compared. A value is the projected content contributed by a position when another position attends to it. These descriptions identify their computational roles; they do not imply that queries, keys, or values are literal symbolic questions, labels, or database records. They are learned vectors whose geometry is shaped by optimization.

The query and key dimensions must agree because their compatibility is computed by a dot product. The value dimension need not equal the key dimension. This distinction allows the model to learn one representation space for deciding where to retrieve information and another for representing the information that is transferred.

2.6.3 Scaled dot-product attention

Compatibility scores. For a query q_i and key k_j, the unnormalized compatibility score is their inner product:

$$s_{ij} = q_i k_j^\top = \sum_{r=1}^{d_k} q_{ir} k_{jr}. \tag{2.273}$$

The dot product is a scalar measure of alignment in the learned query–key space. It is not a probability and is not restricted to the interval $[0, 1]$. It may be positive, negative, or zero. For all query–key pairs, the scores are computed by the matrix multiplication

$$S = QK^\top, \qquad S \in \mathbb{R}^{N \times N}. \tag{2.274}$$

The element S_{ij} is the score assigned by query position i to key position j. Thus, dense self-attention constructs an $N \times N$ matrix of pairwise interactions.

Why the scores are scaled. The magnitude of an unscaled dot product depends on the dimension d_k. A statistical argument explains the scaling used in the standard formulation. Assume, for this calculation, that the components of q_i and k_j are mutually independent, have zero mean, and have unit variance. Then

$$\mathbb{E}[q_i k_j^\top] = 0, \qquad \mathrm{Var}(q_i k_j^\top) = d_k. \tag{2.275}$$

The variance therefore increases linearly with the key dimension. Dividing the dot product by $\sqrt{d_k}$ gives a score with unit variance under these assumptions:

$$\mathrm{Var}\left(\frac{q_i k_j^\top}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1. \tag{2.276}$$

This is a scale-control argument under a simplifying statistical model, not a claim that trained attention scores necessarily have unit variance.

The scaled score matrix is

$$\widetilde{S} = \frac{QK^\top}{\sqrt{d_k}}. \tag{2.277}$$

The scaling reduces the tendency for score magnitudes to grow merely because the projection dimension is larger. This is relevant because softmax can become highly concentrated when its inputs have large differences. A concentrated distribution can yield small derivatives for many score components and can make optimization less well-conditioned. The scaled dot-product formulation is the standard attention computation described in the supplied surveys and in the FlashAttention-2 paper (**ansar2024transformers**; **dao2023flashattention2**).

Softmax normalization. The score matrix is converted into attention weights by applying softmax independently to each row. For a vector $s = (s_1, \dots, s_N)$, softmax is

$$\mathrm{softmax}(s)_j = \frac{\exp(s_j)}{\sum_{r=1}^{N} \exp(s_r)}. \tag{2.278}$$

The attention probability matrix is therefore

$$P = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right), \qquad P \in \mathbb{R}^{N\times N}. \tag{2.279}$$

For every row i,

$$P_{ij} \geq 0, \qquad \sum_{j=1}^{N} P_{ij} = 1. \tag{2.280}$$

Thus, each row is a probability distribution over the key positions available to the corresponding query. The weights are relative: changing one score changes the normalized weights of the other positions in that row as well.

A simple numerical calculation illustrates the normalization. If a query has two scaled scores $s_1 = 0$ and $s_2 = \ln 2$, then

$$\mathrm{softmax}(0, \ln 2) = \left(\frac{1}{3}, \frac{2}{3}\right). \tag{2.281}$$

The second key receives twice the normalized weight of the first key. This does not mean that its value vector is twice as large; it means that its contribution to the weighted aggregation is multiplied by twice the coefficient.

For numerical stability, softmax is commonly evaluated after subtracting the largest score in the row. If $m = \max_j s_j$, then

$$\frac{\exp(s_j)}{\sum_r \exp(s_r)} = \frac{\exp(s_j - m)}{\sum_r \exp(s_r - m)}. \tag{2.282}$$

The equality follows because the same factor $\exp(-m)$ appears in the numerator and denominator and cancels. The transformed exponents are non-positive, which reduces the risk of overflow. This identity is also the basis of the online softmax updates used by FlashAttention (**dao2023flashattention2**).

Weighted aggregation of values. The attention weights determine how value vectors are combined:

$$O = PV, \qquad O \in \mathbb{R}^{N \times d_v}. \tag{2.283}$$

For position i, the output is

$$o_i = \sum_{j=1}^{N} P_{ij} v_j. \tag{2.284}$$

Because the weights are non-negative and sum to one, this output is a convex combination of the value vectors included in that row's normalization. It therefore lies in their convex hull. The output is context-dependent because the weights depend on the current query and all available keys.

Combining the score, normalization, and aggregation stages gives the scaled dot-product attention operation:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V}. \tag{2.285}$$

The equation separates the decision about which positions to retrieve from the transformation of the information being retrieved.

2.6.4 Self-attention, cross-attention, and causal masking

Self-attention. Self-attention is the configuration in which queries, keys, and values are all projected from the same input sequence:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V. \tag{2.286}$$

The output is

$$\text{SelfAttention}(X) = \text{softmax}_{\text{row}}\left(\frac{XW_Q W_K^\top X^\top}{\sqrt{d_k}}\right) XW_V. \tag{2.287}$$

Every position can use its query to identify relevant positions in the same sequence. The interaction pattern is content-dependent because it is calculated from the current learned representations, rather than fixed solely by token distance. Self-attention is a central component of Transformer-based language models and is described as such in the supplied Transformer survey (**ansar2024transformers**).

Cross-attention. Queries do not have to originate from the same sequence as keys and values. Let X_q be a query sequence of length N_q, and let X_c be a context sequence of length N_c. Cross-attention uses

$$Q = X_q W_Q, \qquad Q \in \mathbb{R}^{N_q \times d_k}, \tag{2.288}$$

$$K = X_c W_K, \qquad K \in \mathbb{R}^{N_c \times d_k}, \tag{2.289}$$

$$V = X_c W_V, \qquad V \in \mathbb{R}^{N_c \times d_v}. \tag{2.290}$$

The score matrix then has shape

$$S = QK^\top \in \mathbb{R}^{N_q \times N_c}, \tag{2.291}$$

and the output is

$$O = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{N_q \times d_v}. \tag{2.292}$$

Each query position can therefore retrieve information from the context sequence even when the two sequences have different lengths. This is the form of attention used when a decoder retrieves information from encoder representations in an encoder–decoder arrangement. The supplied GQA study also evaluates multi-query and grouped-query variants in decoder cross-attention as well as decoder self-attention (**ainslie2023gqa**).

Causal masking. Autoregressive language modeling imposes an information constraint: the prediction at position i must not depend on tokens at positions greater than i. Without a mask, ordinary self-attention could use future tokens during training, producing information leakage relative to the next-token prediction task.

Define the additive causal mask $M \in \mathbb{R}^{N \times N}$ by

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \tag{2.293}$$

The masked score matrix is

$$S_{\text{causal}} = \frac{QK^\top}{\sqrt{d_k}} + M. \tag{2.294}$$

Because $\exp(-\infty) = 0$, future positions receive zero attention probability. The row-i distribution is normalized only over positions $1, \dots, i$:

$$P_{ij} = \begin{cases} \dfrac{\exp(\widetilde{S}_{ij})}{\sum_{r=1}^{i} \exp(\widetilde{S}_{ir})}, & j \le i, \\ 0, & j > i, \end{cases} \tag{2.295}$$

where $\widetilde{S} = QK^\top/\sqrt{d_k}$. Causal attention is consequently lower triangular. The first position can use only itself; the second can use the first and second positions; and position i can use the prefix ending at i.

The mask defines the permitted information flow. During autoregressive generation, the model processes the available prefix, predicts a next token, appends that token, and makes its representation available to subsequent positions. The causal restriction is therefore an architectural condition for autoregressive prediction, not merely a numerical optimization. Implementations can exploit the triangular structure by skipping blocks that lie entirely in the masked region, as discussed in the FlashAttention-2 paper (**dao2023flashattention2**).

2.6.5 Multi-head attention

A single attention head uses one learned query–key compatibility space and one value space. A language sequence, however, can contain several simultaneous kinds of relationships. A token may need to retrieve a nearby syntactic dependency, an entity introduced much earlier, or information associated with a task-specific pattern. Requiring all such relationships to be represented through one projection space may constrain the range of interactions the layer can learn.

Multi-head attention addresses this limitation by performing several attention computations in parallel. Let H be the number of heads. For head h, define

$$Q_h = XW_Q^{(h)}, \qquad K_h = XW_K^{(h)}, \qquad V_h = XW_V^{(h)}, \tag{2.296}$$

$$\text{head}_h = \text{softmax}_{\text{row}}\left(\frac{Q_h K_h^\top}{\sqrt{d_k}} + M\right) V_h. \tag{2.297}$$

Each head has its own learned projection matrices and can therefore learn a different compatibility function and value transformation.

In the conventional equal-dimension construction,

$$d_k = d_v = d_h = \frac{d_{\text{model}}}{H}. \tag{2.298}$$

Each head output has shape

$$\text{head}_h \in \mathbb{R}^{N \times d_h}. \tag{2.299}$$

The outputs are concatenated along their feature dimensions:

$$C = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_H\right) \in \mathbb{R}^{N \times H d_h} = \mathbb{R}^{N \times d_{\text{model}}}. \tag{2.300}$$

A learned output projection maps the concatenated result into the model representation space:

$$\boxed{\text{MHA}(X) = CW_O, \qquad W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}.} \tag{2.301}$$

The output projection allows information from different head feature dimensions to be combined. The multi-head construction is summarized in figure 2.13; the supplied Transformer survey describes the same parallel-head, concatenation, and output-projection structure (**ansar2024transformers**).

Parameter count. Assume that all heads have dimension $d_h = d_{\text{model}}/H$, and ignore projection biases. Across all heads, the query projection weights contain

$$P_Q = H d_{\text{model}} d_h = d_{\text{model}}^2 \tag{2.302}$$

parameters. The key and value projections have the same count:

$$P_K = P_V = d_{\text{model}}^2. \tag{2.303}$$

The output projection contributes another d_{model}^2 parameters. Thus, under these assumptions,

$$P_{\text{MHA}} = P_Q + P_K + P_V + P_O = 4 d_{\text{model}}^2. \tag{2.304}$$

This count is independent of H when the total model dimension remains fixed and the per-head dimension is reduced proportionally. Increasing the number of heads therefore changes the partition of the projection space, but does not by itself increase this idealized total projection parameter count.

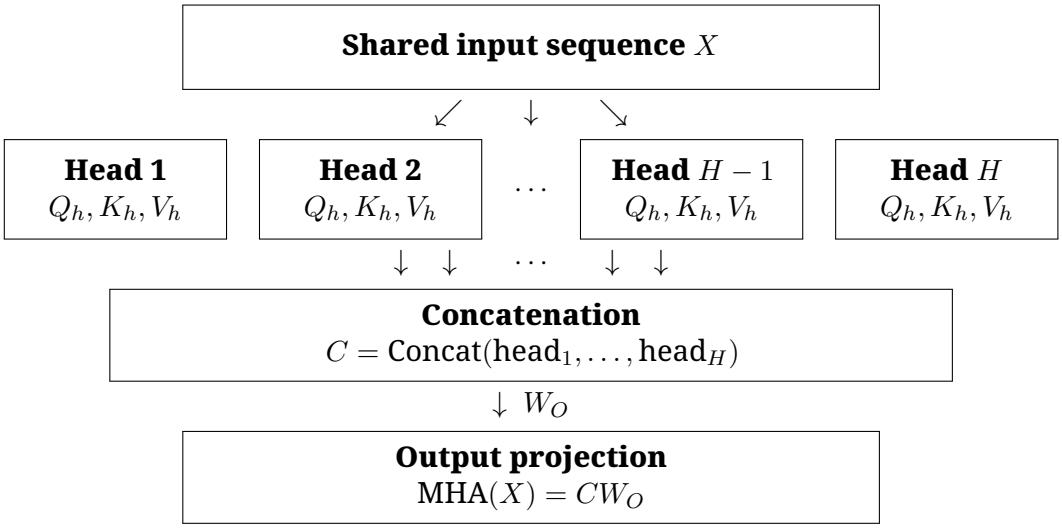


Figure 2.13: Multi-head attention. The same input sequence is projected into separate query, key, and value representations for each head. The head outputs are concatenated and passed through a learned output projection. Based on the multi-head formulation in **ansar2024transformers**.

Arithmetic cost. For one head, computing $Q_h K_h^\top$ requires approximately $2N^2 d_h$ floating-point operations when a multiplication and an addition are counted separately. Computing $P_h V_h$ requires another $2N^2 d_h$. Summing over H heads gives

$$C_{\text{attention}} \approx 4HN^2 d_h = 4N^2 d_{\text{model}} \tag{2.305}$$

for the two principal matrix multiplications of one sequence. This expression excludes the query, key, value, and output projections, the softmax, masking, and other layer operations. Its principal implication is the quadratic dependence on N.

2.6.6 Backpropagation through attention

Attention is a differentiable computation whose projection matrices are learned through the loss function. The backward pass determines how the loss changes when the query, key, and value representations change. This derivation also explains why storing or recomputing attention probabilities is important for memory-efficient training.

Consider the forward equations

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad P = \text{softmax}_{\text{row}}(S), \qquad O = PV. \tag{2.306}$$

Let $\mathcal{L}$ be a scalar loss and define the upstream gradient

$$G = \frac{\partial \mathcal{L}}{\partial O} \in \mathbb{R}^{N \times d_v}. \tag{2.307}$$

The value gradient follows directly from $O = PV$:

$$\frac{\partial \mathcal{L}}{\partial V} = P^\top G. \tag{2.308}$$

The probability gradient is

$$\frac{\partial \mathcal{L}}{\partial P} = GV^\top \in \mathbb{R}^{N\times N}. \tag{2.309}$$

The softmax derivative must account for the coupling introduced by normalization. For one row $p = \text{softmax}(s)$, the Jacobian is

$$J_{\text{softmax}}(p) = \text{diag}(p) - pp^\top. \tag{2.310}$$

This follows from

$$\frac{\partial p_j}{\partial s_k} = p_j(\delta_{jk} - p_k), \tag{2.311}$$

where δ_{jk} is the Kronecker delta. If $D_P = \partial\mathcal{L}/\partial P$, the score gradient for each row is

$$ds = \left(\text{diag}(p) - pp^\top\right) dp. \tag{2.312}$$

Equivalently, in component form,

$$\frac{\partial \mathcal{L}}{\partial s_j} = p_j \left(\frac{\partial \mathcal{L}}{\partial p_j} - \sum_{r=1}^{N} p_r \frac{\partial \mathcal{L}}{\partial p_r}\right). \tag{2.313}$$

The gradient of one score therefore depends on the probability and upstream gradient of every score in the same row.

Let $D = \partial\mathcal{L}/\partial S$ denote the matrix formed by applying this row-wise derivative. The gradients of the scaled query–key product are

$$\frac{\partial \mathcal{L}}{\partial Q} = \frac{DK}{\sqrt{d_k}}, \tag{2.314}$$

$$\frac{\partial \mathcal{L}}{\partial K} = \frac{D^\top Q}{\sqrt{d_k}}. \tag{2.315}$$

For a masked score, the corresponding probability is zero and its gradient contribution is zero. The complete backward pass is consequently

$$dV = P^\top G, \tag{2.316}$$

$$dP = GV^\top, \tag{2.317}$$

$$dS = \text{SoftmaxBackward}(P, dP), \tag{2.318}$$

$$dQ = \frac{dSK}{\sqrt{d_k}}, \tag{2.319}$$

$$dK = \frac{dS^\top Q}{\sqrt{d_k}}. \tag{2.320}$$

The gradients then propagate to the learned projection matrices. Since $Q = XW_Q$, $K = XW_K$, and $V = XW_V$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial W_Q} = X^\top \frac{\partial \mathcal{L}}{\partial Q}, \tag{2.321}$$

$$\frac{\partial \mathcal{L}}{\partial W_K} = X^\top \frac{\partial \mathcal{L}}{\partial K}, \tag{2.322}$$

$$\frac{\partial \mathcal{L}}{\partial W_V} = X^\top \frac{\partial \mathcal{L}}{\partial V}. \tag{2.323}$$

The contribution to the input gradient through these three projections is

$$\frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial Q} W_Q^\top + \frac{\partial \mathcal{L}}{\partial K} W_K^\top + \frac{\partial \mathcal{L}}{\partial V} W_V^\top. \tag{2.324}$$

For multi-head attention, the output projection combines the head-specific contributions before they return to the shared input representation. Training therefore adjusts both the information-selection spaces and the value transformations. The same backward dependencies motivate the recomputation strategy used by FlashAttention (**dao2023flashattention2**).

2.6.7 Attention-head interpretation and functional specialization

The multi-head formulation explains how several attention computations can operate in parallel, but it does not establish what a particular head has learned to do. Investigating that question belongs to mechanistic interpretability: the study of internal computations through parameters, activations, intermediate representations, and controlled interventions. The supplied survey by **zheng2024attentionheads** organizes reported attention-head functions through a four-stage framework: Knowledge Recalling, In-Context Identification, Latent Reasoning, and Expression Preparation. The framework is a way to organize experimental findings; it is not evidence that a Transformer contains four explicit modules corresponding to human cognitive stages.

The QK and OV transformations. For one head h, let $W_Q^{(h)}$, $W_K^{(h)}$, and $W_V^{(h)}$ be its projection matrices. Let $W_O^{(h)}$ denote the slice of the multi-head output projection that maps this head's output into the model dimension. The head's contribution can be expressed as

$$\text{Head}_h(X) = \text{softmax}\left(\frac{X W_Q^{(h)} W_K^{(h)\top} X^\top}{\sqrt{d_h}} + M\right) X W_V^{(h)} W_O^{(h)}. \tag{2.325}$$

This expression exposes two composite transformations. The query–key matrix is

$$W_{QK}^{(h)} = W_Q^{(h)} W_K^{(h)\top}, \tag{2.326}$$

which determines how token representations are compared to produce attention scores. The value–output matrix is

$$W_{OV}^{(h)} = W_V^{(h)} W_O^{(h)}, \tag{2.327}$$

which determines how attended information is transformed before it is written into the model representation space. The QK transformation is therefore associated with information selection, while the OV transformation is associated with information transfer. The distinction is central to the interpretability framework in **zheng2024attentionheads**.

Residual-stream contribution. An attention sublayer usually adds an update to the representation carried through the Transformer by residual connections. Omitting architecture-dependent normalization details, this update can be written as

$$R_\ell^{\text{after}} = R_\ell^{\text{before}} + \sum_{h=1}^{H} \text{Head}_{\ell,h}(R_\ell^{\text{before}}). \tag{2.328}$$

The residual stream is the shared representation pathway through which information from different heads and layers can accumulate. A head reads the representation available in this stream, performs its query–key selection and value transformation, and writes an update back into the stream. Subsequent heads and layers can then use the modified representation. The exact placement of normalization and the ordering of sublayers depend on the Transformer variant; the equation isolates the residual contribution rather than specifying one universal block layout.

Knowledge Recalling. The Knowledge Recalling stage concerns retrieval of information associated with the model's parametric knowledge. The survey identifies the Memory Head as an example: it can retrieve attributes associated with entities present in the input and write the retrieved information into the residual stream. In task-specific settings, the survey also describes Constant Heads, Single Letter Heads, and Negative Heads in multiple-choice or binary-decision contexts. These examples illustrate that some heads are associated with retrieval from learned parameters, while others are associated with the structure of a particular task (**zheng2024attentionheads**).

In-Context Identification. The In-Context Identification stage concerns identifying structural, syntactic, and semantic information in the current context. The survey discusses Previous Heads and Positional Heads as examples that track positional relationships, Duplicate Heads as heads that respond to repeated content, and Retrieval Heads as heads that locate specific information in longer contexts. Syntactic Heads are reported to identify relationships such as subjects, objects, and modifiers. Subword Merge Heads are associated with integrating subword representations, while Name Mover and Letter Mover Heads transfer entity or letter information toward positions used in subsequent computation. Context Heads and Content Gatherer Heads identify task-relevant content; Sentiment Summarizers and Semantic Induction Heads are associated with sentiment and semantic relationships. These are reported functional descriptions from the surveyed studies, not universal properties of every model's attention heads (**zheng2024attentionheads**).

Latent Reasoning. The Latent Reasoning stage concerns the integration of information identified in the context and the use of learned patterns to produce intermediate conclusions. Induction Heads are a representative example. The survey describes a pattern of the form

$$\ldots [A][B] \ldots [A], \tag{2.329}$$

where the first occurrence of $[A]$ is followed by $[B]$. An Induction Head can use information about the earlier continuation to support the prediction of $[B]$ after the later occurrence of $[A]$. The survey also discusses Summary Readers and Function Vectors in task recognition, as well as heads associated with truthfulness, accuracy, consistency, iteration, successor operations, and inhibition of selected signals. These examples show that head functions can involve both the retrieval and combination of information and the suppression of competing information (**zheng2024attentionheads**).

Expression Preparation. The Expression Preparation stage concerns preparing information for expression in the model's output. The survey describes Mixed Heads as

aggregating information passed from other heads, Amplification and Correct Heads as strengthening signals associated with a selected answer, and Coherence and Faithfulness Heads as being associated with linguistic consistency and alignment between reasoning-related information and instructions. The framework also allows that some tasks may not require a distinct expression-preparation operation: information already present in the residual stream may be sufficient to determine the next output token (**zheng2024attentionheads**).

The four stages can interact and need not occur as a rigid one-way sequence. A model may return to contextual identification after retrieving knowledge, or retrieve additional information during reasoning. The framework is therefore best interpreted as a descriptive organization of reported head functions rather than a literal computational pipeline. figure 2.14 summarizes the four categories and illustrative functions discussed in the survey.

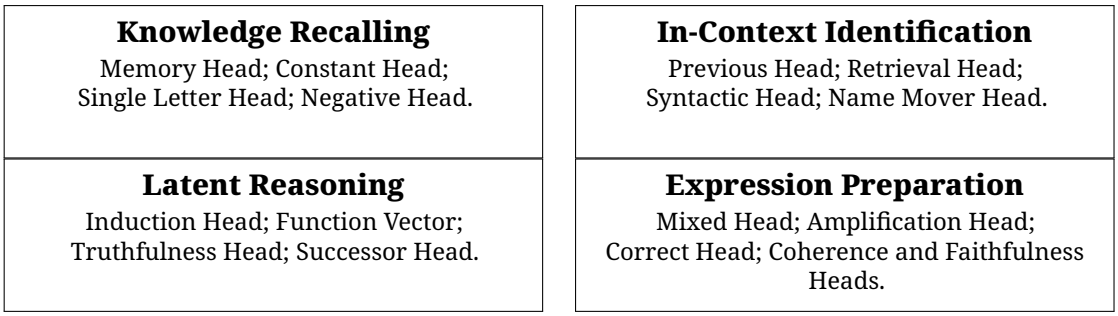


Figure 2.14: Four-stage organization of reported attention-head functions: Knowledge Recalling, In-Context Identification, Latent Reasoning, and Expression Preparation. The categories are an interpretability framework and may interact rather than forming a rigid sequential pipeline. Based on **zheng2024attentionheads**.

Experimental investigation. The survey describes experiments that test whether a proposed head function contributes to model behavior. Activation patching replaces selected activations with values from another input, a baseline, or a perturbed run, and measures the effect on the output. Ablation instead removes or suppresses a component and observes the resulting behavioral change. The two methods are related but not identical: patching substitutes an activation, whereas ablation removes a component's contribution. The survey also describes the logit lens, which uses the unembedding transformation to map intermediate representations to vocabulary logits and thereby examine how an intermediate state relates to candidate output tokens (**zheng2024attentionheads**).

These interventions provide evidence about the role of a component under the tested conditions. They do not, by themselves, establish that a proposed function generalizes to every task, model family, or context. The survey identifies generalizability, transferability, interactions among multiple heads, and the need for stronger theoretical accounts as important limitations in the study of attention-head functions (**zheng2024attentionheads**).

2.6.8 Computational and memory complexity of dense attention

The dimensions of the score and probability matrices explain the principal scaling limitation of dense attention. For one head, the two main matrix multiplications are

$$QK^\top : (N \times d_k)(d_k \times N) \longrightarrow N \times N \tag{2.330}$$

and

$$PV : (N \times N)(N \times d_v) \longrightarrow N \times d_v. \tag{2.331}$$

Their combined arithmetic cost is

$$\mathcal{O}(N^2 d_k + N^2 d_v). \tag{2.332}$$

When $d_k = d_v = d_h$, this is

$$\mathcal{O}(N^2 d_h) \tag{2.333}$$

up to constant factors. Across H heads with $Hd_h = d_{\text{model}}$, the leading sequence-length dependence is

$$\boxed{\mathcal{O}(N^2 d_{\text{model}})}. \tag{2.334}$$

The quadratic dependence can be illustrated numerically. If the sequence length increases from $N = 2{,}048$ to $N = 8{,}192$, the ratio of the numbers of pairwise positions is

$$\frac{8{,}192^2}{2{,}048^2} = 4^2 = 16. \tag{2.335}$$

Thus, at fixed head and model dimensions, the principal dense-attention matrix multiplications require sixteen times as much arithmetic after a fourfold increase in sequence length.

Intermediate memory. The score matrix S and probability matrix P each contain N^2 elements per head and sequence. If a conventional implementation materializes both matrices for a batch of B sequences and H heads, the storage for these two intermediates is

$$M_{\text{intermediate}} = 2BHN^2 b \quad \text{bytes}, \tag{2.336}$$

where b is the number of bytes per stored element. The precise peak allocation depends on the implementation and which intermediates are retained for the backward pass, but the quadratic dependence on N remains the central difficulty. The FlashAttention-2 paper identifies materialization of the score and probability matrices in high-bandwidth memory as a major source of memory use and data movement (**dao2023flashattention2**).

The key–value cache during autoregressive inference. Training processes multiple positions in parallel subject to the causal mask. Autoregressive inference instead generates one token at a time. At each decoding step, the new query must interact with the keys and values of preceding positions. Recomputing all preceding key and value projections at every step would repeat work, so decoder implementations commonly retain the previously computed keys and values in a key–value cache, or KV cache.

Let L be the number of Transformer layers, B the batch size, N the number of cached positions, H_{KV} the number of key–value heads, d_h the dimension of each such head, and b the number of bytes per element. The cache contains keys and values, so its memory requirement is

$$\boxed{M_{\text{KV}} = 2LBNH_{\text{KV}}d_h b}. \tag{2.337}$$

The cache grows linearly with sequence length, but its total size can be large when the model has many layers, a large batch, or many key–value heads. During decoding, the keys and values must also be transferred from memory for the current query computation. Consequently, inference speed can be constrained by memory bandwidth, not only by arithmetic throughput. This is the motivation for reducing the number of key–value heads in multi-query and grouped-query attention (**ainslie2023gqa**).

2.6.9 Multi-query and grouped-query attention

Multi-query attention. Conventional multi-head attention uses a separate key and value head for each query head. Multi-query attention (MQA) retains multiple query heads but shares one key head and one value head across all of them. If there are H query heads, then

$$H_Q = H, \qquad H_K = H_V = 1. \tag{2.338}$$

The output of query head h is

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K^\top}{\sqrt{d_h}} + M\right) V. \tag{2.339}$$

The query projection remains head-specific, but the same key and value representations are used by all query heads.

For conventional MHA, the KV cache contains $2LBNHd_h b$ bytes. For MQA, it contains $2LBNd_h b$ bytes. Under the same dimensions, the reduction factor is

$$R_{\text{MQA}} = \frac{2LBNHd_h b}{2LBNd_h b} = H. \tag{2.340}$$

This reduction decreases the amount of key–value data that must be stored and loaded during autoregressive inference. It does not remove the query-head computations. Sharing key and value representations can, however, reduce representational flexibility and may cause quality degradation or training instability. These trade-offs motivate an intermediate design (**ainslie2023gqa**).

Grouped-query attention. Grouped-query attention (GQA) retains H query heads but partitions them into G groups, each sharing one key head and one value head. Assume G divides H. Each group then contains H/G query heads:

$$H_Q = H, \qquad H_K = H_V = G. \tag{2.341}$$

Let $g(h)$ identify the key–value group assigned to query head h. The head output is

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_{g(h)}^\top}{\sqrt{d_h}} + M\right) V_{g(h)}. \tag{2.342}$$

The KV-cache memory becomes

$$M_{\text{KV,GQA}} = 2LBNGd_h b. \tag{2.343}$$

Relative to MHA, the cache reduction factor is

$$R_{\text{GQA}} = \frac{H}{G}. \tag{2.344}$$

The limiting cases recover the two familiar architectures:

$$G = 1 \Longrightarrow \text{GQA} = \text{MQA}, \qquad G = H \Longrightarrow \text{GQA} = \text{MHA}\,. \tag{2.345}$$

Intermediate values of G trade key–value memory against the number of independently represented key–value groups.

For example, if a model has $H = 32$ query heads and uses $G = 8$ key–value groups, then

$$R_{\text{GQA}} = \frac{32}{8} = 4. \tag{2.346}$$

The KV cache is one quarter the size of the corresponding MHA cache, while remaining eight times the size of an MQA cache under the same head-dimension assumptions. The query-head count remains 32.

Converting and uptraining existing checkpoints. **ainslie2023gqa** propose converting a multi-head checkpoint into an MQA or GQA checkpoint and then performing additional pre-training so that the model can adapt to the changed key–value structure. For GQA, the original key and value projection matrices are mean-pooled within each new group. If $\mathcal{G}_g$ is the set of original heads assigned to group g, the initialization is

$$W_K^{(g)} = \frac{G}{H} \sum_{h \in \mathcal{G}_g} W_K^{(h)}, \tag{2.347}$$

$$W_V^{(g)} = \frac{G}{H} \sum_{h \in \mathcal{G}_g} W_V^{(h)}. \tag{2.348}$$

The group mean retains contributions from the original heads rather than selecting one head or randomly initializing the new shared projection. The converted checkpoint is then uptrained using a fraction of the original pre-training procedure. The authors report experiments with an uptraining proportion of five percent of the original pre-training steps.

Their T5-XXL experiments report an inference time of 1.51 ms per sample and an average task score of 47.2 for MHA-XXL. The uptrained MQA-XXL model reports 0.24 ms and 46.6, while GQA-8-XXL reports 0.28 ms and 47.1. These results illustrate the reported trade-off: GQA-8 approaches the MHA-XXL average score while retaining inference time close to MQA. The values are specific to the models, tasks, and hardware configuration used in the study; they are not universal performance guarantees (**ainslie2023gqa**).

GQA is therefore a compact-attention strategy: it reduces the amount of key–value information stored in the cache through sharing, while retaining the dense query-to-key interaction pattern. It should not be confused with sparse attention, which omits selected query–key interactions, or with linear attention, which changes the mathematical formulation of attention (**ainslie2023gqa**; **sun2026efficientattention**).

Table 2.12: Selected T5-XXL results reported for MHA, MQA, and GQA. Inference time is the reported average time per sample; average performance is the reported average over the evaluated tasks. Source: **ainslie2023gqa**, Table 1.

Configuration	Inference time (ms)	Average performance	Key–value structure
MHA-XXL	1.51	47.2	One key–value head per query head
MQA-XXL	0.24	46.6	One shared key–value head
GQA-8-XXL	0.28	47.1	Eight shared key–value groups

2.6.10 FlashAttention and FlashAttention-2

The preceding methods address the cost of retaining key–value representations during inference. A different problem occurs when the attention score and probability matrices are materialized during training or prompt processing. FlashAttention addresses this problem without approximating the dense attention result. Its central idea is to reorganize the computation to reduce data movement between high-bandwidth memory (HBM) and fast on-chip storage.

Memory hierarchy and the I/O bottleneck. GPUs contain several levels of memory with different capacities and bandwidths. HBM provides comparatively large storage, while on-chip SRAM is smaller and faster. The FlashAttention-2 paper uses NVIDIA A100 hardware as an example and reports HBM bandwidth of approximately 1.5–2.0 TB/s and an estimated on-chip SRAM bandwidth of approximately 19 TB/s. It also describes approximately 192 KB of SRAM per streaming multiprocessor in the hardware setting considered by the paper. These figures are hardware-specific examples, but they illustrate why repeated transfers of large intermediate matrices can limit practical attention speed (**dao2023flashattention2**).

A conventional implementation may compute $S = QK^\top$, write S to HBM, read it for softmax, write P to HBM, and read P again for $O = PV$. The score and probability matrices each have N^2 entries per head. The repeated writes and reads create substantial memory traffic, and the probability matrix may also need to be retained for backpropagation. The runtime is consequently influenced by both arithmetic and memory movement.

Tiling. FlashAttention uses tiling to keep intermediate blocks in on-chip SRAM. The query matrix is divided into row blocks, and the key and value matrices are divided into corresponding blocks. For a query block Q_i and key block K_j, the local score block is

$$S_{ij} = \frac{Q_i K_j^\top}{\sqrt{d_k}}, \qquad S_{ij} \in \mathbb{R}^{B_r \times B_c}, \tag{2.349}$$

where B_r and B_c are the query-row and key-column block sizes. The corresponding value block is $V_j \in \mathbb{R}^{B_c \times d_v}$. The algorithm loads blocks from HBM, computes their contribution in SRAM, updates a running output, and moves to the next block. It avoids writing the full $N \times N$ score and probability matrices to HBM.

The blocks cannot simply be softmax-normalized independently and then concatenated. The softmax denominator for a query row depends on every permitted key posi-

tion, including keys in blocks that have not yet been processed. FlashAttention resolves this dependency through online softmax.

Online softmax derivation. Consider one query-row block and suppose that key–value blocks are processed in sequence. For a given query row, maintain a running maximum m, a running normalization denominator ℓ, and an unnormalized value accumulator U. Before processing any key block, initialize

$$m^{(0)} = -\infty, \qquad \ell^{(0)} = 0, \qquad U^{(0)} = 0. \tag{2.350}$$

For the next score block $S^{(j)}$, update the running maximum:

$$m^{(j)} = \max\left(m^{(j-1)}, \text{rowmax}(S^{(j)})\right). \tag{2.351}$$

The denominator is updated by rescaling the previous contribution to the new maximum and adding the current block's contribution:

$$\boxed{\ell^{(j)} = e^{m^{(j-1)} - m^{(j)}} \ell^{(j-1)} + \text{rowsum}\left(e^{S^{(j)} - m^{(j)}}\right)}. \tag{2.352}$$

The exponentials, maxima, and row sums are evaluated row-wise. The unnormalized value accumulator is updated as

$$\boxed{U^{(j)} = e^{m^{(j-1)} - m^{(j)}} U^{(j-1)} + e^{S^{(j)} - m^{(j)}} V_j}. \tag{2.353}$$

The rescaling factor is necessary because the maximum used to stabilize the exponential may change when a new block is introduced. After all key–value blocks have been processed, the exact normalized output is

$$O = \frac{U^{(T_c)}}{\ell^{(T_c)}}, \tag{2.354}$$

where T_c is the number of key–value blocks. The division is performed row-wise and broadcast across the value dimension.

These updates are algebraically equivalent to full softmax attention. They change the order in which intermediate quantities are computed and stored, not the attention function itself. The running maximum also preserves the numerical-stability benefit of max-shifted softmax. figure 2.15 illustrates the blockwise computation and the compact online-softmax state (**dao2023flashattention2**).

Backward pass and recomputation. A conventional backward pass uses the attention probabilities to calculate gradients. FlashAttention avoids storing the complete probability matrix in HBM. Instead, it retains compact normalization information and recomputes the necessary score and probability blocks during the backward pass. This trades additional arithmetic for reduced memory traffic and intermediate storage. The gradients are still those of the exact attention operation:

$$dV = P^\top dO, \tag{2.355}$$

$$dP = dOV^\top, \tag{2.356}$$

$$dS = \text{SoftmaxBackward}(P, dP), \tag{2.357}$$

$$dQ = \frac{dSK}{\sqrt{d_k}}, \qquad dK = \frac{dS^\top Q}{\sqrt{d_k}}. \tag{2.358}$$

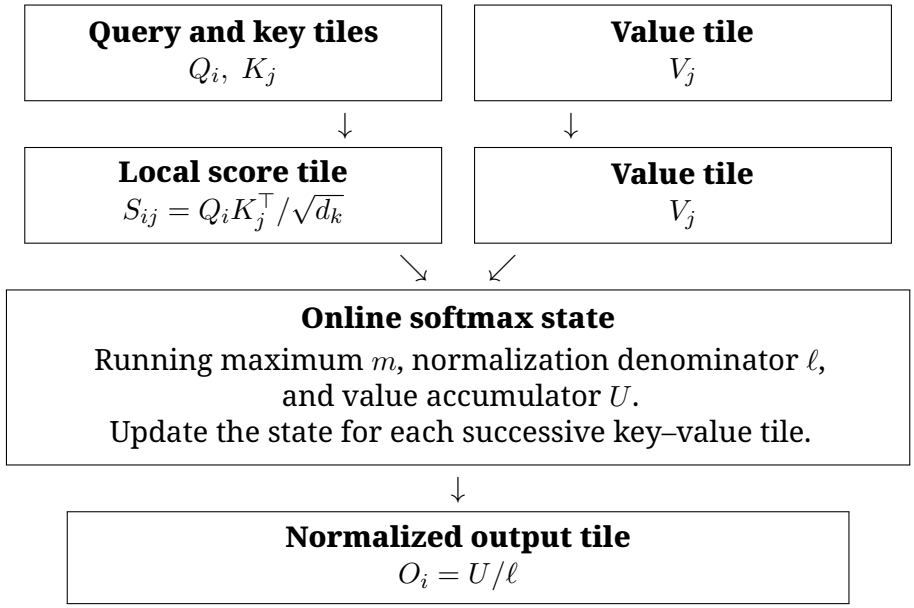


Figure 2.15: Conceptual blockwise attention computation. Query, key, and value tiles are loaded into on-chip memory; each score tile updates the running maximum, normalization denominator, and value accumulator. The full score and probability matrices are not materialized in HBM. Based on **dao2023flashattention2**.

The algorithm recomputes local blocks when required rather than retaining all $N \times N$ intermediates. The additional memory required by the attention operation is thereby reduced from quadratic to linear in sequence length, apart from the storage of the input and output tensors (**dao2023flashattention2**).

FlashAttention-2 and GPU work partitioning. FlashAttention-2 improves the work distribution of the original algorithm. The paper identifies three related changes. It reduces non-matrix-multiplication operations, parallelizes attention across sequence blocks even when a single head is being processed, and changes the division of work among warps within a thread block to reduce communication through shared memory.

The distinction between matrix and non-matrix operations matters because modern GPUs provide specialized matrix-multiplication hardware. The FlashAttention-2 paper gives an A100 example in which the reported theoretical throughput for FP16/BF16 matrix multiplication is 312 TFLOPs/s, compared with 19.5 TFLOPs/s for non-matrix FP32 operations. The ratio is

$$\frac{312}{19.5} = 16. \tag{2.359}$$

This comparison is specific to the hardware and precisions reported in the paper. It illustrates why a kernel that reduces non-matrix work and increases the fraction of time spent in optimized matrix operations can achieve better hardware utilization.

The original FlashAttention work distributes computation across batch and head dimensions. FlashAttention-2 additionally partitions work along the sequence dimension, increasing the number of independent thread blocks when batch size or head count is small. Within a thread block, it also changes the warp-level partitioning to reduce shared-

memory communication and synchronization. For causal attention, blocks entirely outside the permitted triangular region can be skipped rather than computed and discarded.

The paper reports approximately a twofold speedup over the original FlashAttention in its evaluated settings, with 50–73% of the theoretical maximum FLOPs/s on A100. In end-to-end GPT-style training, it reports up to 225 TFLOPs/s per A100 GPU and approximately 72% model FLOPs utilization. These are experimental results tied to the implementations and hardware in the paper; they should not be interpreted as fixed speedups for every model or device (**dao2023flashattention2**).

The central distinction is that FlashAttention-2 does not remove the quadratic arithmetic complexity of dense attention. It changes the execution schedule, intermediate storage, and work partitioning so that the same mathematical operation is performed with less data movement and better hardware utilization.

2.6.11 Linear attention

FlashAttention improves the execution of dense attention without changing its mathematical result. Linear attention takes a different approach: it changes the formulation so that the sequence-length dependence can be linear rather than quadratic. The supplied survey by **sun2026efficientattention** organizes linear attention into kernelized methods, recurrent methods with forgetting mechanisms, and methods that interpret the recurrent state as a fast-weight or in-context-learning mechanism.

Kernelized reformulation. Consider one query q_i and the sequence of keys and values. In the efficient-attention survey's notation, the softmax kernel contributes weights proportional to $\exp(q_i k_j^\top)$. Kernelized linear attention seeks a feature mapping

$$\phi : \mathbb{R}^{d_k} \longrightarrow \mathbb{R}^r \tag{2.360}$$

such that

$$\exp(q_i k_j^\top) \approx \phi(q_i)^\top \phi(k_j). \tag{2.361}$$

If scaled dot products are used, the scaling can be incorporated into the arguments of the feature map. The essential requirement is that the similarity kernel be approximated by an inner product in a feature space.

Substituting the feature approximation into normalized attention gives

$$o_i \approx \frac{\sum_{j=1}^{N} \phi(q_i)^\top \phi(k_j) v_j}{\sum_{j=1}^{N} \phi(q_i)^\top \phi(k_j)}. \tag{2.362}$$

Since $\phi(q_i)$ does not depend on j, associativity and distributivity allow it to be factored out:

$$o_i \approx \frac{\phi(q_i)^\top \left(\sum_{j=1}^{N} \phi(k_j) v_j^\top \right)}{\phi(q_i)^\top \left(\sum_{j=1}^{N} \phi(k_j) \right)}. \tag{2.363}$$

Define the aggregated key–value matrix and key sum as

$$A = \sum_{j=1}^{N} \phi(k_j) v_j^\top, \qquad A \in \mathbb{R}^{r \times d_v}, \tag{2.364}$$

$$z = \sum_{j=1}^{N} \phi(k_j), \qquad z \in \mathbb{R}^r. \tag{2.365}$$

Then

$$\boxed{o_i \approx \frac{\phi(q_i)^\top A}{\phi(q_i)^\top z}}. \tag{2.366}$$

The numerator and denominator are evaluated using the aggregated state. The full $N \times N$ attention matrix is not constructed.

For the complete sequence, let

$$\Phi_Q = \phi(Q) \in \mathbb{R}^{N \times r}, \qquad \Phi_K = \phi(K) \in \mathbb{R}^{N \times r}. \tag{2.367}$$

The output can be written as

$$\boxed{O = \frac{\Phi_Q(\Phi_K^\top V)}{\Phi_Q(\Phi_K^\top \mathbf{1}_N)}}. \tag{2.368}$$

Here, $\mathbf{1}_N$ is an N-dimensional vector of ones, and the division is row-wise, with the denominator broadcast over the value dimension. The matrix product $\Phi_K^\top V$ has shape $r \times d_v$, while $\Phi_Q(\Phi_K^\top V)$ has shape $N \times d_v$.

The first product costs $\mathcal{O}(Nrd_v)$, and the multiplication by Φ_Q costs another $\mathcal{O}(Nrd_v)$. The denominator costs $\mathcal{O}(Nr)$. The total is therefore

$$\mathcal{O}(Nrd_v + Nr), \tag{2.369}$$

which is linear in N when the feature and value dimensions are fixed. If $r = d_v = d_h$, the leading cost is $\mathcal{O}(Nd_h^2)$. The reduction in sequence-length complexity comes from changing the order of matrix multiplication and avoiding the $N \times N$ interaction matrix (**sun2026efficientattention**).

Feature-map choices and approximation. The feature map determines how well the linearized computation approximates the original softmax kernel. The survey describes Linear Transformer as using the positive feature map

$$\phi(x) = \mathrm{ELU}(x) + 1, \tag{2.370}$$

where ELU is the exponential linear unit. The positive outputs are compatible with the non-negative character of the exponential kernel and permit a normalization term.

Performer uses FAVOR+, a positive orthogonal random-feature method designed to provide an unbiased approximation of the softmax kernel. The survey also discusses Random Feature Attention and cosFormer as alternative feature-based constructions. These methods differ in how they represent or approximate the similarity function. Their computational advantage is obtained by replacing exact softmax attention with a feature-space approximation, so approximation quality and the feature dimension are part of the design problem (**sun2026efficientattention**).

Recurrent state formulation. The associative form can be evaluated incrementally. Define the state after token t as

$$A_t = \sum_{j=1}^{t} \phi(k_j) v_j^\top, \qquad z_t = \sum_{j=1}^{t} \phi(k_j). \tag{2.371}$$

The state updates are

$$A_t = A_{t-1} + \phi(k_t) v_t^\top, \tag{2.372}$$

$$z_t = z_{t-1} + \phi(k_t), \tag{2.373}$$

with $A_0 = 0$ and $z_0 = 0$. The current output is

$$o_t = \frac{\phi(q_t)^\top A_t}{\phi(q_t)^\top z_t}. \tag{2.374}$$

The accumulated state summarizes the preceding key–value information. The current query retrieves from that state without separately comparing it with every previously stored key.

The state has fixed dimensions $r \times d_v$ and r, so the update and storage per token do not grow with the number of preceding tokens, provided r and d_v are fixed. This gives linear-time sequence processing and constant-size recurrent memory per step. The recurrent form is useful for autoregressive decoding, while parallel or chunkwise forms can be used to expose more parallelism during training and prompt processing (**sun2026efficientattention**).

Forgetting mechanisms. Basic additive linear attention accumulates previous information without an explicit decay. The survey describes recurrent designs that introduce forgetting mechanisms so that the contribution of older information can decrease over time.

RetNet is presented as a data-independent-decay example. Its state recurrence is

$$S_t = \gamma S_{t-1} + k_t^\top v_t, \qquad 0 < \gamma < 1, \tag{2.375}$$

where γ is a learned decay coefficient, commonly associated with a retention head. The output is

$$o_t = q_t S_t. \tag{2.376}$$

Unrolling the recurrence gives

$$S_t = \sum_{n=1}^{t} \gamma^{t-n} k_n^\top v_n, \qquad o_t = \sum_{n=1}^{t} \gamma^{t-n} q_t k_n^\top v_n. \tag{2.377}$$

The contribution from token n is multiplied by γ^{t-n}. When γ is closer to one, information decays more slowly; a smaller γ produces faster forgetting. The decay is data-independent because the coefficient is not recalculated from the content of each incoming token.

Data-dependent approaches instead use a gate G_t determined by the current input:

$$S_t = G_t S_{t-1} + k_t^\top v_t. \tag{2.378}$$

A gate value near zero suppresses the corresponding part of the previous state, while a value near one preserves it. The survey discusses Mamba, Gated Linear Attention, xLSTM, GateLoop, HGRN, and related architectures as examples of input-dependent or gated recurrent state dynamics. The exact structure of G_t varies across architectures; it should not be assumed to be a single scalar in every model (**sun2026efficientattention**).

Linear attention as an in-context memory. The recurrent state can also be interpreted as an associative memory. The update $A_t = A_{t-1} + k_t^\top v_t$ writes a key–value association into a matrix-valued state, and $o_t = q_t A_t$ retrieves information from that state using the current query. The state is therefore both a compact summary of preceding tokens and a rapidly updated memory. The efficient-attention survey places fast-weight and test-time-regression approaches, including DeltaNet and related methods, within this broader line of work. Their state-update rules modify the simple additive association update to control how new information changes previously stored information (**sun2026efficientattention**).

Linear attention changes the computation rather than merely changing the implementation schedule. Its sequence-length complexity can be linear, but its output is generally an approximation or alternative formulation of softmax attention. The feature map, state dynamics, forgetting mechanism, and ability to retain detailed long-range information therefore affect its modeling behavior.

2.6.12 Sparse attention

Linear attention changes the kernel or recurrent formulation. Sparse attention instead retains softmax-style interactions but restricts the set of key positions available to each query. Its motivation is that many query–key interactions may be unnecessary for a useful output. The survey by **sun2026efficientattention** organizes sparse attention into fixed-pattern attention, block-sparse attention, clustering-based attention, and related bidirectional designs.

Sparse interaction sets. Let $\mathcal{S}(i)$ be the subset of key positions selected for query i. Sparse attention is

$$o_i = \sum_{j \in \mathcal{S}(i)} \alpha_{ij} v_j, \tag{2.379}$$

where

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{r \in \mathcal{S}(i)} \exp(s_{ir})}, \qquad s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}}. \tag{2.380}$$

The softmax denominator includes only the selected positions. If each query attends to at most k keys, the main interaction cost becomes

$$\mathcal{O}(Nkd_h) \tag{2.381}$$

for one head. When k is fixed as N grows, this cost is linear in sequence length. The quality of the result depends on whether the selected set preserves the information needed by the query.

Fixed-pattern sparse attention. A fixed-pattern method defines the permitted interactions before evaluating the input. Local-window attention is one example. In a causal window of width w, query position i attends to recent positions:

$$\mathcal{S}_{\text{local}}(i) = \{j : \max(1, i - w + 1) \le j \le i\}. \tag{2.382}$$

The interaction count is at most Nw, so the leading cost is $\mathcal{O}(Nwd_h)$. A fixed window is efficient when w is small, but a query cannot directly retrieve a token outside the window.

Dilated attention selects positions separated by a fixed interval. For dilation rate r, an illustrative causal set is

$$\mathcal{S}_{\text{dilated}}(i) = \{i, i - r, i - 2r, \ldots\} \cap \{1, \ldots, N\}. \tag{2.383}$$

This pattern expands the temporal or positional range without attending to every intervening token. The survey lists local-window and dilated designs among fixed sparse attention methods (**sun2026efficientattention**).

Block-sparse attention. Token-level sparsity can create irregular memory accesses. Block-sparse attention groups tokens into blocks and selects interactions between blocks. Let the block size be b, and let $T = \lceil N/b \rceil$ be the number of blocks. A block mask

$$M_{\text{block}} \in \{0, 1\}^{T \times T} \tag{2.384}$$

indicates which query blocks may interact with which key blocks. If each query block attends to at most k_b key blocks, then the number of token-level interactions is proportional to

$$\frac{N}{b} k_b b^2 = N k_b b.$$

The leading attention cost is therefore

$$\mathcal{O}(N k_b b d_h). \tag{2.385}$$

For fixed b and k_b, the dependence on N is linear.

Block structure allows hardware kernels to skip entire matrix-multiplication tiles. The efficient-attention survey distinguishes block-sparse methods for prompt prefill from methods designed for decoding. Prefill methods include approaches that select important blocks while processing a long input; decoding methods select blocks from the existing KV cache for each new query. The survey also discusses routing-based block sparsity, including methods that use pooled block representations or learned selection scores (**sun2026efficientattention**).

Clustering and retrieval-based attention. Clustering-based methods organize key representations into groups and retrieve selected groups rather than comparing every query with every key. Suppose the key vectors are partitioned into C clusters with centroids

$$\mu_c = \frac{1}{|\mathcal{C}_c|} \sum_{j \in \mathcal{C}_c} k_j, \qquad c = 1, \ldots, C. \tag{2.386}$$

A query can be compared with the centroids to identify promising clusters, after which the corresponding token-level keys and values are retrieved. The survey describes ClusterKV as using k-means clustering and RetrievalAttention as using approximate nearest-neighbor search. It also discusses Locality-Sensitive Hashing (LSH) as a way to map similar query and key vectors into common hash buckets (**sun2026efficientattention**).

The efficiency of retrieval-based attention depends on the quality of the index or clustering structure and on the cost of selecting candidate groups. If relevant keys are assigned to clusters that are not retrieved, the query can lose access to useful information. The retrieval stage must therefore be included when evaluating the complete computational cost.

Sparsity and hardware alignment. A sparse mathematical pattern does not automatically guarantee a proportional wall-clock speedup. Irregular accesses, selection overhead, small matrix tiles, and insufficient parallel work can reduce the benefit of sparsity. Block-sparse designs attempt to align the pattern with matrix-multiplication tiles so that skipped interactions correspond to operations the hardware can omit efficiently. The survey emphasizes that practical sparse attention must account for block sizes, grouping of key–value heads, and reuse of selected blocks (**sun2026efficientattention**).

The survey also identifies training sparse attention from scratch as a challenge. Restricted connectivity can lead to sparse gradient signals and less effective optimization. Training-aware or trainable sparse mechanisms attempt to learn useful selection patterns, but high sparsity can still harm model quality. Sparse attention is therefore a trade-off between omitted computation and the risk of excluding information needed for accurate prediction (**sun2026efficientattention**).

2.6.13 Hybrid attention architectures

Dense, compact, hardware-efficient, sparse, and linear attention address different constraints. Dense attention offers unrestricted token-to-token interaction. Compact methods reduce the amount of key–value state that must be retained. Hardware-efficient methods reduce data movement and improve the execution of the same mathematical operation. Sparse methods omit selected interactions, while linear methods reformulate the computation to avoid the full pairwise matrix. A hybrid architecture combines more than one of these mechanisms so that different layers or components can provide complementary capabilities.

The efficient-attention survey describes several broad hybrid patterns. A sparse–dense design interleaves dense attention layers with locally banded or otherwise sparse layers. Dense layers provide global interactions, while sparse layers reduce the number of token pairs processed in other parts of the network. The survey discusses GPT-3 as an example of interleaving dense and locally banded sparse attention layers (**sun2026efficientattention**).

A linear–full design uses linear or recurrent attention in many layers and inserts softmax-based full attention at selected depths. The survey discusses Jamba and MiniMax-01 as examples of this pattern. In the reported MiniMax-01 design, Lightning Attention is used across most layers, with full attention inserted periodically. The purpose is to retain efficient state-based processing while preserving opportunities for precise global softmax interactions (**sun2026efficientattention**).

A local–full design alternates sliding-window attention with occasional global attention layers. The survey discusses Gemma 3, Command A, and LLaMA-4 Maverick as examples of architectures using local and global attention components. Local layers emphasize nearby context, while global layers provide longer-range interactions. The frequency and positioning of global layers determine the balance between computation and global information exchange (**sun2026efficientattention**).

These examples demonstrate that efficient attention is not necessarily a choice between one universal mechanism and another. Different architectural components can be combined to distribute the requirements of local processing, global retrieval, recurrent memory, and exact softmax interactions across the network.

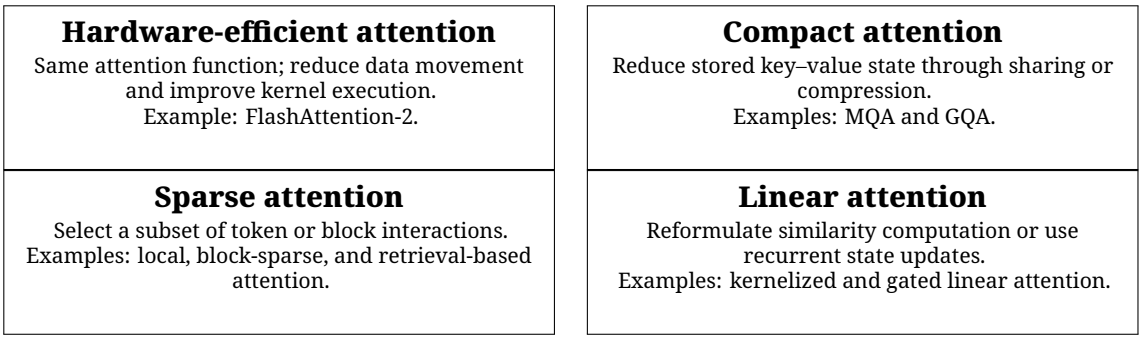


Figure 2.16: Conceptual taxonomy of efficient attention mechanisms. Hardware-efficient attention changes the execution schedule and memory traffic; compact attention reduces stored key–value state; sparse attention selects a subset of interactions; and linear attention changes the mathematical formulation to obtain subquadratic sequence-length dependence. Hybrid architectures combine these mechanisms. Based on **sun2026efficientattention**.

2.6.14 Limitations and connection to the next topic

Attention provides content-dependent information exchange in the Transformer. Its mathematical structure separates compatibility estimation from value aggregation, while multi-head attention provides several learned interaction spaces. Causal masking constrains information flow for autoregressive prediction, and backpropagation optimizes the projection spaces through the loss function.

The architectural and computational variants address different limitations and should not be treated as interchangeable. Multi-query and grouped-query attention reduce key–value cache storage and memory traffic, but retain dense query-to-key interactions. FlashAttention-2 preserves exact dense attention and its quadratic arithmetic dependence while reducing intermediate memory traffic and improving GPU work partitioning. Kernelized linear attention changes the similarity formulation and introduces approximation or alternative recurrent state dynamics. Sparse attention omits selected interactions and depends on the quality and hardware efficiency of its selection pattern. Hybrid architectures combine these mechanisms, but their benefits depend on the layer arrangement, sequence length, task, and hardware environment.

The central engineering lesson is that arithmetic complexity, memory capacity, memory bandwidth, approximation error, and hardware parallelism are distinct quantities. A reduction in one does not guarantee a reduction in all the others. Consequently, an attention architecture must be evaluated in relation to its mathematical formulation and its intended training or inference workload.

This completes the architectural account of how Transformer layers exchange contextual information. The next chapter turns from the architecture of the computation to the optimization procedures used to learn its parameters. Understanding attention's forward and backward computations provides the necessary basis for examining optimization algorithms, loss functions, and the numerical and computational techniques used to train large language models.

Part II

Optimization and Scaling

3. Optimization

3.1 Optimization Algorithms

The preceding discussion established the mathematical components from which a language model is constructed. Those components define a parameterized computational function, but they do not by themselves determine the numerical values of its trainable parameters. Training requires a procedure that uses observed data to modify those parameters so that the model's predictions become progressively more consistent with the training objective. This procedure is the subject of optimization. In a large language model (LLM), optimization is not merely a numerical routine appended to the neural architecture: it is a central part of the training system, because its update rule determines how information contained in the training data is translated into changes in the model.

The central problem is to minimize a loss function over a parameter space whose dimension may reach billions. The objective is generally nonconvex, gradients are usually estimated from minibatches rather than the complete training distribution, and the optimization process must operate within constraints imposed by accelerator memory, numerical precision, computation, and communication. These properties explain why the development of LLM optimizers involves more than selecting a learning rate. It requires decisions about which gradient information to retain, how to transform that information into parameter updates, what approximation to the local geometry is affordable, and how to evaluate progress under a fixed token or wall-clock budget (**Bottou2018Optimization**; **Ranganath2026Navigating**).

3.1.1 Optimization as a Statistical Learning Problem

Let $\boldsymbol{\theta} \in \mathbb{R}^d$ denote the complete vector of trainable parameters of a language model, where d is the number of scalar parameters. For an autoregressive language model and a token sequence $x_{1:T} = (x_1, \ldots, x_T)$, the joint probability is factorized into conditional next-token probabilities:

$$p_{\boldsymbol{\theta}}(x_{1:T}) = \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(x_t \mid x_{<t}), \tag{3.1}$$

where $x_{<t} = (x_1, \ldots, x_{t-1})$ is the context preceding token x_t. Training by next-token prediction minimizes the expected negative log-likelihood under the training distribution $\mathcal{D}$:

$$F(\boldsymbol{\theta}) = -\mathbb{E}_{x_{1:T} \sim \mathcal{D}} \left[\sum_{t=1}^{T} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}) \right]. \tag{3.2}$$

This objective is an instance of the expected-risk formulation used in statistical learning. More generally, let ξ denote a random training observation, which may be a single example or a minibatch, and let $f(\boldsymbol{\theta}; \xi)$ be the loss evaluated at that observation. The expected objective is

$$F(\boldsymbol{\theta}) = \mathbb{E}_{\xi \sim \mathcal{D}} \left[f(\boldsymbol{\theta}; \xi) \right]. \tag{3.3}$$

The distribution $\mathcal{D}$ is not known in complete form. Instead, the training procedure has access to a finite dataset $\{\xi_1, \ldots, \xi_N\}$. The corresponding empirical objective is

$$F_N(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} f(\boldsymbol{\theta}; \xi_i), \tag{3.4}$$

and the finite-data optimization problem is

$$\boldsymbol{\theta}_N^* \in \operatorname*{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} F_N(\boldsymbol{\theta}). \tag{3.5}$$

The expected risk and empirical risk represent different mathematical objects. The former measures expected loss under the underlying distribution; the latter is the average loss over the available observations. Minimizing the empirical objective is therefore a practical approximation to the statistical learning problem, not a guarantee that the expected risk has also been minimized. The distinction between expected and empirical risk is the starting point for the optimization analysis of large-scale machine learning (**Bottou2018Optimization**). The autoregressive formulation in equations (3.1) and (3.2) makes this general framework specific to language-model pretraining (**Ranganath2026Navigating**).

An optimization algorithm constructs a sequence of parameter vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots$ with the intention of reducing the objective. The most basic local information used by gradient-based methods is the gradient:

$$\nabla F(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial F}{\partial \theta_1} & \frac{\partial F}{\partial \theta_2} & \cdots & \frac{\partial F}{\partial \theta_d} \end{bmatrix}^{\mathsf{T}}. \tag{3.6}$$

The gradient belongs to the same d-dimensional vector space as the parameters. Its j-th component measures the local sensitivity of the objective to the j-th parameter. For a sufficiently small displacement $\Delta\boldsymbol{\theta}$, the first-order Taylor approximation gives

$$F(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx F(\boldsymbol{\theta}) + \nabla F(\boldsymbol{\theta})^{\mathsf{T}} \Delta\boldsymbol{\theta}. \tag{3.7}$$

Choosing the displacement in the negative-gradient direction,

$$\Delta\boldsymbol{\theta} = -\eta \nabla F(\boldsymbol{\theta}), \qquad \eta > 0, \tag{3.8}$$

yields the first-order change

$$F(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) - F(\boldsymbol{\theta}) \approx -\eta \|\nabla F(\boldsymbol{\theta})\|_2^2. \tag{3.9}$$

Thus, the negative gradient is a local descent direction whenever the gradient is nonzero. The scalar η is the step size or learning rate; it controls the magnitude of the displacement. This local argument motivates gradient descent, but it does not by itself

establish convergence. A convergence analysis requires assumptions about how rapidly the gradient can change.

A common assumption is that the gradient is Lipschitz continuous. A differentiable function has an L-Lipschitz gradient if

$$\|\nabla F(\boldsymbol{\theta}) - \nabla F(\boldsymbol{\theta}')\|_2 \leq L\|\boldsymbol{\theta} - \boldsymbol{\theta}'\|_2 \tag{3.10}$$

for all relevant parameter vectors. Under this assumption, the descent lemma states that

$$F(\boldsymbol{\theta}') \leq F(\boldsymbol{\theta}) + \nabla F(\boldsymbol{\theta})^{\mathsf{T}}(\boldsymbol{\theta}' - \boldsymbol{\theta}) + \frac{L}{2}\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|_2^2. \tag{3.11}$$

Substituting the gradient-descent update $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla F(\boldsymbol{\theta}_t)$ into equation (3.11) gives

$$\begin{aligned} F(\boldsymbol{\theta}_{t+1}) &\leq F(\boldsymbol{\theta}_t) - \eta_t \|\nabla F(\boldsymbol{\theta}_t)\|_2^2 + \frac{L\eta_t^2}{2}\|\nabla F(\boldsymbol{\theta}_t)\|_2^2 \\ &= F(\boldsymbol{\theta}_t) - \eta_t \left(1 - \frac{L\eta_t}{2}\right) \|\nabla F(\boldsymbol{\theta}_t)\|_2^2. \end{aligned} \tag{3.12}$$

For $0 < \eta_t \leq 1/L$, the coefficient multiplying the squared gradient is positive, and the bound guarantees a decrease in the objective whenever the gradient is nonzero. This explains why an excessively large learning rate can destabilize optimization: the quadratic remainder in the descent bound can offset the improvement predicted by the linear term. An excessively small learning rate, on the other hand, can make progress unnecessarily slow (**Bottou2018Optimization**).

The optimization landscape of a deep neural network is generally nonconvex. Consequently, a stationary point satisfying $\nabla F(\boldsymbol{\theta}) = \mathbf{0}$ need not be a global minimizer. It may be a local minimum, a saddle point, or another stationary point. Theoretical results for nonconvex stochastic optimization therefore often establish convergence toward regions with small gradient norm rather than convergence to a global optimum. This distinction is essential when interpreting guarantees for the training of Transformer language models (**Bottou2018Optimization**; **Ranganath2026Navigating**).

3.1.2 Stochastic and Batch Gradient Methods

Differentiating the empirical objective in equation (3.4) gives the full gradient

$$\nabla F_N(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \nabla f(\boldsymbol{\theta}; \xi_i). \tag{3.13}$$

A batch-gradient method evaluates this complete sum at every iteration:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla F_N(\boldsymbol{\theta}_t). \tag{3.14}$$

The direction is calculated from all training observations and is therefore the exact gradient of the empirical objective. Its disadvantage is computational: each update requires processing the complete dataset. When the dataset contains an enormous number of sequences and tokens, this requirement makes frequent full-gradient evaluations impractical.

Stochastic gradient (SG) methods replace the full gradient with an estimate computed from a randomly selected observation. If i_t is sampled uniformly from $\{1, \dots, N\}$, the stochastic gradient is

$$\mathbf{g}_t = \nabla f(\boldsymbol{\theta}_t; \xi_{i_t}), \tag{3.15}$$

and the update is

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t.} \tag{3.16}$$

Under uniform sampling, the estimator is conditionally unbiased:

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\theta}_t] = \nabla F_N(\boldsymbol{\theta}_t). \tag{3.17}$$

Unbiasedness does not mean that every sampled direction decreases the empirical objective. An individual stochastic gradient may point in a direction that increases the full objective. It means that the expected direction, conditioned on the current parameters, equals the full empirical gradient. The iterates are therefore a stochastic process whose trajectory depends on the sampled observations (**Bottou2018Optimization**).

The difference between the stochastic gradient and its conditional expectation is gradient noise. Its covariance matrix is

$$\boldsymbol{\Sigma}_t = \mathbb{E}\left[(\mathbf{g}_t - \nabla F_N)(\mathbf{g}_t - \nabla F_N)^\mathsf{T} \mid \boldsymbol{\theta}_t\right], \tag{3.18}$$

and its total variance is

$$\sigma_t^2 = \mathrm{Tr}(\boldsymbol{\Sigma}_t) = \mathbb{E}\left[\|\mathbf{g}_t - \nabla F_N\|_2^2 \mid \boldsymbol{\theta}_t\right]. \tag{3.19}$$

The diagonal entries of $\boldsymbol{\Sigma}_t$ describe coordinate-wise variances, while its off-diagonal entries describe correlations between gradient coordinates. In high-dimensional neural networks, these correlations need not vanish. The covariance therefore contains information about the statistical structure of gradient noise, not merely the magnitude of each individual coordinate.

For an unbiased estimator,

$$\mathbb{E}[\|\mathbf{g}_t\|_2^2 \mid \boldsymbol{\theta}_t] = \|\nabla F_N(\boldsymbol{\theta}_t)\|_2^2 + \sigma_t^2. \tag{3.20}$$

Combining this identity with the descent lemma gives, under the same smoothness assumptions,

$$\begin{aligned}\mathbb{E}[F_N(\boldsymbol{\theta}_{t+1}) \mid \boldsymbol{\theta}_t] \le F_N(\boldsymbol{\theta}_t) - \eta_t \left(1 - \frac{L\eta_t}{2}\right) \|\nabla F_N(\boldsymbol{\theta}_t)\|_2^2 \\ + \frac{L\eta_t^2}{2}\sigma_t^2.\end{aligned} \tag{3.21}$$

The negative term represents expected progress in the gradient direction; the positive term represents the adverse contribution of stochastic variance. This inequality makes explicit why learning-rate selection and noise control are central to stochastic optimization.

In large-scale neural-network training, gradients are commonly estimated using a minibatch. For a minibatch $\mathcal{B}_t$ containing B sampled observations,

$$\mathbf{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla f(\boldsymbol{\theta}_t; \xi_i). \tag{3.22}$$

When observations are sampled independently and have finite variance σ^2, the variance of the minibatch mean is

$$\mathrm{Var}(\mathbf{g}_t) = \frac{\sigma^2}{B}. \tag{3.23}$$

Increasing B therefore reduces the variance of the gradient estimate. However, it also increases the work performed to calculate one update. The reduction in noise must be considered together with the computational cost of processing a larger minibatch. Minibatches also provide an important systems advantage: they expose parallel computation and allow accelerators to process many training examples efficiently (**Bottou2018Optimization**; **Ranganath2026Navigating**).

For strongly convex objectives, stochastic-gradient methods with a constant step size generally approach a neighborhood of the minimizer when gradient noise remains nonzero. The neighborhood reflects a balance between contraction toward the optimum and the variance introduced by stochastic updates. A diminishing step-size sequence can reduce the asymptotic effect of this noise. A classical sufficient condition is

$$\sum_{t=1}^{\infty} \eta_t = \infty, \qquad \sum_{t=1}^{\infty} \eta_t^2 < \infty. \tag{3.24}$$

The first condition prevents the cumulative step sizes from becoming finite, while the second controls the accumulated effect of stochastic noise. Under the assumptions developed by Bottou, Curtis, and Nocedal, diminishing step sizes yield an $O(1/t)$ expected optimality-gap rate for strongly convex objectives (**Bottou2018Optimization**).

For nonconvex objectives, the relevant theoretical quantity is often the squared gradient norm. Under the assumptions of the stochastic-gradient analysis in **Bottou2018Optimization**, a fixed sufficiently small step size gives a bound of the general form

$$\frac{1}{K} \sum_{t=1}^{K} \mathbb{E}\left[\|\nabla F(\boldsymbol{\theta}_t)\|_2^2\right] \leq C_1 \eta + \frac{C_2}{K\eta}, \tag{3.25}$$

where C_1 depends on the noise and smoothness bounds and C_2 depends on the initial objective gap and the lower bound on the objective. The first term reflects the residual effect of stochastic noise; the second decreases with the number of iterations. This result does not guarantee a global minimum. It shows that the algorithm spends an increasing fraction of its iterations in regions where the objective has a relatively small gradient.

Noise-reduction methods. The variance in a stochastic direction can be reduced without abandoning the computational advantages of stochastic optimization. Dynamic sample-size methods increase the number of observations used to estimate the gradient as training progresses. They can use inexpensive estimates during early optimization and more accurate estimates when the algorithm approaches a stationary region.

Gradient-aggregation methods retain information from earlier iterations. For example, the stochastic variance-reduced gradient (SVRG) estimator uses a reference point $\widetilde{\boldsymbol{\theta}}$ and the full gradient at that reference point:

$$\mathbf{g}_t^{\text{SVRG}} = \nabla f_{i_t}(\boldsymbol{\theta}_t) - \nabla f_{i_t}(\widetilde{\boldsymbol{\theta}}) + \nabla F_N(\widetilde{\boldsymbol{\theta}}). \tag{3.26}$$

Taking the conditional expectation over i_t gives $\mathbb{E}[\mathbf{g}_t^{\text{SVRG}}] = \nabla F_N(\boldsymbol{\theta}_t)$. The correction term removes the sampled gradient contribution evaluated at the reference point and replaces it with the full reference gradient. When the current iterate is close to the reference point, this construction can substantially reduce variance. SAGA and stochastic average gradient (SAG) are related gradient-aggregation approaches that retain historical gradient information. Iterate averaging is another variance-reduction strategy; it forms an average of iterates rather than relying only on the most recent parameter vector:

$$\overline{\boldsymbol{\theta}}_T = \frac{1}{T}\sum_{t=1}^{T} \boldsymbol{\theta}_t. \tag{3.27}$$

These methods are important in the broader theory of large-scale optimization, although their memory and data-access requirements must be considered before applying them to very large language-model training (**Bottou2018Optimization**).

3.1.3 First-Order and Adaptive Optimization Algorithms

A first-order optimizer constructs its update primarily from gradient information rather than explicitly forming and inverting the Hessian. A general update can be written as

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{u}_t, \tag{3.28}$$

where $\mathbf{u}_t$ is a direction derived from the current gradient and, possibly, information retained from previous iterations. The algorithms in this family differ in how they construct $\mathbf{u}_t$ and how much additional state they store.

Stochastic gradient descent and momentum. The simplest first-order method uses the stochastic gradient directly:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t. \tag{3.29}$$

Its principal advantage is its small memory overhead: it does not require additional per-parameter optimizer state. Its limitation is that one scalar learning rate is applied to all coordinates. In an LLM, embeddings, attention projections, feed-forward matrices, and normalization parameters can have different gradient statistics, so a single global learning rate may not be equally appropriate for every parameter group (**Ranganath2026Navigating**).

Momentum introduces a velocity vector that accumulates gradient information:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \tag{3.30}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{v}_t. \tag{3.31}$$

Here, $\beta \in [0, 1)$ controls the influence of previous gradients. By smoothing short-term fluctuations, momentum can reduce oscillation and improve progress in poorly conditioned landscapes. It does not, however, provide coordinate-wise learning-rate adaptation (**Bottou2018Optimization**; **Ranganath2026Navigating**).

Adaptive diagonal preconditioning. Adaptive optimizers can be interpreted as diagonal preconditioned gradient methods. A diagonal preconditioner rescales each coordinate independently, without representing cross-coordinate correlations.

AdaGrad accumulates squared gradients:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t, \tag{3.32}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon}. \tag{3.33}$$

The symbol $\odot$ denotes element-wise multiplication, and division and square roots in vector expressions are also element-wise. Coordinates with large accumulated squared gradients receive smaller effective learning rates. This behavior is useful when gradient scales are heterogeneous or sparse. Because the accumulator grows monotonically, however, the effective learning rates can become very small over long training runs.

RMSProp replaces the cumulative sum with an exponential moving average:

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\mathbf{g}_t \odot \mathbf{g}_t), \tag{3.34}$$

and uses

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t} + \epsilon}. \tag{3.35}$$

The exponential average gives greater weight to recent gradient magnitudes, allowing the coordinate-wise scaling to adapt as the optimization dynamics change (**Ranganath2026Navigating**).

Adam. Adam combines momentum with adaptive second-moment scaling. It maintains a first-moment estimate and a second-moment estimate:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \tag{3.36}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\mathbf{g}_t \odot \mathbf{g}_t). \tag{3.37}$$

Because both estimates are initialized at zero, they are biased toward zero during early iterations. Adam corrects this initialization bias by using

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \tag{3.38}$$

$$\widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \tag{3.39}$$

The parameter update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}. \tag{3.40}$$

The first moment provides a smoothed update direction, while the second moment provides coordinate-wise scaling based on historical gradient magnitudes. Adam therefore combines two mechanisms that are separate in ordinary SGD: directional smoothing and adaptive effective learning rates (**Ranganath2026Navigating**).

AdamW and decoupled weight decay. Weight decay regularizes parameter magnitudes. In ordinary gradient descent, adding an ℓ_2 penalty $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ to the objective adds $\lambda\boldsymbol{\theta}$ to its gradient. The resulting update can be written as

$$\boldsymbol{\theta}_{t+1} = (1 - \eta_t\lambda)\boldsymbol{\theta}_t - \eta_t \nabla F(\boldsymbol{\theta}_t). \tag{3.41}$$

For an adaptive optimizer, including the penalty in the gradient causes the regularization term to be transformed by the adaptive preconditioner. Its effective strength then depends on the optimizer's coordinate-wise statistics. AdamW separates the decay operation from the adaptive gradient update:

$$\boxed{\boldsymbol{\theta}_{t+1} = (1 - \eta_t\lambda)\boldsymbol{\theta}_t - \eta_t \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}.} \tag{3.42}$$

Here, λ is the decoupled weight-decay coefficient. The separation allows the adaptive update and the regularization strength to be tuned more independently. AdamW is the principal baseline in the supplied LLM-optimizer survey and in the later systematic pretraining comparison (**Ranganath2026Navigating**; **Wen2026Fantastic**).

The optimizer update is used together with a learning-rate schedule. A common schedule begins with warmup and then decays the learning rate. For linear warmup,

$$\eta_t = \eta_{\max} \frac{t}{T_{\text{warmup}}}, \qquad 0 \le t \le T_{\text{warmup}}. \tag{3.43}$$

A cosine decay phase can then be written as

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left[1 + \cos\left(\pi \frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}\right)\right]. \tag{3.44}$$

Warmup is especially relevant to adaptive optimizers because their moment estimates are initially inaccurate. Large early updates can destabilize training in large Transformer models. Gradient clipping is another stability mechanism. For a global clipping threshold τ, the clipped gradient is

$$\widetilde{\mathbf{g}}_t = \mathbf{g}_t \min\left(1, \frac{\tau}{\|\mathbf{g}_t\|_2}\right). \tag{3.45}$$

When the gradient norm is below τ, the gradient is unchanged; when it exceeds τ, its norm is reduced to τ. In practice, AdamW training is also affected by the numerical stabilizer ϵ, the moment coefficients, parameter-specific weight-decay exclusions, and the precision used to store parameters and optimizer states (**Ranganath2026Navigating**).

3.1.4 Memory-Efficient and Alternative Update Rules

The two moment vectors maintained by AdamW require memory proportional to the number of model parameters. If each state element is stored in 32-bit floating-point precision, the two vectors require

$$M_{\text{states}} = 2d \times 4 = 8d \text{ bytes}. \tag{3.46}$$

For a model with one billion parameters, this is approximately 8 gigabytes for the first and second moments alone. This excludes model weights, gradients, activations, temporary buffers, and distributed-training overhead. A simplified mixed-precision accounting that includes low-precision parameters and gradients, 32-bit master parameters, and the two 32-bit Adam states can reach approximately $16d$ bytes (**Ranganath2026Navigating**).

Adafactor and factored second moments. Adafactor reduces the storage required for adaptive second-moment statistics by exploiting the matrix structure of a parameter tensor. For a matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, a full coordinate-wise second-moment accumulator contains mn entries. Adafactor instead maintains row and column statistics with m and n entries, respectively. In a factored approximation, the estimated second-moment matrix is reconstructed from these lower-dimensional statistics. The state storage therefore scales as $O(m + n)$ rather than $O(mn)$. This reduces memory, at the cost of replacing independent statistics for every matrix entry with a structured approximation (**Ranganath2026Navigating**).

Low-precision and grouped optimizer states. Low-bit optimizer methods reduce memory by representing optimizer statistics with fewer bits. Block-wise quantization partitions a state tensor into blocks and quantizes values relative to block-level scales. The resulting approximation error must be controlled because the moment estimates directly influence update magnitudes.

Adam-mini reduces the number of independent adaptive statistics by grouping parameters and sharing adaptive information within groups. Its motivation is that maintaining an independent learning-rate statistic for every parameter can be redundant. LOMO takes a different systems-oriented approach by fusing gradient computation and parameter updates, reducing the need to retain conventional gradient and optimizer state tensors simultaneously. These methods reduce different components of the memory and execution burden, and their trade-offs must be evaluated in the context of the training implementation (**Ranganath2026Navigating**).

Lion and sign-based updates. Lion replaces adaptive second-moment normalization with a sign-based update derived from a momentum-like quantity. A representative form is

$$\mathbf{u}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \tag{3.47}$$

$$\boldsymbol{\theta}_{t+1} = (1 - \eta_t \lambda)\boldsymbol{\theta}_t - \eta_t \operatorname{sign}(\mathbf{u}_t), \tag{3.48}$$

$$\mathbf{m}_t = \beta_2 \mathbf{m}_{t-1} + (1 - \beta_2)\mathbf{g}_t. \tag{3.49}$$

The sign function is applied element-wise. It returns -1, 0, or 1 according to the sign of its argument. Unlike AdamW, which rescales each coordinate according to a second-moment estimate, Lion primarily retains directional information in the update. It therefore removes the need for an Adam-style second-moment vector, but its effective update magnitude is controlled differently and requires its own learning-rate and weight-decay tuning (**Ranganath2026Navigating**; **Wen2026Fantastic**).

Large-batch optimization. Large minibatches can improve accelerator utilization and reduce gradient variance, but the optimization dynamics may change as the batch size increases. LAMB introduces layer-wise trust ratios to scale updates according to the relative norms of parameters and their proposed updates. For a layer with parameters $\mathbf{w}_\ell$ and an Adam-derived update direction $\mathbf{u}_\ell$, the trust ratio has the form

$$r_\ell = \frac{\|\mathbf{w}_\ell\|_2}{\|\mathbf{u}_\ell\|_2}, \tag{3.50}$$

with practical implementations including stabilizing details for small denominators and weight decay. The layer-wise scaling changes the relative magnitude of updates across layers and is intended to support large-batch training. It also illustrates that an optimizer's behavior depends on the interaction between its update rule, batch size, and parameter-group structure (**Ranganath2026Navigating**).

3.1.5 Curvature-Aware and Second-Order Optimization

The gradient describes the first-order sensitivity of the objective, but it does not fully describe the local geometry of the optimization landscape. For a twice-differentiable objective, the second-order Taylor approximation is

$$F(\boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}) \approx F(\boldsymbol{\theta}_t) + \mathbf{g}_t^\mathsf{T}\Delta\boldsymbol{\theta} + \frac{1}{2}\Delta\boldsymbol{\theta}^\mathsf{T}\mathbf{H}_t\Delta\boldsymbol{\theta}, \tag{3.51}$$

where $\mathbf{g}_t = \nabla F(\boldsymbol{\theta}_t)$ and

$$\mathbf{H}_t = \nabla^2_{\boldsymbol{\theta}} F(\boldsymbol{\theta}_t) \tag{3.52}$$

is the Hessian. The Hessian is a $d \times d$ matrix whose (i, j) entry is the second partial derivative $\partial^2 F/(\partial\theta_i \partial\theta_j)$. Its diagonal entries describe curvature along individual coordinates, while its off-diagonal entries describe interactions between parameter coordinates.

If the Hessian is positive definite, minimizing the quadratic approximation in equation (3.51) gives

$$\mathbf{H}_t\Delta\boldsymbol{\theta}_t = -\mathbf{g}_t, \qquad \Delta\boldsymbol{\theta}_t = -\mathbf{H}_t^{-1}\mathbf{g}_t. \tag{3.53}$$

This is the Newton step. The inverse Hessian rescales the gradient according to local curvature. To understand this effect, consider the eigendecomposition of a symmetric positive-definite Hessian:

$$\mathbf{H} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^\mathsf{T}, \qquad \boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_d). \tag{3.54}$$

The columns of $\mathbf{Q}$ are orthonormal curvature directions, and the eigenvalues describe curvature magnitudes along those directions. The condition number is

$$\kappa(\mathbf{H}) = \frac{\lambda_{\max}(\mathbf{H})}{\lambda_{\min}(\mathbf{H})}. \tag{3.55}$$

A large condition number indicates that curvature varies substantially across directions. Gradient descent must use a learning rate small enough for the steep directions,

which can make progress slow in shallow directions. Newton's method attempts to compensate by scaling each eigendirection inversely to its curvature.

Exact Newton optimization is infeasible for a model with billions of parameters. A Hessian with d^2 entries requires $O(d^2)$ storage, and constructing or factorizing it is also computationally prohibitive. For $d = 10^9$, the matrix contains 10^{18} entries; at four bytes per entry, its storage alone would be approximately 4×10^{18} bytes. Large-scale methods must therefore approximate or avoid explicit Hessian construction (**Bottou2018Optimization**; **Ranganath2026Navigating**).

Quasi-Newton and Hessian-free methods. Quasi-Newton methods approximate curvature using changes in parameters and gradients. Define

$$\mathbf{s}_t = \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t, \tag{3.56}$$

$$\mathbf{y}_t = \mathbf{g}_{t+1} - \mathbf{g}_t. \tag{3.57}$$

The pair $(\mathbf{s}_t, \mathbf{y}_t)$ contains information about how the gradient changes along the recent parameter displacement. BFGS uses such pairs to update an approximation of the inverse Hessian. Limited-memory BFGS (L-BFGS) retains only a finite history of these pairs rather than storing a complete dense matrix. Stochastic minibatches complicate these methods because gradient differences contain sampling noise, and line searches or synchronization can introduce additional cost.

Hessian-free methods avoid explicitly forming the Hessian and instead use iterative procedures to approximate a Newton-like direction. Bottou, Curtis, and Nocedal review both stochastic quasi-Newton and Hessian-free approaches as attempts to retain useful curvature information while controlling computational cost (**Bottou2018Optimization**).

Curvature estimation in language-model optimizers. Sophia is a curvature-aware optimizer discussed in the supplied LLM-optimizer survey. It combines momentum information with an estimate of diagonal curvature and clips the resulting curvature-normalized update. The curvature estimate is intended to provide information that is not identical to the historical squared-gradient statistics used by Adam. The clipping operation limits excessively large coordinate-wise updates. Sophia is an example of an approach that introduces approximate second-order information without constructing the complete Hessian (**Ranganath2026Navigating**).

3.1.6 Structured Matrix and Tensor Preconditioning: Shampoo

The previous discussion shows that a full Hessian or full-dimensional preconditioner is too expensive for large neural networks. Diagonal adaptive methods avoid this cost by treating coordinates independently, but this discards correlations between different entries of a parameter matrix. Transformer layers contain matrix-valued parameters, so a natural alternative is to retain matrix structure while approximating the full preconditioner. Shampoo, introduced by Gupta, Koren, and Singer, follows this approach by maintaining one preconditioning matrix for each tensor dimension (**Gupta2018Shampoo**).

Consider a matrix parameter $\mathbf{W} \in \mathbb{R}^{m\times n}$ with gradient $\mathbf{G}_t \in \mathbb{R}^{m\times n}$. A full preconditioner for the vectorized gradient would have dimension $mn \times mn$ and require $O(m^2n^2)$ storage. Shampoo instead maintains a left preconditioner $\mathbf{L}_t \in \mathbb{R}^{m\times m}$ and a right preconditioner $\mathbf{R}_t \in \mathbb{R}^{n\times n}$:

$$\mathbf{L}_t = \mathbf{L}_{t-1} + \mathbf{G}_t\mathbf{G}_t^\mathsf{T}, \tag{3.58}$$

$$\mathbf{R}_t = \mathbf{R}_{t-1} + \mathbf{G}_t^\mathsf{T}\mathbf{G}_t. \tag{3.59}$$

A positive regularization term is commonly used in the initialization, for example $\mathbf{L}_0 = \epsilon\mathbf{I}_m$ and $\mathbf{R}_0 = \epsilon\mathbf{I}_n$. The left matrix accumulates row-wise gradient correlations: its (i, j) entry is the inner product between rows i and j of the gradient. The right matrix similarly accumulates column-wise gradient correlations.

Both matrices are positive semidefinite because, for any $\mathbf{x} \in \mathbb{R}^m$,

$$\mathbf{x}^\mathsf{T}\mathbf{G}_t\mathbf{G}_t^\mathsf{T}\mathbf{x} = \|\mathbf{G}_t^\mathsf{T}\mathbf{x}\|_2^2 \geq 0, \tag{3.60}$$

and the same argument applies to $\mathbf{G}_t^\mathsf{T}\mathbf{G}_t$. Regularization makes the preconditioners positive definite, allowing inverse fractional matrix powers to be defined.

For a symmetric positive-definite matrix $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$, a fractional power is defined spectrally:

$$\mathbf{A}^\alpha = \mathbf{Q}\mathbf{\Lambda}^\alpha\mathbf{Q}^\mathsf{T}. \tag{3.61}$$

In particular, $\mathbf{A}^{-1/4} = \mathbf{Q}\,\mathrm{diag}(\lambda_i^{-1/4})\mathbf{Q}^\mathsf{T}$. This operation rotates into the eigenbasis, rescales each eigendirection, and rotates back. The original matrix form of Shampoo uses the update

$$\boxed{\mathbf{W}_{t+1} = \mathbf{W}_t - \eta_t\mathbf{L}_t^{-1/4}\mathbf{G}_t\mathbf{R}_t^{-1/4}.} \tag{3.62}$$

Left multiplication transforms the row geometry, and right multiplication transforms the column geometry. Unlike a diagonal optimizer, the transformation can mix information across coordinates within each matrix dimension.

The relationship to a full preconditioner can be seen through the Kronecker product. With a conventional column-major vectorization, the identity

$$\mathrm{vec}(\mathbf{AGB}) = (\mathbf{B}^\mathsf{T} \otimes \mathbf{A})\,\mathrm{vec}(\mathbf{G}) \tag{3.63}$$

implies that the Shampoo matrix update applies the structured preconditioner

$$\mathbf{R}_t^{-1/4} \otimes \mathbf{L}_t^{-1/4} \tag{3.64}$$

to the vectorized gradient. The Kronecker product is not explicitly constructed; its action is obtained by two matrix multiplications. The storage requirement is therefore $O(m^2+n^2)$ rather than $O(m^2n^2)$. Exact eigendecompositions or matrix-root computations nevertheless have costs that scale cubically with the individual matrix dimensions, which motivates practical approximations and infrequent updates (**Gupta2018Shampoo**; **Vyas2025SOAP**).

Shampoo generalizes from matrices to a tensor $\mathcal{G}_t \in \mathbb{R}^{n_1\times\cdots\times n_k}$ of order k. Let $\mathbf{G}_t^{(i)}$ denote the matricization of the tensor along mode i, with shape $n_i \times n_{-i}$, where $n_{-i} = \prod_{j\neq i} n_j$. The preconditioner for mode i is

$$\mathbf{H}_{t,i} = \mathbf{H}_{t-1,i} + \mathbf{G}_t^{(i)} \left(\mathbf{G}_t^{(i)}\right)^{\mathsf{T}}. \tag{3.65}$$

The gradient is transformed along each tensor mode:

$$\widetilde{\mathcal{G}}_t = \mathcal{G}_t \times_1 \mathbf{H}_{t,1}^{-1/(2k)} \times_2 \mathbf{H}_{t,2}^{-1/(2k)} \cdots \times_k \mathbf{H}_{t,k}^{-1/(2k)}, \tag{3.66}$$

where $\times_i$ denotes multiplication along mode i. When $k = 2$, the exponent becomes $-1/4$, recovering the matrix formulation. The corresponding parameter update is

$$\mathcal{W}_{t+1} = \mathcal{W}_t - \eta_t \widetilde{\mathcal{G}}_t. \tag{3.67}$$

The preconditioner storage is proportional to $O(\sum_{i=1}^{k} n_i^2)$, rather than the square of the total number of tensor entries. The method can use diagonal preconditioning for dimensions whose full matrix preconditioners would be too expensive. The approximation is structured: it retains correlations within each tensor dimension but does not represent arbitrary correlations between different parameter tensors (**Gupta2018Shampoo**).

The original Shampoo analysis is carried out in online convex optimization. For matrix gradients of rank at most r, Gupta, Koren, and Singer establish a regret bound of the form

$$\sum_{t=1}^{T} f_t(\mathbf{W}_t) - \sum_{t=1}^{T} f_t(\mathbf{W}^*) \le \sqrt{2r}\, D\, \operatorname{Tr}(\mathbf{L}_T^{1/4}) \operatorname{Tr}(\mathbf{R}_T^{1/4}), \tag{3.68}$$

where

$$\mathbf{L}_T = \epsilon \mathbf{I}_m + \sum_{t=1}^{T} \mathbf{G}_t \mathbf{G}_t^{\mathsf{T}}, \tag{3.69}$$

$$\mathbf{R}_T = \epsilon \mathbf{I}_n + \sum_{t=1}^{T} \mathbf{G}_t^{\mathsf{T}} \mathbf{G}_t, \tag{3.70}$$

$$D = \max_{t \in \{1,\dots,T\}} \|\mathbf{W}_t - \mathbf{W}^*\|_F. \tag{3.71}$$

Here, $\mathbf{W}^*$ is a comparator matrix and $\|\cdot\|_F$ denotes the Frobenius norm. Under the paper's assumptions, the regret has the optimal $O(\sqrt{T})$ dependence for online convex optimization. This is a convex online guarantee; it is not a proof of global convergence for nonconvex Transformer training (**Gupta2018Shampoo**).

The original algorithm's matrix-root computations can introduce substantial overhead if performed at every iteration. Practical implementations therefore use engineering choices such as periodic preconditioner-root updates, momentum, and diagonal approximations for very large dimensions. Shampoo's experiments include a language-model setting with an attention model of approximately 9.8 million parameters on the LM1B benchmark. These experiments demonstrate the potential of structured preconditioning, but they should not be interpreted as a direct guarantee of the same speedup at contemporary billion-parameter scales (**Gupta2018Shampoo**).

3.1.7 SOAP: Adam in the Shampoo Eigenbasis

Shampoo uses matrix-valued preconditioners, but recomputing their eigenvectors or matrix roots can be expensive. Computing these quantities less frequently reduces overhead, although the preconditioner basis then becomes stale between updates. SOAP, introduced by Vyas and colleagues, addresses this issue by running Adam in the preconditioner's eigenbasis and continuing to update the Adam second-moment statistics between basis refreshes (**Vyas2025SOAP**).

For a matrix-valued layer, let the batch gradient be $\mathbf{G}_t \in \mathbb{R}^{m\times n}$. Consider the expected left and right gradient covariance matrices

$$\mathbf{L} = \mathbb{E}_B[\mathbf{G}_B\mathbf{G}_B^\top], \tag{3.72}$$

$$\mathbf{R} = \mathbb{E}_B[\mathbf{G}_B^\top\mathbf{G}_B], \tag{3.73}$$

where the expectation is over batches. Let their eigendecompositions be

$$\mathbf{L} = \mathbf{Q}_L\mathbf{\Lambda}_L\mathbf{Q}_L^\top, \tag{3.74}$$

$$\mathbf{R} = \mathbf{Q}_R\mathbf{\Lambda}_R\mathbf{Q}_R^\top. \tag{3.75}$$

The rotated gradient is

$$\mathbf{G}'_t = \mathbf{Q}_L^\top\mathbf{G}_t\mathbf{Q}_R. \tag{3.76}$$

The SOAP paper first establishes a connection between a modified, idealized Shampoo algorithm using a power of $1/2$ and Adafactor operating in the eigenbasis supplied by Shampoo. The equivalence uses dataset-averaged covariance matrices and a scalar correction to the layer learning rate. This is distinct from the original Shampoo exponent of $1/4$. The connection motivates replacing Adafactor with Adam in the rotated coordinate system (**Vyas2025SOAP**).

In the practical SOAP algorithm, the first-moment state is maintained in the original parameter coordinates:

$$\mathbf{M}_t = \beta_1\mathbf{M}_{t-1} + (1-\beta_1)\mathbf{G}_t. \tag{3.77}$$

It is then rotated into the current preconditioner basis:

$$\mathbf{M}'_t = \mathbf{Q}_L^\top\mathbf{M}_t\mathbf{Q}_R. \tag{3.78}$$

The second-moment state is maintained in the rotated coordinates:

$$\mathbf{V}'_t = \beta_2\mathbf{V}'_{t-1} + (1-\beta_2)(\mathbf{G}'_t \odot \mathbf{G}'_t). \tag{3.79}$$

After the appropriate bias corrections, the Adam update in the rotated space is

$$\mathbf{N}'_t = \frac{\widehat{\mathbf{M}}'_t}{\sqrt{\widehat{\mathbf{V}}'_t} + \epsilon}. \tag{3.80}$$

The update is transformed back to the original coordinates:

$$\mathbf{N}_t = \mathbf{Q}_L \mathbf{N}'_t \mathbf{Q}_R^\top, \tag{3.81}$$

and the parameters are updated using the AdamW-style decoupled weight-decay operation:

$$\mathbf{W}_{t+1} = (1 - \eta_t \lambda)\mathbf{W}_t - \eta_t \mathbf{N}_t. \tag{3.82}$$

The left and right covariance estimates are updated online:

$$\mathbf{L}_t = \beta_2 \mathbf{L}_{t-1} + (1 - \beta_2)\mathbf{G}_t \mathbf{G}_t^\top, \tag{3.83}$$

$$\mathbf{R}_t = \beta_2 \mathbf{R}_{t-1} + (1 - \beta_2)\mathbf{G}_t^\top \mathbf{G}_t. \tag{3.84}$$

The eigenvector basis is refreshed every f iterations, where f is the preconditioning frequency. SOAP uses power iteration followed by QR decomposition to update eigenvector estimates. In a simplified power-iteration step, a positive-semidefinite preconditioner $\mathbf{P}$ and a current basis estimate $\mathbf{Q}$ produce

$$\mathbf{S} = \mathbf{P}\mathbf{Q}, \qquad \mathbf{Q}_{\text{new}} = \mathrm{QR}(\mathbf{S}), \tag{3.85}$$

where QR denotes the orthogonal factor from a QR decomposition. The first basis is initialized by an eigendecomposition. For one-dimensional parameters, SOAP uses ordinary AdamW. For very large dimensions, a rotation may be fixed to the identity to avoid prohibitive memory and computation; if both rotations of a matrix layer are identities, the update reduces to Adam-style optimization (**Vyas2025SOAP**).

Vyas and colleagues evaluate SOAP on language-model pretraining runs with approximately 360-million- and 660-million-parameter models. In their large-batch experiments, SOAP reduces the number of optimization iterations by more than 40% and wall-clock time by more than 35% relative to AdamW, and improves both measures by approximately 20% relative to Shampoo. These are results for the experimental configurations in that paper; they do not establish that the same improvements hold at all model scales, hardware configurations, or data-to-model ratios (**Vyas2025SOAP**).

3.1.8 Low-Rank and Matrix-Orthogonalized Optimizers

Shampoo and SOAP exploit matrix structure through covariance preconditioning. Other optimizers exploit different structural properties of neural-network gradients and parameter matrices. Low-rank methods reduce the dimension in which optimizer states are maintained, whereas matrix-orthogonalized methods transform a matrix-valued update according to its singular-value structure (**Ranganath2026Navigating**).

GaLore and low-rank projection. For a gradient matrix $\mathbf{G}_t \in \mathbb{R}^{m \times n}$, a low-rank approximation takes the form

$$\mathbf{G}_t \approx \mathbf{U}_t \mathbf{S}_t \mathbf{V}_t^\top, \qquad r \ll \min(m, n), \tag{3.86}$$

where $\mathbf{U}_t$ and $\mathbf{V}_t$ contain r basis vectors and $\mathbf{S}_t$ contains the corresponding reduced-dimensional coefficients. Projection-based optimizers maintain adaptive states in a low-dimensional subspace rather than in the full matrix space. For a right projection matrix $\mathbf{P}_t \in \mathbb{R}^{n \times r}$, the projected gradient is

$$\widetilde{\mathbf{G}}_t = \mathbf{G}_t \mathbf{P}_t. \tag{3.87}$$

An update computed in the projected space can be mapped back by multiplication with $\mathbf{P}_t^\top$. GaLore uses gradient-derived low-rank subspaces to reduce the memory required for adaptive optimizer states. It is important to distinguish this from parameter-efficient fine-tuning: low-rank optimizer projection can reduce optimizer-state memory while still updating the original model parameters, whereas parameter-efficient fine-tuning restricts the trainable parameterization or subset of parameters (**Ranganath2026Navigating**; **Vyas2025SOAP**).

Muon and matrix orthogonalization. Muon applies an approximate orthogonalization operation to a momentum-based matrix update. Let

$$\mathbf{M}_t = \beta \mathbf{M}_{t-1} + (1-\beta)\mathbf{G}_t \tag{3.88}$$

be a momentum matrix. The update is transformed by an approximate orthogonalization operator:

$$\mathbf{U}_t = \mathrm{NS}(\mathbf{M}_t), \qquad \mathbf{W}_{t+1} = \mathbf{W}_t - \eta_t \mathbf{U}_t, \tag{3.89}$$

where NS denotes a Newton–Schulz-based approximation to matrix orthogonalization. To understand the operation, consider the singular value decomposition

$$\mathbf{M} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top. \tag{3.90}$$

The corresponding polar orthogonal factor is $\mathbf{U}\mathbf{V}^\top$. It retains the left and right singular directions while normalizing the singular values. Direct singular value decomposition at every update can be expensive; Newton–Schulz iterations approximate the transformation using matrix multiplications. Their coefficients and normalization are chosen so that the singular values move toward the desired range. Muon therefore differs from Shampoo: Shampoo accumulates covariance statistics and constructs matrix preconditioners, whereas Muon orthogonalizes a momentum-derived matrix update (**Ranganath2026Navigating**; **Wen2026Fantastic**).

In practical training, matrix-based updates are applied to suitable dense weight matrices, while embeddings, normalization parameters, and other parameter groups may use different update rules. This parameter-group-specific treatment reflects the fact that not every tensor has the same shape or benefits from the same matrix operation (**Ranganath2026Navigating**).

3.1.9 Systems Constraints and Numerical Implementation

The mathematical update equation is only one component of an optimizer's practical cost. Large language models are commonly trained with mixed precision and distributed computation, and the optimizer must operate within the memory and communication limits of the complete training system.

Optimizer-state memory. For an AdamW-style mixed-precision configuration, a simplified accounting may include low-precision model parameters and gradients, 32-bit master parameters, and two 32-bit moment vectors. This gives an approximate upper accounting of

$$M_{\text{training}} \approx 2d + 2d + 4d + 4d + 4d = 16d \text{ bytes}. \tag{3.91}$$

The exact total depends on precision, framework implementation, parameter sharding, and whether master parameters are stored separately. The equation excludes activation memory and temporary buffers. Its purpose is to show why optimizer states become a major memory component as d grows (**Ranganath2026Navigating**).

Mixed precision and numerical stability. LLM training uses numerical formats such as fp16 and bf16 to reduce memory and increase throughput. Reduced precision changes the numerical behavior of optimization. Small gradients may underflow, large values may overflow, and moment estimates may lose precision. Loss scaling can mitigate underflow in fp16 training, while bf16's larger exponent range can reduce the need for explicit scaling. Optimizer states are often retained in fp32 even when model weights and activations use lower precision.

The stabilizer ϵ in Adam-like updates can materially affect the update when second-moment estimates are small. Gradient clipping limits rare large gradients, and the precision used for moment accumulation affects the accuracy of adaptive scaling. Matrix preconditioners introduce further numerical considerations because eigenvectors, matrix powers, and covariance estimates are sensitive to finite-precision arithmetic. Comparisons must therefore specify precision, clipping, loss scaling, and optimizer-state representation (**Ranganath2026Navigating**).

Distributed optimization and communication. In data-parallel training, each worker computes gradients on a local microbatch. The gradients must be synchronized, commonly through all-reduce, before replicas apply the same update. Gradient synchronization can become a communication bottleneck as model size and worker count increase. Gradient accumulation, overlapping communication with backpropagation, bucketed communication, mixed-precision communication, and optimizer-state sharding are system-level techniques used to manage this cost.

Optimizer-state sharding distributes auxiliary states such as Adam's first and second moments across workers. This differs from model sharding, which partitions parameters and may also partition gradients. Structured preconditioners and matrix-level optimizers can introduce additional statistics and synchronization requirements. Consequently, the cost of an optimizer cannot be inferred solely from the arithmetic in its update equation. It must be measured within the distributed layout and hardware configuration in which the model is trained (**Ranganath2026Navigating**).

3.1.10 Empirical Evaluation and Optimizer Selection

An optimizer should not be judged only by the rate at which its training loss decreases during the early part of a run. Two distinct efficiency measures are important. Token efficiency describes validation loss as a function of the number of training tokens:

$$\mathcal{L}_{\text{val}} = \mathcal{L}_{\text{val}}(N_{\text{tokens}}), \tag{3.92}$$

whereas wall-clock efficiency describes validation loss as a function of elapsed training time:

$$\mathcal{L}_{\text{val}} = \mathcal{L}_{\text{val}}(T_{\text{wall}}). \tag{3.93}$$

An optimizer can require fewer tokens to reach a target loss while incurring greater computation per update. Conversely, an optimizer with lower per-step overhead may need more tokens to reach the same loss. Memory usage, throughput, and downstream performance are also relevant. A meaningful comparison therefore considers the complete training budget rather than one isolated metric (**Ranganath2026Navigating**; **Wen2026Fantastic**).

Hyperparameter fairness is equally important. Learning rate, weight decay, warmup duration, momentum coefficients, clipping thresholds, and optimizer-specific parameters interact with the update rule. Applying identical hyperparameters to different optimizers does not ensure a fair comparison. Wen and colleagues show that Lion and AdamW, for example, can require substantially different optimal weight-decay coefficients; their reported comparison includes an optimum near 0.6 for Lion and near 0.1 for AdamW in the studied setting. A poorly tuned AdamW baseline can therefore exaggerate the apparent speedup of an alternative optimizer (**Wen2026Fantastic**).

Model scale and data-to-model ratio also affect optimizer rankings. Wen and colleagues evaluate models from approximately 0.1 billion to 1.2 billion parameters and training regimes corresponding to $1\times$, $2\times$, $4\times$, and $8\times$ the Chinchilla data-to-model ratio. Their results show that the speedup of matrix-based optimizers over a well-tuned AdamW baseline decreases with model size: the reported advantage falls from approximately $1.4\times$ for 0.1-billion-parameter models to approximately $1.1\times$ for 1.2-billion-parameter models in the reported $8\times$ regime. They also find that matrix-based optimizers consistently outperform scalar-based optimizers in their smaller model experiments, while the preferred optimizer can change as the data-to-model ratio increases. Muon performs strongly in smaller-ratio regimes, whereas SOAP and Kron become more competitive at larger ratios (**Wen2026Fantastic**).

Intermediate checkpoints can also produce misleading rankings. As learning rates decay, loss curves for different optimizers can cross, so an optimizer that appears better early in training may not be better at the target training budget. The comparison should therefore be made at the intended end-of-training budget, with hyperparameters tuned separately for each optimizer and model setting. These results do not invalidate the improvements reported in the SOAP experiments; they establish that such improvements must be interpreted in relation to scale, batch size, training horizon, data-to-model ratio, hardware, and tuning methodology (**Vyas2025SOAP**; **Wen2026Fantastic**).

The relationships summarized in figure 3.1 and figure 3.2 show that optimizer design is a choice about which information from the gradient history and parameter geometry is retained, and how that information is used to construct an update. The empirical results show why no single optimizer can be declared universally superior from one experiment. The appropriate choice depends on the model scale, the data budget, the hardware and distributed-training layout, the memory budget, and the quality of hyperparameter tuning (**Ranganath2026Navigating**; **Wen2026Fantastic**).

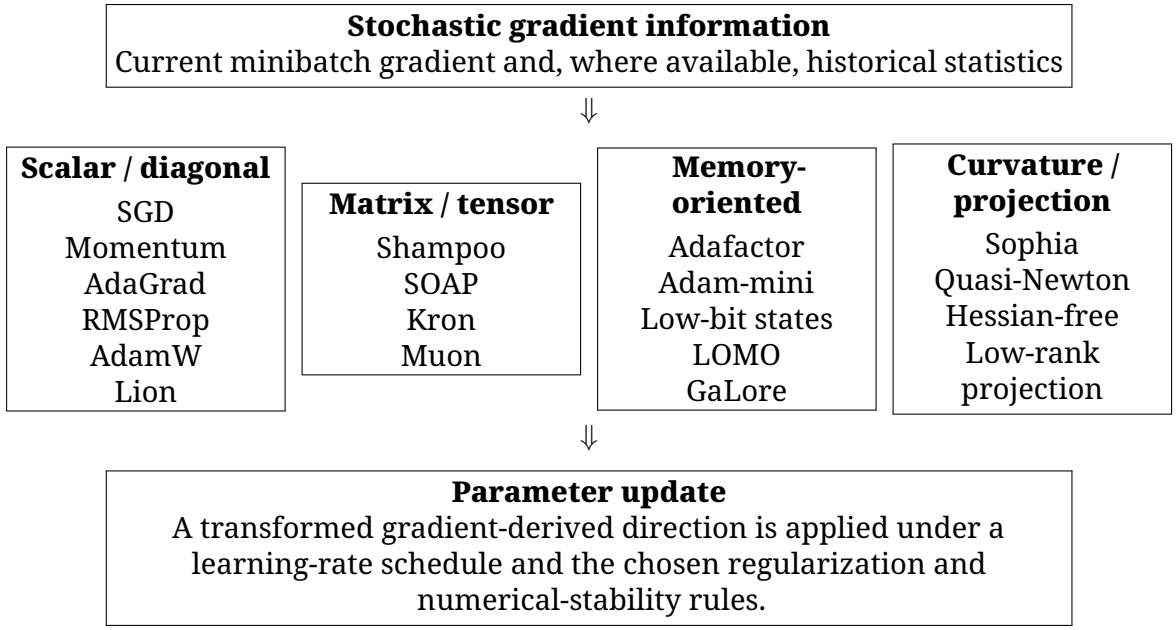


Figure 3.1: A conceptual taxonomy of optimizer families discussed in this section. The categories distinguish coordinate-wise adaptive updates, structured matrix preconditioning, memory-oriented methods, and curvature or projection methods. They are not mutually exclusive: an optimizer can combine more than one design principle.

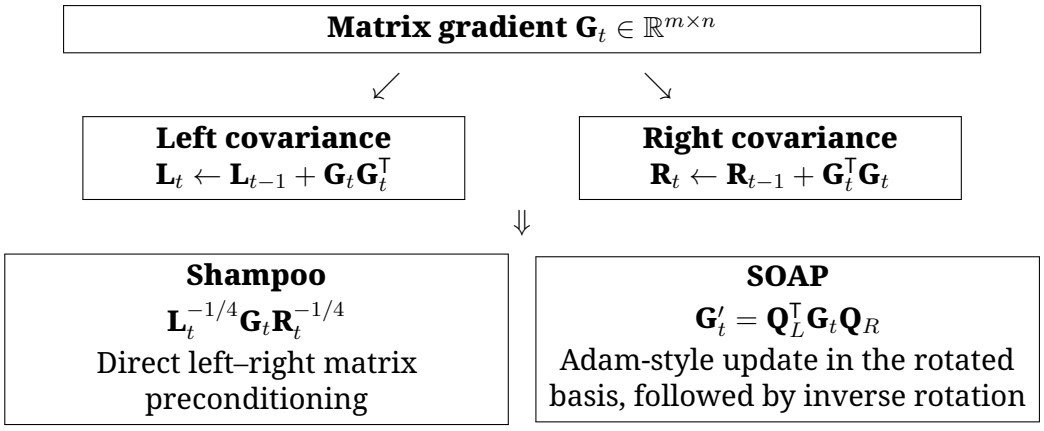


Figure 3.2: Conceptual comparison of Shampoo and SOAP for a matrix-valued parameter. Shampoo applies left and right covariance-derived matrix preconditioners. SOAP rotates the gradient into their eigenbasis, applies Adam-style adaptive updates in that basis, and rotates the update back to the original coordinates.

3.1.11 Limitations and Connection to the Next Topic

The progression from stochastic gradient descent to adaptive, memory-efficient, curvature-aware, and matrix-based optimizers reflects a progression in the information used to construct parameter updates. SGD uses the current stochastic gradient. Momentum retains a smoothed history of gradients. AdamW combines first- and second-moment estimates with decoupled weight decay. Memory-efficient methods reduce the cost of storing optimizer statistics, while curvature-aware and structured methods attempt to represent additional information about the optimization landscape.

Shampoo demonstrates how matrix and tensor structure can be retained without explicitly constructing a full-dimensional preconditioner. SOAP combines this structured coordinate system with Adam-style adaptive statistics. GaLore uses low-rank subspaces to reduce optimizer-state memory, and Muon applies matrix orthogonalization to momentum-derived updates. These methods address different limitations and should be compared using both mathematical and system-level criteria (**Gupta2018Shampoo**; **Vyas2025SOAP**; **Ranganath2026Navigating**).

The theoretical guarantees available for these algorithms also have different scopes. Stochastic-gradient convergence results depend on assumptions such as smoothness, convexity or nonconvex stationarity conditions, and bounded stochastic variance. Shampoo's regret guarantee is established in a stochastic convex online setting. Empirical improvements on nonconvex language-model pretraining tasks are important, but they do not replace these assumptions with a general proof of global optimality. Similarly, empirical speedups measured at one scale or data budget do not establish the same speedup at another scale (**Bottou2018Optimization**; **Gupta2018Shampoo**; **Vyas2025SOAP**; **Wen2026Fantastic**).

The central conclusion is that an optimizer is not simply a rule for subtracting a scaled gradient. It is a numerical procedure for deciding which statistical and geometric properties of the gradient should influence the next parameter update, subject to the memory, precision, computation, and communication constraints of the training system. Having established how the parameters are updated, the next discussion can turn to the objective functions that define what the optimization process is attempting to achieve. The distinction between the objective and the algorithm that minimizes it is necessary for understanding the complete training process of language models.

3.2 Loss Functions

3.3 Quantization

The preceding sections established how an objective function defines the learning problem and how an optimization algorithm uses gradients to update the parameters of a neural network. These operations explain how a language model learns its parameters, but they do not by themselves explain whether the resulting model can be stored and executed within a practical computational budget. As the number of parameters and the length of processed sequences increase, the numerical precision used to represent weights and intermediate tensors becomes an important part of the model's computational design.

Quantization addresses this problem by representing numerical quantities with fewer bits. It can reduce parameter storage, the memory required for intermediate representations, and, when supported by the hardware and kernels, the cost of arithmetic operations. The reduction in precision, however, also reduces the number of representable values and therefore introduces approximation error. The central problem is consequently not merely to compress a model, but to choose numerical representations that reduce computational requirements while preserving the information needed for accurate prediction and stable optimization.

This distinction connects quantization directly to the optimization process. A pretrained model can be quantized after its training has finished, or quantization can be simulated during training so that the model parameters adapt to the numerical constraints that will be present at inference time. The first strategy is generally described as post-training quantization (PTQ), whereas the second is quantization-aware training (QAT). The surveys by **chen2026llmquantization** and **song2026targetcentricqat** provide complementary foundations for this discussion: the former organizes large language model quantization across pretraining, fine-tuning, and inference, while the latter analyzes QAT according to the particular tensors being quantized.

3.3.1 Numerical representation and the need for quantization

A neural network performs its computations using numerical representations of model parameters and intermediate quantities. Consider a linear transformation

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}, \tag{3.94}$$

where $\mathbf{W} \in \mathbb{R}^{m\times n}$ is a weight matrix, $\mathbf{x} \in \mathbb{R}^n$ is an input vector, and $\mathbf{b} \in \mathbb{R}^m$ is a bias vector. The output belongs to $\mathbb{R}^m$. In a Transformer, the same principle appears in the linear projections used by attention and feed-forward sublayers, as well as in the transformations applied to intermediate representations.

In conventional high-precision computation, tensor elements may be represented using formats such as FP32 or FP16. Lower-precision formats include FP8 and FP4, while integer representations include INT8 and INT4. The bit-width b determines the maximum number of distinct binary codewords that can represent a scalar:

$$N_{\text{levels}} = 2^b. \tag{3.95}$$

For an unsigned integer representation, the range is $0, \dots, 2^b - 1$. A common signed two's-complement range is

$$q_{\text{min}} = -2^{b-1}, \qquad q_{\text{max}} = 2^{b-1} - 1. \tag{3.96}$$

A smaller bit-width therefore provides fewer representable values. The result is a more compact numerical representation, but also a greater risk that distinct real values will be mapped to the same discrete level.

The storage implications can be derived directly. Let a model contain N parameters, each represented using b bits. Ignoring scale factors, zero-points, alignment, and other metadata, the parameter storage is

$$M_b = \frac{Nb}{8} \text{ bytes}. \tag{3.97}$$

FP16 storage requires $2N$ bytes, whereas INT4 storage requires $N/2$ bytes. The idealized compression ratio relative to FP16 is therefore

$$C_{\text{FP16}\to b} = \frac{16}{b}. \tag{3.98}$$

For a model with 10^9 parameters, FP16 weights require approximately 2 GB when decimal gigabytes are used, while INT4 weights require approximately 0.5 GB before quantization metadata and implementation overheads are included. This is an idealized weight-storage calculation, not a prediction of the total memory required by a training or inference system. The actual memory footprint also depends on activations, gradients, optimizer states, cached representations, and the storage of quantization parameters (**chen2026llmquantization**).

The distinction between parameter storage and total execution memory is important for large language models. During training, weights are only one part of the memory requirement: activations must be retained or recomputed for backpropagation, gradients must be represented, and optimizers may maintain additional state variables. During autoregressive inference, the key-value (KV) cache grows with the number of retained tokens. Quantizing only the weights therefore does not necessarily resolve every memory bottleneck. This motivates treating quantization according to both the numerical representation and the computational role of the quantity being represented.

3.3.2 Uniform affine quantization

A fundamental formulation used in neural-network quantization is uniform affine quantization. It maps a real-valued tensor to a discrete integer tensor using equally spaced quantization levels. Let

$$\mathbf{x} \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_k} \tag{3.99}$$

be a real-valued tensor. Its integer representation is denoted by $\mathbf{x}_{\text{int}}$. Uniform affine quantization is determined by a positive step size s, an integer zero-point z, and the representable integer interval $[q_{\min}, q_{\max}]$. The step size sets the distance between adjacent reconstruction levels. The zero-point determines which integer code represents the real value zero.

For a scalar x, the quantization operation is

$$x_{\text{int}} = \text{clamp}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \tag{3.100}$$

where $\text{round}(\cdot)$ denotes rounding to the nearest integer and $\text{clamp}(\cdot)$ restricts its argument to the valid integer interval. In explicit piecewise form,

$$\text{clamp}(u, q_{\min}, q_{\max}) = \begin{cases} q_{\min}, & u < q_{\min}, \\ u, & q_{\min} \le u \le q_{\max}, \\ q_{\max}, & u > q_{\max}. \end{cases} \tag{3.101}$$

The integer code is converted back to an approximation in the real domain through dequantization:

$$\widehat{x} = s(x_{\text{int}} - z). \tag{3.102}$$

Combining the two operations gives the quantize–dequantize mapping

$$\boxed{Q_{s,z}(x) = s\left[\text{clamp}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\text{min}}, q_{\text{max}}\right) - z\right].} \tag{3.103}$$

The output $\widehat{x} = Q_{s,z}(x)$ is the reconstructed value used to approximate the original real-valued quantity. The corresponding quantization error is

$$\epsilon = x - \widehat{x}. \tag{3.104}$$

The mapping and its constituent operations are summarized in figure 3.3. This quantize–dequantize form is especially important in QAT, where the forward computation simulates the effect of low-precision representation while the training implementation may continue to use floating-point arithmetic (**song2026targetcentricqat**; **chen2026llmquantization**).

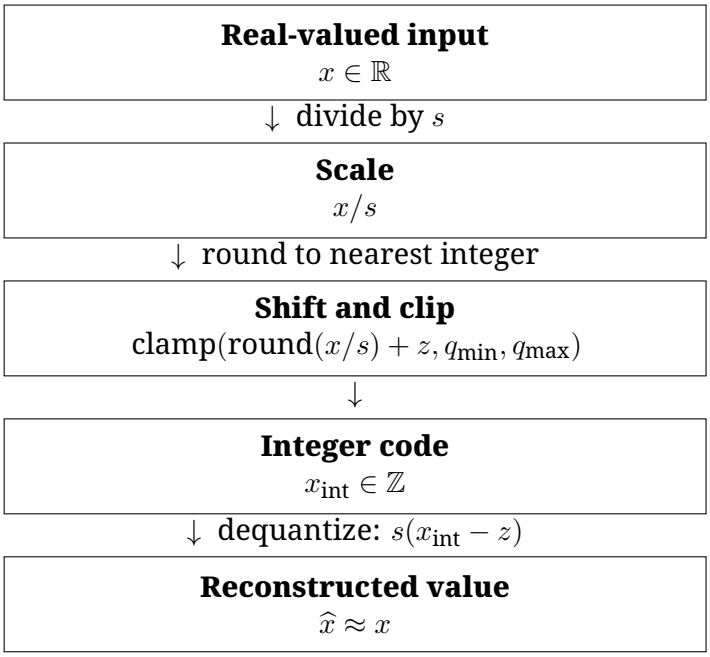


Figure 3.3: Uniform affine quantization and dequantization. A real-valued input is scaled, rounded, shifted by the zero-point, and clipped to the integer range. Dequantization maps the integer code back to an approximation in the real domain.

Symmetric quantization. In symmetric quantization, the zero-point is fixed to zero:

$$z = 0. \tag{3.105}$$

The quantization and dequantization operations consequently simplify to

$$x_{\text{int}} = \text{clamp}\left(\text{round}\left(\frac{x}{s}\right), q_{\text{min}}, q_{\text{max}}\right), \tag{3.106}$$

$$\widehat{x} = s x_{\text{int}}. \tag{3.107}$$

For a symmetric real-valued clipping interval, define

$$\alpha = \max\{|x_{\min}|, |x_{\max}|\}. \tag{3.108}$$

A common symmetric step size is

$$s = \frac{\alpha}{q_{\max}}, \tag{3.109}$$

where $q_{\max} > 0$. In a symmetric signed quantizer, the negative integer endpoint may be restricted to $-q_{\max}$, even when the underlying signed integer data type has one additional negative code. This convention keeps the quantization interval symmetric around zero. Symmetric quantization is commonly used for weight tensors because the zero-point-free formulation simplifies integer arithmetic (**song2026targetcentricqat**; **chen2026llmquantization**).

Asymmetric quantization. Asymmetric quantization permits a nonzero zero-point and can therefore represent a real-valued interval that is not centered around zero. Given a selected clipping interval $[x_{\min}, x_{\max}]$ and integer interval $[q_{\min}, q_{\max}]$, the step size is

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}. \tag{3.110}$$

The zero-point is chosen so that real zero is represented as accurately as possible:

$$z = \text{round}\left(q_{\min} - \frac{x_{\min}}{s}\right), \tag{3.111}$$

and is clipped to the valid integer range when necessary. Asymmetric quantization can be useful for activation distributions that are shifted or predominantly nonnegative. The choice between symmetric and asymmetric quantization is therefore related to the statistical distribution of the tensor and to the arithmetic supported by the target implementation (**song2026targetcentricqat**; **chen2026llmquantization**).

Quantization granularity. The number of tensor elements that share the same quantization parameters determines the quantization granularity. Per-tensor quantization uses one scale and zero-point for an entire tensor. Per-channel quantization assigns separate parameters to channels, which allows different channel distributions to be represented using different ranges. Per-group quantization partitions a tensor into smaller groups and assigns quantization parameters to each group. Per-token quantization applies parameters separately to token representations and is relevant when activation or KV-cache distributions vary across token positions. Per-layer terminology is also used when quantization parameters are shared at the layer level; the exact distinction between per-layer and per-tensor terminology depends on which tensors are included in the implementation.

Finer granularity can reduce distortion because the scale and zero-point adapt to more local distributions. It also requires additional metadata and can increase indexing, memory, and computational overhead. Coarser granularity reduces this overhead but may force numerically heterogeneous values to share a quantization range. Group size, tensor layout, and the availability of efficient kernels must therefore be considered together (**song2026targetcentricqat**; **chen2026llmquantization**).

3.3.3 Quantization error and its propagation through neural networks

Quantization error has two principal sources. Rounding error occurs because a real-valued quantity within the representable interval must be mapped to one of the discrete quantization levels. Clipping error occurs when the quantity lies outside the representable interval and must be saturated at a boundary. With a consistent sign convention, the total error can be decomposed into rounding and clipping contributions:

$$\epsilon = \epsilon_{\text{round}} + \epsilon_{\text{clip}}. \tag{3.112}$$

For a value that is not clipped, nearest-level rounding gives the absolute-error bound

$$|\epsilon_{\text{round}}| \leq \frac{s}{2}. \tag{3.113}$$

This bound does not apply to clipped values, whose error can be much larger than half a quantization step.

A numerical example makes the distinction explicit. Consider a symmetric INT4 quantizer with the symmetric integer interval $[-7, 7]$ and real clipping interval $[-1, 1]$. The step size is

$$s = \frac{1}{7} \approx 0.142857. \tag{3.114}$$

For $x = 0.42$, the integer code is

$$x_{\text{int}} = \text{round}(0.42/0.142857) = \text{round}(2.94) = 3. \tag{3.115}$$

The reconstructed value is approximately

$$\widehat{x} = 3(0.142857) = 0.428571, \tag{3.116}$$

so the signed error is

$$\epsilon = 0.42 - 0.428571 \approx -0.008571. \tag{3.117}$$

This is a rounding error. For $x = 1.2$, the unbounded rounded code would be 8, but the largest permitted code is 7. The value is therefore clipped, giving

$$x_{\text{int}} = 7, \qquad \widehat{x} = 1, \qquad \epsilon = 1.2 - 1 = 0.2. \tag{3.118}$$

The example illustrates the trade-off in selecting a clipping range: expanding the range reduces clipping but increases the spacing between quantization levels, whereas narrowing the range improves resolution for values near zero but can clip more extreme values (**song2026targetcentricqat**; **chen2026llmquantization**).

Quantization errors also propagate through the network. Consider the linear transformation in equation (3.94). Let the quantized weight matrix and input vector be

$$\widehat{\mathbf{W}} = \mathbf{W} + \Delta\mathbf{W}, \tag{3.119}$$

$$\widehat{\mathbf{x}} = \mathbf{x} + \Delta\mathbf{x}, \tag{3.120}$$

where $\Delta\mathbf{W}$ and $\Delta\mathbf{x}$ are their quantization perturbations. The quantized output is

$$\widehat{\mathbf{y}} = (\mathbf{W} + \Delta\mathbf{W})(\mathbf{x} + \Delta\mathbf{x}) + \mathbf{b}. \tag{3.121}$$

Expanding the product and subtracting the original output gives

$$\boxed{\Delta \mathbf{y} = \Delta \mathbf{W}\mathbf{x} + \mathbf{W}\Delta \mathbf{x} + \Delta \mathbf{W}\Delta \mathbf{x}.} \tag{3.122}$$

If only weights are quantized, then $\Delta \mathbf{x} = \mathbf{0}$, and

$$\Delta \mathbf{y} = \Delta \mathbf{W}\mathbf{x}. \tag{3.123}$$

If only activations are quantized, then $\Delta \mathbf{W} = \mathbf{0}$, and

$$\Delta \mathbf{y} = \mathbf{W}\Delta \mathbf{x}. \tag{3.124}$$

When both are quantized, the interaction term $\Delta \mathbf{W}\Delta \mathbf{x}$ also contributes. This is one reason joint weight–activation quantization is more complicated than weight-only quantization.

In a deep network, perturbations can pass through successive linear transformations, nonlinear activation functions, attention operations, normalization, and residual connections. The error at a later layer therefore depends on both the local approximation and the transformations applied to the perturbed representation. Weight errors are relatively static after offline quantization, whereas activation errors depend on the input and runtime tensor distributions. The LLM quantization survey describes this distinction as static weight error and dynamic activation error, with activation errors potentially accumulating across layers (**chen2026llmquantization**). The QAT survey further emphasizes that error characteristics depend on the quantization target and its lifetime, reuse frequency, and deployment constraints (**song2026targetcentricqat**).

3.3.4 Quantization as part of the optimization process

Post-training quantization. Post-training quantization applies low-precision representations to a model after high-precision training has been completed. The pretrained parameters are analyzed, quantization parameters are selected, and the model tensors are mapped to their lower-precision representations. Calibration data can be used to estimate activation ranges, while reconstruction objectives or sensitivity information can be used to choose quantization parameters. PTQ avoids repeating the original pretraining process and is therefore attractive when training the model again would be prohibitively expensive.

Its principal limitation is that the original model was optimized without explicitly accounting for the approximation errors introduced by quantization. A low-bit representation can consequently alter sensitive weights or activations and degrade the model's performance. This issue is especially important when activation distributions contain outliers: a small number of large values can expand the quantization range and leave fewer effective levels for the majority of values. PTQ methods therefore differ in how they estimate ranges, reconstruct weights, use calibration data, and account for sensitivity (**chen2026llmquantization**).

Quantization-aware training. Quantization-aware training incorporates quantization effects into the training or fine-tuning computation. Instead of optimizing a model

only under high-precision forward computation, QAT inserts fake-quantization operations so that the forward pass reflects the approximation that will be present in the deployed model. If a layer is

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b}), \tag{3.125}$$

a QAT forward pass can use

$$\widehat{\mathbf{y}} = f\left(Q(\mathbf{W})Q(\mathbf{x}) + \mathbf{b}\right), \tag{3.126}$$

where $Q(\cdot)$ denotes the quantize–dequantize operator and only the selected tensors are quantized. The forward computation is commonly performed in floating-point arithmetic while the low-precision effects are emulated by the fake quantizer.

The underlying trainable parameters can remain in full precision as master weights. Their fake-quantized versions are used in the forward pass, while the optimizer updates the master parameters. A basic gradient-descent update is

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \nabla_{\mathbf{W}} \mathcal{L}, \tag{3.127}$$

where η is the learning rate and $\mathcal{L}$ is the training loss. The updated master weights are quantized again during the next forward pass. Maintaining continuous master weights preserves small parameter updates that could be lost if optimization were performed directly on a coarse discrete grid. The distinction between the full-precision parameter state and its fake-quantized forward proxy is central to common QAT formulations (**song2026targetcentricqat**).

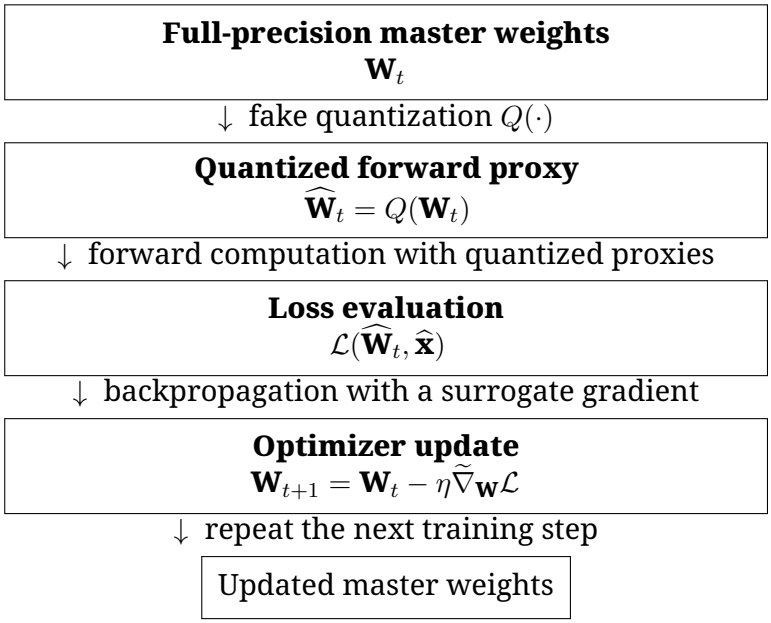


Figure 3.4: Simplified quantization-aware training loop. Master weights are fake-quantized for the forward computation; the loss is differentiated using a surrogate gradient, and the optimizer updates the underlying master weights.

The straight-through estimator. The central mathematical difficulty in QAT is that rounding is non-differentiable. The function

$$q(x) = \text{round}(x) \tag{3.128}$$

is piecewise constant. Its derivative is zero almost everywhere and undefined at rounding thresholds. Direct differentiation through the rounding operation would therefore fail to provide useful gradients for updating the underlying continuous parameters.

The straight-through estimator (STE) replaces the true derivative with a surrogate derivative. For a quantized scalar with clipping interval $[x_{\text{min}}, x_{\text{max}}]$, a common clipped STE approximation is

$$\frac{\partial \widehat{x}}{\partial x} \approx \begin{cases} 1, & x_{\text{min}} \leq x \leq x_{\text{max}}, \\ 0, & \text{otherwise.} \end{cases} \tag{3.129}$$

Using the chain rule with this approximation gives

$$\frac{\partial \mathcal{L}}{\partial x} \approx \frac{\partial \mathcal{L}}{\partial \widehat{x}} \mathbb{I}(x_{\text{min}} \leq x \leq x_{\text{max}}), \tag{3.130}$$

where $\mathbb{I}(\cdot)$ is the indicator function. Inside the clipping interval, the quantizer is treated as if it had the derivative of the identity function. Outside the interval, the gradient is masked. This is a surrogate-gradient rule, not the true derivative of the discrete quantization operator. It makes gradient-based optimization practical but introduces a mismatch between the forward quantizer and the backward approximation (**song2026targetcentricqat**).

The clipped STE also has a limitation: saturated values receive zero gradient through the quantizer and may consequently have difficulty returning to the valid range. This motivates methods that learn or adapt clipping thresholds, scales, and offsets. PACT learns activation clipping thresholds, LSQ treats the step size as a learnable parameter, and LSQ+ introduces learnable offsets for asymmetric activation distributions. Gradient scaling and magnitude-aware differentiation address the scale of updates to quantization parameters, while oscillation-aware methods attempt to reduce repeated threshold crossings. Differentiable soft quantization replaces hard rounding during some or all of training with a smooth relaxation, but a relaxation can introduce a discrepancy between the training operator and the final hard quantizer. These alternatives are reviewed in **song2026targetcentricqat**.

Learning the quantization step size. A fixed quantization grid may not remain suitable as the weight and activation distributions change during training. Learnable step-size quantization treats the scale s as an optimization variable. For symmetric quantization, the fake-quantized scalar is

$$\widehat{x} = s \ \text{clamp}\left(\text{round}\left(\frac{x}{s}\right), q_{\text{min}}, q_{\text{max}}\right). \tag{3.131}$$

Let $v = x/s$. Under the STE approximation, the derivative with respect to s is represented piecewise as

$$\frac{\partial \widehat{x}}{\partial s} \approx \begin{cases} q_{\text{min}}, & v \leq q_{\text{min}}, \\ \text{round}(v) - v, & q_{\text{min}} < v < q_{\text{max}}, \\ q_{\text{max}}, & v \geq q_{\text{max}}. \end{cases} \tag{3.132}$$

The loss gradient is then

$$\frac{\partial \mathcal{L}}{\partial s} = \frac{\partial \mathcal{L}}{\partial \widehat{x}} \frac{\partial \widehat{x}}{\partial s}. \tag{3.133}$$

The learned step size changes the trade-off between clipping-induced distortion and rounding-induced error. LSQ additionally scales the gradient of the step-size parameter to make its update magnitude compatible with updates to the model weights (**song2026targetcentricqat**).

Quantization oscillation and optimization stability. The forward quantizer is piecewise constant, so small changes in a full-precision parameter can cause its quantized value to jump between adjacent levels. For a uniform quantizer with step size s, a rounding threshold between levels k and $k+1$ is

$$w_{\text{threshold}} = \left(k + \frac{1}{2}\right) s. \tag{3.134}$$

A parameter moving back and forth across this threshold can repeatedly change its quantized value even when its continuous updates are small. This behavior is called quantization oscillation and can impair convergence, particularly at low bit-widths. Oscillation-aware optimization, adaptive quantization parameters, staged precision changes, and selected weight-freezing strategies are among the stabilization directions discussed in **song2026targetcentricqat**.

The optimization sensitivity of parameters also explains why equal quantization errors need not produce equal changes in model loss. Let $\mathbf{w}$ be the parameter vector and $\Delta\mathbf{w}$ its quantization perturbation. A second-order Taylor approximation gives

$$\mathcal{L}(\mathbf{w} + \Delta\mathbf{w}) \approx \mathcal{L}(\mathbf{w}) + \nabla\mathcal{L}(\mathbf{w})^\top \Delta\mathbf{w} + \frac{1}{2}\Delta\mathbf{w}^\top \mathbf{H} \Delta\mathbf{w}, \tag{3.135}$$

where $\mathbf{H} = \nabla^2\mathcal{L}(\mathbf{w})$ is the Hessian. Near a stationary point, the gradient term may be small, and the quadratic term describes how the curvature of the loss surface weights different perturbation directions. This provides a mathematical motivation for sensitivity-aware reconstruction and mixed-precision allocation. The Hessian expression is a local approximation; it does not by itself specify a complete quantization algorithm.

3.3.5 Quantization targets and the lifecycle of a language model

Quantization can be applied at different stages of the language-model lifecycle. The target-centric perspective is important because weights, activations, gradients, and cached representations have different statistical properties, lifetimes, reuse frequencies, and computational roles. **song2026targetcentricqat** organizes QAT around weights, activations, joint weight–activation quantization, KV cache, and gradients. **chen2026llmquantization** additionally organizes large language model quantization around pretraining, fine-tuning, and inference.

Weight quantization. Weights are learned parameters and remain relatively stationary during inference. They are therefore natural targets for offline quantization. If the quantized weight matrix is

$$\widehat{\mathbf{W}} = \mathbf{W} + \Delta\mathbf{W}, \tag{3.136}$$

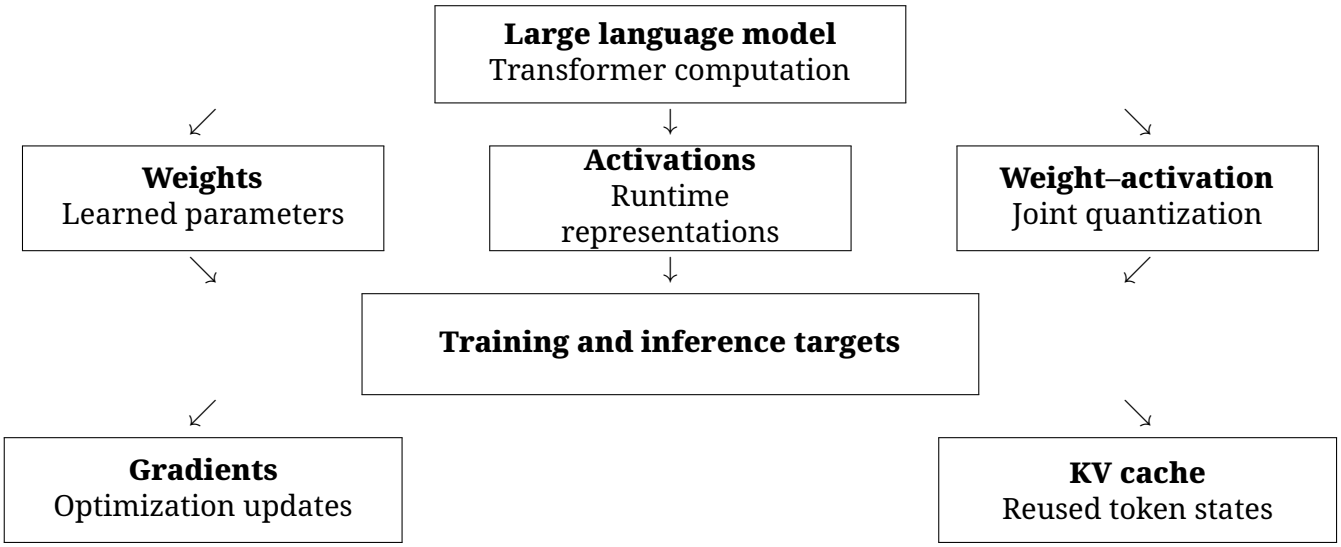


Figure 3.5: Principal quantization targets in large language models. Weights and activations participate in the forward computation; gradients and optimizer-related quantities arise during training; and the key-value cache stores representations reused during autoregressive decoding. The categories identify different quantization targets rather than mutually exclusive model stages.

then the error in a linear layer is

$$\Delta \mathbf{y} = \Delta \mathbf{W} \mathbf{x}. \tag{3.137}$$

The effect of weight error depends on the activation vector with which the weights are multiplied. Weight-only quantization can reduce model storage and memory traffic without requiring the activations to use the same low-precision format.

GPTQ and AWQ are representative weight-only PTQ methods discussed by **chen2026llmquantizatio** GPTQ uses second-order information to guide weight reconstruction, whereas AWQ uses activation information to identify salient weight channels and guide scaling. Group-wise quantization assigns separate scales to smaller groups of weights, allowing local distributions to be represented more closely than with a single global scale. At very low bit-widths, selective preservation of sensitive weights or channels can reduce the loss of model capability (**chen2026llmquantization**; **song2026targetcentricqat**).

Activation quantization. Activations are generated dynamically from the input data. Their distributions can vary across layers, examples, and token positions. This makes their quantization ranges more difficult to determine than those of fixed inference weights. Outliers are particularly important: a small number of large values can increase the scale of a symmetric quantizer and reduce the effective resolution available for most values. Learnable clipping, per-token or per-channel scales, and mixed-precision handling of outliers are strategies for addressing this issue. SmoothQuant and RPTQ are representative weight–activation quantization methods discussed in the LLM quantization survey (**chen2026llmquantization**).

Joint weight–activation quantization. A notation such as W4A4 indicates that both weights and activations are represented using four bits. Joint quantization can reduce memory traffic and enable low-precision matrix multiplication, but the two error sources interact through equation (3.122). Moreover, quantizing upstream weights can change the distributions of downstream activations. Consequently, calibration ranges and quantization parameters may need to account for the quantized computational graph rather than being selected independently for each tensor. Integer-compatible execution also requires attention to nonlinear and normalization operations such as Softmax, GELU, and LayerNorm. A model with quantized linear layers is not necessarily an integer-only model if other operations still require floating-point execution (**chen2026llmquantization**).

Key-value cache quantization. Autoregressive language models generate tokens sequentially. During self-attention, key and value representations for previously processed tokens are retained so that later decoding steps can reuse them. Let L denote the number of Transformer layers, B the batch size, T the number of cached tokens, H_{KV} the number of key-value heads, and d_h the dimension of each head. The number of scalar values stored in the key-value cache is approximately

$$N_{\mathrm{KV}} = 2LBTH_{\mathrm{KV}}d_h, \tag{3.138}$$

where the factor of two accounts for keys and values. If each scalar uses b bits, the idealized cache storage is

$$M_{\mathrm{KV}} = \frac{2LBTH_{\mathrm{KV}}d_h b}{8} \text{ bytes.} \tag{3.139}$$

The cache therefore grows linearly with sequence length and batch size. Quantization can reduce this memory requirement, but its error has a distinctive consequence: cached representations are reused in subsequent attention computations, so an approximation can affect multiple future decoding steps. Per-token, per-channel, adaptive, and non-uniform approaches are discussed as ways of accommodating variation in cached key and value distributions (**song2026targetcentricqat**; **chen2026llmquantization**).

Gradient quantization. Gradients determine the updates applied to model parameters. Let $\mathbf{g}_t = \nabla_{\mathbf{w}}\mathcal{L}$ be the full-precision gradient and let its quantized approximation be

$$\widehat{\mathbf{g}}_t = \mathbf{g}_t + \boldsymbol{\epsilon}_{g,t}. \tag{3.140}$$

A gradient-descent update using the quantized gradient is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\left(\mathbf{g}_t + \boldsymbol{\epsilon}_{g,t}\right). \tag{3.141}$$

The update perturbation is therefore

$$\Delta\mathbf{w}_{t+1} = -\eta\boldsymbol{\epsilon}_{g,t}. \tag{3.142}$$

Gradient quantization can change both the magnitude and direction of the update. Its evaluation should therefore consider gradient norms, directional agreement, and the stability of the optimization trajectory, rather than tensor reconstruction error alone (**song2026targetcentricqat**).

Quantization during pretraining and fine-tuning. During pretraining, low-precision computation can be applied to forward activations, weights, gradients, and communication quantities, while selected persistent states retain higher precision. FP8 formats such as E4M3 and E5M2 allocate bits differently between exponent and mantissa: E4M3 provides more mantissa precision over a more restricted range, whereas E5M2 provides a larger exponent range. This distinction is relevant because weights, activations, and gradients can have different magnitude distributions. Dynamic scaling and block-wise scaling are used to adapt the numerical range to tensor or block statistics. The LLM quantization survey discusses FP8, INT8, INT4, and FP4 training directions, as well as mixed-precision training systems and low-bit pretraining (**chen2026llmquantization**).

Fine-tuning quantization includes PTQ and QAT. PTQ applies a low-bit representation to an existing pretrained model, while QAT simulates quantization during fine-tuning so that the model can adapt to the deployment representation. QAT can be combined with distillation or parameter-efficient adaptation. LLM-QAT, QA-LoRA, LoftQ, and QEFT are examples of approaches discussed in the supplied surveys. Their detailed relationship to parameter-efficient fine-tuning is developed in the subsequent PEFT section; here, their relevance is that quantization can be integrated with adaptation rather than treated only as a final compression step (**song2026targetcentricqat**; **chen2026llmquantization**).

3.3.6 Numerical formats, mixed precision, and hardware efficiency

Integer and floating-point formats distribute their representational capacity differently. Integer quantization uses a discrete range with a fixed step size over the selected interval. Floating-point formats represent values through sign, exponent, and mantissa fields, so the spacing between representable values varies with magnitude. FP8 and FP4 formats are therefore not interchangeable with INT8 and INT4 merely because their nominal bit-widths are equal. The suitable format depends on the dynamic range and precision requirements of the target tensor and on the arithmetic supported by the hardware (**chen2026llmquantization**).

Non-uniform quantization uses unequally spaced representational levels. It can allocate more levels to regions where values are concentrated or where approximation is more important. NF4 is an example discussed in the LLM quantization survey for representing weights whose distribution is approximately normal. Such a representation may reduce distortion for the intended distribution, although its implementation and hardware support must also be considered (**chen2026llmquantization**).

Mixed-precision quantization assigns different bit-widths to different model components. Suppose a model is divided into K parameter groups, where group k contains N_k parameters and uses b_k bits per parameter. Ignoring metadata, the total storage is

$$M = \frac{1}{8} \sum_{k=1}^{K} N_k b_k. \tag{3.143}$$

A simplified precision-allocation problem can be written as

$$\begin{aligned} \min_{\{b_k\}} \quad & \mathcal{L}_{\text{quantized}}(\{b_k\}) \\ \text{subject to} \quad & \sum_{k=1}^{K} N_k b_k \le B, \end{aligned} \tag{3.144}$$

where B is a bit budget. This formulation expresses the objective of minimizing quantization-induced task degradation under a storage constraint. In practice, the loss as a function of bit allocation is not generally known in closed form. Sensitivity estimates, local reconstruction error, activation statistics, and hardware constraints are therefore used to guide allocation (**song2026targetcentricqat**; **chen2026llmquantization**).

The practical benefit of quantization depends on whether a workload is limited by data movement or arithmetic throughput. A memory-bound workload is constrained primarily by moving weights or activations between memory and compute units. A compute-bound workload is constrained primarily by the rate of arithmetic operations. Weight-only quantization can reduce the amount of weight data transferred, whereas joint weight–activation quantization can additionally exploit low-precision arithmetic units when suitable kernels are available. A reduction in bit-width does not, by itself, guarantee a proportional reduction in latency: dequantization, memory access patterns, kernel availability, operator fallbacks, and hardware architecture all affect the realized result (**chen2026llmquantization**).

Accumulator precision is another constraint. In an integer matrix multiplication, an output element is an accumulated sum:

$$y_{ij} = \sum_{k=1}^{n} w_{ik} x_{kj}. \tag{3.145}$$

Even if the operands use low-bit integers, the sum can require a wider representation. For example, if symmetric INT4 operands lie between -7 and 7, the maximum magnitude of one product is 49. An accumulation of 4096 such maximum positive products would be

$$4096 \times 49 = 200{,}704. \tag{3.146}$$

This value exceeds the maximum positive value of a signed 18-bit integer, $2^{17}-1 = 131{,}071$, but is within the maximum positive value of a signed 19-bit integer, $2^{18}-1 = 262{,}143$. This worst-case calculation illustrates why operand precision and accumulator precision must be considered separately. Actual implementations choose accumulator formats according to their supported arithmetic and overflow constraints (**song2026targetcentricqat**; **chen2026llmquantization**).

Hardware–algorithm co-design is consequently part of quantization. A low-bit representation must be supported by kernels that efficiently load data, apply scales, perform arithmetic, accumulate products, and write outputs. A weight-only kernel may reduce memory traffic but still incur dequantization overhead. A weight–activation quantized model may require low-precision matrix multiplication and compatible implementations of nonlinear and normalization operations. Hand-tuned kernels, compiler-based generation, and mixed-precision execution therefore influence whether a theoretical compression benefit becomes a practical speed or energy benefit (**chen2026llmquantization**).

3.3.7 Evaluation, limitations, and connection to the next topic

The evaluation of a quantized model must distinguish numerical reconstruction, preservation of model behavior, and deployment efficiency. A common tensor-level measure is normalized mean squared error. For an original tensor $\mathbf{X}$ and its quantized reconstruction $Q(\mathbf{X})$, it is

$$\mathrm{nMSE}(\mathbf{X}) = \frac{\|\mathbf{X} - Q(\mathbf{X})\|_F^2}{\|\mathbf{X}\|_F^2}, \tag{3.147}$$

where $\|\cdot\|_F$ is the Frobenius norm. For a matrix $\mathbf{A}$,

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} a_{ij}^2}. \tag{3.148}$$

A smaller nMSE indicates a closer tensor reconstruction. It should be complemented by cosine similarity, clipping and saturation ratios, and layer-wise error profiles because a single aggregate error can conceal distortion concentrated in a small number of sensitive modules (**song2026targetcentricqat**).

For Transformer models, local tensor error should be paired with measurements of block-output drift, hidden-state similarity, and divergence between the output distributions of the full-precision and quantized models. One distributional measure is the Kullback–Leibler divergence:

$$D_{\mathrm{KL}}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}, \tag{3.149}$$

where P and Q are probability distributions. In this setting, they can represent the token distributions produced by the original and quantized models on the same input. Cross-entropy and perplexity also measure the effect of quantization on language modeling. For a sequence of T tokens, the average negative log-likelihood is

$$\mathcal{L}_{\mathrm{NLL}} = -\frac{1}{T} \sum_{t=1}^{T} \log P(x_t \mid x_{<t}), \tag{3.150}$$

and perplexity is

$$\mathrm{PPL} = \exp(\mathcal{L}_{\mathrm{NLL}}). \tag{3.151}$$

Perplexity is useful for language modeling but does not, by itself, establish that reasoning, mathematical problem solving, instruction following, or code generation capabilities have been preserved.

The QAT survey reports that LLM evaluation uses language-modeling datasets and task benchmarks that include WikiText-2, C4, MMLU, ARC, GSM8K, MATH or BBH, and HumanEval. These evaluate different aspects of model behavior, and results should be compared only with care because the evaluated backbones, training budgets, prompts, decoding settings, and benchmark protocols can differ. The survey also notes that reasoning and code-generation evaluation is less consistently reported than conventional language-modeling and zero-shot evaluation in the methods it reviews (**song2026targetcentricqat**).

Deployment evaluation must report the actual memory footprint, latency, throughput, and hardware conditions. Prefill and autoregressive decoding should be distinguished because their computational and memory behavior differs. The nominal bit-width is not a substitute for measured efficiency: two models with the same weight bit-width may use different activation formats, group sizes, accumulators, kernels, or floating-point fallbacks. QAT-specific stability diagnostics can additionally include gradient cosine similarity, gradient sign agreement, gradient-norm drift, and the fraction of weights whose quantized values repeatedly change across adjacent updates (**song2026targetcentricqat**; **chen2026llmquantization**).

Quantization nevertheless involves unavoidable trade-offs. Lower bit-widths reduce storage and can enable more efficient computation, but they reduce representational capacity and can increase rounding, clipping, and propagation errors. Fine-grained scales can improve local approximation at the cost of metadata and implementation overhead. QAT can recover performance by adapting a model to quantization constraints, but its surrogate gradients are approximate and its optimization can be unstable. Hardware support determines whether the numerical savings translate into real latency, throughput, or energy improvements.

The central result is that quantization is not a single compression operation. It is a design problem involving numerical representation, statistical distributions, optimization dynamics, model components, and hardware execution. PTQ and QAT provide different ways of introducing low-precision representations; target-specific methods address the distinct behavior of weights, activations, gradients, and KV-cache entries; and mixed-precision allocation balances accuracy against a resource budget.

This discussion also establishes a connection to the next optimization topic. Quantization changes how numerical information is represented, whereas knowledge distillation transfers predictive information from a teacher model to a student model. The two can be combined, as the QAT literature illustrates, but they solve different parts of the problem. The next section therefore develops distillation as a complementary mechanism for transferring model behavior under computational and representational constraints.

3.4 Knowledge Distillation

The preceding discussion of optimization established how a language model learns a predictive function by adjusting its parameters against a training objective. That formulation, however, leaves an important deployment question unresolved. A model may achieve strong predictive performance and yet remain too expensive to store, execute, or serve at scale. Increasing model capacity can improve the quality of the learned distribution, but it also increases parameter memory, arithmetic operations, inference latency, and the resources required to serve long sequences. The optimization problem therefore has a second dimension: how can the information acquired by a computationally expensive model be transferred to a model whose execution is less demanding?

Knowledge distillation addresses this question by training a *student* model under the guidance of a *teacher* model. The teacher is usually larger or more capable, while the student is designed to have lower storage and inference costs. The objective is not necessarily to copy the teacher's parameters or architecture. Instead, the student is trained to reproduce selected aspects of the teacher's behavior, such as its output probability distributions, intermediate representations, generated responses, or reasoning traces. The information made available by the teacher and the student's ability to absorb it determine the form of the distillation objective. Knowledge distillation is consequently a family of optimization procedures rather than a single loss function. This characterization is consistent with the LLM distillation surveys (**Yang2024SurveyKD**; **Fang2026KnowledgeDatasetDistillation**).

The distinction between model compression and knowledge transfer is central to this chapter. A smaller parameter count does not by itself guarantee that the student retains the teacher's useful behavior. The student must learn a suitable approximation from

finite data and within a finite optimization budget. The following development begins with the probability distributions produced by autoregressive language models, establishes the mathematical relationship between those distributions, and then examines the different interfaces through which a teacher can supervise a student.

3.4.1 Predictive distributions and the mathematical objective of distillation

An autoregressive language model predicts a distribution over the next token conditioned on the tokens already observed. Let the vocabulary be $\mathcal{V} = \{v_1, \dots, v_C\}$, where C is the vocabulary size. At position t, the context is $c_t = x_{<t} = (x_1, \dots, x_{t-1})$. A model with parameters θ maps this context to a logit vector

$$\mathbf{z}_\theta(c_t) = \big(z_{\theta,1}(c_t), \dots, z_{\theta,C}(c_t)\big)^\mathsf{T} \in \mathbb{R}^C. \tag{3.152}$$

The logits are unrestricted real-valued scores; they are not probabilities and need not sum to one. The softmax transformation converts them into a categorical probability distribution:

$$p_\theta(v_i \mid c_t) = \frac{\exp(z_{\theta,i}(c_t))}{\sum_{j=1}^{C} \exp(z_{\theta,j}(c_t))}. \tag{3.153}$$

The resulting probability vector belongs to the probability simplex

$$\Delta^{C-1} = \left\{ \mathbf{p} \in \mathbb{R}^C : p_i \geq 0,\ \sum_{i=1}^{C} p_i = 1 \right\}. \tag{3.154}$$

For a sequence $\mathbf{x} = (x_1, \dots, x_T)$, the autoregressive factorization gives

$$P_\theta(\mathbf{x}) = \prod_{t=1}^{T} P_\theta(x_t \mid x_{<t}). \tag{3.155}$$

Taking the negative logarithm converts the product into a sum. The next-token negative log-likelihood (NLL) objective is therefore

$$\mathcal{L}_{\text{LM}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}). \tag{3.156}$$

This is also the token-averaged cross-entropy between the empirical target and the model prediction. To see this, let $\mathbf{e}_{x_t}$ be the one-hot vector whose component corresponding to the observed token is one. Then

$$\mathcal{L}_{\text{NLL}}^{(t)} = -\sum_{i=1}^{C} e_{x_t,i} \log p_\theta(v_i \mid c_t) = -\log p_\theta(x_t \mid c_t). \tag{3.157}$$

A one-hot target identifies the observed token, but it does not directly express how plausible the model considers the other vocabulary items. A teacher's full predictive distribution contains this additional information. For example, suppose the observed token is v_1 and the teacher's probability vector is

$$\mathbf{p}_T = (0.70, 0.20, 0.08, 0.02)^\mathsf{T}. \tag{3.158}$$

The distribution communicates not only that v_1 is preferred, but also that v_2 is considered more plausible than v_3, and v_3 more plausible than v_4. These relative preferences among alternatives are commonly called the teacher's *dark knowledge* in the distillation literature (**Yang2024SurveyKD**; **Fang2026KnowledgeDatasetDistillation**).

The uncertainty represented by a categorical distribution can be quantified using Shannon entropy. For a probability vector $\mathbf{p}$,

$$H(\mathbf{p}) = -\sum_{i=1}^{C} p_i \log p_i. \tag{3.159}$$

A distribution concentrated on one category has low entropy, whereas the uniform distribution has entropy $\log C$. Thus, teacher predictions with the same most probable token can still convey different information when their probability mass is distributed differently across the remaining vocabulary. Distillation uses this richer distribution as a learning target rather than reducing teacher behavior to its top prediction.

Let $\mathbf{p}_T(\cdot \mid c)$ and $\mathbf{p}_S(\cdot \mid c)$ denote teacher and student distributions at the same context. Their cross-entropy is

$$H(\mathbf{p}_T, \mathbf{p}_S) = -\sum_{i=1}^{C} p_{T,i} \log p_{S,i}, \tag{3.160}$$

whereas their Kullback–Leibler (KL) divergence is

$$D_{\mathrm{KL}}(\mathbf{p}_T \| \mathbf{p}_S) = \sum_{i=1}^{C} p_{T,i} \log \frac{p_{T,i}}{p_{S,i}}. \tag{3.161}$$

Expanding the logarithm gives

$$\begin{aligned} D_{\mathrm{KL}}(\mathbf{p}_T \| \mathbf{p}_S) &= \sum_i p_{T,i} \log p_{T,i} - \sum_i p_{T,i} \log p_{S,i} \\ &= -H(\mathbf{p}_T) + H(\mathbf{p}_T, \mathbf{p}_S). \end{aligned} \tag{3.162}$$

Consequently,

$$H(\mathbf{p}_T, \mathbf{p}_S) = H(\mathbf{p}_T) + D_{\mathrm{KL}}(\mathbf{p}_T \| \mathbf{p}_S). \tag{3.163}$$

When the teacher is fixed, $H(\mathbf{p}_T)$ is constant with respect to the student parameters. Minimizing cross-entropy against the teacher is therefore equivalent to minimizing forward KL divergence. KL divergence is nonnegative and is zero if the two distributions agree, subject to the usual support conditions. It is not symmetric: in general, $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$.

This asymmetry changes the approximation favored when the student cannot represent the teacher distribution exactly. Forward KL, $D_{\mathrm{KL}}(\mathbf{p}_T\|\mathbf{p}_S)$, weights the alternatives according to teacher probability and penalizes the student for failing to cover teacher-supported alternatives. Reverse KL, $D_{\mathrm{KL}}(\mathbf{p}_S\|\mathbf{p}_T)$, weights the alternatives according to student probability and penalizes student mass on alternatives that the teacher considers unlikely. In a multimodal target and a restricted student family, the two objectives can favor different approximations. The Jensen–Shannon divergence provides a symmetric alternative. With $\mathbf{m} = (\mathbf{p}_T + \mathbf{p}_S)/2$, it is defined as

$$D_{\mathrm{JS}}(\mathbf{p}_T\|\mathbf{p}_S) = \frac{1}{2} D_{\mathrm{KL}}(\mathbf{p}_T\|\mathbf{m}) + \frac{1}{2} D_{\mathrm{KL}}(\mathbf{p}_S\|\mathbf{m}). \tag{3.164}$$

The divergence is symmetric and bounded. The choice of discrepancy is thus part of the optimization design, not a notational variation (**Song2026OnPolicyDistillation**).

Temperature-scaled targets. The teacher's original distribution may be sharply concentrated. Temperature scaling changes its concentration by dividing logits by a positive parameter τ before applying softmax:

$$p_{T,i}^{(\tau)} = \frac{\exp(z_{T,i}/\tau)}{\sum_{j=1}^{C} \exp(z_{T,j}/\tau)}, \qquad p_{S,i}^{(\tau)} = \frac{\exp(z_{S,i}/\tau)}{\sum_{j=1}^{C} \exp(z_{S,j}/\tau)}. \tag{3.165}$$

For $0 < \tau < 1$, differences between logits are amplified relative to the temperature and the distribution becomes sharper. For $\tau > 1$, the distribution becomes smoother. At $\tau = 1$, the original softmax is recovered. If the maximum logit is unique,

$$\lim_{\tau \to 0^+} p_{T,i}^{(\tau)} = \begin{cases} 1, & i = \arg\max_j z_{T,j}, \\ 0, & \text{otherwise}, \end{cases} \qquad \lim_{\tau \to \infty} p_{T,i}^{(\tau)} = \frac{1}{C}. \tag{3.166}$$

Temperature therefore controls how much of the teacher's relative preference among nonmaximal alternatives is exposed to the student.

For the temperature-scaled cross-entropy

$$\mathcal{L}_{\text{KD}}^{(\tau)} = -\sum_{i=1}^{C} p_{T,i}^{(\tau)} \log p_{S,i}^{(\tau)}, \tag{3.167}$$

differentiation with respect to a student logit gives

$$\frac{\partial \mathcal{L}_{\text{KD}}^{(\tau)}}{\partial z_{S,j}} = \frac{1}{\tau} \left(p_{S,j}^{(\tau)} - p_{T,j}^{(\tau)} \right). \tag{3.168}$$

In the high-temperature regime, the difference between the softened probabilities is itself approximately proportional to $1/\tau$. The unscaled gradient can therefore shrink approximately as $1/\tau^2$. A conventional formulation compensates for this scale change by multiplying the KL term by τ^2:

$$\mathcal{L}_{\text{KD}} = \tau^2 D_{\text{KL}} \left(\mathbf{p}_T^{(\tau)} \| \mathbf{p}_S^{(\tau)} \right). \tag{3.169}$$

The factor is a gradient-scale compensation, not a requirement that every distillation implementation must use the same temperature convention. The temperature also controls the dispersion of the transferred distribution and is discussed in Bayesian interpretations of distillation (**Fang2026KnowledgeDatasetDistillation**; **Yang2024SurveyKD**).

The optimization objective. Let $f_T(c; \theta_T)$ and $f_S(c; \theta_S)$ denote teacher and student logits. In conventional distillation, the teacher parameters θ_T are fixed and the student parameters θ_S are optimized. For a context distribution $\mathcal{D}$, the basic logits-based objective is

$$\theta_S^* = \arg\min_{\theta_S} \mathbb{E}_{c \sim \mathcal{D}} \left[D_{\text{KL}} \left(\mathbf{p}_T^{(\tau)}(\cdot \mid c) \| \mathbf{p}_S^{(\tau)}(\cdot \mid c; \theta_S) \right) \right]. \tag{3.170}$$

The teacher provides a target distribution; the student receives gradients through its own predictions. Distillation does not replace gradient-based optimization. It changes the information that defines the target of that optimization.

The original next-token target can be retained because a teacher may be incorrect, biased, or poorly calibrated. A common combined objective is

$$\mathcal{L}_{\text{total}} = (1-\alpha)\mathcal{L}_{\text{LM}} + \alpha\mathcal{L}_{\text{KD}}, \qquad 0 \le \alpha \le 1, \tag{3.171}$$

where α controls the relative contribution of the original language-modeling target and teacher supervision. At $\alpha = 0$, the student is trained only on the observed tokens; at $\alpha = 1$, the specified objective uses only the distillation term. Intermediate values combine both sources of supervision. This formulation is the basis for the pre-training distillation experiments discussed later (**Peng2024PretrainingDistillation**).

3.4.2 Knowledge interfaces: white-box and black-box distillation

The output distribution is only one possible representation of a teacher's knowledge. A language model's predictive behavior is produced by a sequence of internal transformations, and information relevant to the final prediction is distributed across its hidden states, attention computations, and output layer. The form of distillation is therefore constrained by which parts of the teacher are accessible. The survey by **Yang2024SurveyKD** distinguishes white-box methods, which can access internal model information, from black-box methods, which rely on observable outputs such as generated text.

Logits-based white-box distillation. Logits-based distillation transfers the teacher's final vocabulary probabilities or logits. At each token position, the student is trained to minimize a discrepancy such as the forward KL divergence in equation (3.170). It provides information about the teacher's relative preferences over alternative tokens without requiring the student to reproduce the teacher's hidden-state dimensions or layer structure. Its limitation is that supervision is applied at the output interface. Different internal computations can produce similar output distributions, so output matching alone does not force the student to reproduce the teacher's intermediate representations.

Hint-based and representation distillation. Hint-based methods add supervision at intermediate layers. Let the teacher hidden representation at a selected layer be $H_T^{(l)} \in \mathbb{R}^{T\times d_T}$, and let the corresponding student representation be $H_S^{(k)} \in \mathbb{R}^{T\times d_S}$. The sequence length is T, while d_T and d_S are the teacher and student hidden widths. Since the dimensions and layer counts need not agree, a projection $\phi : \mathbb{R}^{d_S} \to \mathbb{R}^{d_T}$ can map the student features into the teacher feature space. A squared-error feature objective is

$$\mathcal{L}_{\text{feat}} = \frac{1}{Td_T}\left\| H_T^{(l)} - \phi(H_S^{(k)}) \right\|_F^2, \tag{3.172}$$

where the squared Frobenius norm is the sum of squared entries of a matrix. This objective aligns selected representations, but it also imposes an architectural correspondence that must be chosen carefully. Matching identically indexed layers is not automatically appropriate when teacher and student depths differ.

Patient knowledge distillation uses several teacher layers rather than only the final layer. TinyBERT aligns several levels of the Transformer computation, including embedding outputs, hidden states, and attention-related representations. MiniLM focuses on relations encoded by self-attention computations, allowing transfer through attention

relationships rather than requiring every hidden coordinate to match. These methods illustrate that representation transfer can be designed around the structures that are most useful to preserve, but retaining intermediate activations increases memory and computation during teacher inference (**Yang2024SurveyKD**).

Black-box distillation through generated text. A black-box teacher exposes responses but not necessarily its logits, parameters, or intermediate activations. The student must then learn from examples generated by the teacher. In-context learning distillation uses a task instruction and a set of demonstrations. Let $D_K = \{(x_i, y_i)\}_{i=1}^{K}$ be the demonstrations and let I be the instruction. The teacher generates a response according to

$$\widehat{y} = T\left(I, (x_1, y_1), \ldots, (x_K, y_K), x_{K+1}\right). \tag{3.173}$$

The generated input–output examples are subsequently used to fine-tune the student. The student is thereby trained on demonstrations of how the teacher responds to instructions and contextual examples, without requiring access to the teacher's internal probability distribution.

Chain-of-thought distillation extends the target from a final answer to an intermediate rationale and a final answer. Let r_i be a teacher-generated rationale for input x_i, and let y_i be its final answer. The student can be trained to model the joint conditional sequence:

$$\mathcal{L}_{\text{CoT}} = -\frac{1}{N}\sum_{i=1}^{N} \log P_{\theta_S}(r_i, y_i \mid x_i). \tag{3.174}$$

For an autoregressive model, the joint probability factorizes into the conditional probabilities of the rationale tokens followed by the answer tokens. Rationale supervision therefore teaches an intermediate sequence, not merely the final output. Its effectiveness depends on the correctness and usefulness of the teacher's rationales; a fluent explanation is not by itself evidence that the reasoning is valid (**Yang2024SurveyKD**; **Fang2026KnowledgeDatasetDistillation**).

Instruction-following distillation constructs examples containing an instruction I_i, an optional input x_i, and a teacher response y_i^T. The student objective is

$$\mathcal{L}_{\text{inst}} = -\frac{1}{N}\sum_{i=1}^{N} \log P_{\theta_S}(y_i^T \mid I_i, x_i). \tag{3.175}$$

The diversity and difficulty of the instruction set determine the range of behaviors represented in the training data. A narrow collection of simple instructions may yield a student that reproduces those examples but does not generalize to complex or unfamiliar requests. Self-instruction approaches expand the dataset by asking a teacher to generate additional instructions and corresponding responses, followed by quality filtering (**Yang2024SurveyKD**).

Uncertainty-aware and multi-teacher distillation. A single predictive distribution does not necessarily express all forms of uncertainty relevant to a task. Uncertainty-aware distillation can use ensembles or probabilistic teachers to expose predictive variability and train the student to approximate a teacher predictive distribution. At a state x, a generic objective remains

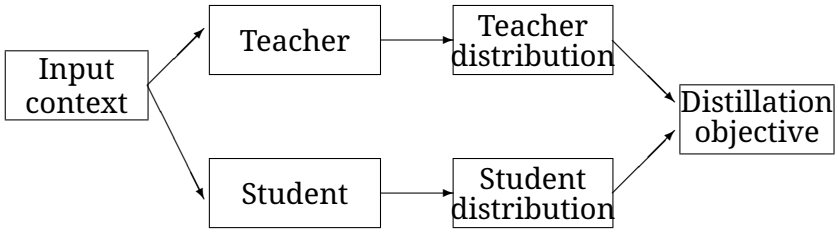
$$\mathcal{L}_{\text{unc}} = D_{\text{KL}}\left(p_T(\cdot \mid x) \| p_S(\cdot \mid x)\right), \tag{3.176}$$

but the teacher distribution can now reflect uncertainty represented by multiple models or posterior samples. The Bayesian interpretation discussed in the survey treats teacher-induced regularization as information about a prior over student behavior; temperature controls the dispersion of the teacher-informed target. The additional sampling or ensemble computation must be weighed against the value of improved calibration and robustness (**Fang2026KnowledgeDatasetDistillation**).

For K teachers with predictive distributions $p_k(y \mid x)$, a simple multi-teacher target is the weighted mixture

$$\bar{p}(y \mid x) = \sum_{k=1}^{K} w_k(x) p_k(y \mid x), \qquad w_k(x) \geq 0, \quad \sum_{k=1}^{K} w_k(x) = 1. \tag{3.177}$$

The student can be trained to match $\bar{p}$. Uniform weights assume equal teacher usefulness, whereas input-dependent weights can reflect specialization or reliability. Multi-teacher distillation can combine complementary expertise, but it increases teacher inference cost and can introduce conflicts among teachers with different domains, vocabularies, or prediction preferences. Dynamic and adaptive methods allow the supervisor or the relative contributions of teachers to change during training. Such methods can support evolving data or continual learning, but co-evolution can make convergence and error control more difficult (**Fang2026KnowledgeDatasetDistillation**).



The teacher and student receive the same context.
The loss compares predictive distributions; student parameters receive the update.

Figure 3.6: Output-level knowledge distillation. The same context is presented to teacher and student models. Their predictive distributions are compared by a distillation objective, which supplies the student update without requiring the student to copy the teacher parameters.

3.4.3 Post-training and pre-training distillation

Distillation can be applied at different stages of language-model construction. Post-training distillation generally starts from a pretrained student and transfers task-specific behavior through teacher-generated instructions, answers, or rationales. Pre-training distillation instead uses teacher predictions over the pre-training corpus, so that the student receives a teacher-derived learning signal while acquiring general language modeling capabilities. The two settings differ in their data distributions, objectives, target-generation costs, and intended transfer (**Peng2024PretrainingDistillation**; **Yang2024SurveyKD**).

In post-training distillation, let $D_T = \{(x_i, y_i^T)\}_{i=1}^N$ be a collection of prompts and teacher-generated responses. Teacher-forced student training minimizes

$$\mathcal{L}_{\text{post}} = -\frac{1}{N}\sum_{i=1}^{N} \log P_{\theta_S}(y_i^T \mid x_i). \tag{3.178}$$

For a response $y_i^T = (y_{i,1}^T, \ldots, y_{i,L_i}^T)$, the conditional sequence probability is

$$P_{\theta_S}(y_i^T \mid x_i) = \prod_{t=1}^{L_i} P_{\theta_S}(y_{i,t}^T \mid x_i, y_{i,<t}^T). \tag{3.179}$$

This procedure is applicable when the teacher is accessible only through a text-generation interface. Its limitation is that the student is trained on fixed response prefixes. During deployment, the student conditions on its own previous predictions, which can take it away from the histories present in the teacher demonstrations. This discrepancy motivates on-policy distillation, developed in section 3.4.4.

In pre-training distillation, teacher and student process contexts from a pre-training corpus. Let $x_{<t}$ be a corpus prefix and let $p_T(\cdot \mid x_{<t})$ and $p_S(\cdot \mid x_{<t})$ be their next-token distributions. A generic objective combines the original token loss with a teacher-matching term:

$$\mathcal{L}_{\text{PD}} = (1-\alpha)\mathcal{L}_{\text{LM}} + \alpha\frac{1}{T}\sum_{t=1}^{T} D_{\text{KL}}\left(p_T(\cdot \mid x_{<t}) \| p_S(\cdot \mid x_{<t})\right). \tag{3.180}$$

The central motivation is that the teacher distribution contains information about alternatives that the observed next token does not reveal. The pre-training design-space study by **Peng2024PretrainingDistillation** examines the processing of teacher logits, the loss function, the allocation of distillation weight, model and data scaling, and offline versus online generation of teacher targets.

Truncating teacher logits. A full teacher distribution has C entries at every token position. If there are N training tokens and each stored logit uses b bytes, the storage required for all teacher logits is approximately

$$M_{\text{full}} = NCb. \tag{3.181}$$

For billion-token corpora and vocabularies with hundreds of thousands of entries, this can require petabytes of storage. Peng et al. report that storing full float32 logits for their 100-billion-token preliminary setting would require approximately 58.6 petabytes. Their two-stage truncation procedure applies top-p selection followed by top-k selection and renormalizes the retained probability mass. In top-p selection, sort the teacher probabilities in descending order and retain the smallest set S_p satisfying

$$\sum_{i \in S_p} p_{T,(i)} \geq p, \tag{3.182}$$

where $p_{T,(i)}$ denotes the i-th largest teacher probability. Top-k then caps the number of retained entries at k. If the retained index set is S, the normalized truncated target is

$$\widetilde{p}_{T,i} = \frac{p_{T,i}\mathbb{I}[i \in S]}{\sum_{j \in S} p_{T,j}}, \tag{3.183}$$

where $\mathbb{I}[\cdot]$ is the indicator function. Renormalization is necessary because truncation removes probability mass. The reported top-0.95-100 configuration reduces storage by roughly a factor of 4,000, from about 58.6 petabytes to approximately 15 terabytes in the preliminary experiment. Truncation therefore reduces the engineering burden, although it also changes the target by removing the low-probability tail (**Peng2024PretrainingDistillation**).

Loss selection and scheduling. The pre-training study compares negative log-likelihood, KL divergence, and mean squared error for distillation. KL divergence and negative log-likelihood perform similarly in the reported experiments, whereas mean squared error is substantially less effective. This distinction follows from the object being matched: a probability distribution has a categorical normalization constraint, while squared error measures Euclidean distance between numerical coordinates and does not directly express a distributional discrepancy. The coefficient α in equation (3.180) also affects performance. The reported experiments find that increasing the distillation contribution helps up to a point, after which further increases can reduce performance.

Rather than holding α constant, the study also examines a warmup–stable–decay (WSD) schedule. During warmup, α increases from zero to one; it remains at one during the stable phase and then follows a cosine decay toward zero. The reported configuration uses a warmup proportion of approximately ten percent and a decay proportion of approximately one percent, paired with a WSD learning-rate schedule. The schedule coordinates the strength of teacher supervision with the stage of student optimization. It is an empirical configuration, not a universal schedule that must be optimal for every teacher, student, corpus, or training budget (**Peng2024PretrainingDistillation**).

Offline and online targets. Offline distillation computes teacher logits using an already pretrained teacher and then uses those targets to train the student. Online distillation stores logits while the teacher itself is being pretrained. Online storage can avoid a separate teacher-inference pass and can make one teacher run useful for a family of students. However, the quality of online targets depends on the teacher's training stage. Peng et al. find that logits from early, non-converged teacher checkpoints can be noisy and yield weak student performance. Later-stage online logits are more useful, although their reported student remains below the student trained from offline logits of a pretrained teacher. Their practical conclusion is conditional: when one student is required, offline targets from a pretrained teacher are preferable in the studied setting; when multiple student sizes are to be trained, online logits stored during the largest model's pretraining can amortize the target generation cost (**Peng2024PretrainingDistillation**).

The same study reports that larger students generally benefit more from pre-training distillation, while increasing teacher size does not guarantee better student performance. Its preliminary experiment distills a 1.9-billion-parameter student from GLM-4-9B over 100 billion tokens and reports an average relative improvement of approximately 1.6 percent over the language-modeling baseline across the selected evaluations. In its larger-scale experiments, improvements remain visible over a 500-billion- token training run. These findings support the effectiveness of pre-training distillation in the investigated settings, but do not establish a universal gain at arbitrary model sizes or compute budgets (**Peng2024PretrainingDistillation**).

3.4.4 On-policy distillation and the student-state distribution

The fixed-prefix objectives above define what the student should predict at a given context, but they do not determine where that context comes from. Autoregressive generation makes this distinction consequential. Each student-generated token becomes part of the next context, so an early prediction changes the states at which later predictions are made. A student trained only on reference or teacher-generated prefixes may therefore be poorly supervised on the histories it encounters during deployment. On-policy distillation (OPD) addresses this mismatch by generating student responses and querying the teacher along the resulting student-generated prefixes (**Song2026OnPolicyDistillation**).

For a prompt x and a response $y = (y_1, \dots, y_T)$, define the state at position t as

$$c_t = (x, y_{<t}), \qquad y_{<t} = (y_1, \dots, y_{t-1}). \tag{3.184}$$

The next token is the action. The student policy is $\pi_\theta(a \mid c_t)$, and the teacher target is $q(a \mid c_t^T)$, where c_t^T denotes the context available to the teacher. In the simplest case, teacher and student are evaluated on the same text prefix. A trajectory consists of the sequence of states and actions generated by the student. Because each action changes the next prefix, the state distribution is induced by the policy that generates the response.

Under off-policy distillation, training states can come from reference responses, teacher responses, or trajectories generated by an earlier student. Under strict on-policy distillation, the states are sampled from the current student policy. A trajectory collected by an earlier student was on-policy when it was generated, but becomes stale relative to a later student after the parameters change. Re-scoring a stored trajectory can refresh teacher supervision without refreshing the state distribution. This distinction between the source of states and whether a dataset is stored or processed offline is important when designing replay and asynchronous training systems (**Song2026OnPolicyDistillation**).

A common distribution-matching formulation is

$$\mathcal{L}_{\text{match}}(\theta) = \mathbb{E}_{s \sim d_\mu}\left[w(s) D_f\left(q(\cdot \mid s^T) \| \pi_\theta(\cdot \mid s) \right) \right], \tag{3.185}$$

where d_μ is the state distribution induced by collection policy μ, $w(s)$ is a nonnegative state weight, and D_f is an f-divergence. For a convex generator f satisfying $f(1) = 0$,

$$D_f(P \| Q) = \mathbb{E}_{a \sim Q}\left[f\left(\frac{P(a)}{Q(a)} \right) \right]. \tag{3.186}$$

Choosing $f(u) = u \log u$ yields forward KL, while $f(u) = -\log u$ yields reverse KL, wherever the required probability ratios are defined. Equation equation (3.185) separates three decisions: which states are visited, which target the teacher provides at those states, and which discrepancy is used to compare teacher and student distributions. Strict on-policy collection corresponds to $\mu = \pi_\theta$.

Sampled-token objectives and variance reduction. Full-vocabulary matching uses teacher probabilities for all possible next tokens at a visited prefix. A sampled-token surrogate can instead use only the token drawn from the student. Let

$$r_t = \log \frac{p_T(y_t \mid c_t)}{p_\theta(y_t \mid c_t)}. \tag{3.187}$$

A detached reverse-KL surrogate is

$$\ell_t(\theta) = -\,\mathrm{sg}[r_t] \log p_\theta(y_t \mid c_t), \tag{3.188}$$

where $\mathrm{sg}[\cdot]$ denotes stop-gradient: the coefficient is held fixed while differentiating the student log-probability. If $r_t > 0$, the teacher assigns the sampled token greater probability than the student and the update reinforces that token. If $r_t < 0$, the update suppresses it. This estimator avoids an explicit sum over the full vocabulary, but it does not directly provide a positive correction for every teacher-preferred alternative that the student did not sample. Restricting supervision to top-K tokens, discarding the tail, renormalizing it, or representing it as a residual probability bucket changes the effective support of the objective (**Song2026OnPolicyDistillation**).

A baseline can reduce sampling variance without changing the expected conditional score-function gradient when it depends on the prefix but not on the sampled action. The conditional mean log-ratio baseline is

$$\begin{aligned} b_t(c_t) &= \mathbb{E}_{a \sim p_\theta(\cdot \mid c_t)} \left[\log \frac{p_T(a \mid c_t)}{p_\theta(a \mid c_t)} \right] \\ &= -D_{\mathrm{KL}} \left(p_\theta(\cdot \mid c_t) \| p_T(\cdot \mid c_t) \right). \end{aligned} \tag{3.189}$$

Subtracting this action-independent baseline centers the immediate conditional learning signal. It is a control variate: it changes the variance of the estimator without changing its expectation under the stated sampling conditions. It should not be confused with a value function that estimates cumulative future discrepancies (**Song2026OnPolicyDistillation**).

Sequence-level reverse KL and temporal credit. A local conditional objective treats a visited prefix as fixed. It does not necessarily assign credit to an earlier token for changing the prefixes that will be visited later. Sequence-level reverse KL accounts for this temporal dependence. Let $p_\theta(y)$ and $p_T(y)$ be the student and teacher sequence distributions, and let the student trajectory be sampled from p_θ. For a fixed teacher with support on the student-generated responses, the ideal student-sampled gradient can be written as

$$\nabla_\theta D_{\mathrm{KL}}(p_\theta \| p_T) = -\mathbb{E}_{y \sim p_\theta} \left[\sum_{t=1}^{|y|} (R_t - 1) \nabla_\theta \log p_\theta(y_t \mid c_t) \right], \tag{3.190}$$

where the return-to-go of log-probability ratios is

$$R_t = \sum_{t'=t}^{|y|} \log \frac{p_T(y_{t'} \mid c_{t'})}{p_\theta(y_{t'} \mid c_{t'})}. \tag{3.191}$$

The constant score contribution vanishes in expectation, whereas R_t allows an earlier action to receive credit for teacher–student discrepancies at later positions. MiniLLM uses sequence-level reverse KL and approximates the practical update using teacher–student mixed sampling, per-token importance ratios, and length normalization. These implementation choices matter: without suitable sampling and length control, reverse-KL training can reduce its objective while producing short, repetitive, or otherwise unhelpful responses (**Song2026OnPolicyDistillation**).

A discounted temporal objective makes the horizon of credit assignment explicit. Define the local log-probability discrepancy

$$\Delta_t = \log p_\theta(y_t \mid c_t) - \log p_T(y_t \mid c_t). \tag{3.192}$$

The discounted return-like quantity is

$$A_t^{(\gamma)} = -\sum_{t'=t}^{T} \gamma^{t'-t} \Delta_{t'}, \qquad 0 \le \gamma \le 1. \tag{3.193}$$

At $\gamma = 0$, the signal is local to the current token. At $\gamma = 1$, it includes the undiscounted future discrepancies through the end of the sequence. Intermediate values trade off local correction against longer-range temporal credit. Under the uniform second-moment assumption analyzed in the on-policy survey, fixed $\gamma < 1$ gives a horizon-independent upper bound on the variance of this advantage estimate; that statement is not, by itself, a variance bound for the complete parameter-gradient estimator (**Song2026OnPolicyDistillation**).

The teacher's reliability at student-generated prefixes is a separate condition. A prefix may contain an incorrect intermediate step or a context that the teacher would not ordinarily generate. High standalone teacher accuracy does not guarantee reliable guidance at every such state. Teacher adaptation, eligibility tests, target repair, and selective supervision are therefore relevant extensions. On-policy distillation solves an important state-distribution mismatch, but it does not make every teacher prediction correct or every teacher target learnable by the student (**Song2026OnPolicyDistillation**).

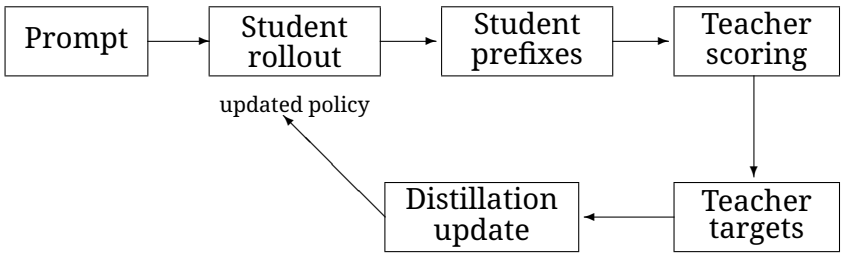


Figure 3.7: The on-policy distillation loop. The student generates its own response prefixes; the teacher evaluates those prefixes and supplies targets; the resulting update changes the student that generates the next trajectories. The state distribution therefore changes as the student is optimized.

3.4.5 Distillation scaling laws and compute allocation

A distillation objective specifies what the student should imitate, but it does not guarantee that the student has enough representational capacity, data, or compute to learn the target. The teacher may be more capable than the student can approximate. Conversely, a teacher that is too weak may provide little information beyond

the original training targets. The teacher–student capacity gap must therefore be considered jointly with the student's training budget. The distillation scaling study by **Busbridge2025DistillationScalingLaws** makes this relationship quantitative through validation cross-entropy.

A conventional supervised scaling relation expresses validation loss as a function of parameter count N and training tokens D:

$$L(N, D) = E + \left(\frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \right)^{\gamma}, \tag{3.194}$$

where E is an irreducible-error term under the fitted model, $A, B, \alpha, \beta, \gamma$ are positive fitted coefficients, and N, D represent model capacity and data scale. Increasing either model size or training data generally reduces loss with diminishing marginal returns. Under a fixed compute budget, compute-optimal training chooses the model size and token count that minimize validation loss subject to the training FLOP constraint.

For distillation, let N_S be the number of student non-embedding parameters, D_S the number of distillation tokens, and L_T the teacher validation cross-entropy. Let

$$\widetilde{L}_S = L(N_S, D_S) \tag{3.195}$$

be the cross-entropy expected for a student of that size and token budget under the supervised scaling relation. Busbridge et al. fit the following distillation scaling law:

$$\begin{aligned} L_S(N_S, D_S, L_T) = L_T + c_0 \left[1 + \left(\frac{L_T}{\widetilde{L}_S} \right)^{1/f_1} \right]^{-c_1 f_1} \\ \times \left(\frac{A}{N_S^{\alpha'}} + \frac{B}{D_S^{\beta'}} \right)^{\gamma'}, \end{aligned} \tag{3.196}$$

where $c_0, c_1, f_1, A, B, \alpha', \beta', \gamma'$ are positive coefficients fitted to the experimental data. The first term is the teacher cross-entropy. The second term models the student's ability to approximate the teacher and depends on student size and distillation data. The ratio $L_T/\widetilde{L}_S$ captures the relative learning abilities of teacher and student on the reference task. The fitted transition produces a capacity gap: at fixed student size and data budget, improving the teacher initially helps, but a teacher that becomes too difficult for the student to imitate can eventually produce a worse student. The optimal teacher is therefore not necessarily the strongest available teacher.

The scaling law is empirical and was fitted to a particular family of language models and training conditions. It should not be interpreted as a universal law independent of architecture, data distribution, tokenization, or optimization. Its main contribution is to make teacher quality, student capacity, and the number of distillation tokens explicit in one predictive relation (**Busbridge2025DistillationScalingLaws**).

Compute accounting. Distillation consumes compute for student training, teacher inference, and, when the teacher does not already exist, teacher pre-training. Let $F(N)$ be the forward-pass FLOPs per token for a model with N parameters. Busbridge et al. approximate the total cost as

$$C_{\text{distill}} \approx 3F(N_S)D_S + F(N_T)\left(\delta_T^{\text{logit}} D_S + 3\delta_T^{\text{pre}} D_T \right), \tag{3.197}$$

where N_T and D_T are teacher parameter count and pre-training tokens. The factor of three in the student term approximates forward and backward training cost; the teacher-logit term accounts for teacher forward passes used to produce targets; and the teacher-pretraining term accounts for training the teacher. The indicators δ_T^{logit} and δ_T^{pre} specify which costs are charged to the current budget. A fully amortized teacher can make both indicators zero; if the teacher already exists but must be queried for this student, only the logit cost is counted; if teacher training is required and its logits are stored for reuse, the pre-training cost is counted; if both teacher training and student-specific inference are charged, both terms are included.

This accounting changes the comparison with supervised learning. When a suitable teacher already exists or will be reused across many students, distillation can amortize the teacher cost and outperform supervised training over a range of budgets. When a teacher must be trained solely for one student, the teacher's pre-training and inference costs can make supervised learning preferable. Busbridge et al. find that the compute regime in which distillation is advantageous depends on student size and teacher reuse; distillation is not automatically more efficient merely because the final student is smaller (**Busbridge2025DistillationScalingLaws**).

3.4.6 Dataset distillation and its integration with model distillation

Knowledge distillation compresses a model's learned behavior. Dataset distillation addresses a complementary problem: it attempts to construct a small synthetic dataset that preserves information needed to train a model. Let the original dataset be $D_R = \{(x_i, y_i)\}_{i=1}^N$, and let a compact synthetic dataset be $D_S = \{(\widetilde{x}_j, \widetilde{y}_j)\}_{j=1}^M$, with $M \ll N$. Unlike ordinary data selection, which retains existing examples, dataset distillation can optimize or generate synthetic examples specifically to preserve the learning signal of the original corpus (**Fang2026KnowledgeDatasetDistillation**).

Gradient-matching formulations. Let $\mathcal{L}(D;\theta)$ be the training loss of a model with parameters θ on dataset D. At optimization state θ_t, define the original-data and synthetic-data gradients as

$$g_R^{(t)} = \nabla_{\theta_t}\mathcal{L}(D_R;\theta_t), \qquad g_S^{(t)} = \nabla_{\theta_t}\mathcal{L}(D_S;\theta_t). \tag{3.198}$$

A gradient-matching objective seeks a synthetic dataset whose updates resemble those induced by the original data:

$$\min_{D_S} \sum_{t=0}^{K-1} \|\nabla_{\theta_t}\mathcal{L}(D_S;\theta_t) - \nabla_{\theta_t}\mathcal{L}(D_R;\theta_t)\|_2^2 . \tag{3.199}$$

The Euclidean norm measures the discrepancy between the parameter updates induced by the two datasets. If the synthetic examples produce similar gradients over the relevant training trajectory, training on the compact set may approximate aspects of training on the full dataset. The method can be computationally expensive because synthetic samples must be optimized against model-training dynamics, often involving repeated higher-order or unrolled computations.

Generative synthesis and selection. A generative approach learns a mapping G from a latent variable $z \sim p(z)$ to a synthetic example:

$$\widetilde{x} = G(z), \qquad z \sim p(z). \tag{3.200}$$

The generator is trained to produce representative examples of the original data distribution or examples that preserve task-relevant information. The survey also distinguishes dataset synthesis from data selection: selection identifies a high-quality subset of existing examples, whereas synthesis constructs new examples through optimization, generative models, or heuristics. These methods trade off computational cost, sample diversity, and the fidelity with which rare or difficult patterns are preserved (**Fang2026KnowledgeDatasetDistillation**).

Combining dataset and knowledge distillation. The two forms of compression can be integrated. A teacher is first trained on the original dataset; a compact synthetic coreset is then constructed; and teacher predictions can be used as soft targets on the synthetic examples. If $C_{\text{syn}} = \{(\widetilde{x}_i, \widetilde{y}_i)\}_{i=1}^{M}$ is the synthetic coreset, a student can be trained using

$$\mathcal{L}_{C_{\text{syn}}}(\theta_S) = -\frac{1}{M} \sum_{i=1}^{M} \sum_{v \in \mathcal{V}} \widetilde{y}_{i,v} \log p_{\theta_S}(v \mid \widetilde{x}_i), \tag{3.201}$$

where $\widetilde{y}_i$ is the teacher-derived target distribution. This combines compression of the training examples with compression of the teacher's predictive behavior. The central risk is that aggressive dataset compression can remove rare linguistic structures, facts, or reasoning patterns that occupy little volume in the original corpus but matter for generalization. The size of the synthetic set is therefore not sufficient evidence of its information coverage (**Fang2026KnowledgeDatasetDistillation**).

3.4.7 Evaluation of distilled language models

A distilled model must be evaluated along several dimensions because predictive quality, teacher fidelity, efficiency, and robustness are not equivalent objectives. A student can reproduce teacher outputs closely without generalizing well to unseen tasks. It can also outperform the teacher on a particular benchmark while differing substantially from the teacher's original distribution. Evaluation must therefore compare the student both with the teacher and with independent task targets (**Yang2024SurveyKD**; **Fang2026KnowledgeDatasetDistillation**).

For language modeling, perplexity is the exponential of token-averaged negative log-likelihood:

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \log p_{\theta_S}(x_t \mid x_{<t}) \right). \tag{3.202}$$

Lower perplexity means that the model assigns higher average probability to the observed evaluation tokens. For task-based evaluation, accuracy, exact match, and F1-score measure different aspects of answer correctness. Benchmarks for knowledge, reasoning, mathematics, commonsense, and instruction following are needed to determine whether the student preserves capabilities beyond the examples used for distillation. A benchmark result is meaningful only with its task definition, data split, prompting protocol, and scoring procedure specified.

Teacher fidelity can be measured directly through a distributional comparison such as

$$\mathcal{F}_{\text{KD}} = \mathbb{E}_{x \sim \mathcal{D}_{\text{eval}}} \left[D_{\text{KL}} \left(p_T(\cdot \mid x) \| p_S(\cdot \mid x) \right) \right]. \tag{3.203}$$

A lower value indicates closer predictive distributions under the selected evaluation contexts. Representation-based methods can additionally compare hidden states using cosine similarity or representation-similarity measures. These metrics assess transfer fidelity, not necessarily task correctness.

Efficiency evaluation should include model parameter count, storage, peak memory, inference latency, throughput, and arithmetic cost. A simple parameter compression ratio is

$$R_{\text{compression}} = \frac{N_T}{N_S}, \tag{3.204}$$

but parameter count alone does not determine wall-clock latency or memory consumption. Architecture, sequence length, hardware, batching, and implementation all influence actual deployment cost. Training efficiency must also account for teacher inference and, where relevant, teacher pre-training, as in equation (3.197).

Robustness and calibration are also relevant when the student is expected to operate under noisy or shifted inputs. Predictive entropy equation (3.159) describes the uncertainty of a single categorical distribution, while calibration compares stated confidence with empirical accuracy. One common calibration statistic is expected calibration error (ECE). If evaluation examples are partitioned into confidence bins B_m, then

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \text{acc}(B_m) - \text{conf}(B_m) \right|. \tag{3.205}$$

Here $\text{acc}(B_m)$ is empirical accuracy in bin m, and $\text{conf}(B_m)$ is its average predicted confidence. Robustness testing should also consider distribution shifts and adversarial perturbations where appropriate. No single metric captures all these properties; a distillation method should be assessed against the deployment requirements it is intended to satisfy (**Yang2024SurveyKD**; **Fang2026KnowledgeDatasetDistillation**).

3.4.8 Limitations and connection to parameter-efficient fine-tuning

Knowledge distillation reduces deployment cost only when the student preserves the capabilities needed by its application. Several limitations follow from the mechanisms developed above. A narrow distillation dataset may omit capabilities that are not expressed in its prompts or examples. Teacher-generated rationales may be fluent but incorrect, and a student can inherit teacher biases or systematic errors. Multi-teacher training can combine complementary expertise but also increases inference cost and creates conflicts among heterogeneous supervisors. Hint-based methods require decisions about which layers or representations should be aligned, and intermediate activation storage can become expensive. On-policy methods expose the teacher to student-generated histories, but teacher reliability at erroneous or unfamiliar prefixes remains a separate problem. Finally, a teacher that is substantially more capable than the student may be difficult to imitate, while the cost of producing teacher targets can outweigh the savings from a smaller student (**Yang2024SurveyKD**; **Peng2024PretrainingDistillation**; **Busbridge2025DistillationScalingLaws**; **Song2026OnPolicyDistillation**).

These limitations show why distillation is not a universal replacement for conventional pre-training or fine-tuning. Its value depends on the information available from the teacher, the student's capacity, the data coverage, the optimization objective, and the total compute budget. Dataset distillation can reduce the volume of training information that must be processed, while quantization reduces the numerical precision required for model storage and computation. Distillation addresses a different source of cost: it transfers learned behavior to a model with a smaller or more suitable parameterization. These strategies can be combined, but their benefits must be measured at the level of the complete training and inference pipeline.

The next section turns from transferring behavior between models to adapting an existing model with a restricted set of trainable parameters. Parameter-efficient fine-tuning investigates how a pretrained model can be specialized without updating all of its parameters. It therefore continues the chapter's central optimization question from a complementary direction: how can useful model behavior be obtained and adapted while controlling the computational and memory costs of learning?

3.5 Parameter-Efficient Fine-Tuning

The preceding sections have established the optimization procedures and training objectives used to construct and improve language models. They have also considered techniques such as quantization and distillation that reduce the computational or representational cost of training and deployment. A closely related problem arises after pretraining: a model that has acquired general-purpose linguistic representations must often be adapted to a particular domain, task, or application. The central question is whether this adaptation requires changing every parameter of the pretrained model.

Full fine-tuning provides a direct answer by allowing all model parameters to change. For modern large language models, however, the resulting memory, computation, and storage requirements can be substantial. Moreover, when one pretrained model must serve many downstream tasks, maintaining a complete fine-tuned copy for every task duplicates most of the original model. Parameter- efficient fine-tuning (PEFT) addresses this problem by restricting the task-specific optimization to a small subset of existing parameters, by introducing lightweight trainable modules, or by reparameterizing the update so that it is represented with fewer degrees of freedom. The objective is not to replace the pretrained model with a smaller network, but to adapt its existing computational structure with a smaller task-specific optimization problem (**prottasha2025peft**; **he2025smt**).

3.5.1 From Full Fine-Tuning to Parameter-Efficient Adaptation

A language model can be represented as a parameterized function

$$f_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Y}, \qquad \boldsymbol{\theta} \in \mathbb{R}^P, \tag{3.206}$$

where $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces, $\boldsymbol{\theta}$ is the vector containing the model parameters, and P is the number of scalar parameters. In a large language model, P may reach billions. The pretrained parameter vector is denoted by $\boldsymbol{\theta}_0$. It is obtained by

optimizing a pretraining objective over a large corpus. For an autoregressive language model, a representative objective for a token sequence $x_1, \dots, x_T$ is

$$\mathcal{L}_{\text{pretrain}}(\boldsymbol{\theta}) = -\sum_{t=1}^{T} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}), \tag{3.207}$$

where $x_{<t}$ denotes the tokens preceding position t. The resulting $\boldsymbol{\theta}_0$ provides an initialization that already encodes statistical regularities learned from the pretraining distribution.

For a downstream task, let

$$\mathcal{D}_{\text{task}} = \{(x_i, y_i)\}_{i=1}^{N} \tag{3.208}$$

be a dataset of N input-target pairs. The task-specific empirical risk is

$$\mathcal{L}_{\text{task}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_{\boldsymbol{\theta}}(x_i), y_i)\,, \tag{3.209}$$

where ℓ is the loss appropriate to the task. For classification it may be cross-entropy; for autoregressive generation it may be token-level negative log-likelihood. Full fine-tuning solves

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^P} \mathcal{L}_{\text{task}}(\boldsymbol{\theta}), \qquad \boldsymbol{\theta}^{(0)} = \boldsymbol{\theta}_0. \tag{3.210}$$

The complete pretrained parameter vector is therefore the optimization variable. At iteration t, a gradient method evaluates

$$\mathbf{g}^{(t)} = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{task}}\big(\boldsymbol{\theta}^{(t)}\big) \tag{3.211}$$

and, in the case of gradient descent, updates

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \mathbf{g}^{(t)}, \tag{3.212}$$

where $\eta > 0$ is the learning rate. In full fine-tuning, every component of $\boldsymbol{\theta}$ is eligible for an update.

The memory cost is not limited to the model weights. Training also requires gradient storage, optimizer state, and intermediate activations needed by backpropagation. For Adam, the first and second moment estimates are

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t, \tag{3.213}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^{\odot 2}, \tag{3.214}$$

where $\beta_1, \beta_2 \in [0, 1)$ are decay coefficients and $\odot$ denotes element-wise multiplication. The bias-corrected estimates are

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \qquad \widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \tag{3.215}$$

and the Adam update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}, \tag{3.216}$$

where ϵ is a small positive constant for numerical stability. Consequently, the optimizer maintains additional quantities whose sizes scale with the number of trainable parameters.

A useful first-order memory accounting model is

$$M_{\text{FT}} \approx P(b_w + b_g + b_m + b_v) + M_{\text{act}}, \tag{3.217}$$

where b_w, b_g, b_m, and b_v are the bytes per parameter used for weights, gradients, first moments, and second moments, respectively, and M_{act} is the activation memory. This is an accounting model, not a universal implementation-specific formula: precision, optimizer implementation, activation checkpointing, gradient accumulation, and offloading change the actual footprint.

The scale of the problem is illustrated by the LLaMA-7B example reported by **he2025smt**. In their stated configuration, full fine-tuning without CPU offloading requires approximately 58 GB of GPU memory: about 13.6 GB for trainable parameters, 40 GB for Adam optimizer states and gradients, and 2 GB for activations. The exact values depend on the implementation and precision, but the example shows why weight storage alone is an inadequate estimate of training memory.

Full fine-tuning also has a multi-task storage cost. If a pretrained model with P parameters is independently fine-tuned for K tasks, storing all task-specific copies requires approximately

$$S_{\text{FT}} = KP. \tag{3.218}$$

If the base model is shared and each task stores only q adaptation parameters, where $q \ll P$, the corresponding storage is

$$S_{\text{PEFT}} = P + Kq. \tag{3.219}$$

The ratio is

$$\frac{S_{\text{PEFT}}}{S_{\text{FT}}} = \frac{1}{K} + \frac{q}{P}. \tag{3.220}$$

Thus, the storage advantage becomes increasingly important as the number of downstream tasks grows.

The statistical motivation is related but distinct. A model with a very large number of adjustable parameters can fit complex patterns, but a downstream dataset may contain relatively few examples. In such a setting, unrestricted adaptation can increase estimation variance and overfitting. The PEFT survey describes restricted, structured updates as a form of implicit regularization that can be beneficial in low-resource settings (**prottasha2025peft**). Restricting updates can also reduce interference with pretrained knowledge and thereby mitigate, but does not guarantee the elimination of, catastrophic forgetting. The central design question is therefore how to restrict the update space without preventing the model from learning the task-specific information it needs.

General formulation and taxonomy. The common mathematical structure of PEFT is to replace unrestricted optimization over $\boldsymbol{\theta} \in \mathbb{R}^P$ with optimization over a smaller task-specific parameter vector $\boldsymbol{\phi} \in \mathbb{R}^q$, where $q \ll P$. A general update parameterization is

$$\Delta\boldsymbol{\theta} = g(\boldsymbol{\phi}), \qquad \boldsymbol{\theta}_{\text{eff}} = \boldsymbol{\theta}_0 + g(\boldsymbol{\phi}), \tag{3.221}$$

where g maps the lower-dimensional adaptation variables into an update of the pretrained parameters. The optimization problem becomes

$$\phi^* = \arg\min_{\phi \in \mathbb{R}^q} \frac{1}{N} \sum_{i=1}^{N} \ell\big(f_{\theta_0 + g(\phi)}(x_i), y_i\big)\,. \tag{3.222}$$

The pretrained parameters may remain fixed, while gradients are propagated through the model to optimize ϕ. In methods that add modules rather than explicitly reparameterize the original weights, the model can be written more directly as $f_{\theta_0,\phi}(x)$.

Selective fine-tuning can be represented using a binary mask $\mathbf{M} \in \{0,1\}^P$:

$$\theta_{\text{eff}} = \theta_0 + \mathbf{M} \odot \Delta\theta, \qquad q = \|\mathbf{M}\|_0, \tag{3.223}$$

where $\|\mathbf{M}\|_0$ counts the selected scalar parameters. A value of one permits an update, while zero freezes the corresponding parameter. Other PEFT families do not necessarily use a binary mask, but they share the principle of reducing the number of independent adaptation variables.

The survey by **prottasha2025peft** organizes PEFT techniques around additive, selective, reparameterized, hybrid, and unified approaches, and separately discusses mixture-of-experts-based variants. These categories describe different ways of controlling the adaptation space. Additive methods introduce task-specific modules or representations. Selective methods choose a subset of existing parameters. Reparameterized methods express weight changes through lower-dimensional variables. Hybrid and unified frameworks combine multiple mechanisms, while mixture-of-experts designs route inputs or tokens among specialized adaptation components. Figure 3.8 summarizes this conceptual organization.

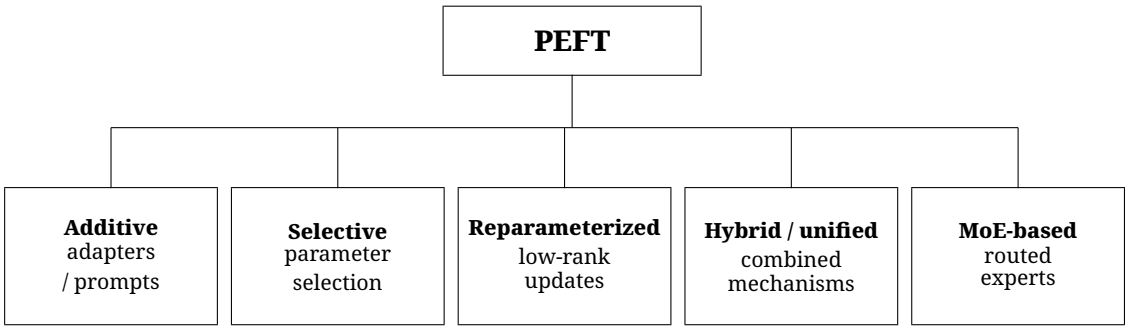


Figure 3.8: Conceptual taxonomy of parameter-efficient fine-tuning. The branches summarize the principal families discussed by **prottasha2025peft**; the diagram is an original schematic rather than a reproduction of the survey figure.

The categories should not be interpreted as mutually exclusive mathematical classes. A method can combine an additive module with a low-rank update, or use a sparse selection rule to determine where a reparameterized update is applied. The classification is useful because it identifies the principal mechanism that controls the trainable parameter space.

3.5.2 Additive and Prompt-Based Adaptation

Additive fine-tuning introduces trainable modules or representations into the existing model while retaining the pretrained backbone. The added parameters encode task-specific transformations, and the original model parameters can remain frozen. Adapters and soft prompts are two important examples (**prottasha2025peft**).

Serial adapters. A serial adapter is commonly implemented as a bottleneck transformation. Let $\mathbf{h} \in \mathbb{R}^d$ be a hidden representation, and let $d_b \ll d$ be the bottleneck dimension. The down-projection is

$$\mathbf{h}_{\text{down}} = \text{ReLU}\left(\mathbf{W}_{\text{down}}\mathbf{h} + \mathbf{b}_{\text{down}}\right), \qquad \mathbf{W}_{\text{down}} \in \mathbb{R}^{d_b \times d}, \tag{3.224}$$

and the up-projection is

$$\mathbf{h}_{\text{up}} = \mathbf{W}_{\text{up}}\mathbf{h}_{\text{down}} + \mathbf{b}_{\text{up}}, \qquad \mathbf{W}_{\text{up}} \in \mathbb{R}^{d \times d_b}. \tag{3.225}$$

The output is added through a residual connection:

$$\mathbf{h}_{\text{out}} = \mathbf{h} + \mathbf{h}_{\text{up}}. \tag{3.226}$$

Ignoring biases, one adapter contains

$$P_{\text{adapter}} = dd_b + d_b d = 2dd_b \tag{3.227}$$

trainable parameters. A dense $d \times d$ transformation contains d^2 parameters, so the ratio is $2d_b/d$. If one adapter is inserted in each of L layers, the approximate count is $2Ldd_b$, subject to the particular placement and architecture. The bottleneck creates a capacity-efficiency trade-off: a larger d_b permits a richer transformation but increases parameters and computation.

Parallel and hybrid adapters. A parallel adapter processes a representation alongside the main model pathway. If its output is

$$\mathbf{h}_{\text{parallel}} = \mathbf{W}_{\text{parallel}}\mathbf{h} + \mathbf{b}_{\text{parallel}}, \tag{3.228}$$

the combined representation can be written as

$$\mathbf{h}_{\text{out}} = \mathbf{h}_{\text{main}} + \alpha\mathbf{h}_{\text{parallel}}, \tag{3.229}$$

where α controls the contribution of the parallel pathway. Hybrid adapters combine serial and parallel pathways, for example,

$$\mathbf{h}_{\text{out}} = \beta\mathbf{h}_{\text{serial}} + \gamma\mathbf{h}_{\text{parallel}}, \tag{3.230}$$

with coefficients β and γ. The survey discusses serial, parallel, and hybrid designs, as well as single-task and multi-task adapter use cases (**prottasha2025peft**). Figure 3.9 contrasts the broad idea of inserting a trainable adapter with that of introducing trainable prompt representations.

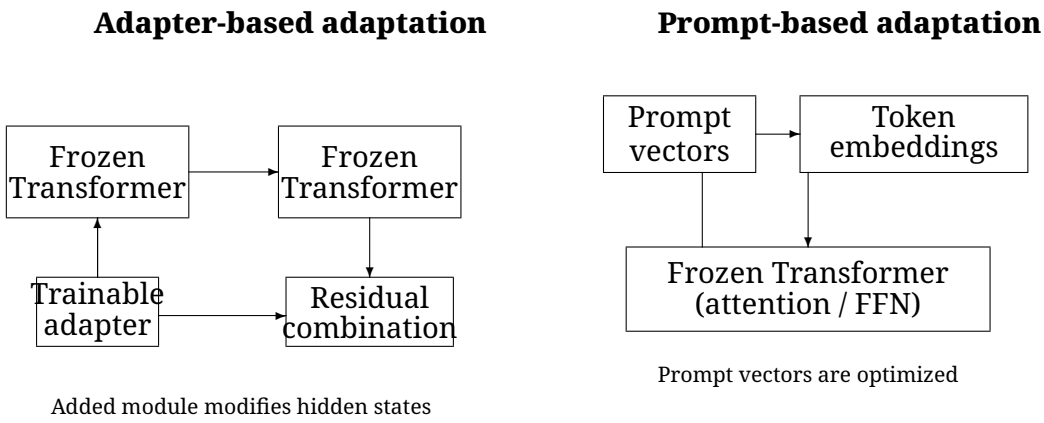


Figure 3.9: Conceptual comparison of adapter-based and prompt-based adaptation. Adapters add trainable transformations within the model pathway, whereas prompt methods add trainable representations to the input or attention context. Original schematic based on the mechanisms surveyed by **prottasha2025peft**.

Soft prompt tuning. Let the token embedding sequence for an input x be $\mathbf{E}(x) \in \mathbb{R}^{n \times d}$, where n is the number of input tokens and d is the embedding dimension. Soft prompt tuning introduces m trainable vectors,

$$\mathbf{P} \in \mathbb{R}^{m \times d}, \tag{3.231}$$

and concatenates them with the input embeddings:

$$\mathbf{X}_{\text{prompt}} = [\mathbf{P}; \mathbf{E}(x)] \in \mathbb{R}^{(m+n) \times d}. \tag{3.232}$$

The pretrained model is frozen and the prompt is optimized:

$$\mathbf{P}^* = \arg\min_{\mathbf{P}} \mathcal{L}\left(f_{\boldsymbol{\theta}_0}([\mathbf{P}; \mathbf{E}(x)]), y\right). \tag{3.233}$$

The prompt contains md trainable scalar values. Unlike a discrete prompt, whose elements are vocabulary tokens, a soft prompt is a continuous matrix of trainable embeddings and is not required to correspond to ordinary words.

Prefix tuning. Prefix tuning extends the prompt principle into the attention computation. For query, key, and value matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, introduce trainable prefix key and value representations $\mathbf{P}_K$ and $\mathbf{P}_V$. The attention operation becomes

$$\text{Attention}\left(\mathbf{Q}, [\mathbf{P}_K; \mathbf{K}], [\mathbf{P}_V; \mathbf{V}]\right) = \text{softmax}\left(\frac{\mathbf{Q}[\mathbf{P}_K; \mathbf{K}]^\top}{\sqrt{d_k}}\right)[\mathbf{P}_V; \mathbf{V}], \tag{3.234}$$

where d_k is the key dimension. Prompt tuning modifies the input embedding sequence, whereas prefix tuning makes trainable contextual representations available within attention. The survey also discusses multi-layer continuous prompts such as P-Tuning v2, as well as discrete prompt optimization methods such as RLPrompt and sparse prompt selection (**prottasha2025peft**).

For a discrete prompt, token identities are not ordinary continuous optimization variables. One alternative is to optimize a policy over candidate token sequences. If ψ denotes policy parameters and $R(P)$ is the reward associated with a prompt P, a policy-gradient update has the form

$$\psi_{t+1} = \psi_t + \eta \nabla_{\psi} \mathbb{E}[R(P)]. \tag{3.235}$$

Sparse prompt selection can instead be expressed as

$$P^*_{\text{sparse}} = \arg\min_P \|P\|_0 \quad \text{subject to} \quad R(P) \geq R_{\text{threshold}}, \tag{3.236}$$

where $\|P\|_0$ counts the selected prompt elements. These formulations illustrate that prompt-based PEFT can optimize either continuous embeddings or a policy that generates discrete prompts.

Scaling-based adaptation. The survey also describes Propulsion, which modifies feature representations through trainable scaling values. For feature groups $\mathbf{V} = [v_1, \ldots, v_s]$, its transformation is expressed as

$$\mathbf{V}' = [v_1 \odot z_1^k,\ v_2 \odot z_2^k,\ \ldots,\ v_s \odot z_s^k], \tag{3.237}$$

where z_i is a trainable scaling parameter, k is a scaling exponent, and $\odot$ denotes element-wise multiplication. This mechanism changes the relative contribution of feature components without introducing a complete dense transformation.

Additive and prompt-based methods are parameter-efficient, but they do not eliminate all computation. Adapters add operations to the forward path. Prompt and prefix methods increase the effective context length; if the attention sequence grows from n to $n+m$, the dense attention score matrix grows from $n \times n$ to $(n+m) \times (n+m)$. Its leading pairwise interaction cost therefore changes from $\mathcal{O}(n^2)$ to $\mathcal{O}((n+m)^2)$. A small trainable parameter count should consequently not be confused with a proportionate reduction in inference computation.

3.5.3 Selective and Reparameterized Adaptation

Selective fine-tuning does not introduce a new transformation as its defining operation. Instead, it chooses which existing parameters may change. Partition the pretrained parameters into selected and frozen components,

$$\theta = \{\theta_s, \theta_f\}, \qquad \theta_f = \text{constant}. \tag{3.238}$$

The optimization is

$$\theta_s^* = \arg\min_{\theta_s} \mathcal{L}\left(f_{\theta_s, \theta_f}(x), y\right). \tag{3.239}$$

The key problem is the selection rule.

Gradient and Fisher-information criteria. For parameter θ_i, the local sensitivity of the loss is described by

$$g_i = \frac{\partial \mathcal{L}}{\partial \theta_i}. \tag{3.240}$$

A simple importance score is $C(\theta_i) = |g_i|$. Parameters with large scores may be selected, subject to a trainable-parameter budget. This score is local: it depends on the current model, data batch, and loss, and is not a task-independent measure of universal parameter importance.

A statistical alternative is Fisher information. For a model distribution $p(x;\boldsymbol{\theta})$, the diagonal Fisher information associated with parameter θ_i is

$$I(\theta_i) = \mathbb{E}\left[\left(\frac{\partial \log p(x;\boldsymbol{\theta})}{\partial \theta_i}\right)^2\right]. \tag{3.241}$$

An empirical approximation over N observations is

$$\widehat{I}(\theta_i) = \frac{1}{N}\sum_{j=1}^{N}\left(\frac{\partial \log p(x_j;\boldsymbol{\theta})}{\partial \theta_i}\right)^2. \tag{3.242}$$

Large diagonal Fisher values indicate that the log-likelihood is sensitive to the corresponding parameters under the sampled distribution. The survey discusses Fisher-based selection methods such as FishMask and Adafish, as well as gradient-based selection methods (**prottasha2025peft**).

Unstructured and structured selection. Unstructured selection chooses individual scalar parameters, potentially at irregular positions throughout the model. BitFit, Child-Tuning, and other sparsity-based methods illustrate this family. Structured selection chooses coherent groups such as layers, heads, rows, columns, or blocks. A binary mask can enforce a selection pattern:

$$\boldsymbol{\theta}_s = \mathbf{M} \odot \boldsymbol{\theta}, \qquad \mathbf{M} \in \{0,1\}^P. \tag{3.243}$$

Structured patterns can be easier to implement efficiently because they preserve contiguous parameter groups. Unstructured sparsity may select more precisely, but irregular memory access and indexing overhead can reduce the hardware benefit. The survey discusses methods including UBitFit, LT-SFT, Child-Tuning, RoCoFT, Xattn Tuning, and structured masking approaches (**prottasha2025peft**).

Low-rank reparameterization. Reparameterized PEFT constrains the weight update rather than selecting individual original parameters. Consider a linear transformation

$$\mathbf{y} = \mathbf{W}_0\mathbf{x}, \qquad \mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}. \tag{3.244}$$

Full fine-tuning permits an arbitrary matrix update $\Delta\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, containing $d_{\text{out}}d_{\text{in}}$ scalar degrees of freedom. Low-Rank Adaptation (LoRA) parameterizes the update as

$$\Delta\mathbf{W} = \mathbf{BA}, \qquad \mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \tag{3.245}$$

where $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The effective weight is

$$\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}. \tag{3.246}$$

The number of trainable parameters is

$$P_{\text{LoRA}} = r(d_{\text{out}} + d_{\text{in}}), \tag{3.247}$$

instead of $d_{\text{out}}d_{\text{in}}$. For a square $d \times d$ matrix, the ratio is

$$\frac{P_{\text{LoRA}}}{P_{\text{FT}}} = \frac{2dr}{d^2} = \frac{2r}{d}. \tag{3.248}$$

For example, with $d = 4096$ and $r = 8$, full fine-tuning of this matrix requires $4096^2 = 16\,777\,216$ trainable values, whereas LoRA requires $2(4096)(8) = 65\,536$, approximately 0.39% as many.

The matrix rank of a product cannot exceed the rank of either factor:

$$\text{rank}(\Delta\mathbf{W}) = \text{rank}(\mathbf{BA}) \leq r. \tag{3.249}$$

LoRA therefore restricts the update to a low-dimensional matrix subspace. This reduces parameter count and can act as an implicit regularizer, but it also limits the set of updates that can be represented. The rank r controls the capacity-efficiency trade-off. The survey describes an initialization in which one factor is initialized to zero so that the initial low-rank update is zero, preserving the pretrained transformation at the start of training (**prottasha2025peft**).

Dynamic rank and LoRA variants. A fixed rank allocates the same maximum update dimension throughout training. Dynamic methods instead adapt rank or rank allocation according to training signals or task-specific criteria. The survey discusses DyLoRA, AdaLoRA, SLORA, CapaBoost, and AutoLoRA. A schematic representation is

$$r_t = f(\|\nabla_t\|), \qquad t \in \{1, \ldots, T\}, \tag{3.250}$$

where f denotes a rank-allocation rule and ∇_t denotes gradient information at iteration t. This equation expresses the general idea of adaptive rank selection; it is not a universal update rule shared by every dynamic-rank method.

The survey also reviews regularized and task-specific variants, including LoRA Dropout, Laplace LoRA, Predict LoRA, LoRA++ and continual-learning variants. Quantization can be combined with low-rank adaptation. QLoRA, for example, keeps a quantized frozen base model and trains low-rank adaptation parameters. SVDQUANT combines singular-value decomposition and quantization. Variational LoRA introduces a probabilistic treatment of the adapted weights, with the Bayesian relation

$$p(\mathbf{W} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{W})p(\mathbf{W}), \qquad \mathbf{W} = \mathbf{W}_0 + \mathbf{BA}. \tag{3.251}$$

These variants target different limitations, including rank allocation, regularization, quantized storage, uncertainty, and continual adaptation (**prottasha2025peft**).

Hybrid, unified, and expert-based approaches. Hybrid methods combine adaptation mechanisms. UniPELT, for example, combines LoRA, prefix tuning, and adapters within one framework. A general weighted combination of n adaptation contributions can be written as

$$\Delta\boldsymbol{\theta}_{\text{hybrid}} = \sum_{i=1}^{n} \alpha_i \Delta\boldsymbol{\theta}_i, \tag{3.252}$$

where α_i controls the contribution of mechanism i. Unified frameworks seek to coordinate or select among parameter-efficient modules rather than committing to one fixed adaptation mechanism.

Mixture-of-experts-based PEFT extends this idea by maintaining several specialized adaptation components. If expert i supplies a low-rank update $\mathbf{B}_i\mathbf{A}_i$, a routed update can be represented as

$$\Delta\mathbf{W}(x) = \sum_{i=1}^{E} \alpha_i(x)\mathbf{B}_i\mathbf{A}_i, \qquad \sum_{i=1}^{E} \alpha_i(x) = 1, \tag{3.253}$$

where the routing coefficients depend on the input or token representation. Sparse routing can activate only a subset of experts. The survey discusses MoE LoRA, MixLoRA, MoLoRA, MOA, and MoLE as examples of expert-based adaptation (**prottasha2025peft**).

3.5.4 Sparse Matrix Tuning and Comparative Analysis

Low-rank methods constrain the rank of the update matrix. Sparse Matrix Tuning (SMT), introduced by **he2025smt**, takes a different approach: it selects a small number of submatrices in the original weight matrices and updates those blocks directly. The selected blocks need not be low-rank. SMT therefore constrains the support of the update, rather than requiring the entire update matrix to have low rank.

Task objective and block partitioning. Let $P_{\Phi_0}(y \mid x)$ be a pretrained autoregressive language model with parameters Φ_0, and let $\mathcal{Z} = \{(x_i, y_i)\}_{i=1}^{N}$ be a downstream dataset. A conditional language-modeling objective is

$$\max_{\Theta} \sum_{(x,y)\in\mathcal{Z}} \sum_{t=1}^{|y|} \log P_{\Phi_0+\Delta\Phi(\Theta)}(y_t \mid x, y_{<t}), \tag{3.254}$$

where Θ denotes the trainable adaptation variables. For LoRA, Θ contains low-rank factors. For SMT, Θ contains the selected submatrices.

Consider a pretrained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$. Partition it into non-overlapping blocks of size $l \times l$, assuming for the moment that l divides both dimensions. The number of blocks is

$$N_{\text{blocks}} = \frac{d}{l}\frac{k}{l} = \frac{dk}{l^2}. \tag{3.255}$$

If $\mathcal{S}$ is the selected set of m blocks, the adapted weight is

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W}_{\mathcal{S}}, \qquad \Delta\mathbf{W}_{\mathcal{S},ij} = 0 \quad \text{outside } \mathcal{S}. \tag{3.256}$$

The number of trainable values in this matrix is

$$P_{\text{SMT}} = ml^2, \qquad m \ll N_{\text{blocks}}. \tag{3.257}$$

Within a selected block, the update can be a general $l \times l$ matrix. Thus, a selected block can retain full local rank even though only a small fraction of the complete matrix is trainable. Figure 3.10 contrasts the low-rank and submatrix update principles.

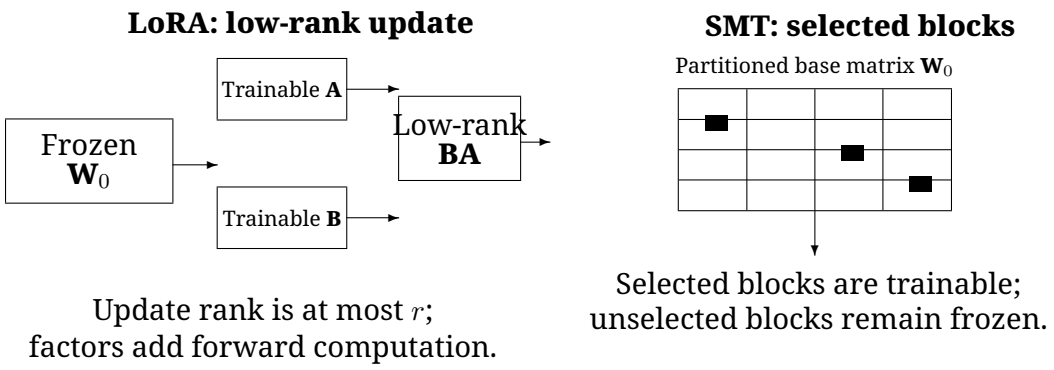


Figure 3.10: Conceptual comparison of LoRA and Sparse Matrix Tuning. LoRA represents the update as a product of low-rank factors, whereas SMT partitions the original matrix and updates selected blocks. Original schematic based on **he2025smt**.

Gradient-aware block selection. SMT must identify which blocks are most relevant to the downstream objective. The paper compares activation-aware selection with gradient-aware selection. In the reported experiments, gradient-aware selection is preferred. During a warm-up phase, gradients from the relevant attention linear layers are accumulated. Let $\mathbf{G}^{(t)}$ be the weight-gradient matrix at warm-up iteration t. The accumulated gradient is

$$\mathbf{G}^{\text{acc}} = \sum_{t=1}^{T_w} \mathbf{G}^{(t)}, \tag{3.258}$$

where T_w is the number of warm-up iterations. For block B_j, the score used to rank blocks can be represented as

$$s_j = \frac{1}{l^2} \sum_{(a,b)\in B_j} \left| G^{\text{acc}}_{ab} \right|. \tag{3.259}$$

The absolute value prevents positive and negative entries from cancelling when the magnitude of accumulated gradient information is assessed. The blocks are ranked by s_j, and the highest-scoring blocks are retained until the specified trainable-parameter budget is reached. The selection budget is a method configuration; the exact trainable fraction varies across the reported experiments.

The paper reports fewer than 100 warm-up iterations in its experiments. It uses 100 warm-up iterations for the commonsense experiments and 25 for the Math10K experiments. The block dimension is 256×256 in the reported LLaMA experiments. This dimension is selected because it is a common factor of the row and column dimensions of the relevant LLaMA linear layers, avoiding remainder blocks. The authors also report that activation-aware selection substantially underperforms gradient-aware selection in their ablation study (**he2025smt**).

Partial weight-gradient computation. The computational advantage of SMT depends on changing the backward calculation, not merely freezing parameters after a conventional full backward pass. Consider a linear layer

$$\mathbf{Z} = \mathbf{W}\mathbf{X}, \tag{3.260}$$

where $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\mathbf{X} \in \mathbb{R}^{d_{\text{in}} \times n}$, and $\mathbf{Z} \in \mathbb{R}^{d_{\text{out}} \times n}$. Define $\mathbf{G} = \partial \mathcal{L} / \partial \mathbf{Z}$. The standard matrix derivatives are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{G}\mathbf{X}^{\mathsf{T}}, \tag{3.261}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \mathbf{W}^{\mathsf{T}}\mathbf{G}. \tag{3.262}$$

These derivatives have different roles. The first determines the weight update; the second propagates the loss gradient to earlier layers. Freezing a weight matrix removes the need to calculate its weight gradient, but the input gradient may still be necessary if earlier components are trainable.

For a selected block with row indices I and column indices J, the required weight gradient is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{I,J}} = \mathbf{G}_{I,:}\mathbf{X}^{\mathsf{T}}_{J,:}. \tag{3.263}$$

Only the corresponding rows of the output gradient and columns of the activation are required for this block's weight-gradient calculation. SMT therefore saves selected activation components and computes only the selected weight gradients. It does not eliminate every backward operation: the activation-gradient calculations that remain necessary for propagation through the computational graph are retained.

The implementation replaces selected linear layers with customized sparse linear layers. After selection, the chosen blocks are gathered and concatenated into a dense tensor of shape $m \times l \times l$. The method does not rely on general sparse matrix-matrix multiplication for these updates; it uses dense matrix multiplication within the selected blocks. This is a hardware-aware choice because irregular sparse indexing can introduce overhead, whereas dense matrix multiplication is highly optimized on contemporary accelerators. Figure 3.11 summarizes the selection and update process.

Why attention and value projections? The SMT experiments freeze MLP layers and apply their selected updates to the query, key, and value projections in attention. The authors compare parameter budgets allocated to MLPs and attention. On LLaMA-7B, with a reported trainable budget of approximately 0.84%, allocating all selected parameters to attention produces an average commonsense accuracy of 78.7%, compared with 76.7% when the same budget is allocated to MLPs. Intermediate allocations produce intermediate results in the reported ablation (**he2025smt**).

The authors further compare the query, key, and value projections. Their reported average accuracies are 73.9% for key-only tuning, 75.3% for query-only tuning, 78.5% for value-only tuning, and 78.7% for joint query-key-value selection. SMT assigns approximately 95.17% of its selected trainable parameters to value projections in the reported visualization. These findings are empirical observations for the studied LLaMA configurations and downstream benchmarks; they should not be generalized into a universal

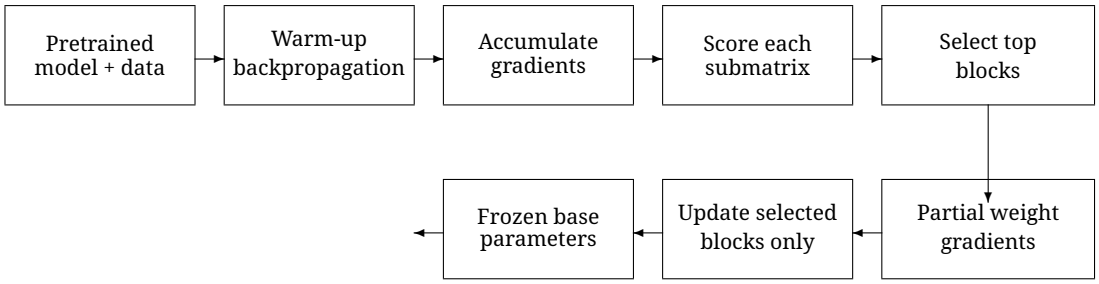


Figure 3.11: SMT workflow. A short warm-up phase accumulates gradients, submatrix scores determine the selected blocks, and a customized backward implementation computes and updates only the selected submatrix gradients. The diagram is an original schematic based on **he2025smt**.

claim that value projections are always the most important parameters in every Transformer.

The attention computation helps explain why gradient magnitudes can differ among these projections. For scaled dot-product attention,

$$\mathbf{S} = \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}, \qquad \mathbf{P} = \mathrm{softmax}(\mathbf{S}), \qquad \mathbf{O} = \mathbf{P}\mathbf{V}. \tag{3.264}$$

For a row of softmax probabilities,

$$\frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j). \tag{3.265}$$

If one probability approaches one and the others approach zero, the corresponding softmax derivatives approach zero. This is softmax saturation. The gradient with respect to the value matrix follows directly through the weighted aggregation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{V}} = \mathbf{P}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{O}}. \tag{3.266}$$

By contrast, gradients with respect to $\mathbf{Q}$ and $\mathbf{K}$ pass through the softmax Jacobian. For one query vector $\mathbf{q}$ and key vectors $\mathbf{k}_j$, the score derivative is

$$\frac{\partial s_j}{\partial \mathbf{q}} = \frac{\mathbf{k}_j}{\sqrt{d_k}}, \qquad \frac{\partial s_j}{\partial \mathbf{k}_j} = \frac{\mathbf{q}}{\sqrt{d_k}}. \tag{3.267}$$

Consequently, saturation can attenuate the gradients reaching query and key projections. This provides a mechanism consistent with the gradient differences reported by the SMT authors, although the exact magnitude also depends on the data, model state, and attention distribution. The scaling factor $1/\sqrt{d_k}$ controls score variance under the usual independent, zero-mean component assumptions. If the query and key components have variances σ_Q^2 and σ_K^2, respectively, then

$$\mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = d_k \sigma_Q^2 \sigma_K^2, \qquad \mathrm{Var}\left(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}\right) = \sigma_Q^2 \sigma_K^2. \tag{3.268}$$

The scaling controls the growth of score variance with d_k, but does not guarantee that attention probabilities cannot become saturated.

Reported performance and system efficiency. The commonsense results reported by **he2025smt** show that SMT can approach full fine-tuning while updating a small fraction of parameters. For LLaMA-7B, the reported average accuracy is 81.4% for full fine-tuning. At approximately 0.84% trainable parameters, SMT obtains 78.7%, compared with 76.3% for LoRA, 78.4% for DoRA, and 76.5% for SpIEL. With a larger SMT budget of approximately 4.91%, the reported average reaches 81.4%. These figures are benchmark-specific and should be read together with the stated model, dataset, and hyperparameter configuration.

Table 3.1: LLaMA-7B average accuracy on the commonsense reasoning benchmark reported by **he2025smt**. The trainable-parameter percentages and accuracy values are reproduced from the paper's comparison table.

Method	Trainable parameters (%)	Average accuracy (%)
Full fine-tuning	100.00	81.4
LoRA	0.83	76.3
DoRA	0.84	78.4
SpIEL	0.84	76.5
SMT	0.84	78.7
SMT (larger budget)	4.91	81.4

For the LLaMA-7B timing comparison, the paper reports experiments on four NVIDIA A100 40 GB GPUs. Under that configuration, full fine-tuning takes 243.84 seconds, while SMT with a 1.26% trainable-parameter budget takes 16.68 seconds, corresponding to a reported $14.6\times$ speedup. LoRA, DoRA, and SpIEL take 17.82, 18.04, and 25.45 seconds, respectively, under the reported comparison settings. These wall-clock values are configuration-specific, not universal speedup guarantees. The paper also reports a 67% reduction in GPU memory footprint relative to full fine-tuning and a 99.5% reduction in optimizer memory under its stated analysis (**he2025smt**).

Complexity comparison. The parameter count is only one part of the cost of an adaptation method. A useful comparison distinguishes trainable parameters, additional parameters, forward-pass overhead, backward-pass work, and the storage required by optimizer state. Table 3.2 summarizes representative parameter-count scalings. These expressions are schematic: the exact count depends on how many layers and matrices are adapted, whether biases are included, and the implementation details.

For LoRA and adapters, additional trainable modules are introduced and executed in the forward path unless they are merged or otherwise transformed for inference. Selective tuning and SMT update selected portions of the original parameters. Prompt methods add trainable representations and may increase the effective sequence length. Quantization-based methods address the numerical precision and storage of the frozen base model, which is a different axis of efficiency from reducing the number of trainable parameters. The survey's complexity discussion distinguishes total trainable parameters

Table 3.2: Representative trainable-parameter scalings for PEFT methods. Here P is the base-model parameter count, L is the number of adapted layers, d is the hidden dimension, d_b is the adapter bottleneck, m is the prompt length, and r is the LoRA rank.

Method	Approximate trainable parameters	Principal restriction
Full fine-tuning	P	No restriction on the update
Serial adapters	$2Ldd_b$	Bottleneck transformations
Soft prompt tuning	md	Input-level prompt vectors
Prefix tuning	$\mathcal{O}(Lmd)$	Layer-wise attention prefixes
LoRA	$r(d_{\text{in}} + d_{\text{out}})$ per matrix	Low-rank update
Selective tuning	$q \ll P$	Selected original parameters
SMT	ml^2 per matrix	Selected full-rank blocks
Propulsion	$\mathcal{O}(d)$	Feature-wise scaling

from additional parameters and emphasizes that parameter count alone does not capture all deployment costs (**prottasha2025peft**).

3.5.5 Limitations, Research Directions, and Connection to Scaling

PEFT trades unrestricted adaptation capacity for a smaller optimization problem. This trade-off can improve efficiency and may reduce estimation variance on low-resource tasks, but a restrictive parameterization can increase approximation error when the downstream task requires substantial changes to the pretrained representation. The survey notes that PEFT can underperform when a task demands significant adaptation or differs sharply from the pretraining application. Adapter dimensions, prompt lengths, rank values, layer placements, and selection thresholds can also be sensitive hyperparameters (**prottasha2025peft**).

The statistical trade-off can be expressed conceptually as a balance between approximation error and estimation error. A highly flexible full fine-tuning procedure can represent a broad set of task-specific changes, but estimating a large number of parameters from limited data can increase variance. A constrained PEFT method reduces the dimension of the adaptation problem, but may not contain an update that adequately represents the target task. The best method is therefore not necessarily the one with the fewest trainable parameters. It is the method that meets the task's performance requirement within the available compute, memory, storage, and latency budgets.

Several research questions follow from this perspective. A theoretical account of parameter influence could connect the mutual information between adaptation modules and predictions with the curvature of the loss landscape. For example, the Hessian

$$\mathbf{H} = \nabla^2_{\boldsymbol{\theta}} \mathcal{L}_{\text{task}} \tag{3.269}$$

describes local second-order curvature. Studying how low-dimensional adaptation subspaces align with sensitive directions of this curvature could help explain why some small updates are effective. The survey identifies theoretical understanding of parameter influence as an open research direction, rather than presenting a completed general theory (**prottasha2025peft**).

Layer-wise sensitivity is another open problem. If q_l parameters are assigned to layer l, a simplified allocation problem is

$$\max_{\{q_l\}_{l=1}^{L}} \sum_{l=1}^{L} \Delta\mathcal{P}_l(q_l) \quad \text{subject to} \quad \sum_{l=1}^{L} q_l \leq Q, \tag{3.270}$$

where $\Delta\mathcal{P}_l(q_l)$ denotes the performance contribution associated with the parameter budget assigned to layer l, and Q is the total budget. This formulation makes explicit why uniform allocation need not be optimal. Jacobian analysis and Fisher-information estimates are possible tools for identifying sensitive layers. SMT provides one empirical example of nonuniform allocation by selecting blocks according to their gradient scores (**prottasha2025peft**; **he2025smt**).

Task-aware and domain-specific adaptation also remains important. Legal, medical, mathematical, and multilingual tasks can impose different representational requirements. Future PEFT designs may use structure-aware prompts, hierarchical adapters, or task-conditioned reparameterization. Multimodal and non-Transformer architectures introduce further challenges: an adaptation must preserve alignment between modalities while modifying only selected components. The survey identifies vision-language, speech, video, graph, and recurrent architectures as areas in which PEFT principles can be developed beyond conventional text-only Transformers (**prottasha2025peft**).

Continual learning requires a model to acquire new tasks without losing previous capabilities. Dynamically growing adapter banks, adapter composition, sparse adapter stacking, and delta compression are possible directions, but they introduce decisions about module selection, interference, and storage. Interpretability is also unresolved: a small adapter or prompt is not automatically understandable. Attribution methods and activation visualization are needed to explain how task-specific modules affect model predictions.

Privacy-preserving and federated PEFT may reduce communication by exchanging small adaptation updates rather than complete model copies. However, parameter efficiency alone does not provide a privacy guarantee; adaptation parameters can still reveal information about training examples. Differential privacy and secure aggregation must be treated as additional mechanisms. Standardized benchmarks are likewise necessary so that methods are compared under consistent models, datasets, trainable budgets, hardware, latency, and memory measurements. Hardware-aware and energy-aware design is essential because mathematical sparsity or a reduced parameter count does not automatically translate into faster execution. Finally, meta-PEFT aims to learn how to select adapter placements, prompt structures, rank values, or other adaptation configurations across tasks, reducing manual hyperparameter search (**prottasha2025peft**).

The role of PEFT in the optimization narrative is now clear. Pretraining constructs a reusable parameterized model; full fine-tuning adapts that model by optimizing its complete parameter vector; and PEFT restricts or reparameterizes the task-specific update to reduce the cost of adaptation. The different methods make distinct compromises between flexibility, regularization, modularity, optimizer memory, and hardware efficiency. SMT illustrates that the organization of the selected parameters and the implementation of their gradients can matter as much as the abstract parameter count.

This completes the discussion of optimization-side adaptation mechanisms. The next stage of the book moves from the question of how a model is optimized or adapted to the

broader question of how model performance, data, and computational resources relate as model scale changes. That question leads naturally to the study of scaling laws.

4. Scaling Laws

The preceding discussions of language-model architecture, optimization, and training objectives establish how an autoregressive model is parameterized and how its parameters are adjusted to improve next-token prediction. A further question follows naturally from these foundations: how should the size of the model, the quantity of training data, and the computational resources be increased as the intended system becomes larger? Answering this question requires more than observing that larger models often achieve lower predictive loss. It requires a quantitative account of how performance changes with scale, how the different resources interact, and how a limited computational budget should be allocated.

Training a large language model is expensive not only because the final model may contain billions or trillions of parameters, but also because identifying a suitable configuration requires experiments with different architectures, training-data quantities, optimization settings, and training durations. Exhaustively testing every configuration at the target scale is generally impractical. Scaling laws address this difficulty by fitting mathematical relationships to observations obtained from a manageable collection of experiments and using those relationships to predict the behavior of larger systems. They therefore connect empirical machine-learning practice with statistical estimation, numerical optimization, and constrained resource allocation (**Sengupta2025ScalingSurvey**; **Li2025MisFitting**).

For autoregressive language models, the central object is usually a predictive loss measured on a specified data distribution. The scaling variables traditionally include the number of model parameters, the number of training tokens, and the computational expenditure required to process those tokens. The conventional framework is subsequently extended when inference latency, architectural shape, or sparse conditional computation becomes important. These extensions are necessary because a model that is optimal in pre-training FLOPs need not be optimal in memory use, inference latency, throughput, or deployment cost.

A scaling law is an empirical predictive model, not a universal identity. Its coefficients and exponents depend on the model family, data distribution, training procedure, evaluation metric, experimental range, and fitting methodology. The objective of this section is to develop the foundations in a logical order: first defining the predictive quantity and scale variables, then deriving power-law and compute-optimal relationships, explaining how these relationships are estimated and validated, and finally examining inference-aware, architecture-aware, and mixture-of-experts (MoE) extensions.

4.0.1 Predictive loss and the fundamental scaling variables

Autoregressive prediction and cross-entropy. An autoregressive language model factorizes the probability of a token sequence into conditional next-token probabilities. For a sequence

$$X = (x_1, x_2, \ldots, x_T),$$

where T is the sequence length, the model with parameters $\boldsymbol{\theta}$ represents

$$p_{\boldsymbol{\theta}}(X) = \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(x_t \mid x_{<t}), \tag{4.1}$$

where $x_{<t} = (x_1, \ldots, x_{t-1})$ denotes the context preceding position t. This factorization follows from the chain rule of probability and reduces sequence modeling to a succession of conditional prediction problems.

Let the vocabulary contain V tokens. At position t, the model produces a probability vector

$$\mathbf{p}_{\boldsymbol{\theta}}^{(t)} = [p_{\boldsymbol{\theta}}(1 \mid x_{<t}), \ldots, p_{\boldsymbol{\theta}}(V \mid x_{<t})]^{\mathsf{T}} \in \mathbb{R}^V.$$

The target token can be represented by a one-hot vector $\mathbf{y}^{(t)} \in \{0,1\}^V$, whose component corresponding to the observed token is one. The cross-entropy at this position is

$$\ell_t(\boldsymbol{\theta}) = -\sum_{v=1}^{V} y_v^{(t)} \log p_{\boldsymbol{\theta}}(v \mid x_{<t}) = -\log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}). \tag{4.2}$$

The second equality follows from the one-hot structure of the target distribution.

For D token-prediction targets, the empirical average negative log-likelihood is

$$\widehat{\mathcal{L}}_{\text{train}}(\boldsymbol{\theta}) = -\frac{1}{D} \sum_{t=1}^{D} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}). \tag{4.3}$$

Minimizing this quantity encourages the model to assign higher probability to the observed next tokens. At the population level, the corresponding predictive risk is an expectation over the data-generating distribution $\mathcal{D}$:

$$\mathcal{L}_{\mathcal{D}}(\boldsymbol{\theta}) = \mathbb{E}_{X \sim \mathcal{D}} \left[-\frac{1}{T} \sum_{t=1}^{T} \log p_{\boldsymbol{\theta}}(x_t \mid x_{<t}) \right]. \tag{4.4}$$

The empirical training loss and the population-level risk are not interchangeable. Training loss is computed on observations used for optimization, whereas validation or test loss is computed on separate observations to estimate generalization. A scaling study must therefore state which loss is being fitted and must not silently treat training and evaluation measurements as equivalent.

For natural-log cross-entropy, perplexity is

$$\text{PPL} = \exp(\mathcal{L}). \tag{4.5}$$

Because the exponential function is strictly increasing, minimizing cross-entropy and minimizing perplexity induce the same ordering of models, although their numerical

scales differ. Cross-entropy is particularly useful for scaling analysis because it is a continuous measure of probabilistic prediction. A discrete downstream metric, such as accuracy, can remain unchanged over a range of loss improvements and then change abruptly.

This distinction matters when extrapolating pre-training behavior to downstream tasks. A reduction in language-model loss does not imply a fixed or universal increase in task accuracy. The relationship between loss and accuracy can vary by task, as illustrated in the downstream comparisons reported by **Bian2025InferenceEfficient**. Accordingly, a scaling law must always be interpreted as a model of the metric that was actually fitted.

Model size. Let

$$N = |\boldsymbol{\theta}| \tag{4.6}$$

denote the number of trainable scalar parameters under a specified counting convention. Parameters are the learned numerical entries of weight matrices, bias vectors, embedding matrices, and other trainable tensors. In a decoder-only Transformer, the parameter count depends on such architectural quantities as hidden dimension, number of layers, attention projections, and feed-forward dimensions.

The phrase "number of parameters" must be defined carefully. Some studies count embedding and output-projection parameters, whereas others report non-embedding parameters. A useful bookkeeping identity is

$$N_{\text{total}} = N_{\text{nonembed}} + N_{\text{embed}}, \tag{4.7}$$

where the exact components included in N_{embed} depend on the study's convention. These alternatives can assign different numerical sizes to the same architecture and can therefore change fitted scaling coefficients and resource ratios. The distinction becomes even more important for sparse models, in which stored parameters and parameters activated for a token are not equal (**Li2025MisFitting**; **Ludziejewski2025JointMoE**).

Training-data volume. Let D denote the number of training tokens processed during pre-training. The token count depends on the tokenizer and preprocessing pipeline; a fixed quantity of raw text can yield different token counts under different tokenization procedures. It is therefore necessary to distinguish raw corpus size from tokenized training volume.

It is also useful to distinguish unique tokens from processed token exposures. If a corpus contains D_{unique} tokens and is traversed for E_{epoch} complete epochs, then, under the simplifying assumption that every epoch processes the entire corpus,

$$D_{\text{processed}} = E_{\text{epoch}} D_{\text{unique}}. \tag{4.8}$$

The processed-token count measures training exposures; it does not measure the quantity of unique information available to the model. Repeated exposure to the same observations can behave differently from adding new data. Moreover, datasets with equal token counts need not have equal statistical value: domain composition, repetition, noise, and information content can affect the learning outcome. Data volume is therefore a useful scale variable, but not a complete description of data quality (**Sengupta2025ScalingSurvey**; **Li2025MisFitting**).

Training computation. Let C denote training computation, commonly approximated in floating-point operations (FLOPs). A FLOP is a floating-point arithmetic operation. FLOPs approximate arithmetic work but do not directly determine wall-clock time, energy use, monetary cost, or hardware utilization.

For a conventional dense Transformer, a widely used approximation is

$$C(N, D) \approx 6ND. \tag{4.9}$$

The factor of approximately six is a conventional accounting of forward and backward arithmetic per parameter per processed token: roughly two operations per parameter for the forward computation and approximately four additional operations for the backward computation. Thus, the approximate training computation per token is $6N$, and processing D tokens gives $C \approx 6ND$.

For example, a dense model with $N = 10^9$ parameters trained on $D = 20 \times 10^9$ tokens has approximate training computation

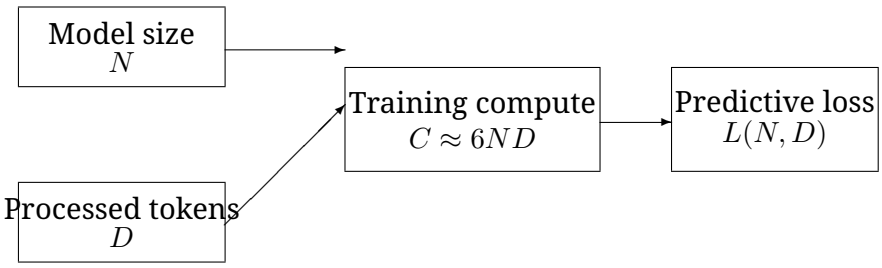
$$\begin{aligned} C &\approx 6(10^9)(20 \times 10^9) \\ &= 1.2 \times 10^{20} \text{ FLOPs}. \end{aligned} \tag{4.10}$$

This estimate is an approximation, not an exact accounting of every implementation. Sequence-length-dependent attention computation, embedding operations, architectural details, and implementation-specific work can introduce additional terms. Long-context models are a particularly important case because attention-related computation can depend explicitly on context length (**Li2025MisFitting**).

The three conventional variables are therefore linked by

$$C \approx 6ND. \tag{4.11}$$

Under a fixed compute budget, increasing N requires decreasing D; increasing D requires either decreasing N or increasing C. The central resource-allocation question is consequently not whether parameters or data are independently beneficial, but how they should be balanced to minimize predictive loss under a specified computational constraint.



Dense-model approximation; architecture and data quality also matter.

Figure 4.1: The conventional scaling variables and their relationship. Model size N and processed training tokens D determine approximate dense-model training computation $C \approx 6ND$, while predictive loss is modeled as a function of these variables.

4.0.2 Power-law formulations and compute-optimal allocation

Univariate power laws. A power law describes a relationship of the form

$$y = ax^k, \tag{4.12}$$

where a is a positive coefficient and k is a scaling exponent. When y is a loss that decreases with scale, a common form is

$$L(x) = L_\infty + ax^{-\alpha}, \qquad a > 0, \quad \alpha > 0, \tag{4.13}$$

where L_∞ is the asymptotic component under the assumed model and α controls the rate at which the scale-dependent excess loss decreases.

Differentiating with respect to x gives

$$\frac{dL}{dx} = -a\alpha x^{-\alpha-1} < 0, \tag{4.14}$$

so increasing x decreases the modeled loss. The second derivative is

$$\frac{d^2L}{dx^2} = a\alpha(\alpha+1)x^{-\alpha-2} > 0, \tag{4.15}$$

which expresses diminishing absolute improvements as scale increases.

Subtracting the asymptotic term and taking logarithms yields

$$\log(L - L_\infty) = \log a - \alpha \log x. \tag{4.16}$$

The slope in log-log coordinates is $-\alpha$. Equivalently,

$$\frac{d\log(L - L_\infty)}{d\log x} = -\alpha,$$

so the exponent is the elasticity of excess loss with respect to scale. If the scale is multiplied by $q > 0$, then

$$\frac{L(qx) - L_\infty}{L(x) - L_\infty} = q^{-\alpha}. \tag{4.17}$$

Thus, the exponent quantifies the proportional reduction in excess loss associated with a proportional increase in the scaling variable.

Multivariable power laws. Language-model loss depends on more than one resource. A general additive form is

$$L(P_1, \ldots, P_m) = L_\infty + \sum_{i=1}^{m} A_i P_i^{-\alpha_i}, \tag{4.18}$$

where P_i is a scaling variable, A_i is its coefficient, and α_i is its exponent. For model size N and training tokens D, a conventional formulation is

$$\boxed{L(N, D) = E + AN^{-\alpha} + BD^{-\beta}} \tag{4.19}$$

with positive coefficients A, B, positive exponents α, β, and asymptotic term E.

The limiting cases clarify the interpretation. If model size increases without bound while data volume is fixed, then

$$\lim_{N\to\infty} L(N, D) = E + BD^{-\beta}. \tag{4.20}$$

The model-size contribution vanishes, but the finite-data contribution remains. Conversely,

$$\lim_{D\to\infty} L(N, D) = E + AN^{-\alpha}, \tag{4.21}$$

and if both variables increase without bound,

$$\lim_{\substack{N\to\infty \\ D\to\infty}} L(N, D) = E. \tag{4.22}$$

The additive form is an empirical decomposition of the modeled excess loss; it does not establish that these are the only possible sources of error.

The additive assumption also differs from a model with explicit interaction terms. A more general illustrative expression is

$$L(N, D) = E + AN^{-\alpha} + BD^{-\beta} + GN^{-\mu}D^{-\nu}, \tag{4.23}$$

where the final term represents a possible interaction. The exact functional form must be supported by the experimental observations and the scientific hypothesis. The Kaplan-style and Chinchilla-style formulations discussed in the scaling-law literature encode different assumptions about the interaction between model size and data volume; their fitted extrapolations can therefore differ even when both describe a finite set of observations reasonably well (**Li2025MisFitting**; **Sengupta2025ScalingSurvey**).

Compute-optimal resource allocation. Once a loss function and a computational constraint are specified, the resource-allocation problem becomes a constrained optimization problem:

$$\begin{aligned} \min_{N,D} \quad & L(N, D) \\ \text{subject to} \quad & 6ND = C. \end{aligned} \tag{4.24}$$

Substituting $D = C/(6N)$ into the additive loss in equation (4.19) gives

$$L_C(N) = E + AN^{-\alpha} + B\left(\frac{C}{6}\right)^{-\beta} N^{\beta}. \tag{4.25}$$

The second term decreases as N increases, while the third increases because a larger model leaves fewer tokens available under the fixed computational budget. The optimum balances these opposing effects.

Differentiating and setting the derivative to zero gives

$$\begin{aligned} \frac{dL_C}{dN} &= -\alpha AN^{-\alpha-1} + \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} = 0, \\ \alpha A &= \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\alpha+\beta}, \\ N^{\alpha+\beta} &= \frac{\alpha A}{\beta B}\left(\frac{C}{6}\right)^{\beta}. \end{aligned} \tag{4.26}$$

Hence the compute-optimal parameter count is

$$\boxed{N^*(C) = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}}} \tag{4.27}$$

and the corresponding token count, obtained from the compute constraint, is

$$\boxed{D^*(C) = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}}}. \tag{4.28}$$

Therefore,

$$N^*(C) \propto C^{\frac{\beta}{\alpha+\beta}}, \qquad D^*(C) \propto C^{\frac{\alpha}{\alpha+\beta}}. \tag{4.29}$$

The optimal allocation depends on the relative loss sensitivities encoded by α and β, as well as on the coefficients A and B.

Dividing the two optimal quantities gives the optimal token-to-parameter ratio:

$$\boxed{\frac{D^*(C)}{N^*(C)} = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\alpha-\beta}{\alpha+\beta}}}. \tag{4.30}$$

If $\alpha = \beta$, the ratio is independent of compute:

$$\frac{D^*}{N^*} = \left(\frac{B}{A}\right)^{\frac{1}{2\alpha}}, \qquad N^*(C) \propto C^{1/2}, \qquad D^*(C) \propto C^{1/2}. \tag{4.31}$$

If $\alpha > \beta$, the optimal token-to-parameter ratio increases with compute; if $\alpha < \beta$, it decreases. A constant ratio is therefore a consequence of a particular relationship between fitted exponents, not a universal property of language-model training.

Kaplan-style and Chinchilla-style conclusions. The scaling literature reports different empirical conclusions about how optimal data volume should grow with model size. The Kaplan-style relationship is commonly summarized as

$$D^* \propto (N^*)^{0.74}, \tag{4.32}$$

In this relationship, parameter count increases faster than data volume. A later Chinchilla-style result instead reports approximately proportional growth:

$$D^* \propto N^*, \qquad D^* \approx 20N^* \tag{4.33}$$

when N^* is expressed as a parameter count and D^* as a token count. The latter is an empirical rule of thumb associated with the studied conditions, not a universal prescription.

The difference between these conclusions should not be reduced to a claim that one ratio is correct for every architecture and data regime. The replication and survey analysis of **Li2025MisFitting** identifies training hyperparameters, learning-rate schedules, parameter-counting conventions, FLOP calculations, checkpoint selection, and fitting procedures as factors that can alter the estimated relationship. The fitted optimum is conditional on the experimental design and on the validity of the chosen functional form.

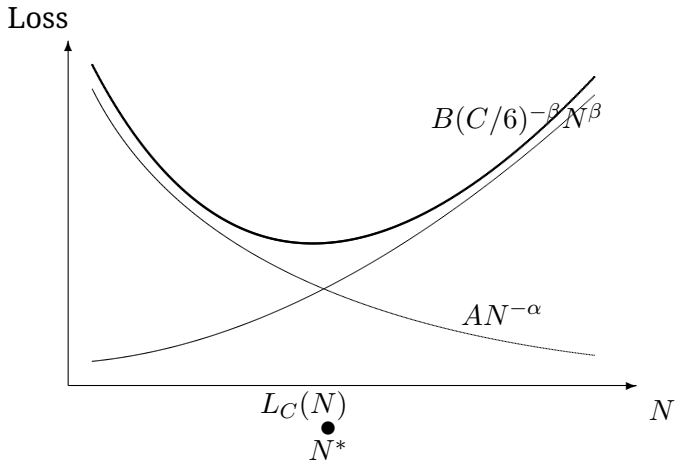


Figure 4.2: Schematic fixed-compute trade-off. Increasing parameter count reduces the model-capacity contribution to loss but increases the data-limitation contribution because fewer tokens can be processed. Their sum has an interior minimum under the assumptions of the additive scaling model.

IsoFLOPs analysis. The compute-optimal derivation assumes that the scaling function is known. In practice, its coefficients and exponents must be estimated. An IsoFLOPs experiment fixes a computational budget C_i and evaluates several configurations satisfying

$$6ND = C_i, \qquad D = \frac{C_i}{6N}. \tag{4.34}$$

Each configuration is trained and its loss measured. The lowest observed loss along the fixed-compute profile identifies the empirical optimum for that budget. Repeating the experiment for several budgets yields observations of the form

$$\{(C_i, N_i^*, D_i^*)\}_{i=1}^{K},$$

which can be used to estimate how the optimal configuration changes with compute.

IsoFLOPs analysis directly investigates resource allocation, but it can be expensive because multiple configurations may need to be trained at each budget. Intermediate checkpoints and interpolation can reduce the number of full training runs, although the resulting estimates depend on the appropriateness of the interpolation procedure. The distinction between predicting loss for arbitrary configurations and estimating an optimal resource ratio is important: the former requires a predictive loss surface, whereas the latter focuses on the minimizing configuration (**Li2025MisFitting**).

4.0.3 Empirical fitting and validation

Experimental observations and fitting objective. A scaling law is estimated from a dataset of trained and evaluated models. For the additive form, write

$$\widehat{L}(N, D; \boldsymbol{\phi}) = E + AN^{-\alpha} + BD^{-\beta}, \qquad \boldsymbol{\phi} = (E, A, B, \alpha, \beta). \tag{4.35}$$

If the i-th experiment uses (N_i, D_i) and produces measured loss L_i, the fitting dataset is

$$\mathcal{S} = \{(N_i, D_i, L_i)\}_{i=1}^{m}. \tag{4.36}$$

The parameter-estimation problem is

$$\widehat{\phi} = \arg\min_{\phi} \sum_{i=1}^{m} \ell\Big(L_i, \widehat{L}(N_i, D_i; \phi)\Big), \tag{4.37}$$

where ℓ is a chosen discrepancy function. The number and distribution of observations matter. If all experiments use nearly the same parameter-to-token ratio, it can be difficult to identify the separate effects of model size and data volume. A narrow range of model sizes also provides weak evidence for extrapolation to substantially larger models.

Training conditions should be controlled or reported. Architecture, batch size, learning-rate schedule, optimizer, data composition, training duration, and evaluation procedure can all affect measured loss. In particular, a learning rate that is appropriate for a small model may be suboptimal for a larger model. If optimization quality changes systematically with model size, the fitted scaling law can mistakenly attribute optimization differences to scale itself. The survey by **Li2025MisFitting** emphasizes the sensitivity of scaling fits to these experimental decisions.

Checkpoint selection and training trajectories. A training run provides more than one final observation. Let $\boldsymbol{\theta}_i(t)$ denote the parameters of model i after processing t tokens, and let

$$L_i(t) = \mathcal{L}_{\text{eval}}(\boldsymbol{\theta}_i(t)) \tag{4.38}$$

denote its evaluated loss. Intermediate checkpoints can provide observations at different training durations and can be useful when constructing fixed-compute profiles. Interpolation between checkpoints can estimate a loss at an intermediate token count, but this introduces assumptions about the smoothness of the learning trajectory. The choice between final checkpoints, multiple checkpoints, smoothed trajectories, and interpolated observations can affect the fitted law (**Li2025MisFitting**).

Least-squares and robust objectives. A common fitting objective is mean squared error:

$$J_{\text{MSE}}(\phi) = \frac{1}{m} \sum_{i=1}^{m} \Big[L_i - \widehat{L}(N_i, D_i; \phi)\Big]^2. \tag{4.39}$$

Squaring residuals prevents positive and negative errors from cancelling and penalizes large residuals more heavily. Mean absolute error is

$$J_{\text{MAE}}(\phi) = \frac{1}{m} \sum_{i=1}^{m} \Big|L_i - \widehat{L}(N_i, D_i; \phi)\Big|. \tag{4.40}$$

Its linear penalty makes it less sensitive than MSE to individual large residuals.

The Huber loss combines quadratic and linear behavior. For residual r and threshold $\delta > 0$,

$$\ell_\delta(r) = \begin{cases} \frac{1}{2} r^2, & |r| \le \delta, \\ \delta\left(|r| - \frac{\delta}{2}\right), & |r| > \delta. \end{cases} \tag{4.41}$$

For small residuals it behaves quadratically, while for large residuals it grows linearly. The threshold δ determines the transition. The scaling-law literature uses variants of MAE, MSE, Huber loss, and custom objectives; the selected objective should be reported because it changes the relative influence of observations (**Li2025MisFitting**).

Nonlinear numerical optimization. The exponents in a power law are unknown parameters, so fitting is generally a nonlinear optimization problem. BFGS is a quasi-Newton method that uses gradient information and an approximation to the Hessian. A Newton-type update is

$$\phi_{k+1} = \phi_k - H_k^{-1} \nabla J(\phi_k), \tag{4.42}$$

where H_k is the Hessian or its approximation. BFGS updates an approximation to curvature information from successive parameter and gradient changes. L-BFGS stores only a limited history of these changes, reducing memory requirements. These methods are used in the scaling-law literature, including in fitting procedures surveyed by **Li2025MisFitting**.

Levenberg–Marquardt is another method for nonlinear least-squares optimization. If $\mathbf{r}_k$ is the residual vector and J_k is its Jacobian with respect to the fitted parameters, a damped update is

$$\phi_{k+1} = \phi_k - \left(J_k^\top J_k + \lambda I \right)^{-1} J_k^\top \mathbf{r}_k, \tag{4.43}$$

where $\lambda > 0$ is a damping parameter and I is the identity matrix. Larger damping makes the update more gradient-like, whereas smaller damping approaches a Gauss–Newton step. Levenberg–Marquardt is used in the architecture-related scaling analyses of **Bian2025InferenceEfficient** and **Bian2026ArchitectureScaling**.

Initialization is consequential because nonlinear objectives can contain flat regions, poorly conditioned directions, or multiple local minima. Grid search, random initializations, and initial values informed by prior fits are alternative strategies. The replication analysis in **Li2025MisFitting** shows that initialization can materially affect the fitted result. An optimization report should therefore state the algorithm, objective, initialization, parameter constraints, stopping criterion, and any model or observation exclusions.

Logarithmic transformations. The simple power law $y = ax^{-\alpha}$ can be linearized as

$$\log y = \log a - \alpha \log x. \tag{4.44}$$

For a loss with an asymptotic term, the corresponding transformation is

$$\log(L - E) = \log a - \alpha \log x,$$

so the asymptotic term must be accounted for. More importantly, minimizing squared error in the original loss space is not equivalent to minimizing squared error after a logarithmic transformation:

$$\sum_i (L_i - \widehat{L}_i)^2 \neq \sum_i (\log L_i - \log \widehat{L}_i)^2. \tag{4.45}$$

The transformation changes the error weighting and can give greater relative influence to errors at smaller loss values. Linearization may be computationally convenient, but it changes the statistical objective and should not be assumed to be neutral (**Li2025MisFitting**).

Validation and predictive reliability. A function that fits the observations used for estimation is not necessarily reliable outside that dataset. Residuals are

$$r_i = L_i - \widehat{L}(N_i, D_i; \widehat{\phi}). \tag{4.46}$$

Systematic residual patterns can indicate that the assumed functional form omits an important variable or interaction. The mean squared error and coefficient of determination are

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m} r_i^2, \tag{4.47}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{m}(L_i - \widehat{L}_i)^2}{\sum_{i=1}^{m}(L_i - \overline{L})^2}, \qquad \overline{L} = \frac{1}{m}\sum_{i=1}^{m} L_i. \tag{4.48}$$

The R^2 statistic compares squared prediction error with a baseline that predicts the sample mean. A high value does not, by itself, establish accurate extrapolation.

When the objective is to rank candidate architectures, rank agreement can also matter. For untied ranks R_i and S_i, Spearman's rank correlation is

$$r_s = 1 - \frac{6\sum_{i=1}^{m}(R_i - S_i)^2}{m(m^2-1)}. \tag{4.49}$$

A value close to one indicates strong agreement between the two rankings. The inference- and architecture-aware studies use measures including MSE, R^2, relative prediction error, and Spearman correlation (**Bian2025InferenceEfficient**; **Bian2026ArchitectureScaling**).

Confidence intervals, goodness-of-fit measures, and out-of-sample evaluation provide complementary evidence. A particularly informative extrapolation test fits a law on smaller models and evaluates its predictions on larger models not used during fitting. Successful extrapolation over a finite range is useful evidence, but it does not prove validity at arbitrary scales.

Reproducibility. The survey by **Li2025MisFitting** examines 51 scaling-law studies, of which 45 quantify scaling trends using power laws. The survey reports substantial omissions in the description of training, equation variables, evaluation, fitting procedures, optimization, parameter counting, and FLOP accounting. It also reports limited availability of analysis code, checkpoints, and metric scores. These observations show that a compact equation is not a complete scientific result: the experimental and numerical pipeline that produced its coefficients must also be documented.

A reproducible scaling study should specify the functional form, definitions of all variables, parameter-counting convention, FLOP calculation, model architecture, training data, optimizer, batch size, learning-rate schedule, checkpoint selection, evaluation metric, fitting objective, optimization algorithm, initialization strategy, parameter constraints, and validation procedure. Without these details, independently fitting an equation of the same nominal form need not reproduce the original conclusion.

4.0.4 Inference-efficient and architecture-aware scaling

Training optimality versus deployment optimality. The conventional framework minimizes pre-training loss under a training compute constraint. Deployment introduces

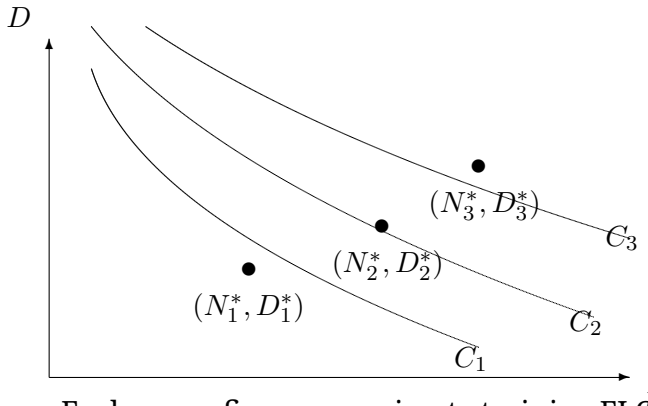


Figure 4.3: Schematic IsoFLOPs analysis. Each curve represents configurations with the same approximate training-compute budget. The minimum-loss configuration on each curve defines an empirical compute-optimal point.

additional objectives, including inference latency, throughput, and memory use. A model that is optimal in training FLOPs need not be optimal for autoregressive generation.

Parameter count alone does not determine inference latency. In a decoder-only Transformer, successive layers are executed sequentially during generation. The number of layers therefore contributes to the sequential depth of the computation, while hidden dimension affects projection sizes, matrix multiplication shapes, and memory access. Models with similar parameter counts can consequently exhibit different inference behavior. The measurements reported by **Bian2025InferenceEfficient** show substantial latency differences between models of comparable size and illustrate the importance of model shape.

Inference-constrained scaling. An inference-aware optimization can be expressed as

$$\begin{aligned} \min_{N,D} \quad & L(N, D) \\ \text{subject to} \quad & N \le N_C, \\ & D \le D_C, \\ & T_{\text{inf}} \le T_C, \end{aligned} \tag{4.50}$$

where N_C and D_C are the available model-size and training-data budgets, and T_C is the maximum permitted inference latency. The latency T_{inf} must be measured or modeled under a specified hardware, batch-size, sequence-length, and decoding configuration; it is not a universal property of parameter count alone.

Bian2025InferenceEfficient introduce the aspect ratio

$$R = \frac{d_{\text{model}}}{n_{\text{layers}}}, \tag{4.51}$$

where d_{model} is the hidden dimension and n_{layers} is the number of Transformer layers. Their inference-efficient loss formulation is

$$\boxed{L(N, D, R) = \left(E + AN^{-\alpha} + BD^{-\beta}\right)(1 + \epsilon R^{\gamma})} \tag{4.52}$$

where ϵ and γ, together with the conventional scaling coefficients, are estimated from observations. The first factor represents the conventional size-and-data relationship, while the second introduces an aspect-ratio-dependent adjustment.

The authors train 63 models across different parameter counts, data budgets, and architectural configurations. Their experiments report that models with similar parameter counts can have different inference latencies and that wider, shallower configurations can offer throughput advantages under the studied settings. They use the fitted relationship together with latency measurements to select candidate architectures. Their reported Morph-1B result demonstrates an inference-latency improvement relative to comparable open-source models under the paper's evaluation conditions. These findings support the broader principle that model selection should account for deployment constraints rather than optimizing pre-training loss in isolation (**Bian2025InferenceEfficient**).

Conditional architecture-aware scaling. Architecture is not merely a fixed background condition. Hidden dimension, layer count, and the allocation of parameters between attention and feed-forward networks are also optimization variables. In a decoder-only Transformer, let

$$r_{\text{MLP/attn}} = \frac{N_{\text{MLP}}}{N_{\text{attn}}} \tag{4.53}$$

denote the ratio of feed-forward parameters to attention parameters. Grouped-query attention (GQA) changes the relationship between query heads and key-value heads and can therefore affect key-value-cache memory and inference throughput.

The architecture-aware study of **Bian2026ArchitectureScaling** reports approximately U-shaped relationships between predictive loss and normalized hidden dimension, and between loss and the MLP-to-attention ratio. This behavior differs from a monotonically decreasing power law: under a fixed parameter budget, an architectural variable can have an interior optimum. The paper uses a representative calibration form

$$f(x) = c_0 + c_1 \log x + \frac{c_2}{x}, \tag{4.54}$$

where $x > 0$. Its derivative is

$$f'(x) = \frac{c_1}{x} - \frac{c_2}{x^2}.$$

For positive c_1, c_2, the stationary point is

$$x^* = \frac{c_2}{c_1}, \qquad f''(x^*) = \frac{c_1^3}{c_2^2} > 0, \tag{4.55}$$

so it is a local minimum.

The normalized hidden dimension is defined in the study as

$$x_d = \frac{d_{\text{model}}}{\sqrt{N_{\text{nonembed}}}}, \tag{4.56}$$

where N_{nonembed} is the non-embedding parameter count. Let $r = r_{\text{MLP/attn}}$. The conditional calibration is represented as

$$L(x_d, r \mid N, D) = \left(a_0 + a_1 \log x_d + \frac{a_2}{x_d}\right) \times \left(b_0 + b_1 \log r + \frac{b_2}{r}\right) L_{\text{opt}}(N, D), \tag{4.57}$$

where

$$L_{\text{opt}}(N, D) = E + AN^{-\alpha} + BD^{-\beta}. \tag{4.58}$$

The coefficients are fitted to experimental observations. The multiplicative form assumes that the two architectural calibration factors can be separated; it is a modeling choice rather than a general property of Transformer architectures.

Architecture selection can then be written as an efficiency optimization subject to a loss constraint:

$$\begin{aligned} \max_{\mathcal{P}} \quad & I_N(\mathcal{P}) \\ \text{subject to} \quad & L(\mathcal{P} \mid N, D) \leq L_t, \end{aligned} \tag{4.59}$$

where $\mathcal{P}$ denotes an architectural configuration, $I_N(\mathcal{P})$ is its inference-efficiency measure, and L_t is the maximum acceptable loss. GQA can be treated as a discrete architectural choice and searched over feasible configurations rather than forced into the same continuous calibration function.

The study trains more than 200 models, spanning approximately 80 million to 3 billion non-embedding parameters and training-data budgets from 8 billion to 100 billion tokens. It reports up to 42% greater inference throughput and 2.1% higher average downstream accuracy than its specified LLaMA-3.2 baselines under the reported conditions. The authors also show that using models closer to the target scale can improve prediction accuracy. These results reinforce that the scale range used for fitting remains important even when architecture is explicitly modeled (**Bian2026ArchitectureScaling**).

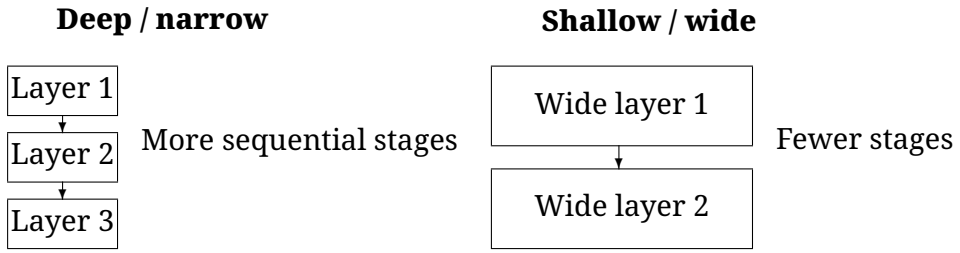


Figure 4.4: Schematic architecture-aware scaling. At a fixed parameter budget, different allocations between depth and width can have different inference costs and predictive losses; an architecture-aware objective searches for efficient configurations subject to a loss constraint.

4.0.5 Joint scaling for mixture-of-experts models

Stored parameters and active parameters. In a dense Transformer, the same feed-forward parameters are applied to each token. A mixture-of-experts model replaces a dense feed-forward component with multiple expert networks and a routing mechanism. The router selects which expert or experts process a token. In the top-one routing configuration considered in the Switch-style setting, a token is routed to one expert.

Let N_{total} be the total number of stored parameters and N_{act} the number of parameters activated for a token. For a dense model, these quantities are approximately equal under a consistent counting convention. For an MoE model,

$$N_{\text{act}} < N_{\text{total}}, \tag{4.60}$$

because only a subset of expert parameters participates in a token's computation. Active parameters are relevant to arithmetic work per token, whereas total parameters are important for weight storage. The number of experts, denoted by E, is an additional architectural variable.

Joint MoE scaling law. **Ludziejewski2025JointMoE** extend the Chinchilla-style relationship to include active parameters, training tokens, and the number of experts. Their joint formulation is

$$\begin{aligned} L(N_{\text{act}}, D, \widehat{E}) = a\widehat{E}^{\delta} N_{\text{act}}^{\alpha+\gamma\ln(\widehat{E})} \\ + b\widehat{E}^{\omega} D^{\beta+\zeta\ln(\widehat{E})} + c, \end{aligned} \tag{4.61}$$

where $\widehat{E}$ is the transformed expert-count variable defined by the study, and $a, b, c, \delta, \alpha, \beta, \gamma, \omega, \zeta$ are fitted parameters. The effective exponents are

$$\mu(E) = \alpha + \gamma\ln(\widehat{E}), \qquad \nu(E) = \beta + \zeta\ln(\widehat{E}). \tag{4.62}$$

Thus, the influence of active parameters and data volume can change with the expert count. For a fixed expert configuration, the expression retains a Chinchilla-style dependence on active parameters and training tokens.

The transformation $\widehat{E}$ is part of the authors' specified functional form and should be implemented according to their definition; it should not be replaced by an assumed transformation when reproducing their fit. The formulation is empirical and its coefficients and exponents must be estimated from observations.

Compute- and memory-constrained MoE scaling. Under the conventional approximation used for the studied sparse models, training computation is expressed using active parameters:

$$F_{\text{train}} \approx 6N_{\text{act}}D. \tag{4.63}$$

The compute-optimal problem is therefore

$$\begin{aligned} \min_{N_{\text{act}}, D, E} \quad & L(N_{\text{act}}, D, E) \\ \text{subject to} \quad & 6N_{\text{act}}D = F, \end{aligned} \tag{4.64}$$

where F is the available training-compute budget. Unlike the dense formulation, the expert count now influences the optimal balance between active parameters and tokens.

The results reported by **Ludziejewski2025JointMoE** show that, for the studied settings and fixed training-compute budget, increasing the number of experts leads to an optimum with fewer active parameters and more training tokens. Their reported configurations for a budget of 10^{20} FLOPs are shown in table 4.1.

Table 4.1: Reported compute-optimal MoE configurations at a training budget of 10^{20} FLOPs. Values are reproduced from the results reported by **Ludziejewski2025JointMoE**.

Number of experts	Optimal active parameters	Optimal training tokens
1	1.7B	9.7B
2	1.5B	11.4B
4	1.2B	13.9B
8	990M	17B
16	810M	20.7B

The optimal ratio of tokens to active parameters changes with expert count. A single fixed ratio cannot describe every MoE configuration.

Training compute is not the only constraint. The total parameter count determines weight-storage requirements, and autoregressive inference also requires memory for the key-value cache. For a decoder with N_{blocks} blocks, hidden dimension d_{model}, and T cached tokens, a simplified cache-element count is proportional to

$$M_{\text{KV}} \propto 2TN_{\text{blocks}}d_{\text{model}}, \tag{4.65}$$

where the factor of two accounts for keys and values. The exact memory in bytes additionally depends on batch size, the number of key-value heads, storage precision, and implementation details. Under a simplified parameter-equivalent accounting, the combined constraint can be represented as

$$N_{\text{total}} + 2TN_{\text{blocks}}d_{\text{model}} \leq M, \tag{4.66}$$

where M is the corresponding memory budget. This expression is a simplified representation of the weight and cache requirements, not a hardware-independent byte-accurate memory model.

The MoE analysis therefore distinguishes three quantities that must not be collapsed into one: total stored parameters, active parameters per token, and memory required by the key-value cache. The study argues that MoE models can be memory-efficient under the investigated constraints even though their total parameter count is larger than that of a corresponding dense model. This conclusion depends on the studied expert configurations and accounting assumptions; FLOPs alone do not capture all communication, memory-access, or hardware execution costs (**Ludziejewski2025JointMoE**).

4.0.6 Limitations, synthesis, and transition

The scaling relationships developed in this section provide a quantitative framework for reasoning about model size, training data, computational expenditure, inference efficiency, and architectural allocation. Their predictive validity is nevertheless conditional.

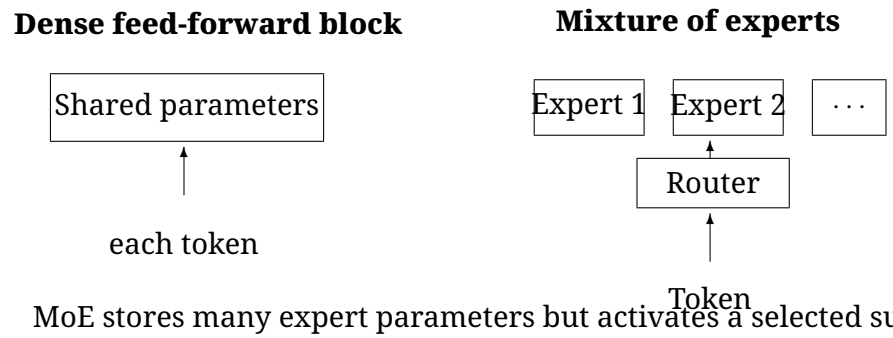


Figure 4.5: Schematic distinction between dense and mixture-of-experts parameter accounting. A dense model activates its shared parameters for each token, whereas an MoE model stores multiple experts and activates only a selected subset for a token.

A scaling law fitted for one architecture should not automatically be transferred to another architecture, because equal parameter counts can correspond to different hidden dimensions, layer counts, attention configurations, and allocations between attention and feed-forward components. The same caution applies to data: equal token counts do not imply equal information content, domain coverage, noise, or repetition.

Extrapolation is another fundamental limitation. A power law fitted over a restricted range can fail when the system enters a different computational or statistical regime. Piecewise or broken-scaling formulations have been proposed to represent changes in scaling behavior, but such extensions require evidence for a transition and additional parameters to estimate. Likewise, predictive loss does not uniquely determine downstream accuracy. The mapping from pre-training loss to task performance can differ across tasks, and downstream evaluation remains necessary when task performance is the actual objective (**Sengupta2025ScalingSurvey**; **Bian2025InferenceEfficient**).

The conventional dense-model formulation begins with $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$ and the approximate computational constraint $C \approx 6ND$. Their combination yields compute-optimal parameter and token allocations. Empirical estimation then requires a carefully designed collection of training runs, explicit definitions of variables, numerical fitting, and validation beyond the observations used for optimization. Inference-aware and architecture-aware formulations add latency and architectural variables, while MoE scaling distinguishes stored parameters from active parameters and introduces memory constraints.

The unifying principle is that scale is multidimensional. Increasing one resource changes the value and cost of the others, and the best configuration depends on the objective being optimized. Scaling laws are useful precisely because they turn these interactions into testable and predictive relationships, but their conclusions remain tied to their assumptions, measurements, and experimental domains. This foundation prepares the subsequent discussion of how training objectives, optimization procedures, and architectural choices interact in the construction of increasingly capable language models.

Part III

Reinforcement Learning and Neuroscience

5. Reinforcement Learning

5.1 Theoretical Foundations

The preceding discussion of model architecture, optimization, and scaling establishes how a language model can be represented and trained as a predictive system. It does not, by itself, specify how the model should change its behavior when the quality of a generated sequence is evaluated through a reward signal. That problem introduces a different mathematical object: a decision-making policy whose actions affect not only the immediate outcome but also the states and decisions that follow. Reinforcement learning provides a framework for reasoning about this sequential dependence.

The purpose of this section is to establish the mathematical foundations needed to understand reinforcement learning as an optimization problem over policies. The development begins with Markov decision processes and Bellman operators, then proceeds to policy improvement and approximation error, information geometry and natural gradients, trust-region optimization, and regularized Markov decision processes. These topics are connected by a common question: how can a policy be improved while controlling the consequences of imperfect value estimates, changes in the distribution of visited states, and finite optimization steps? The cited papers provide the main theoretical structure for this development; the final connection to autoregressive language models is an application of that structure to the subject of this book.

5.1.1 Markov decision processes, policies, and value functions

A reinforcement-learning problem is a sequential decision problem. At each decision time, an agent receives information about the current situation, selects an action, and observes the consequences of that action. The next situation depends on the current situation and action, and the reward may be delayed. Thus, maximizing the reward at the current time is generally not equivalent to maximizing the total reward accumulated over an interaction. The mathematical formulation used throughout the cited work is the discounted Markov decision process (MDP).

For the finite-state and finite-action setting used to develop the matrix identities, an MDP is specified by

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0), \tag{5.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition probability, $r(s, a)$ is the expected immediate reward, $\gamma \in (0, 1)$ is the discount factor, and ρ_0 is the distribution of the initial state. For every state–action pair,

$$P(s' \mid s, a) \geq 0, \qquad \sum_{s' \in \mathcal{S}} P(s' \mid s, a) = 1. \tag{5.2}$$

The finite-state assumption is used only when vectors and matrices are written explicitly. The same definitions extend to general state spaces by replacing finite sums with integrals and transition matrices with probability kernels, as in the formulation considered by **scherrer2014approximate**.

At time t, the agent is in state s_t, chooses action a_t, receives reward r_t, and the environment produces state s_{t+1}. The transition model is

$$P(s' \mid s, a) = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a). \tag{5.3}$$

The Markov property states that, conditioned on the current state and action, the next-state distribution does not depend on the preceding history:

$$\begin{aligned} &\Pr(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0) \\ &\qquad = P(s_{t+1} \mid s_t, a_t). \end{aligned} \tag{5.4}$$

This is a conditional-independence assumption, not an assertion that the environment is deterministic. The transition may remain stochastic. The assumption says that the state representation contains the information needed to determine the conditional distribution of the next state. If the chosen state representation omits information relevant to future transitions or rewards, the process represented by that state need not be Markov.

The policy specifies how the agent selects actions. A stochastic stationary policy is a conditional probability distribution

$$\pi(a \mid s) = \Pr(a_t = a \mid s_t = s), \tag{5.5}$$

with $\pi(\cdot \mid s)$ belonging to the probability simplex

$$\Delta_{\mathcal{A}} = \left\{ p \in \mathbb{R}^{|\mathcal{A}|} : p(a) \geq 0,\ \sum_{a \in \mathcal{A}} p(a) = 1 \right\}. \tag{5.6}$$

A deterministic policy is a special case in which the distribution assigns probability one to a single action. The stochastic formulation is more general and is necessary for the probability-distribution optimization developed later in this section.

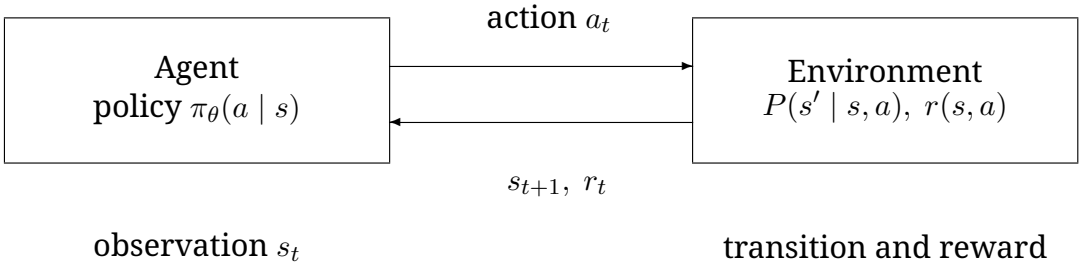


Figure 5.1: The interaction represented by a Markov decision process. The agent selects an action from its policy, and the environment returns the next state and reward according to its transition and reward model.

The objective must account for the complete future reward sequence. For an infinite-horizon discounted problem, the return beginning at time t is

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}. \tag{5.7}$$

The discount factor assigns weight one to the immediate reward and weight γ^k to a reward received k steps later. If rewards are bounded, $|r_t| \le R_{\max}$, then

$$|G_t| \le R_{\max} \sum_{k=0}^{\infty} \gamma^k = \frac{R_{\max}}{1-\gamma}. \tag{5.8}$$

The geometric-series identity requires $0 < \gamma < 1$ and guarantees a bounded return under the bounded-reward assumption. As γ approaches one, more distant rewards receive greater relative weight and the bound $R_{\max}/(1-\gamma)$ increases.

The state-value function of a policy is the conditional expected return when the process starts at state s and subsequently follows that policy:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right]. \tag{5.9}$$

The expectation is over both the policy's action distribution and the environment's transition distribution. The state–action value function conditions additionally on the first action:

$$Q^{\pi}(s,a) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s,\ a_t = a,\ a_{t+j} \sim \pi \ (j \ge 1) \right]. \tag{5.10}$$

Thus, $V^{\pi}(s)$ evaluates the policy's action distribution at the current state, whereas $Q^{\pi}(s,a)$ evaluates a specified first action followed by the policy thereafter. Their relationship is

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^{\pi}(s,a). \tag{5.11}$$

The advantage function compares an action with the average value of the current policy at the same state:

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s). \tag{5.12}$$

A positive advantage means that the specified action has a greater expected return than the policy average at that state; a negative advantage means that its expected return is lower. From equation (5.11) and the normalization of the policy,

$$\sum_{a} \pi(a \mid s) A^{\pi}(s,a) = \sum_{a} \pi(a \mid s) Q^{\pi}(s,a) - V^{\pi}(s) \sum_{a} \pi(a \mid s) \tag{5.13}$$

$$= V^{\pi}(s) - V^{\pi}(s) = 0. \tag{5.14}$$

The zero-mean identity is central to policy optimization: the expected advantage under the current policy is zero at every state. The performance analysis of trust-region methods uses this identity when constructing a local surrogate objective (**schulman2015trpo**).

5.1.2 Bellman operators and dynamic programming

The value functions are not only expectations over trajectories; they satisfy recursive equations. Decomposing the return into the immediate reward and the discounted return from the next time step gives

$$G_t = r_t + \gamma G_{t+1}. \tag{5.15}$$

Taking conditional expectations and using the Markov property yields the Bellman expectation equation:

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s') \right]. \tag{5.16}$$

It states that the value of a state is the expected immediate reward plus the discounted value of the next state.

For a finite state space with $n = |\mathcal{S}|$ and $m = |\mathcal{A}|$, define the policy reward vector $r_\pi \in \mathbb{R}^n$ and transition matrix $P_\pi \in \mathbb{R}^{n \times n}$ by

$$r_\pi(s) = \sum_a \pi(a \mid s) r(s,a), \qquad P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s,a). \tag{5.17}$$

Each row of P_π is a probability distribution. Hence P_π is row-stochastic, has nonnegative entries, and satisfies $\|P_\pi\|_\infty = 1$ and spectral radius $\rho(P_\pi) \leq 1$. In vector form, equation (5.16) becomes

$$V^\pi = r_\pi + \gamma P_\pi V^\pi. \tag{5.18}$$

Rearranging gives

$$(I - \gamma P_\pi) V^\pi = r_\pi, \qquad V^\pi = (I - \gamma P_\pi)^{-1} r_\pi. \tag{5.19}$$

The inverse exists because $\rho(\gamma P_\pi) \leq \gamma < 1$. Equivalently, the Neumann series converges:

$$(I - \gamma P_\pi)^{-1} = \sum_{k=0}^{\infty} (\gamma P_\pi)^k, \qquad V^\pi = \sum_{k=0}^{\infty} \gamma^k P_\pi^k r_\pi. \tag{5.20}$$

The term $P_\pi^k r_\pi$ is the expected reward vector after k transitions under the policy. The Neumann expansion therefore recovers the discounted trajectory interpretation of the value function through linear algebra. The inverse in equation (5.19) is a theoretical characterization, not a recommendation to form a dense matrix inverse numerically. A practical implementation solves the linear system with a sparse solver or uses iterative Bellman evaluation. The bound

$$\|(I - \gamma P_\pi)^{-1}\|_\infty \leq \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma} \tag{5.21}$$

shows why the discounted system can become more sensitive as γ approaches one: the inverse norm bound grows with the effective horizon. Iterative evaluation remains contractive, but its worst-case contraction factor is γ, so the number of iterations required for a fixed error tolerance also increases as γ approaches one.

For an arbitrary candidate value vector $v \in \mathbb{R}^n$, define the Bellman evaluation operator

$$[T_\pi v](s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s') \right]. \tag{5.22}$$

The value function is its fixed point, $V^\pi = T_\pi V^\pi$. The operator is a contraction in the supremum norm. For any v_1, v_2,

$$\|T_\pi v_1 - T_\pi v_2\|_\infty = \gamma \|P_\pi (v_1 - v_2)\|_\infty \tag{5.23}$$
$$\leq \gamma \|P_\pi\|_\infty \|v_1 - v_2\|_\infty \tag{5.24}$$
$$= \gamma \|v_1 - v_2\|_\infty. \tag{5.25}$$

Because $\gamma < 1$, repeated evaluation $v_{k+1} = T_\pi v_k$ converges to the unique fixed point V^π for any initial vector v_0.

Policy evaluation alone does not optimize the policy. To express optimization, replace the action average under a fixed policy with a maximum over actions. The Bellman optimality operator is

$$[T^* v](s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s, a) v(s') \right]. \tag{5.26}$$

Its unique fixed point is the optimal value function V^*, and an optimal deterministic policy is greedy with respect to V^*:

$$V^* = T^* V^*, \qquad \pi^*(s) \in \arg\max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s, a) V^*(s') \right]. \tag{5.27}$$

The inequality $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$ together with the stochastic normalization of P gives

$$\|T^* v_1 - T^* v_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty. \tag{5.28}$$

Thus, value iteration $v_{k+1} = T^* v_k$ converges to V^*.

Policy iteration organizes the same fixed-point problem differently. It first evaluates a current policy, then improves it by choosing actions that maximize the one-step return computed from the current value function. If $G(v) = \arg\max_\pi T_\pi v$ denotes the greedy policy set, the exact update is

$$v_k = V^{\pi_k}, \qquad \pi_{k+1} \in G(v_k). \tag{5.29}$$

Modified policy iteration interpolates between a single optimality update and complete policy evaluation:

$$\pi_{k+1} \in G(v_k), \qquad v_{k+1} = (T_{\pi_{k+1}})^m v_k. \tag{5.30}$$

For $m = 1$, the evaluation and greedy steps combine into a value-iteration update. As m increases, the new policy is evaluated more fully; in the limit of exact evaluation, the update becomes policy iteration. This continuum is the starting point for the approximate and regularized schemes considered below (**scherrer2014approximate**; **geist2019regularized**).

5.1.3 Policy improvement, performance difference, and approximation

The Bellman optimality equation characterizes an optimal value function, but policy optimization also requires a relation between the performance of two policies. Let ρ_0 be the initial-state distribution and define

$$\eta(\pi) = \mathbb{E}_{s_0 \sim \rho_0}[V^\pi(s_0)]. \tag{5.31}$$

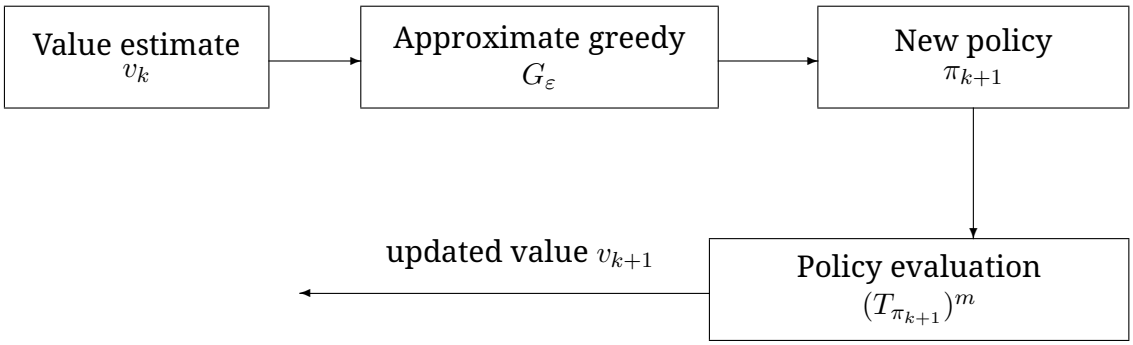


Figure 5.2: The modified policy-iteration cycle. A value estimate is passed to an approximate greedy step, the resulting policy is evaluated for m Bellman steps, and the updated value estimate begins the next iteration.

For a policy π, its unnormalized discounted state-visitation frequency is

$$\rho_\pi(s) = \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s \mid s_0 \sim \rho_0, \pi). \tag{5.32}$$

The normalized discounted occupancy distribution is $d_{\pi,\rho_0} = (1-\gamma)\rho_\pi$ and, in row-vector notation,

$$d_{\pi,\rho_0} = (1-\gamma)\rho_0(I - \gamma P_\pi)^{-1}. \tag{5.33}$$

The normalization follows from $(1-\gamma)\sum_{t=0}^{\infty}\gamma^t = 1$. This distribution weights states according to how often they are encountered under the policy, with later visits discounted.

For two policies π and $\widetilde{\pi}$, the performance difference identity is

$$\eta(\widetilde{\pi}) = \eta(\pi) + \sum_s \rho_{\widetilde{\pi}}(s) \sum_a \widetilde{\pi}(a \mid s) A^\pi(s,a). \tag{5.34}$$

To see the source of the identity, write $A^\pi(s_t,a_t) = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$ in conditional expectation, then sum the discounted terms along a trajectory generated by $\widetilde{\pi}$. The value terms telescope, leaving the difference between the new and old expected returns. Equation equation (5.34) shows that policy improvement depends on two coupled quantities: the advantages of the new policy's actions and the state-visitation distribution induced by the new policy. If the expected advantage is nonnegative at every state, the new policy cannot have lower performance. Exact greedy improvement satisfies this condition; approximate improvement need not.

In approximate policy iteration, the exact greedy operator is replaced by an approximate one. For a reference distribution ν, an approximate greedy policy can be characterized by

$$\nu(Tv - T_\pi v) \le \varepsilon, \qquad \nu f := \mathbb{E}_{s\sim\nu}[f(s)], \tag{5.35}$$

where $\varepsilon \ge 0$ is the greedy-step error. This is an average error under ν, not a uniform guarantee over every state. In the exact case, $\varepsilon = 0$, the policy is greedy on the support of ν. The distinction matters because the distribution used to fit or evaluate the approximate greedy step need not match the state distribution induced by the resulting policy.

The mismatch can be quantified using concentrability coefficients. Given a evaluation distribution μ and a reference distribution ν, let $c(i)$ be a coefficient satisfying, componentwise,

$$\mu P_{\pi_1} P_{\pi_2} \cdots P_{\pi_i} \leq c(i)\nu \tag{5.36}$$

for all admissible sequences of policies of length i. A small $c(i)$ means that distributions reached after i transitions remain covered by the reference distribution. A large or infinite coefficient indicates that the evaluation distribution can place substantial mass on states that are poorly represented under ν. Weighted sums of the coefficients, with powers of γ, appear in performance bounds because errors propagated through many transitions are discounted but are not eliminated. Thus, an approximation error alone does not characterize the quality of an approximate policy-iteration method; the distribution mismatch appearing in its bound is also essential (**scherrer2014approximate**).

The standard Approximate Policy Iteration update is

$$\pi_{k+1} = G_{\varepsilon_{k+1}}(\nu, V^{\pi_k}), \tag{5.37}$$

where the notation makes explicit that the approximate greedy operator depends on both the value estimate and the distribution under which its error is measured. Conservative Policy Iteration controls the change of policy by mixing the current policy with an approximately greedy policy:

$$\pi_{k+1} = (1 - \alpha_{k+1})\pi_k + \alpha_{k+1} G_{\varepsilon_{k+1}}(d_{\pi_k,\rho_0}, V^{\pi_k}). \tag{5.38}$$

Here $\alpha_{k+1} \in (0, 1]$ is a step size. The occupancy distribution is used in the greedy step, and a sufficiently conservative step can be selected to obtain an improvement guarantee. The API(α) variant also mixes policies but uses the reference distribution ν in its approximate greedy step:

$$\pi_{k+1} = (1 - \alpha)\pi_k + \alpha G_{\varepsilon_{k+1}}(\nu, V^{\pi_k}). \tag{5.39}$$

This is simpler to implement when sampling from the current policy's occupancy distribution is inconvenient.

A different construction retains a sequence of policies rather than replacing the current policy by a mixture. Given stationary policies $\pi_1, \ldots, \pi_k$, define the finite nonstationary sequence $\sigma_k = \pi_k \pi_{k-1} \cdots \pi_1$, where the first action uses π_k, the next uses π_{k-1}, and so forth. Its value is

$$V^{\sigma_k} = T_{\pi_k} T_{\pi_{k-1}} \cdots T_{\pi_1} r. \tag{5.40}$$

The infinite-horizon extension called PSDP$_\infty$ constructs the sequence iteratively through

$$\pi_{k+1} = G_{\varepsilon_{k+1}}(\nu, V^{\sigma_k}), \tag{5.41}$$

and can return a periodic policy that repeats the retained finite sequence. The Non-Stationary Policy Iteration method with period m retains only the most recent m policies and repeatedly executes $\sigma_k^m = \pi_k \pi_{k-1} \cdots \pi_{k-m+1}$. Its next policy is obtained from the value of the periodic extension of this sequence:

$$\pi_{k+1} = G_{\varepsilon_{k+1}}\big(\nu, V^{(\sigma_k^m)^\infty}\big). \tag{5.42}$$

It therefore creates a continuum between the one-policy memory of API and the longer memory required by PSDP$_\infty$.

Table 5.1: Qualitative comparison of the approximate policy-iteration schemes analyzed by **scherrer2014approximate**. The guarantees depend on the concentrability coefficients and approximation assumptions, not only on the algorithm name.

Scheme	Policy update structure	Principal theoretical or computational trade-off
API	Replace the current policy with an approximately greedy policy.	Constant memory; its bounds use more restrictive concentrability coefficients.
CPI	Mix the current policy with an approximately greedy policy computed under its occupancy distribution.	Can have stronger performance guarantees, but may require substantially more iterations and occupancy-distribution sampling.
PSDP_∞	Improve a growing non-stationary sequence of policies.	Combines a strong guarantee with an API-like iteration requirement, at the cost of memory growing with the number of iterations.
NSPI(m)	Repeat a sequence containing only the most recent m policies.	Makes memory proportional to m and exposes a trade-off between memory and the performance bound.

The comparison in table 5.1 should not be read as a universal ranking independent of the MDP. Scherrer's analysis shows that the hierarchy of concentrability coefficients can make one guarantee finite while another is infinite. Under the bounds analyzed in that paper, CPI can have a substantially stronger guarantee than API, but its iteration requirement can grow on the order of $1/\varepsilon^2$, whereas API has a logarithmic iteration dependence on $1/\varepsilon$. The PSDP_∞ construction combines a CPI-like guarantee with an API-like iteration requirement, while requiring storage for the policies accumulated along the sequence. NSPI(m) limits that storage to a fixed window. The conclusion is methodological: performance, iteration complexity, memory, and state-distribution coverage must be analyzed together (**scherrer2014approximate**).

5.1.4 Information geometry and covariant policy search

When a policy is represented by a differentiable model, its probability function depends on a parameter vector $\theta \in \mathbb{R}^d$: $\pi_\theta(a \mid s)$. The objective is to maximize the expected return $J(\theta) = \eta(\pi_\theta)$. Ordinary gradient ascent takes the form

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_k). \tag{5.43}$$

This update measures step size using Euclidean distance in parameter space. However, the parameter vector is a coordinate description of a policy, not the behavioral object itself. Two different parameterizations can represent the same family of probability distributions while assigning different Euclidean lengths and directions to corresponding changes in policy behavior.

Suppose that another parameterization is defined by a differentiable invertible coordinate transformation $\phi = f(\theta)$. For a small displacement, $\Delta\phi \simeq J_f(\theta)\Delta\theta$, where J_f is the Jacobian of the coordinate transformation. A geometric update should transform consistently with this change of coordinates. An ordinary Euclidean gradient generally does not have this property. This is the non-covariance problem addressed by **bagnell2003covariant**.

A Riemannian metric assigns an inner product to local tangent directions. In parameter coordinates, let $F(\theta)$ be a positive-definite metric tensor on the identifiable tangent directions. The local squared length of an update is

$$\|\Delta\theta\|_F^2 = \Delta\theta^{\mathsf{T}} F(\theta)\Delta\theta. \tag{5.44}$$

The direction of steepest first-order ascent under a fixed metric length is the solution of

$$\max_{\Delta\theta} \quad \nabla_\theta J(\theta)^{\mathsf{T}}\Delta\theta \quad \text{subject to} \quad \Delta\theta^{\mathsf{T}} F(\theta)\Delta\theta \le \delta^2. \tag{5.45}$$

Introducing a Lagrange multiplier and differentiating with respect to $\Delta\theta$ gives $\nabla_\theta J - \lambda F\Delta\theta = 0$. Therefore, up to a positive step length, the natural-gradient direction is

$$\widetilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta). \tag{5.46}$$

If the metric is singular because the parameterization contains redundant directions, the inverse is understood on the identifiable subspace or replaced by a suitable generalized inverse. The metric changes the direction, not merely the scalar learning rate.

The metric used by covariant policy search is derived from the probability distribution over trajectories induced by the stochastic controller. For a finite trajectory $\tau = (s_0, a_0, \dots, s_T)$, its probability is

$$p_\theta(\tau) = \rho_0(s_0)\prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t)P(s_{t+1} \mid s_t, a_t). \tag{5.47}$$

When the initial-state distribution and environment dynamics do not depend on θ, the trajectory score is

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t). \tag{5.48}$$

The Fisher information matrix of the trajectory distribution is

$$F_T(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)^{\mathsf{T}}\right]. \tag{5.49}$$

It is positive semidefinite because, for every vector x, $x^{\mathsf{T}}F_T x$ is the expectation of the nonnegative random variable $(x^{\mathsf{T}}\nabla_\theta \log p_\theta(\tau))^2$. The Fisher matrix measures how sensitively the trajectory distribution changes when the policy parameters change. The same trajectory score gives the likelihood-ratio policy-gradient identity. For an episodic trajectory return $R(\tau)$,

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau)R(\tau)\,d\tau \tag{5.50}$$

$$= \int p_\theta(\tau)R(\tau)\nabla_\theta \log p_\theta(\tau)\,d\tau \tag{5.51}$$

$$= \mathbb{E}_{\tau\sim p_\theta}\left[R(\tau)\nabla_\theta \log p_\theta(\tau)\right]. \tag{5.52}$$

The second equality follows from $\nabla p = p \nabla \log p$ and requires that differentiation can be interchanged with integration. Substituting the trajectory score from equation (5.48) expresses the gradient as a sum of per-decision score terms. A state-dependent baseline can be subtracted without changing the expectation because $\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$. Choosing a value baseline gives the advantage-weighted form used in policy optimization:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t) A^{\pi_\theta}(s_t, a_t)\right], \tag{5.53}$$

with the trajectory weighting and discount convention chosen consistently with J. This identity explains why advantage estimation is central to the actor–critic methods developed in the next section.

For the discounted start-state formulation, the corresponding state weighting is the normalized discounted occupancy distribution. The per-decision policy Fisher matrix can consequently be written as

$$F(\theta) = \mathbb{E}_{\substack{s \sim d_{\pi_\theta, \rho_0} \\ a \sim \pi_\theta(\cdot|s)}}\left[\nabla_\theta \log \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)^{\mathsf{T}}\right], \tag{5.54}$$

up to the normalization convention used for the trajectory objective. In an ergodic average-reward formulation, the analogous weighting is the stationary state distribution. This construction makes the metric depend on the probabilistic behavior of the controller rather than on an arbitrary Euclidean coordinate system (**bagnell2003covariant**).

The Fisher metric is locally related to the Kullback–Leibler divergence. For nearby parameters and a regular differentiable model,

$$D_{\mathrm{KL}}(p_\theta \| p_{\theta + \Delta\theta}) = \frac{1}{2} \Delta\theta^{\mathsf{T}} F_T(\theta) \Delta\theta + o(\|\Delta\theta\|^2). \tag{5.55}$$

Thus, the Fisher matrix is the local quadratic form of statistical distinguishability between nearby trajectory distributions. This explains why the natural gradient is a steepest-ascent direction when policy changes are measured through their effect on probability distributions.

The same geometry connects to compatible actor–critic approximation. Define the policy score feature $\psi_\theta(s, a) = \nabla_\theta \log \pi_\theta(a \mid s)$ and a compatible advantage approximator

$$A_w(s, a) = w^{\mathsf{T}} \psi_\theta(s, a). \tag{5.56}$$

Under the same state–action weighting used to define the policy gradient, the least-squares normal equations for this compatible approximation take the form $F(\theta) w = g$, where $g = \nabla_\theta J(\theta)$, provided the compatible features and the policy-gradient identity are used consistently. Hence

$$w = F(\theta)^{-1} g. \tag{5.57}$$

This result explains the relationship between the geometry of the policy family and actor–critic estimation discussed by **bagnell2003covariant**. It does not imply that an arbitrary critic architecture is compatible; the feature form in equation (5.56) is the relevant condition.

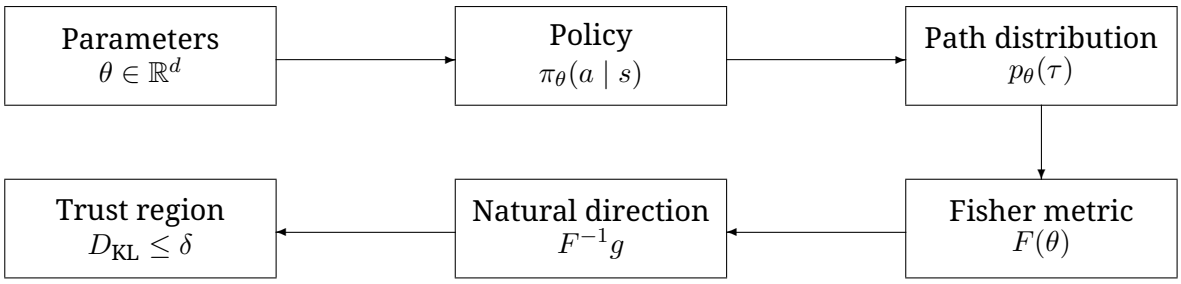


Figure 5.3: The information-geometric route from policy parameters to a trajectory distribution, its Fisher metric, and a trust-region-limited natural-gradient update.

5.1.5 Trust-region policy optimization

The natural gradient supplies a geometrically meaningful direction, but a direction alone does not specify a safe step length. A large update can leave the region in which a local approximation is reliable. Trust Region Policy Optimization (TRPO) addresses this issue by maximizing a surrogate objective while constraining the divergence between the old and new policies (**schulman2015trpo**).

For a current policy π, the performance difference identity in equation (5.34) depends on the state-visitation frequency $\rho_{\widetilde{\pi}}$ of the new policy. Directly optimizing this expression is difficult because the state distribution changes with the policy. TRPO replaces this new-policy occupancy with the old-policy occupancy and defines

$$L_\pi(\widetilde{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \widetilde{\pi}(a \mid s) A^\pi(s,a). \tag{5.58}$$

At the old policy, $L_\pi(\pi) = \eta(\pi)$ because the expected advantage is zero. For a differentiable parameterized policy, the surrogate and the true objective also have the same first derivative at the old parameter value. This makes the surrogate a first-order local approximation, but it does not justify arbitrarily large policy changes.

To bound the error caused by the changed state distribution, define total variation divergence for discrete distributions by

$$D_{\mathrm{TV}}(p,q) = \frac{1}{2}\sum_a |p(a) - q(a)|, \qquad \alpha = \max_s D_{\mathrm{TV}}(\pi(\cdot \mid s), \widetilde{\pi}(\cdot \mid s)), \tag{5.59}$$

and let $\epsilon = \max_{s,a} |A^\pi(s,a)|$. The policy-improvement theorem proved by **schulman2015trpo** states

$$\eta(\widetilde{\pi}) \ge L_\pi(\widetilde{\pi}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2. \tag{5.60}$$

The quadratic dependence on α reflects the fact that two policies can be coupled to select the same action with high probability when their state-wise total variation distance is small. Their trajectories remain identical until a disagreement occurs; the resulting occupancy mismatch is therefore controlled by a second-order term.

Using the inequality $D_{\mathrm{TV}}(p,q)^2 \le D_{\mathrm{KL}}(p\|q)$, define

$$D_{\mathrm{KL}}^{\max}(\pi,\widetilde{\pi}) = \max_s D_{\mathrm{KL}}(\pi(\cdot \mid s)\|\widetilde{\pi}(\cdot \mid s)), \qquad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}. \tag{5.61}$$

Then the lower bound becomes

$$\eta(\widetilde{\pi}) \ge L_\pi(\widetilde{\pi}) - CD_{\mathrm{KL}}^{\max}(\pi,\widetilde{\pi}). \tag{5.62}$$

The maximum state-wise divergence is the quantity required by this theoretical bound. Maximizing the right-hand side at every iteration produces a non-decreasing sequence of true policy performances under the theorem's assumptions. In particular, the current policy is feasible and gives equality in the lower bound, so maximizing the lower-bound objective cannot reduce its value.

The exact maximum-KL constraint is difficult to enforce over a large state space. TRPO therefore uses a practical average-divergence approximation. For a state distribution ρ, define

$$D_{\mathrm{KL}}^{\rho}(\theta_{\text{old}},\theta) = \mathbb{E}_{s\sim\rho}\left[D_{\mathrm{KL}}\big(\pi_{\theta_{\text{old}}}(\cdot \mid s)\|\pi_\theta(\cdot \mid s)\big)\right]. \tag{5.63}$$

The practical constrained problem is

$$\begin{aligned} \max_\theta \quad & L_{\theta_{\text{old}}}(\theta) \\ \text{subject to} \quad & D_{\mathrm{KL}}^{\rho_{\text{old}}}(\theta_{\text{old}},\theta) \le \delta. \end{aligned} \tag{5.64}$$

Replacing the maximum state-wise constraint by an average is a practical approximation; it does not, without additional assumptions, retain the exact monotonic-improvement guarantee of equation (5.62). This distinction between the theorem and the implementable approximation is essential when interpreting TRPO.

The objective can be estimated from trajectories sampled under the old policy. An importance-sampling representation of the action expectation is

$$\mathbb{E}_{a\sim\pi_{\text{old}}(\cdot|s)}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)} A^{\pi_{\text{old}}}(s,a)\right], \tag{5.65}$$

provided the sampling distribution has support wherever the updated policy assigns probability. TRPO describes a single-path estimator, which samples trajectories from the old policy and estimates action values from discounted rewards along each trajectory. It also describes a vine estimator, which selects states from sampled trajectories, evaluates multiple actions from each selected state, and can use common random numbers across rollouts to reduce the variance of action-value comparisons. The single-path method is applicable in model-free settings where the system cannot be reset to an arbitrary state; the vine method is particularly natural in a simulator that permits such resets (**schulman2015trpo**).

The local constrained problem connects TRPO directly to the natural gradient. Let $g = \nabla_\theta L_{\theta_{\text{old}}}(\theta)$ evaluated at θ_{old}. A second-order expansion of the KL constraint gives

$$D_{\mathrm{KL}}^{\rho_{\text{old}}}(\theta_{\text{old}},\theta_{\text{old}}+\Delta\theta) \simeq \frac{1}{2}\Delta\theta^{\mathsf{T}} F \Delta\theta, \tag{5.66}$$

where F is the Fisher information matrix under the old-policy state weighting. The local optimization is therefore

$$\max_{\Delta\theta} g^{\mathsf{T}}\Delta\theta \quad \text{subject to} \quad \frac{1}{2}\Delta\theta^{\mathsf{T}} F \Delta\theta \le \delta. \tag{5.67}$$

The Lagrange-multiplier solution is proportional to $F^{-1}g$. If $s = F^{-1}g$, the step length that saturates the quadratic constraint is

$$\beta = \sqrt{\frac{2\delta}{s^{\mathsf{T}}Fs}}, \qquad \Delta\theta = \beta s. \tag{5.68}$$

Thus, the Fisher matrix determines the direction and the trust-region radius determines the scale.

For a neural policy with d parameters, explicitly storing a dense Fisher matrix requires $O(d^2)$ memory, and a dense factorization or inverse can require $O(d^3)$ arithmetic operations. TRPO avoids forming the dense inverse by solving $Fs = g$ approximately with conjugate gradients. Conjugate gradients requires repeated matrix–vector products Fv, which can be obtained through Hessian–vector products of the KL objective using automatic differentiation. The number of conjugate-gradient iterations controls the accuracy of the linear-system solution. After obtaining a candidate direction, a backtracking line search evaluates the actual nonlinear surrogate and KL divergence; it reduces the step until the candidate improves the surrogate and satisfies the practical divergence constraint. The line search is needed because the first-order objective approximation and quadratic KL approximation are local, not globally exact.

Algorithm 2: Sample-based trust-region policy optimization

Input: Initial policy parameters θ_0, KL radius δ, trajectory sampler, conjugate-gradient tolerance, and line-search rule.
Output: Updated policy parameters θ.

Set $\theta \leftarrow \theta_0$;
while *the stopping condition is not satisfied* **do**
  Set $\theta_{\text{old}} \leftarrow \theta$;
  Collect trajectories using $\pi_{\theta_{\text{old}}}$ and estimate advantages;
  Estimate the surrogate gradient g and Fisher-vector products $v \mapsto Fv$;
  Approximately solve $Fs = g$ with conjugate gradients;
  Scale s using the quadratic KL constraint to obtain a candidate step;
  Evaluate backtracking candidates using the sampled surrogate and average KL;
  if *a candidate improves the surrogate and satisfies the KL constraint* **then**
    Accept the candidate as the new θ;
  else
    Retain θ_{old} and terminate if the line search fails;

The algorithm in algorithm 2 separates statistical estimation from numerical optimization. Its sample quality depends on the trajectory estimator and advantage estimates; its local optimization quality depends on the Fisher-vector products and conjugate-gradient accuracy; and its final accepted step depends on the actual surrogate and KL evaluations. TRPO is therefore not an exact solution of the original constrained problem, but a practical approximation to a theoretically motivated policy-improvement procedure (**schulman2015trpo**).

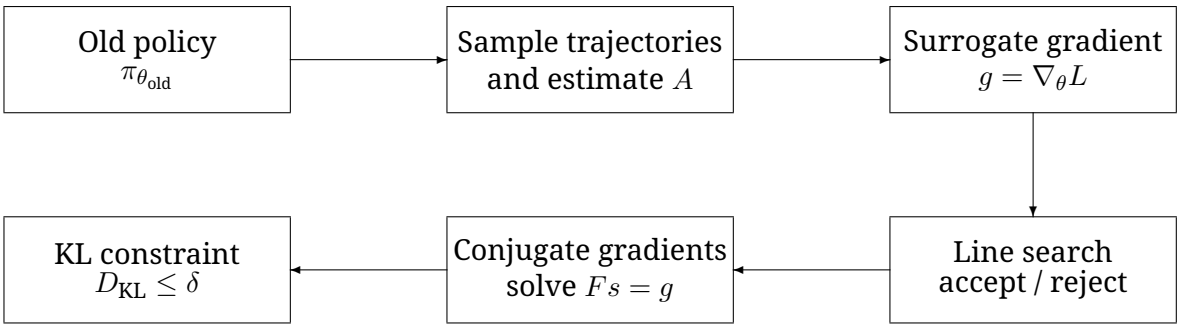


Figure 5.4: The sample-based TRPO pipeline, from old-policy trajectories and advantage estimates through the Fisher-system solve to a line-searched update.

5.1.6 Regularized Markov decision processes

Trust-region optimization limits the change between successive policies. A related but more general strategy modifies the policy optimization objective through a regularizer. The theory developed by **geist2019regularized** treats regularized Markov decision processes using convex analysis and a generalized Bellman framework. The central idea is to define the regularizer over action distributions at each state and then incorporate it into the Bellman evaluation and optimality operators.

Let $\Omega : \Delta_{\mathcal{A}} \to \mathbb{R}$ be a strongly convex regularizer. For a value function v, define the one-step action-value vector

$$q_v(s,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) v(s'). \tag{5.69}$$

The regularized greedy distribution at state s solves

$$\pi_s^* = \arg\max_{p \in \Delta_{\mathcal{A}}} \left[\langle p, q_v(s,\cdot) \rangle - \Omega(p) \right]. \tag{5.70}$$

The linear term favors actions with high expected return; the regularizer changes the preferred distribution according to the chosen convex geometry. Strong convexity makes the maximizer unique under the framework's assumptions.

The Legendre–Fenchel convex conjugate of Ω is

$$\Omega^*(q) = \max_{p \in \Delta_{\mathcal{A}}} \left[\langle p, q \rangle - \Omega(p) \right]. \tag{5.71}$$

When the conjugate is differentiable, its gradient returns the unique maximizing distribution:

$$\pi_s^* = \nabla \Omega^*(q_v(s,\cdot)). \tag{5.72}$$

The convex conjugate therefore converts a regularized optimization over the probability simplex into a value function of the action-value vector.

For negative Shannon entropy,

$$\Omega(p) = \sum_a p(a) \log p(a), \tag{5.73}$$

with the convention $0 \log 0 = 0$, the conjugate is the log-sum-exp function:

$$\Omega^*(q) = \log \sum_a e^{q(a)}, \qquad \nabla\Omega^*(q)(a) = \frac{e^{q(a)}}{\sum_b e^{q(b)}}. \tag{5.74}$$

Thus, negative-entropy regularization yields a softmax policy. If the regularizer is scaled by a temperature $\tau > 0$, so that $\Omega_\tau(p) = \tau \sum_a p(a) \log p(a)$, its conjugate is $\Omega_\tau^*(q) = \tau \log \sum_a e^{q(a)/\tau}$ and the policy is $\pi(a) = e^{q(a)/\tau} / \sum_b e^{q(b)/\tau}$. For example, the quadratic regularizer $\Omega(p) = \frac{1}{2}(\|p\|_2^2 - 1)$ is associated with the order-two Tsallis construction and yields a sparsemax-type solution, which can assign exactly zero probability to some actions. These examples show that the regularizer determines the form of the regularized greedy policy (**geist2019regularized**).

The regularized Bellman evaluation operator is

$$[T_{\pi,\Omega} v](s) = \langle \pi(\cdot \mid s), q_v(s, \cdot) \rangle - \Omega(\pi(\cdot \mid s)), \tag{5.75}$$

and the regularized optimality operator is

$$[T_\Omega^* v](s) = \max_{p \in \Delta_\mathcal{A}} [\langle p, q_v(s, \cdot) \rangle - \Omega(p)] = \Omega^*(q_v(s, \cdot)). \tag{5.76}$$

The regularized value functions are fixed points of these operators: $V^{\pi,\Omega} = T_{\pi,\Omega} V^{\pi,\Omega}$ and $V^{*,\Omega} = T_\Omega^* V^{*,\Omega}$. The regularized greedy policy is $G_\Omega(v) = \nabla\Omega^*(q_v)$.

The regularized operators preserve the key properties needed for dynamic programming. They are monotone: if $v_1 \geq v_2$ componentwise, then $T_{\pi,\Omega} v_1 \geq T_{\pi,\Omega} v_2$ and $T_\Omega^* v_1 \geq T_\Omega^* v_2$. They also satisfy constant-shift identities,

$$T_{\pi,\Omega}(v + c\mathbf{1}) = T_{\pi,\Omega} v + \gamma c \mathbf{1}, \qquad T_\Omega^*(v + c\mathbf{1}) = T_\Omega^* v + \gamma c \mathbf{1}, \tag{5.77}$$

because transition probabilities sum to one and the regularizer does not depend on v. Finally, both operators are γ-contractions in the supremum norm:

$$\|T_{\pi,\Omega} v_1 - T_{\pi,\Omega} v_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty, \qquad \|T_\Omega^* v_1 - T_\Omega^* v_2\|_\infty \leq \gamma \|v_1 - v_2\|_\infty. \tag{5.78}$$

The regularized Bellman operators therefore have unique fixed points, and regularized value iteration converges under the same discounted contraction logic as its unregularized counterpart (**geist2019regularized**).

A fixed regularizer changes the objective, so the optimal regularized policy need not be optimal for the original unregularized MDP. If the regularizer is bounded on the simplex,

$$L_\Omega \leq \Omega(p) \leq U_\Omega \qquad \text{for all } p \in \Delta_\mathcal{A}, \tag{5.79}$$

then the regularized and unregularized values differ by a bounded amount. In particular, the unregularized value of the regularized optimal policy satisfies

$$V^* - V^{\pi^*,\Omega} \leq \frac{U_\Omega - L_\Omega}{1 - \gamma} \mathbf{1}. \tag{5.80}$$

The range of the regularizer and the discount horizon therefore determine a bound on the cost of solving the regularized rather than the original problem. This is distinct from a claim that regularization always improves the unregularized optimum; it controls the difference between the two objectives under the stated boundedness assumptions (**geist2019regularized**).

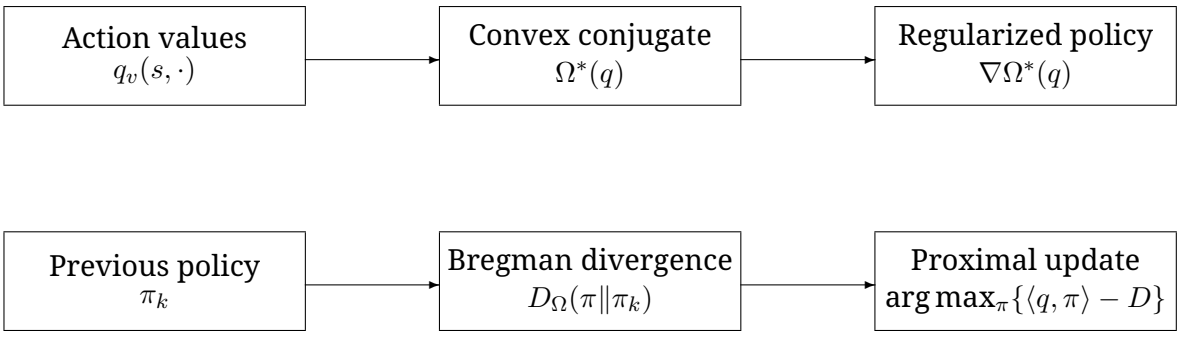


Figure 5.5: Regularized policy improvement. The action-value vector is passed through the convex conjugate to obtain a regularized policy; in the proximal form, the new policy is additionally kept close to the previous policy by a Bregman divergence.

5.1.7 Mirror descent and modified policy iteration

A fixed regularizer modifies the optimal solution of the MDP. An alternative is to regularize each update by penalizing the divergence between the new policy and the policy from the previous iteration. This makes the regularization iteration-dependent and connects policy improvement to proximal convex optimization.

For a differentiable convex function Ω, the Bregman divergence from q to p is

$$D_\Omega(p\|q) = \Omega(p) - \Omega(q) - \langle \nabla\Omega(q), p - q\rangle. \tag{5.81}$$

Convexity implies $D_\Omega(p\|q) \geq 0$ and $D_\Omega(q\|q) = 0$. In general, it is not symmetric and therefore is not a metric. For negative Shannon entropy, the Bregman divergence is the KL divergence:

$$D_\Omega(p\|q) = \sum_a p(a)\log\frac{p(a)}{q(a)} = D_{\mathrm{KL}}(p\|q). \tag{5.82}$$

The proximal, mirror-descent policy update is

$$\pi_{k+1} = \arg\max_{\pi\in\Delta_{\mathcal{A}}} \left[\langle q_k, \pi\rangle - D_\Omega(\pi\|\pi_k)\right]. \tag{5.83}$$

The negative action-value vector plays the role of a gradient in the usual minimization form of mirror descent. The Bregman term controls the change from the previous policy in the geometry induced by Ω.

For the KL case, the update can be derived explicitly. Substitute $D_{\mathrm{KL}}(\pi\|\pi_k)$ into equation (5.83). Introducing a Lagrange multiplier for the normalization constraint $\sum_a \pi(a) = 1$ gives the stationarity condition

$$q_k(a) - \log\frac{\pi_{k+1}(a)}{\pi_k(a)} - 1 + \lambda = 0. \tag{5.84}$$

Solving and normalizing yields

$$\pi_{k+1}(a) = \frac{\pi_k(a)e^{q_k(a)}}{\sum_b \pi_k(b)e^{q_k(b)}}. \tag{5.85}$$

The old policy is therefore reweighted by exponentiated action values. Unlike an independently recomputed softmax, this update retains the previous policy as its reference distribution.

Mirror Descent Modified Policy Iteration (MD-MPI) combines this proximal greedy step with partial policy evaluation. Its two forms differ in the evaluation operator:

$$\begin{aligned} &\text{Type 1:} \quad \pi_{k+1} = G_{\Omega^{\pi_k}}(v_k), && \text{Type 2:} \quad \pi_{k+1} = G_{\Omega^{\pi_k}}(v_k), \\ &\qquad\quad v_{k+1} = (T_{\pi_{k+1},\Omega^{\pi_k}})^m v_k; && \qquad\quad v_{k+1} = (T_{\pi_{k+1}})^m v_k. \end{aligned} \tag{5.86}$$

Both types regularize the greedy step relative to the previous policy. Type 1 also uses the corresponding regularized evaluation operator; Type 2 uses the ordinary evaluation operator. The distinction changes where approximation errors enter the iteration and consequently changes the details of the error analysis.

The framework provides a useful unification. In the formulation discussed by **geist2019regularized**, Type 2 with complete policy evaluation and a KL divergence recovers the structure of TRPO, up to the practical distinction between a hard divergence constraint and a divergence penalty in the proximal objective. The same framework relates other algorithms to regularized modified policy iteration, but those algorithms are not developed further here because the present section is restricted to the four supplied papers.

The analysis of MD-MPI introduces a cumulative regret quantity

$$L_K = \sum_{k=1}^{K} (V^* - V^{\pi_k}) \tag{5.87}$$

and the initial Bregman radius

$$R_\Omega^{\pi_0} = \sup_\pi D_\Omega(\pi \| \pi_0), \tag{5.88}$$

with the appropriate state-wise norm convention. The key telescoping term is

$$\begin{aligned} \sum_{k=0}^{K-1} \left[D_\Omega(\pi^* \| \pi_k) - D_\Omega(\pi^* \| \pi_{k+1}) \right] &= D_\Omega(\pi^* \| \pi_0) - D_\Omega(\pi^* \| \pi_K) \\ &\le D_\Omega(\pi^* \| \pi_0). \end{aligned} \tag{5.89}$$

This is the mechanism by which the initial divergence controls the cumulative contribution of the proximal updates. With a uniform initial policy and KL divergence over a finite action set,

$$\sup_{\pi \in \Delta_{\mathcal{A}}} D_{\mathrm{KL}}(\pi \| \pi_0) = \log |\mathcal{A}|. \tag{5.90}$$

The logarithmic dependence on the number of actions follows because the maximum is attained by a distribution concentrated on one action.

In the exact setting, the regret bound in the MD-MPI analysis implies a bound of the form

$$\frac{1}{K} \| L_K \|_\infty \le \frac{1 - \gamma^K}{K(1-\gamma)^2} 2\gamma \| V^* - v_0 \|_\infty + \frac{R_\Omega^{\pi_0}}{K}. \tag{5.91}$$

Approximate versions add terms for the greedy and evaluation errors propagated through policy transition kernels. The regret bound controls the average loss across

iterates; it does not necessarily say that the last iterate is the best one. The analysis consequently gives a bound on the best policy generated so far through

$$\min_{1\le k\le K} \|V^* - V^{\pi_k}\|_{1,\rho} \le \frac{1}{K}\|L_K\|_{p,\rho}, \tag{5.92}$$

for the norm and distribution conditions specified in the paper. This is why the best iterate can be a more appropriate output than the final iterate (**geist2019regularized**).

5.1.8 Worked example and connection to autoregressive language models

A two-action calculation makes the KL-proximal update concrete. Consider one state with actions a_1 and a_2, previous policy $\pi_k = (0.5, 0.5)$, and action-value vector $q_k = (1, 0)$. Under the KL mirror update in equation (5.85),

$$\pi_{k+1}(a_1) = \frac{0.5e^1}{0.5e^1 + 0.5e^0} = \frac{e}{e+1} \simeq 0.7311, \qquad \pi_{k+1}(a_2) \simeq 0.2689. \tag{5.93}$$

The policy assigns more probability to the action with the larger value, but it retains nonzero probability for the other action. The KL divergence from the updated policy to the previous uniform policy is

$$\begin{aligned} D_{\mathrm{KL}}(\pi_{k+1}\|\pi_k) &\simeq 0.7311 \log \frac{0.7311}{0.5} + 0.2689 \log \frac{0.2689}{0.5} \\ &\simeq 0.1109 \text{ nats}. \end{aligned} \tag{5.94}$$

The unit is nats because the logarithm is natural. This example illustrates the mechanism of the proximal update: action values shift the distribution, while the divergence is measured relative to the previous policy. A larger action value difference produces a more concentrated update; a stronger divergence penalty, if included as a coefficient, would make the update more conservative.

For an autoregressive language model, a natural MDP representation takes the generated token prefix as the state and the next token as the action. If the current prefix is $s_t = (x_1, \dots, x_t)$, the action is $a_t = x_{t+1}$, and the transition appends the selected token:

$$s_{t+1} = s_t \,\|\, a_t, \qquad \pi_\theta(a_t \mid s_t) = p_\theta(x_{t+1} \mid x_1, \dots, x_t). \tag{5.95}$$

Here $\|$ denotes sequence concatenation. If the state contains the complete prefix and any other context needed to determine the reward and transition, this representation is Markov: the next state is determined by the current prefix and selected token. A reward may be assigned at each token step or at the end of the generated sequence. With a terminal sequence reward, the intermediate rewards can be zero and the final transition carries the sequence-level outcome. The horizon is the maximum generated length or a stopping time determined by the generation process.

This mapping is an application of the MDP definitions developed above; the four cited theoretical papers do not themselves establish language-model- specific reinforcement-learning results. Its value for the present book is conceptual and mathematical. It makes explicit why token generation can be viewed as sequential decision-making, why a sequence-level reward creates a delayed-credit-assignment problem, and why policy optimization changes a distribution over complete generated sequences rather than merely changing an isolated token prediction. The policy is the language model's conditional next- token distribution, the trajectory is a generated sequence, and the return is the discounted or finite-horizon accumulation of rewards associated with that sequence.

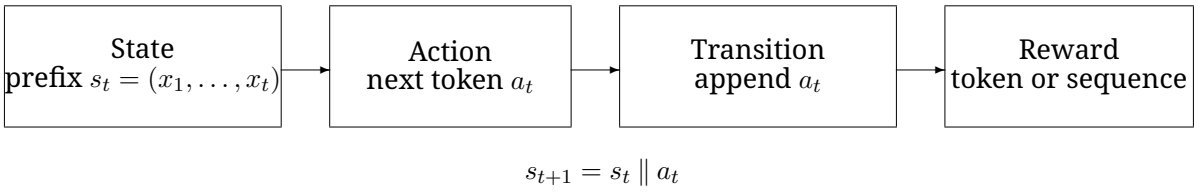


Figure 5.6: A natural MDP interpretation of autoregressive generation. The current token prefix is the state, the next token is the action, appending the token produces the next state, and a reward may be attached to a token step or to the completed sequence.

5.1.9 Limitations and connection to the next topic

The results in this section rely on assumptions that must remain visible when they are applied to large neural policies. The contraction arguments use a discount factor strictly below one and bounded rewards. The finite-dimensional matrix derivations assume finite state and action spaces, although the operator formulation extends to probability kernels. Exact policy-improvement claims assume accurate value functions and greedy updates. Approximate methods replace those operations with estimates, so their guarantees depend on both the approximation errors and the state-distribution coefficients that propagate those errors.

The natural-gradient construction depends on the chosen trajectory probability model and on the regularity of the policy parameterization. A singular Fisher matrix indicates redundant or locally unidentifiable parameter directions. TRPO's monotonic-improvement theorem uses a maximum state-wise KL constraint, whereas the practical algorithm uses an average KL constraint and sample-based estimates. Its practical stability should therefore not be confused with an unconditional theorem for every neural policy and sampling regime. Fixed regularization also changes the optimization objective, while Bregman regularization controls updates relative to a previous policy and introduces its own dependence on the initial divergence and approximation errors.

Taken together, the four supplied papers establish a progression from exact dynamic programming to approximate policy iteration, from coordinate-dependent gradients to information-geometric updates, and from unconstrained policy improvement to trust-region and Bregman-regularized optimization. The common mathematical structure is the control of policy change under imperfect estimation. For language models, this structure explains how next-token probability distributions can be optimized against sequence-level feedback without treating each generated token as an independent supervised target. The next section can now introduce actor–critic architectures, in which a policy model and a value estimator are trained together to estimate the advantages needed for policy improvement.

5.2 Actor–Critic Methods

The preceding discussion established reinforcement learning as an optimization problem in which an agent improves a policy through interaction with an environment and feedback from a reward signal. The next question is how the learning algorithm can determine which individual decisions deserve reinforcement. An observed return evaluates the consequences of a sequence of decisions, whereas policy optimization must update the probability of the individual actions that produced that sequence. This distinction becomes especially important in autoregressive language generation: a response is formed through many token-level decisions, but its quality may only be observable after a substantial part, or all, of the response has been generated.

Policy-gradient methods provide a direct way to optimize a parameterized stochastic policy. Their practical effectiveness, however, depends on the quality of the signal used to evaluate sampled actions. Returns can vary substantially across trajectories that begin in similar states, and this variation can make policy-gradient estimates noisy. The actor–critic architecture addresses this problem by separating policy representation from value estimation. The actor represents the policy that selects actions; the critic estimates the expected future return associated with the states in which those actions are selected. The critic's estimate supplies a baseline for constructing advantages, allowing the actor to learn from the relative quality of an action rather than from its raw return alone.

This separation is central to understanding Proximal Policy Optimization (PPO), which combines a policy objective with a learned value function (**schulman2017ppo**). It is also important for understanding why later language-model methods investigate alternatives to a separately trained critic. The present section develops the actor–critic formulation from its probabilistic and statistical foundations, derives the principal estimators used by PPO, and then maps these components onto autoregressive language models.

5.2.1 Sequential decision-making and the policy represented by the actor

A sequential decision problem can be represented by a Markov decision process (MDP),

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \tag{5.96}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition distribution, r is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The Markov property means that, conditional on the current state and action, the distribution of the next state does not depend on the full history preceding that state. The state is therefore intended to retain the information needed to characterize the future evolution of the decision process.

At time t, the agent occupies state s_t, samples an action a_t, receives reward r_t, and moves to state s_{t+1}. A trajectory over a finite horizon T can be written as

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T). \tag{5.97}$$

The discounted return from time t is

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k. \tag{5.98}$$

The discount factor determines how strongly rewards further in the future contribute to the return. When $\gamma = 0$, only the immediate reward contributes. As γ approaches one,

later rewards receive greater weight. For an episodic finite-horizon problem, the sum terminates at the episode boundary.

The agent's behavior is described by a policy. In the stochastic, parameterized setting considered by policy-gradient methods, the policy is

$$\pi_\theta(a_t \mid s_t), \tag{5.99}$$

where θ denotes the trainable policy parameters. For a fixed state, $\pi_\theta(\cdot \mid s)$ is a probability distribution over the available actions. The policy objective is the expected return under trajectories generated by that policy:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G_0], \qquad \theta^\star = \arg\max_\theta J(\theta). \tag{5.100}$$

The actor is the parameterized function that implements this policy. In a neural implementation, it maps a state representation to the parameters of an action distribution. For a discrete action space with K actions, let $\mathbf{z}_\theta(s) \in \mathbb{R}^K$ denote the actor's logits. The probability assigned to action k is obtained through the softmax map:

$$\pi_\theta(a = k \mid s) = \frac{\exp(z_{\theta,k}(s))}{\sum_{j=1}^{K} \exp(z_{\theta,j}(s))}. \tag{5.101}$$

The resulting vector belongs to the probability simplex: every component is nonnegative and the components sum to one. The dimensionality of the actor's output is therefore determined by the size of the discrete action space.

For a differentiable stochastic policy, the policy-gradient estimator used in PPO has the form

$$\widehat{\mathbf{g}} = \widehat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \widehat{A}_t \right], \tag{5.102}$$

where $\widehat{\mathbb{E}}_t$ denotes an empirical average over sampled time steps and $\widehat{A}_t$ is an estimate of the advantage. The log-probability gradient describes how the probability of the sampled action changes as the policy parameters change. The advantage supplies the evaluation signal: a positive estimate encourages an increase in the probability of the sampled action under gradient ascent, whereas a negative estimate encourages a decrease. The policy-gradient paper formulates the corresponding surrogate objective as the empirical expectation of $\log \pi_\theta(a_t \mid s_t) \widehat{A}_t$ **(schulman2017ppo)**.

For a language model, the state at generation step t consists of the prompt and the tokens generated so far. If x denotes the prompt and $y_{<t} = (y_1, \ldots, y_{t-1})$ denotes the generated prefix, then

$$s_t = (x, y_{<t}), \qquad a_t = y_t, \qquad \pi_\theta(a_t \mid s_t) = \pi_\theta(y_t \mid x, y_{<t}). \tag{5.103}$$

The action is the next token selected from the vocabulary. If the vocabulary contains $|\mathcal{V}|$ tokens, the actor's language-model output distribution has $|\mathcal{V}|$ components. The probability of a complete response $y = (y_1, \ldots, y_T)$ follows from the autoregressive factorization:

$$\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t}), \qquad \log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t}). \tag{5.104}$$

Thus, language-model policy optimization is a sequence of token-level stochastic decisions, even when the reward is assigned to the completed response.

5.2.2 Value functions, the critic, and the advantage

The actor determines which actions are sampled, but the policy-gradient estimator also requires an estimate of how well a sampled action performed. The value-function formulation provides this evaluation. The state-value function of a policy π is the expected discounted return conditional on the current state:

$$V^{\pi}(s_t) = \mathbb{E}_{\pi}\left[G_t \mid s_t\right]. \tag{5.105}$$

It is an expectation over possible future trajectories, not the return of one particular sampled trajectory. Different trajectories can begin from the same state and produce different returns because the policy is stochastic and the transition process may be stochastic.

The action-value function conditions on both the current state and the action selected at that state:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{\pi}\left[G_t \mid s_t, a_t\right]. \tag{5.106}$$

The state-value function is the policy-weighted average of the action-value function:

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^{\pi}(s, a), \tag{5.107}$$

with an integral replacing the summation for a continuous action space. The corresponding Bellman expectation equation is

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[r(s, a, s') + \gamma V^{\pi}(s')\right]. \tag{5.108}$$

This equation expresses the expected value of a state in terms of the immediate reward and the discounted value of the next state, averaged over the policy's actions and the environment's transitions.

The advantage function compares the value of a particular action with the average value of the current policy at the same state:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \tag{5.109}$$

A positive advantage means that the action has a greater expected return than the policy's average action at that state. A negative advantage means that it has a lower expected return. The comparison is state-conditional: an absolute return that is large in one state may be poor in another state if the latter state normally leads to even larger returns.

The advantage has zero conditional expectation under the policy:

$$\begin{aligned} \mathbb{E}_{a \sim \pi(\cdot \mid s)}[A^{\pi}(s, a)] &= \mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q^{\pi}(s, a) - V^{\pi}(s)] \\ &= \mathbb{E}_{a \sim \pi(\cdot \mid s)}[Q^{\pi}(s, a)] - V^{\pi}(s) \\ &= V^{\pi}(s) - V^{\pi}(s) = 0. \end{aligned} \tag{5.110}$$

This property is central to the statistical role of the critic.

In a practical actor–critic system, the exact value function is generally not available. The critic therefore learns a parameterized approximation:

$$V_{\phi}(s) \approx V^{\pi}(s), \tag{5.111}$$

where ϕ denotes the critic parameters. Unlike the actor, whose output is a probability distribution over actions, the critic produces a scalar estimate of expected future return. If the state representation has dimension d_s, a conventional critic can be viewed as a function $V_\phi : \mathbb{R}^{d_s} \to \mathbb{R}$.

The critic is trained by regression against a target return or a return estimate. For a target $\widehat{V}_t^{\text{target}}$, the squared value-function error is

$$\mathcal{L}_V(\phi) = \frac{1}{2}\mathbb{E}\left[\left(V_\phi(s_t) - \widehat{V}_t^{\text{target}}\right)^2\right]. \tag{5.112}$$

When the empirical return G_t is used as the target, this becomes

$$\mathcal{L}_V(\phi) = \frac{1}{2}\mathbb{E}\left[\left(V_\phi(s_t) - G_t\right)^2\right]. \tag{5.113}$$

The factor $1/2$ is a convenient scaling factor. Differentiating the objective gives

$$\nabla_\phi \mathcal{L}_V(\phi) = \mathbb{E}\left[\left(V_\phi(s_t) - G_t\right)\nabla_\phi V_\phi(s_t)\right]. \tag{5.114}$$

Under squared-error minimization, the optimal prediction for a given state is the conditional expectation of the target. The critic is consequently a regression model for expected future return, rather than a predictor of the exact future trajectory.

The value function also provides a baseline for policy-gradient estimation. To see why subtracting a state-dependent baseline does not change the expected policy gradient, consider an arbitrary function $b(s_t)$ that does not depend on the sampled action once the state is fixed. Its conditional contribution to the gradient is

$$\begin{aligned}
&\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) b(s_t)\right] \\
&\quad = b(s_t)\sum_{a_t} \pi_\theta(a_t \mid s_t)\nabla_\theta \log \pi_\theta(a_t \mid s_t) \\
&\quad = b(s_t)\nabla_\theta \sum_{a_t} \pi_\theta(a_t \mid s_t) = b(s_t)\nabla_\theta 1 = 0.
\end{aligned} \tag{5.115}$$

The identity uses $\pi_\theta(a \mid s)\nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta \pi_\theta(a \mid s)$ and the normalization of the policy distribution. Therefore, replacing G_t by $G_t - b(s_t)$ preserves the expected policy gradient. Choosing $b(s_t) = V_\phi(s_t)$ gives the practical return-minus-value advantage estimate

$$\widehat{A}_t = G_t - V_\phi(s_t). \tag{5.116}$$

The baseline does not need to be exact for the baseline identity to hold; it must, however, be a function of the state and not of the sampled action conditional on that state. The accuracy of the critic still matters for the variance of the estimator and, when bootstrapped targets are used, for the bias of the resulting advantage estimate.

5.2.3 Temporal-difference estimation and generalized advantage estimation

The empirical return in equation (5.113) uses the rewards observed over the subsequent trajectory. This Monte Carlo target does not require a value prediction for the next state, but it can have substantial variance because it depends on all subsequent rewards. An

alternative is temporal-difference (TD) estimation, which uses the immediate reward and the critic's estimate of the next state's value.

The one-step TD residual is

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t). \tag{5.117}$$

The first term is the observed immediate reward. The second is the discounted estimate of the next state's value. The final term is the critic's current estimate. A positive residual indicates that the immediate reward plus the estimated continuation value exceeds the critic's current prediction; a negative residual indicates the opposite. Because the next-state value is supplied by the critic, this estimator uses bootstrapping.

Generalized Advantage Estimation (GAE), used in the PPO formulation, combines TD residuals across future time steps (**schulman2017ppo**). For a finite trajectory segment ending at time T, its estimator is

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{\ell=0}^{T-t-1} (\gamma\lambda)^\ell \delta_{t+\ell}, \qquad \delta_{t+\ell} = r_{t+\ell} + \gamma V_\phi(s_{t+\ell+1}) - V_\phi(s_{t+\ell}), \tag{5.118}$$

where $\lambda \in [0, 1]$ controls the weighting of future TD residuals. The discount factor γ controls the contribution of future rewards, whereas λ controls the degree to which the estimator relies on longer sequences of residuals rather than short-horizon bootstrapping.

When $\lambda = 0$, the estimator reduces to the one-step TD residual:

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,0)} = \delta_t. \tag{5.119}$$

When $\lambda = 1$, the residuals telescope:

$$\begin{aligned} \widehat{A}_t^{\mathrm{GAE}(\gamma,1)} &= \sum_{\ell=0}^{T-t-1} \gamma^\ell \left[r_{t+\ell} + \gamma V_\phi(s_{t+\ell+1}) - V_\phi(s_{t+\ell}) \right] \\ &= \sum_{\ell=0}^{T-t-1} \gamma^\ell r_{t+\ell} + \gamma^{T-t} V_\phi(s_T) - V_\phi(s_t). \end{aligned} \tag{5.120}$$

For a completed episode with terminal value $V_\phi(s_T) = 0$, this is the Monte Carlo return minus the current value estimate. For a truncated trajectory segment that has not reached a terminal state, the final bootstrap term $\gamma^{T-t} V_\phi(s_T)$ remains. This is why a finite-horizon implementation can use a value estimate at the end of a collected segment rather than requiring the complete episode to be observed.

The parameter λ controls a bias–variance trade-off. Smaller values place more weight on short-horizon TD residuals and therefore rely more heavily on the critic's bootstrap predictions. This can reduce sampling variance, but errors in the critic can then influence the advantage estimate more directly. Larger values incorporate more sampled rewards and rely less on intermediate bootstrap predictions, but the estimate can inherit more of the variance of the sampled trajectory. The choice of λ therefore balances the statistical properties of the sampled returns against the approximation quality of the critic.

For example, suppose $r_t = 0$, $\gamma = 1$, $V_\phi(s_t) = 0.60$, and $V_\phi(s_{t+1}) = 0.80$. The first TD residual is

$$\delta_t = 0 + 0.80 - 0.60 = 0.20. \tag{5.121}$$

If the next residual is $\delta_{t+1} = -0.10$ and $\lambda = 0.5$, the two-residual estimate is

$$\widehat{A}_t = \delta_t + \gamma\lambda\delta_{t+1} = 0.20 + (1)(0.5)(-0.10) = 0.15. \tag{5.122}$$

The positive result indicates that the sampled action performed better than the critic's current expectation, after incorporating the specified future residual. The calculation also shows how a later negative residual can reduce, but need not reverse, the estimated advantage.

5.2.4 The actor–critic objective in Proximal Policy Optimization

The critic supplies an advantage estimate, but it does not by itself constrain how far the actor moves during optimization. If the actor is updated repeatedly using the same sampled trajectories, its action probabilities can move substantially away from those of the policy that generated the data. PPO addresses this issue with a clipped surrogate objective (**schulman2017ppo**).

Let θ_{old} denote the policy parameters used to collect the current batch of trajectories. The probability ratio for a sampled state–action pair is

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{5.123}$$

At the beginning of the update, when $\theta = \theta_{\text{old}}$, the ratio equals one. Values above one mean that the current policy assigns a larger probability to the sampled action than the data-collection policy; values below one mean that it assigns a smaller probability.

The unclipped surrogate term is $\rho_t(\theta)\widehat{A}_t$. PPO modifies it by clipping the ratio to an interval around one:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \widehat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta)\widehat{A}_t,\ \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)\widehat{A}_t \right) \right], \tag{5.124}$$

where $\epsilon > 0$ is a hyperparameter controlling the clipping interval. The minimum makes the objective conservative: it removes the incentive for a probability-ratio change beyond the clipping threshold when that change would otherwise improve the surrogate objective. It does not impose a hard guarantee that every policy distribution remains inside a fixed distance of the old policy; it modifies the optimization objective so that certain large changes no longer provide additional improvement.

The effect depends on the sign of the advantage. If $\widehat{A}_t > 0$, increasing the probability of the sampled action improves the unclipped term, but the clipped term stops providing additional incentive once the ratio exceeds $1 + \epsilon$. If $\widehat{A}_t < 0$, decreasing the probability of the sampled action improves the objective, but the lower clipping threshold prevents further surrogate improvement once the ratio falls below $1 - \epsilon$.

For a numerical illustration, let $\epsilon = 0.2$, $\rho_t(\theta) = 1.35$, and $\widehat{A}_t = 0.4$. The unclipped contribution is $1.35(0.4) = 0.54$, whereas the clipped contribution is $1.2(0.4) = 0.48$. The minimum is therefore 0.48. The objective does not reward the additional probability increase beyond the upper clipping threshold.

The critic is trained alongside the actor. Let V_t^{target} denote the return or bootstrapped target used for value regression. The value-function error is

$$\mathcal{L}_t^{\text{VF}}(\phi) = \left(V_\phi(s_t) - V_t^{\text{target}} \right)^2. \tag{5.125}$$

PPO can also include an entropy bonus to discourage premature concentration of the policy distribution. For a discrete action distribution, its entropy is

$$\mathcal{H}(\pi_\theta(\cdot \mid s_t)) = -\sum_{a \in \mathcal{A}} \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t). \tag{5.126}$$

The combined actor–critic objective, written as an objective to maximize, can be expressed as

$$\begin{aligned} \mathcal{J}^{\text{PPO}}(\theta, \phi) = \widehat{\mathbb{E}}_t \Big[& \ell_t^{\text{CLIP}}(\theta) - c_1 \mathcal{L}_t^{\text{VF}}(\phi) \\ & + c_2 \mathcal{H}(\pi_\theta(\cdot \mid s_t)) \Big], \end{aligned} \tag{5.127}$$

where ℓ_t^{CLIP} is the per-sample term inside the expectation in equation (5.124), and c_1, c_2 weight the value error and entropy bonus. The negative sign on the value error reflects the fact that the combined objective is maximized, whereas the critic's regression error is minimized.

The actor and critic may use separate parameter sets or share part of a neural representation. When parameters are shared, the policy surrogate and value error both contribute gradients to the shared representation. The PPO paper explicitly describes this combined objective and the possibility of shared parameters between the policy and value function (**schulman2017ppo**). The relative weighting of the two losses then matters because the actor and critic impose different learning signals on the shared representation.

The resulting training cycle alternates between data collection and optimization. The current policy generates trajectories; the algorithm computes reward and value information; advantage estimates are constructed; and the actor and critic are optimized over minibatches for several epochs. The policy used for the next collection phase is then updated to the newly optimized policy. The following algorithm summarizes this actor–critic cycle in the notation used for language-model training.

Algorithm 3: PPO-style actor–critic optimization for autoregressive language models

Input: Initial policy π_θ, reference policy π_{ref}, reward model R_ψ, prompt distribution $\mathcal{D}$, discount γ, GAE parameter λ, clipping parameter ϵ, and optimization budget

Output: Updated policy parameters θ

Initialize or load the critic V_ϕ;
for *each training iteration until the optimization budget is exhausted* **do**
  Sample a batch of prompts from $\mathcal{D}$;
  Set $\theta_{\text{old}} \leftarrow \theta$;
  Generate responses from $\pi_{\theta_{\text{old}}}$;
  Evaluate completed responses with R_ψ;
  Construct token-level rewards, including the reference-policy penalty;
  Evaluate the critic on the prompt and each response prefix;
  Compute TD residuals and GAE advantages;
  Construct return targets for value-function regression;
  for *each optimization epoch and minibatch* **do**
    Compute the probability ratios using $\pi_\theta / \pi_{\theta_{\text{old}}}$;
    Update the actor using the clipped surrogate objective;
    Update the critic using the value-function regression objective;

The actor in this algorithm is the language model's policy. This should not be confused with the parallel data-collection actors sometimes used in conventional PPO implementations: those are rollout workers that execute the policy to collect trajectories. Schulman et al. describe the alternation between parallel trajectory collection and multiple epochs of minibatch optimization in their actor–critic-style PPO algorithm (**schulman2017ppo**).

5.2.5 Mapping the actor–critic architecture onto language models

The MDP representation of language generation makes the roles of the actor and critic explicit. A prompt x initiates an episode, a partial response $x \circ y_{<t}$ is the state, and the next token y_t is the action. After the token is appended, the next state is $x \circ y_{\le t}$. Ivison et al. describe language generation in this way when detailing PPO for preference-based language-model training (**ivison2024unpacking**).

At the actor's output interface, a language-model hidden representation $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ is mapped to a vector of vocabulary logits $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$. A softmax converts these logits into the token distribution. The critic, in contrast, maps the representation of the same prompt–prefix state to a scalar:

$$V_\phi(x \circ y_{<t}) \in \mathbb{R}. \tag{5.128}$$

In an implementation with a scalar regression head, the critic can use a representation of dimension d_{model} and map it to one real-valued prediction. The regression head can be evaluated at each response-token position, so the critic estimates the expected return of incomplete responses rather than assigning a value only to the completed sequence. Ivison et al. report that their value model uses an architecture similar to the reward model, with a regression head that can be applied at response-token positions (**ivison2024unpacking**).

The reward model and the reference model have different roles from the critic. A reward model $R_\psi(x, y)$ assigns a scalar score to a completed response. In preference-based training, it is learned from comparisons between responses. The reference policy π_{ref} is a fixed policy against which the optimized actor is regularized. The critic $V_\phi(x \circ y_{<t})$ estimates expected future return from an intermediate state. Although the reward model and critic both produce scalar outputs, they do not estimate the same quantity: one scores a response, while the other predicts the expected return from a partial response under the active policy.

In the PPO formulation described by Ivison et al., the sequence-level reward is applied at the final response token, while a token-level regularization term penalizes divergence from the reference policy. A corresponding token-level reward can be written as

$$r_t = -\beta \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} + \mathbb{I}[t = T] R_\psi(x, y), \tag{5.129}$$

where β controls the strength of the reference-policy penalty and $\mathbb{I}[t = T]$ is one at the final response token and zero otherwise. The discounted return from a response prefix is then

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k, \tag{5.130}$$

with the terminal indexing understood according to the response boundary. The critic estimates the expected value of this return from the current prompt–prefix state. Its estimate is used to form token-level advantages, which then guide the actor's update.

figure 5.7 summarizes the information flow. The actor generates a token from the current state. The resulting response is evaluated through the reward process. In parallel, the critic estimates the value of the current state. The reward and value estimate jointly determine the advantage signal, which is used to update the actor. The diagram is a conceptual synthesis of the PPO actor–critic formulation and its language-model interpretation (**schulman2017ppo**; **ivison2024unpacking**).

The value model is trained by minimizing a regression error against empirical or bootstrapped returns. Ivison et al. describe the value loss as a mean-squared error between the predicted value of each incomplete response and its empirical return. They also report implementation choices in which the value model is initialized from the reward model, while other implementations initialize it from a supervised-fine-tuned or base model and replace the language-model output head with a regression head (**ivison2024unpacking**). These are implementation choices, not defining properties of the actor–critic architecture.

5.2.6 Computational limitations and the transition beyond a learned critic

The critic provides a principled baseline for variance-reduced policy-gradient estimation, but it introduces a second learned model into the training system. In large language models, a value model with a parameter scale comparable to that of the policy can impose substantial memory and computational costs. The cost is not limited to storing the additional parameters: the critic must also be evaluated on response prefixes and trained through a value-regression objective. These requirements become more consequential when responses are long and the critic must produce estimates at many token positions.

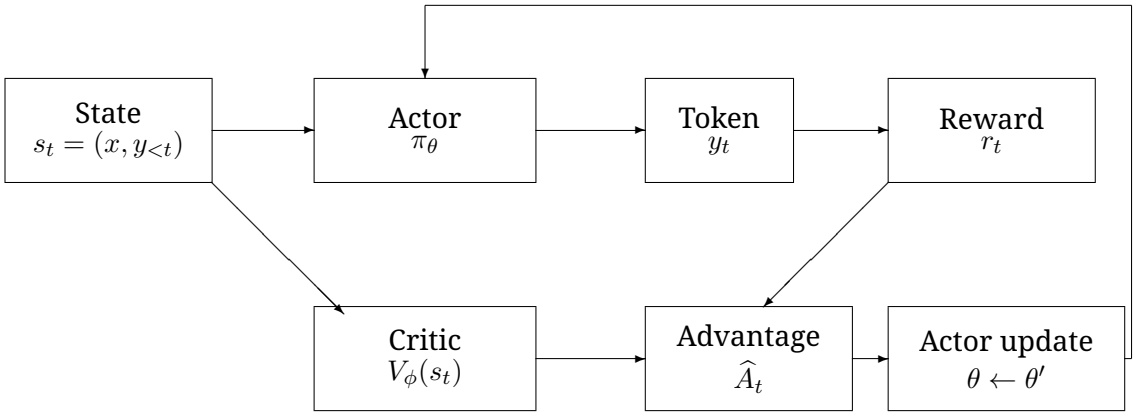


Figure 5.7: Information flow in an actor–critic language-model training step. The actor maps the prompt and generated prefix to a token distribution, while the critic estimates the value of that prefix. Reward information and the value estimate are combined to construct an advantage used in the policy update. Conceptual synthesis based on **schulman2017ppo** and **ivison2024unpacking**.

A further difficulty arises when the reward model supplies a sequence-level score primarily at the end of a response. The critic must then learn to predict the expected eventual outcome from intermediate prefixes, even though the decisive reward signal may not be available until generation finishes. Shao et al. identify both the computational burden of a value model comparable in size to the policy and the difficulty of training accurate token-level values when the reward is concentrated at the final token (**shao2024deepseekmath**).

These limitations motivate methods that estimate a baseline without training a separate value model. One example is Group Relative Policy Optimization (GRPO), introduced by Shao et al. as a PPO variant. For a prompt q, the method samples several responses $\{y_1, \ldots, y_G\}$ from the old policy and evaluates their rewards $\{R_1, \ldots, R_G\}$. A group mean can be formed as

$$\overline{R} = \frac{1}{G}\sum_{i=1}^{G} R_i, \qquad \sigma_R = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(R_i - \overline{R})^2}. \tag{5.131}$$

The normalized relative reward used as a group-based advantage is

$$\widehat{A}_i = \frac{R_i - \overline{R}}{\sigma_R}, \tag{5.132}$$

when the group standard deviation is nonzero. The response-level estimate can be associated with the tokens of the corresponding response. This construction replaces the learned state-value baseline with a baseline calculated from the rewards of multiple responses to the same prompt. Shao et al. use this group-relative comparison to remove the additional value-function approximation in GRPO (**shao2024deepseekmath**).

A group-relative baseline and a learned critic are not identical estimators. The critic predicts a state-dependent expected return for an individual prompt–prefix state. A group baseline compares completed responses generated for the same prompt. The latter avoids training an additional value model, but it does not provide the same token-level state-value approximation. If all responses in a group receive the same reward, then $\sigma_R = 0$ and the normalized expression in equation (5.132) is undefined; such a group contains no within-group reward variation from which to construct a relative ranking. The group-based construction therefore changes the statistical source of the baseline rather than eliminating the need to estimate relative performance.

The actor–critic architecture establishes how a learned value function can convert delayed trajectory outcomes into state-conditioned advantage estimates, and how those estimates can be combined with a constrained policy update. Its limitations also clarify why later language-model algorithms investigate alternative estimators and reward structures. The next section turns from the architecture used to estimate value toward the reward signal itself, with particular attention to reinforcement learning in which outcomes can be checked against verifiable task criteria.

5.3 Reinforcement Learning with Verifiable Rewards

Having established the actor–critic framework and the roles of the policy and value functions, we can now examine how a language model can use an external criterion of correctness to improve its generated responses. The central question is how a scalar assessment of a completed response can be converted into a learning signal for the token-level decisions that produced it. This question motivates *Reinforcement Learning with Verifiable Rewards* (RLVR), a post-training approach in which the reward is obtained from a verification procedure rather than exclusively from a learned model of human preferences.

The distinction matters because not every desirable property of a language model is equally difficult to evaluate. Helpfulness, style, and conversational quality often require contextual judgments. By contrast, the final answer to a well-specified mathematical problem can frequently be checked against a known answer, and explicitly stated instruction constraints can sometimes be tested programmatically. When a task admits a reliable verifier, the model can be optimized against the outcome of its own generated response. The Tülu 3 training framework provides a concrete implementation of this idea for mathematical reasoning and precise instruction following (**lambert2025tulu3**).

RLVR is best understood as a change in the source of the reward, not as a replacement for the probabilistic generation mechanism of a language model. The model remains an autoregressive policy, and its parameters are still updated through reinforcement learning. What changes is the function that evaluates a completed response. The following development begins with the connection between autoregressive generation and episodic decision-making, then formulates the verifiable reward objective, and finally explains how PPO-based optimization, data construction, and the Tülu 3 experiments instantiate the method.

5.3.1 From autoregressive language modeling to outcome-based learning

A language model represents a conditional probability distribution over possible token sequences. Let x denote an input prompt and let $y = (y_1, y_2, \ldots, y_T)$ denote a response consisting of T generated tokens. The model is parameterized by θ, and its autoregressive factorization is

$$\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t}), \qquad y_{<t} = (y_1, \ldots, y_{t-1}). \tag{5.133}$$

The notation π_θ emphasizes that the language model can be interpreted as a stochastic policy. At each generation step, the model assigns a probability to every token in its vocabulary, conditioned on the prompt and the tokens already generated. A token is sampled or selected according to this conditional distribution, appended to the sequence, and used as part of the context for the next prediction. Generation ends when an end-of-sequence (EOS) token is produced or when a prescribed generation limit is reached.

This process has the structure of an episodic decision problem. In reinforcement learning, an agent observes a state, selects an action according to a policy, and moves to a subsequent state. For autoregressive generation, the state at step t can be written as

$$s_t = (x, y_{<t}), \qquad a_t = y_t. \tag{5.134}$$

The prompt and the previously generated tokens form the state, while the next token is the action. Once the action is selected, the next state is obtained by appending that token to the sequence. Conditional on the selected token, this transition is deterministic at the level of the token history; the stochasticity lies in the policy's selection of the token. A complete response therefore corresponds to a trajectory through the state space of the generation process.

The distinction between a token-level action and a sequence-level outcome is central to RLVR. The verifier ordinarily evaluates the completed response, not each token independently. A mathematical answer may be judged correct only after the model has produced its final result, and an instruction may be judged satisfied only after the complete response can be checked against its constraints. Thus, a scalar reward can be assigned at the end of a trajectory even though the policy made a sequence of token-level decisions.

In supervised fine-tuning, a model is trained to increase the likelihood of a supplied reference response. If $y^\star$ is the reference sequence, the usual maximum-likelihood objective is to minimize the negative conditional log-likelihood,

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y^\star)} \left[\log \pi_\theta(y^\star \mid x) \right]. \tag{5.135}$$

This objective directly rewards agreement with the demonstrated sequence. It does not, by itself, distinguish between alternative responses that are all correct. RLVR instead evaluates sampled responses according to whether they satisfy a specified outcome criterion. Different reasoning trajectories can consequently receive the same reward if they lead to an answer accepted by the verifier.

The objective is therefore not to reproduce one prescribed token sequence. It is to increase the probability that the policy generates a response belonging to the set of responses accepted by the verification procedure. This changes the source of supervision while preserving the autoregressive structure of the language model.

5.3.2 Verifiable rewards and the KL-regularized objective

In conventional reinforcement learning from human feedback (RLHF), a learned reward model commonly maps a prompt–response pair to a scalar score. Let

$$r_\phi(x, y) \in \mathbb{R} \tag{5.136}$$

denote such a reward model, where ϕ denotes its parameters. Its output is an estimate of response quality learned from preference information. Because it is a learned statistical model, its predictions can contain estimation error and can reflect limitations in the preference data and the reward-model training procedure.

RLVR replaces this learned estimate with a verification function. Let

$$C(x, y) \in \{0, 1\} \tag{5.137}$$

denote a binary verifier, where $C(x, y) = 1$ means that response y satisfies the criterion associated with prompt x, and $C(x, y) = 0$ means that it does not. The corresponding reward is

$$v(x, y) = \begin{cases} \alpha, & C(x, y) = 1, \\ 0, & C(x, y) = 0, \end{cases} \tag{5.138}$$

where $\alpha > 0$ is the reward magnitude. Equivalently,

$$v(x, y) = \alpha\, \mathbb{I}[C(x, y) = 1], \tag{5.139}$$

where $\mathbb{I}[\cdot]$ is the indicator function. In the Tülu 3 implementation, the authors set $\alpha = 10$, based on pilot experiments, and did not tune it further (**lambert2025tulu3**).

A binary verifier maps a potentially complex response space to two reward values. All responses that fail the specified condition receive zero, regardless of how close they are to satisfying it. Responses that pass receive the same positive reward, even if their reasoning paths or writing styles differ. The reward therefore represents satisfaction of the specified criterion, not a continuous estimate of general response quality.

For a fixed prompt x, the expected verifiable reward under the current policy is

$$\begin{aligned} \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[v(x, y)] &= \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\alpha\, \mathbb{I}[C(x, y) = 1]\right] \\ &= \alpha \Pr_{y\sim\pi_\theta(\cdot|x)}[C(x, y) = 1]. \end{aligned} \tag{5.140}$$

The equality follows from the definition of the expectation of an indicator function. It gives the central interpretation of binary RLVR: maximizing expected reward is equivalent to maximizing the probability that the policy generates a response accepted by the verifier, up to the positive scaling factor α.

Let $\mathcal{D}$ denote the distribution of training prompts. Averaging over this distribution gives

$$J_{\text{ver}}(\theta) = \alpha\, \mathbb{E}_{x\sim\mathcal{D}}\left[\Pr_{y\sim\pi_\theta(\cdot|x)}[C(x, y) = 1]\right]. \tag{5.141}$$

An unconstrained maximization of this objective can move the policy substantially away from the distribution learned during pre-training and earlier post-training. Such

movement may improve the targeted verifier score while degrading behavior that is not measured by that verifier. RLVR therefore uses a regularization term based on the Kullback–Leibler (KL) divergence between the current policy and a reference policy.

For discrete probability distributions P and Q, the KL divergence is

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{z} P(z) \log \frac{P(z)}{Q(z)}. \tag{5.142}$$

It is the expected logarithmic probability ratio under P. The divergence is nonnegative, provided the distributions satisfy the relevant support conditions, and it is zero when the two distributions are identical. For a prompt-conditioned language model, the relevant quantity is

$$D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \parallel \pi_{\mathrm{ref}}(\cdot \mid x)\right), \tag{5.143}$$

where π_{ref} is the reference policy. The current policy π_θ is trainable, whereas the reference policy supplies the distributional anchor against which the change in the current policy is measured.

The KL-regularized RLVR objective used in Tülu 3 can be written as

$$\boxed{\begin{aligned} \max_{\theta} \quad & J_{\mathrm{RLVR}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}}\Big[\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[v(x,y)] \\ & \qquad - \beta D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \parallel \pi_{\mathrm{ref}}(\cdot \mid x)\right)\Big], \end{aligned}} \tag{5.144}$$

where $\beta \geq 0$ controls the strength of the reference-policy regularization. A larger β makes divergence from the reference policy more costly; a smaller β permits greater movement in pursuit of the verification reward. The coefficient therefore controls a trade-off between targeted outcome optimization and preservation of the reference distribution.

The KL term can also be expressed at the token level. From the autoregressive factorization in equation (5.133),

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t}). \tag{5.145}$$

Applying the same identity to the reference policy gives

$$\log \frac{\pi_\theta(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})}. \tag{5.146}$$

Taking the expectation under the current policy yields

$$D_{\mathrm{KL}}\left(\pi_\theta(\cdot \mid x) \parallel \pi_{\mathrm{ref}}(\cdot \mid x)\right) = \mathbb{E}_{y \sim \pi_\theta}\left[\sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\mathrm{ref}}(y_t \mid x, y_{<t})}\right]. \tag{5.147}$$

Thus, a sequence-level divergence can be understood as the expected sum of token-level log-probability differences along trajectories sampled from the current policy. The verifier evaluates the complete response, while the KL regularizer accumulates differences between the conditional token distributions of the current and reference policies.

The reward scale and the KL coefficient must be interpreted jointly. In Tülu 3, a successful verification receives a reward of ten, while the KL coefficient is varied across experiments. Changing the relative scale of these terms changes the optimization trade-off even when the verifier itself is unchanged.

The policy-gradient formulation explains why a nondifferentiable verifier can still be used for learning. For a fixed prompt, define

$$J(\theta) = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[v(x,y)] = \sum_y \pi_\theta(y \mid x) v(x,y). \tag{5.148}$$

Differentiating and applying the logarithmic derivative identity gives

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_y \nabla_\theta \pi_\theta(y \mid x) v(x,y) \\
&= \sum_y \pi_\theta(y \mid x) \nabla_\theta \log \pi_\theta(y \mid x) v(x,y) \\
&= \mathbb{E}_{y\sim\pi_\theta(\cdot|x)} \left[v(x,y) \nabla_\theta \log \pi_\theta(y \mid x) \right].
\end{aligned} \tag{5.149}$$

The derivative is taken through the policy probability, not through the verifier. The verifier may perform exact answer comparison or evaluate a constraint using a discrete programmatic test; it does not need to be differentiable with respect to θ. Using the autoregressive factorization, the sequence-level gradient becomes

$$\nabla_\theta J(\theta) = \mathbb{E}_{y\sim\pi_\theta} \left[v(x,y) \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \right]. \tag{5.150}$$

A scalar outcome reward can therefore influence the parameters involved in all token decisions along the sampled trajectory. For the full KL-regularized objective, the gradient also contains the derivative of the KL term. The score-function derivation above isolates the reward component and shows why nondifferentiable outcome evaluation is compatible with policy-gradient learning.

5.3.3 PPO optimization, value estimation, and implementation

The policy-gradient estimator in equation (5.149) can have high variance because a single response-level reward is associated with many token-generation decisions. Actor–critic methods address this problem by learning a value function that estimates the expected return from an intermediate state. This is the role of the value model in PPO-based RLVR.

Let $V_\psi(s_t)$ denote a value function parameterized by ψ. It estimates the expected return from state $s_t = (x, y_{<t})$ under the policy being considered. The value function is not the verifier and does not define correctness. The verifier supplies the task reward; the value function estimates expected returns and supports the construction of advantage estimates.

The action-value function $Q(s_t, a_t)$ represents the expected return after taking action a_t in state s_t. The advantage function is

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t). \tag{5.151}$$

An advantage is positive when the action performs better than the state-value baseline and negative when it performs worse. In sampled training, an estimated return $\widehat{G}_t$ can be used to construct

$$\widehat{A}_t = \widehat{G}_t - V_\psi(s_t). \tag{5.152}$$

Subtracting a state-dependent baseline can reduce the variance of the policy-gradient estimator without changing its expectation, provided that the baseline does not depend on the sampled action. In particular,

$$\begin{aligned}\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] &= \sum_{a_t} \pi_\theta(a_t \mid s_t)\nabla_\theta \log \pi_\theta(a_t \mid s_t) \\ &= \sum_{a_t} \nabla_\theta \pi_\theta(a_t \mid s_t) \\ &= \nabla_\theta \sum_{a_t} \pi_\theta(a_t \mid s_t) = 0.\end{aligned} \tag{5.153}$$

It follows that subtracting a state-only baseline does not change the expected policy gradient. A learned value function supplies such a baseline while adapting it to the generation state.

A common estimator used with PPO is Generalized Advantage Estimation (GAE). Define the temporal-difference residual as

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t), \tag{5.154}$$

where r_t is the reward at step t, γ is the discount factor, and V_ψ is the value function. GAE combines residuals from multiple future steps:

$$\widehat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \tag{5.155}$$

where λ controls the weighting of longer-horizon residuals. Lower values of λ emphasize shorter-horizon estimates, while values closer to one incorporate more of the later trajectory. The discount factor γ controls the relative contribution of future rewards. Tülu 3 uses $\gamma = 1.0$ and $\lambda = 0.95$ in its reported PPO configuration (**lambert2025tulu3**). With $\gamma = 1$, future rewards are not geometrically discounted in this episodic formulation.

PPO updates the policy using trajectories sampled from a preceding policy. Let θ_{old} denote the parameters of the policy that generated the current rollout, and let θ denote the parameters being optimized. For a sampled state–action pair, define the probability ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{5.156}$$

A ratio greater than one means that the current policy assigns a higher probability to the sampled action than the rollout policy did; a ratio below one means that the probability has decreased. An unclipped surrogate objective would use $\rho_t(\theta)\widehat{A}_t$. Large changes in the ratio, however, can produce overly aggressive updates and make the optimization rely on samples generated by a substantially different policy.

PPO controls this incentive through the clipped surrogate objective,

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \widehat{A}_t,\ \text{clip} \left(\rho_t(\theta), 1-\epsilon, 1+\epsilon \right) \widehat{A}_t \right) \right],} \tag{5.157}$$

where ϵ is the clipping coefficient. When the advantage is positive, the objective favors increasing the probability of the sampled action, but clipping limits the incentive for an excessively large increase. When the advantage is negative, the objective favors decreasing that probability, while clipping limits the incentive for an excessively large decrease. Tülu 3 sets $\epsilon = 0.2$, giving the ratio interval $[0.8, 1.2]$ in the clipping operation.

The PPO clipping mechanism and the KL regularization in equation (5.144) should not be conflated. The ratio in equation (5.156) compares the current policy with the policy that generated the present rollout. The KL term compares the current policy with the reference policy. The two mechanisms use different reference distributions and serve different purposes: clipping controls the local policy update, whereas KL regularization anchors the policy to the selected reference distribution.

The value model is trained to predict returns. A squared-error value loss can be written as

$$L_V(\psi) = \mathbb{E}_t \left[\left(V_\psi(s_t) - \widehat{G}_t \right)^2 \right], \tag{5.158}$$

where $\widehat{G}_t$ is the return target used by the implementation. The policy and value model therefore have distinct roles: the policy learns which token actions should become more probable, while the value model estimates expected returns to support advantage estimation.

The Tülu 3 implementation uses a value-function coefficient $c_1 = 0.1$, a gradient-norm threshold of 1.0, and a linear learning-rate schedule. It also normalizes the advantage estimates by subtracting their batch mean and dividing by their batch standard deviation:

$$\widehat{A}_t^{\text{norm}} = \frac{\widehat{A}_t - \mu_A}{\sigma_A}, \tag{5.159}$$

where μ_A and σ_A are the mean and standard deviation of the advantages over the normalization batch. This operation controls the numerical scale of the policy-gradient signal. The implementation sets dropout to zero during reward-model and reinforcement-learning training so that rollout-time and learning-time token log probabilities remain consistent. If stochastic dropout causes these probabilities to differ, the probability ratio can depart from one even at the start of the first PPO epoch, leading to excessive clipping and potentially eliminating useful gradients (**lambert2025tulu3**).

The response termination condition is also treated explicitly. PPO samples responses up to a maximum token length, and a response that reaches this limit without an EOS token may be incomplete. Tülu 3 applies an additional penalty of -10 to responses without an EOS token. This termination penalty is an implementation detail in addition to the binary verification reward; it encourages the policy to complete its responses rather than repeatedly generating to the maximum length.

The complete relationship between generation, verification, regularization, value estimation, and policy updating is summarized in figure 5.8. The policy generates a response from a prompt, the verifier evaluates the completed response, and the resulting reward is used together with the KL term and the advantage estimates in the PPO update. The

reference policy supplies the distribution used for the KL regularizer, while the value model supplies estimates used to construct advantages. The verifier and value model are therefore not interchangeable: one defines the task outcome, and the other supports optimization.

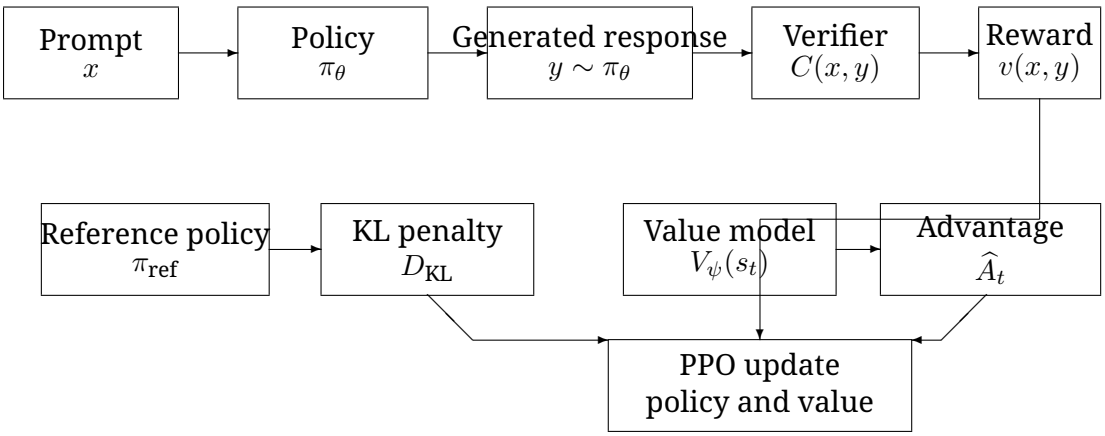


Figure 5.8: Conceptual structure of PPO-based reinforcement learning with verifiable rewards. The policy generates a response that is evaluated by a verifier. The resulting reward, the KL penalty relative to the reference policy, and advantage estimates derived from the value model inform the PPO update. Original schematic based on the RLVR framework described by **lambert2025tulu3**.

5.3.4 Verifiable data, training recipe, and empirical behavior

An RLVR dataset must provide more than input prompts. Each prompt must be associated with a verification procedure that can evaluate a generated response. A convenient representation is

$$\mathcal{D}_{\text{RLVR}} = \{(x_i, C_i)\}_{i=1}^{N}, \tag{5.160}$$

where x_i is a training prompt, C_i is its associated verifier, and N is the number of prompts. The response itself is sampled from the current policy during training; the dataset supplies the prompt and the procedure for evaluating the response.

The verifier is task-dependent. In mathematical reasoning, it can extract the final answer and compare it with a known answer. In precise instruction following, it can test whether a completion satisfies a specified constraint. The same binary reward interface can therefore be used across tasks whose underlying verification procedures differ.

For GSM8K, Tülu 3 uses the training set of 7,473 prompts. Each example is augmented with the standard eight-shot prompt used during evaluation to encourage chain-of-thought generation. The verifier extracts the final number from the generated response and compares it with the ground-truth label. In abstract form,

$$C_{\text{GSM8K}}(x, y) = \mathbb{I}\left[E_{\text{num}}(y) = a^\star(x)\right], \tag{5.161}$$

where E_{num} is the answer-extraction procedure and $a^\star(x)$ is the ground-truth answer.

For MATH, the training mixture contains 7,500 prompts. The examples are augmented with the standard three-shot chain-of-thought prompt used during evaluation. The generated answer is extracted and checked using the flexible MATH evaluation logic. This is important because mathematically equivalent answers can have different textual representations. A verifier based only on literal string equality could incorrectly reject two representations of the same mathematical result. The MATH verification procedure is therefore represented more generally as

$$C_{\text{MATH}}(x, y) = \mathbb{I}\left[E_{\text{MATH}}(y) \equiv a^{\star}(x)\right], \tag{5.162}$$

where $\equiv$ denotes the comparison logic used by the evaluation procedure.

The instruction-following component is constructed differently. Tülu 3 randomly samples instructions from the Tülu 2 supervised fine-tuning mixture and combines them with constraints drawn from the taxonomy used for IFEval. A prompt-specific verifier is constructed for each constraint template. If $g_j(x, y)$ denotes the test associated with constraint template j, then the verification function can be represented as

$$C_{\text{IF}}(x, y) = \mathbb{I}[g_j(x, y) = 1]. \tag{5.163}$$

The condition $g_j(x, y) = 1$ indicates that the generated completion satisfies the corresponding constraint. Unlike mathematical answer verification, this procedure need not extract one numerical answer; it checks the property specified by the instruction template.

The resulting Tülu 3 training mixture contains 29,946 prompts, divided equally between mathematical tasks and verifiable instruction following. The composition is shown in table 5.2.

Table 5.2: Composition of the Tülu 3 verifiable-reward prompt dataset. Source: **lambert2025tulu3**.

Training domain	**Prompts**	**Verification procedure**
GSM8K	7,473	Exact match against the extracted answer
MATH	7,500	Extracted-answer verification using flexible MATH evaluation logic
Verifiable instruction following	14,973	Prompt-specific constraint verifiers
Total	**29,946**	**Binary verification**

The mathematical portion contains

$$N_{\text{math}} = 7{,}473 + 7{,}500 = 14{,}973 \tag{5.164}$$

prompts, which equals the 14,973 prompts in the verifiable instruction- following component. The mixture therefore contains equal numbers of mathematical and instruction-following prompts. Although the reward interface is shared, the meaning of a successful response remains domain-dependent because each task uses its own verifier.

The Tülu 3 procedure applies RLVR after supervised fine-tuning and preference fine-tuning. The starting policy is therefore already an instruction-tuned model that has undergone preference optimization. In the reported experiments, the authors use DPO

checkpoints as the principal starting points and also investigate initialization from a weaker supervised fine-tuning checkpoint. This ordering separates the roles of the post-training stages. Supervised fine-tuning teaches the model from demonstrations, preference fine-tuning optimizes relative response preferences, and RLVR optimizes sampled responses according to verifiable outcomes.

The PPO configuration used in the experiments is summarized in table 5.3. The values distinguish the standard configuration reported for RLVR from settings that vary with model scale or the particular run.

Table 5.3: Selected PPO hyperparameters reported for the Tülu 3 RLVR experiments. Values are transcribed from the RLVR configuration in **lambert2025tulu3**.

Hyperparameter	**Reported value**	**Role or qualification**
Discount factor γ	1.0	No geometric discounting of future rewards
GAE parameter λ	0.95	Controls the temporal weighting in advantage estimation
PPO clipping coefficient ϵ	0.2	Clips the probability ratio to $[0.8, 1.2]$ in the surrogate objective
Value-function coefficient c_1	0.1	Weights the value-function loss
Gradient-norm threshold	1.0	Limits the gradient norm
Learning-rate schedule	Linear	Linear schedule during training
Generation temperature	1.0	Sampling temperature for response generation
Maximum prompt length	2,048 tokens	Maximum prompt context length in the reported configuration
Maximum token length	2,048 tokens	Maximum token limit in the reported configuration
Response length	2,048 tokens	1,024 tokens for GSM8K in the standard RLVR configuration
Learning rate	3×10^{-7}	1×10^{-7} for the reported 70B configuration
Effective batch size	224	640 for the reported 70B configuration
PPO update iterations	4	Number of PPO update iterations in the RLVR column
Total episodes	100,000	The final 70B run is reported separately with 400,000 episodes
KL coefficient β	$\{0.1, 0.05, 0.03, 0.01\}$	Sweep values in the standard RLVR configuration
Warmup ratio ω	0 or 0.1	Final 8B setting: 0; Table 21 70B setting: 0.07
Non-EOS penalty	-10	Additional penalty for responses without an EOS token

The paper reports a numerical inconsistency for the final 70B KL coefficient. Table 21 gives $\beta = 0.07$, while the prose describing the final 70B run in Section 6.4 reports $\beta = 0.7$. These values differ by a factor of ten, and the paper does not reconcile them. They should therefore not be silently treated as equivalent when reproducing the experiment. The same final-run discussion reports a learning rate of 1×10^{-7}, a warmup ratio of 0.1, a response length of 2,048 tokens, 400,000 episodes, and an effective batch size of 640. The warmup value also differs from the 0.07 listed in Table 21, so the prose and table should be consulted separately when reconstructing the exact run (**lambert2025tulu3**).

Several implementation decisions are important for interpreting these settings. The value model is initialized from a general reward model rather than directly from the DPO policy. The general reward model is trained on UltraFeedback data, but its predicted score is not used as the primary task reward in RLVR. It initializes the value model, whose purpose is to estimate returns for PPO. This distinction is important: replacing a learned reward model with a verifier does not remove the need for a value function in an actor–critic implementation.

The authors disable dropout during reward-model and RL training. As explained above, PPO compares token log probabilities obtained during rollout and learning. If dropout changes the forward computation between these phases, the probability ratio may not equal one at the start of the first update. This can lead to excessive clipping and weak or vanishing policy gradients. Disabling dropout makes the probability calculations consistent under the same model parameters and inputs.

The available prompt set is reused across multiple training epochs. In one GSM8K ablation, approximately 100,000 episodes are run using 7,473 prompts, which corresponds to

$$\frac{100{,}000}{7{,}473} \approx 13.38 \tag{5.165}$$

passes over the prompt set if reuse is approximately uniform. The prompts are shuffled between epochs. The authors also evaluate checkpoints during training and select checkpoints using development-set performance rather than relying only on the final training reward.

A simple numerical example clarifies the expected-reward interpretation. Suppose, hypothetically, that a policy samples eight responses to one prompt and that five pass the verifier. With $\alpha = 10$, the empirical success frequency is $5/8 = 0.625$, and the empirical mean verification reward is

$$\widehat{\mathbb{E}}[v] = \frac{5(10) + 3(0)}{8} = 6.25. \tag{5.166}$$

This value is an estimate of the expected reward for that prompt under the sampling policy. It is not a direct estimate of general reasoning ability: it measures the probability of satisfying the particular verifier under the particular prompt and generation procedure. A different verifier or prompt distribution can yield a different expected reward.

The Tülu 3 experiments examine several factors that influence the final model. Initializing the value model from a general reward model performs better in the reported GSM8K ablation than initializing it from the DPO policy. Starting RLVR from a weaker SFT model can reach a similar level of training verification reward, but it incurs greater KL divergence than starting from the stronger DPO model; the authors also find that a stronger starting model generally produces better test performance. These findings show that reaching a particular training reward and generalizing well are not identical outcomes.

The authors also test whether reward-model scores should be added to verifiable rewards. In their GSM8K experiments, using the verifiable reward alone performs better than combining it with reward-model scores, while the combined signal is noisier. This result supports the use of a direct verification signal when a reliable verifier is available, rather than assuming that an additional learned score must improve the optimization signal.

The final performance comparison is reported for the 8B and 70B Tülu 3 models. Selected results are reproduced in table 5.4. The comparison is between the DPO starting point and the RLVR-trained model; the base instruction-tuned model is included in the source paper but is not needed to isolate the effect of the RLVR stage.

Table 5.4: Selected benchmark results for the Tülu 3 DPO starting points and RLVR-trained models. Scores are reproduced from Table 23 of **lambert2025tulu3**.

Evaluation	8B DPO	8B RLVR	70B DPO	70B RLVR
Average score	64.4	64.8	75.9	76.0
MATH	42.0	43.7	62.3	63.0
GSM8K	84.3	87.6	93.5	93.5
IFEval (strict)	81.1	82.4	82.6	83.2
BigBenchHard	65.8	66.0	81.8	82.0
DROP	62.5	62.6	74.1	74.3
Safety6 task average	87.2	85.5	89.0	88.3

At the 8B scale, the average score increases from 64.4 to 64.8. The GSM8K score rises by 3.3 percentage points, from 84.3 to 87.6, while MATH increases by 1.7 points and strict IFEval by 1.3 points. These changes demonstrate targeted gains on the mathematical and instruction-following tasks used to construct the verifiable training mixture. The average improvement is nevertheless modest, and not every evaluation improves: the Safety6 task average decreases from 87.2 to 85.5.

At the 70B scale, the average score increases from 75.9 to 76.0. MATH increases from 62.3 to 63.0, and strict IFEval increases from 82.6 to 83.2, while GSM8K remains at 93.5. The authors suggest that the lack of GSM8K improvement is related to the strong initial score, which leaves less room for improvement on that benchmark. The Safety6 task average also decreases slightly, from 89.0 to 88.3. The results therefore do not support the claim that RLVR improves every capability of a general-purpose language model. They support the narrower conclusion that it can improve selected capabilities while other dimensions remain unchanged or decline.

The implementation also has substantial computational requirements. The Tülu 3 infrastructure uses ZeRO Stage 3 to distribute model parameters and applicable optimizer states across devices. The RLVR setup contains a policy model, a reference policy model, and a value model. The policy and value models are trained, whereas the reference policy is used for inference. The authors allocate dedicated inference GPUs and use Ray for resource allocation and vLLM with PagedAttention for efficient generation. PagedAttention uses paging-based memory management to reduce memory fragmentation and redundant duplication, enabling larger inference batches.

The authors further use asynchronous reinforcement learning. In a synchronous setup, inference GPUs first generate rollout data and learner GPUs subsequently optimize the policy. These stages can leave one set of GPUs idle while the other is working. The Tülu 3 implementation overlaps inference and training to reduce this idle time. Asynchronous generation, however, can produce stale trajectories if inference generates data faster than the learner consumes it. To improve reproducibility, the implementation trains on the second most recent inference data rather than arbitrarily old rollouts. The reported RL run times are approximately 65 hours for the 8B run on 8 GPUs and 60 hours

for the 70B run on 48 GPUs (**lambert2025tulu3**). These figures are implementation-specific measurements, not universal estimates of the cost of RLVR.

5.3.5 Limitations, interpretation, and connection to GRPO

The first limitation of RLVR is its dependence on the quality of the verification function. The policy is optimized against the criterion implemented by the verifier, not against an abstract notion of correctness that is independent of that implementation. If the verifier captures the intended objective reliably, maximizing its expected reward can improve the corresponding capability. If the verifier is incomplete or incorrectly specified, the model can learn to satisfy the implemented condition without achieving the intended outcome.

For example, a verifier that checks only whether a required phrase appears in a response can reward a completion containing that phrase even if the response does not provide the explanation that the task requires. In such a case, policy optimization can be successful with respect to the formal reward and unsuccessful with respect to the intended task. This is not a failure specific to PPO; it follows from the definition of the optimization objective. The verifier defines the outcome being optimized, while the optimizer determines how the policy is changed to increase that outcome.

A second limitation follows from the sparse binary reward. In a mathematical task, a response with largely correct intermediate reasoning but an incorrect final answer can receive the same zero reward as a response with entirely incorrect reasoning. Conversely, two correct responses receive the same positive reward even if one is more concise or uses a more efficient derivation. The mapping

$$v(x, y) \in \{0, \alpha\} \tag{5.167}$$

discards information about degrees of partial correctness, reasoning quality, and the efficiency of the solution. The value function and advantage estimator can improve the statistical use of sampled trajectories, but they do not add information that is absent from the verifier's reward.

A third limitation is overoptimization. Reducing β makes divergence from the reference policy less costly in the objective and generally permits the policy to move farther from its initial distribution. This can increase the targeted verification reward, but it can also damage performance on other evaluations or produce undesirable forms of specialization. The Tülu 3 experiments report that higher KL divergence is often associated with lower average scores in some settings, although this relationship is not universal. The authors also identify overoptimized outputs in some higher-KL instruction-following runs. Thus, KL divergence is a useful diagnostic, but neither a large nor a small value should be interpreted without considering task performance and independent evaluations.

A fourth limitation concerns generalization. The training objective is defined over a finite distribution of prompts and associated verifiers. High reward on this distribution does not mathematically guarantee transfer to new prompts, new task formulations, or capabilities not measured by the verifier. The Tülu 3 results show that training improvements can translate to corresponding test benchmarks, but they also show that broader evaluation metrics may remain unchanged or decline. Training reward must therefore be considered together with held-out evaluations and broader capability measurements.

The experiments also show that a verifier-based reward is not automatically improved by combining it with a learned reward-model score. In the reported GSM8K ablation, the verifiable reward alone performs better, while the combined signal is noisier. This is a useful methodological result, but it should not be generalized to every possible task or verifier. The conclusion supported by the experiment is that adding a learned score is not inherently beneficial when a direct verification signal is already available.

Finally, PPO-based RLVR has nontrivial computational demands. Response generation, policy optimization, reference-policy evaluation, and value estimation must be coordinated. Distributed model placement and asynchronous rollouts can improve hardware utilization, but asynchronous execution introduces the possibility of stale data and reproducibility difficulties. The training recipe is consequently part of the method: a mathematically appropriate reward objective is not sufficient if its implementation produces unstable probability ratios, unreliable advantage estimates, or rollouts that are too stale for the intended update.

The central result of this section is that verifiable outcomes can provide a direct reinforcement-learning signal for language models whenever a reliable evaluator exists. RLVR retains the autoregressive policy, uses a binary or otherwise explicitly verifiable reward, regularizes policy movement relative to a reference distribution, and employs an optimizer such as PPO to convert sampled outcomes into parameter updates. The Tülu 3 experiments demonstrate the approach on mathematical reasoning and precise instruction following, while also exposing the importance of verifier quality, value estimation, KL control, initialization, generalization, and computational infrastructure.

This formulation separates the definition of the reward from the choice of the policy-optimization algorithm. PPO uses a learned value function to construct advantages and a clipped probability-ratio objective to control updates. The next section examines Group Relative Policy Optimization (GRPO), an alternative optimization approach for verifiable-reward training. It builds on the same general setting of sampled responses and verifiable outcomes but changes how relative performance information is used to construct the policy update.

5.4 Group Relative Policy Optimization for Verifiable Rewards

6. Neuroscience and Cognitive Foundations

6.1 Introduction

6.2 From System 1 to System 2

Part IV

Data and Training

7. Training Data

7.1 Pre-Training Data

7.2 Post-Training Data

8. Training Methods

8.1 Pre-Training

8.2 Post-Training

8.3 Machine Unlearning

Part V

Evaluation, Impact, Philosophy, and Challenges

9. Evaluation

10. Environmental Impacts

11. Philosophy

12. Challenges